Neural Network Architextures and Optimizations

Table of Contents

Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift **2**

Layer Normalization **11**

Instance Normalization: The Missing Ingredient for Fast Stylization **23**

Improved Texture Networks: Maximizing Quality and Diversity in Feed-forward Stylization and Texture Synthesis **29**

Improving predictive inference under covariate shift by weighting the log-likelihood function **38**

A Literature Survey on Domain Adaptation of Statistical Classifiers **55**

On the importance of initialization and momentum in deep learning **64**

Adam: A Method for Stochastic Optimization **72**

Adaptive Subgradient Methods for Online Learning and Stochastic Optimization* **86**

END **121**

Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift

Sergey Ioffe
Google Inc., *sioffe@google.com*

Christian Szegedy
Google Inc., *szegedy@google.com*

Abstract

Training Deep Neural Networks is complicated by the fact that the distribution of each layer's inputs changes during training, as the parameters of the previous layers change. This slows down the training by requiring lower learning rates and careful parameter initialization, and makes it notoriously hard to train models with saturating nonlinearities. We refer to this phenomenon as *internal covariate shift*, and address the problem by normalizing layer inputs. Our method draws its strength from making normalization a part of the model architecture and performing the normalization *for each training mini-batch*. Batch Normalization allows us to use much higher learning rates and be less careful about initialization. It also acts as a regularizer, in some cases eliminating the need for Dropout. Applied to a state-of-the-art image classification model, Batch Normalization achieves the same accuracy with 14 times fewer training steps, and beats the original model by a significant margin. Using an ensemble of batch-normalized networks, we improve upon the best published result on ImageNet classification: reaching 4.9% top-5 validation error (and 4.8% test error), exceeding the accuracy of human raters.

1 Introduction

Deep learning has dramatically advanced the state of the art in vision, speech, and many other areas. Stochastic gradient descent (SGD) has proved to be an effective way of training deep networks, and SGD variants such as momentum (Sutskever et al., 2013) and Adagrad (Duchi et al., 2011) have been used to achieve state of the art performance. SGD optimizes the parameters Θ of the network, so as to minimize the loss

$$\Theta = \arg\min_{\Theta} \frac{1}{N} \sum_{i=1}^{N} \ell(\mathrm{x}_i, \Theta)$$

where $\mathrm{x}_{1\ldots N}$ is the training data set. With SGD, the training proceeds in steps, and at each step we consider a *mini-batch* $\mathrm{x}_{1\ldots m}$ of size m. The mini-batch is used to approximate the gradient of the loss function with respect to the parameters, by computing

$$\frac{1}{m} \frac{\partial \ell(\mathrm{x}_i, \Theta)}{\partial \Theta}.$$

Using mini-batches of examples, as opposed to one example at a time, is helpful in several ways. First, the gradient of the loss over a mini-batch is an estimate of the gradient over the training set, whose quality improves as the batch size increases. Second, computation over a batch can be much more efficient than m computations for individual examples, due to the parallelism afforded by the modern computing platforms.

While stochastic gradient is simple and effective, it requires careful tuning of the model hyper-parameters, specifically the learning rate used in optimization, as well as the initial values for the model parameters. The training is complicated by the fact that the inputs to each layer are affected by the parameters of all preceding layers – so that small changes to the network parameters amplify as the network becomes deeper.

The change in the distributions of layers' inputs presents a problem because the layers need to continuously adapt to the new distribution. When the input distribution to a learning system changes, it is said to experience *covariate shift* (Shimodaira, 2000). This is typically handled via domain adaptation (Jiang, 2008). However, the notion of covariate shift can be extended beyond the learning system as a whole, to apply to its parts, such as a sub-network or a layer. Consider a network computing

$$\ell = F_2(F_1(\mathrm{u}, \Theta_1), \Theta_2)$$

where F_1 and F_2 are arbitrary transformations, and the parameters Θ_1, Θ_2 are to be learned so as to minimize the loss ℓ. Learning Θ_2 can be viewed as if the inputs $\mathrm{x} = F_1(\mathrm{u}, \Theta_1)$ are fed into the sub-network

$$\ell = F_2(\mathrm{x}, \Theta_2).$$

For example, a gradient descent step

$$\Theta_2 \leftarrow \Theta_2 - \frac{\alpha}{m} \sum_{i=1}^{m} \frac{\partial F_2(\mathrm{x}_i, \Theta_2)}{\partial \Theta_2}$$

(for batch size m and learning rate α) is exactly equivalent to that for a stand-alone network F_2 with input x. Therefore, the input distribution properties that make training more efficient – such as having the same distribution between the training and test data – apply to training the sub-network as well. As such it is advantageous for the distribution of x to remain fixed over time. Then, Θ_2 does

not have to readjust to compensate for the change in the distribution of x.

Fixed distribution of inputs to a sub-network would have positive consequences for the layers *outside* the subnetwork, as well. Consider a layer with a sigmoid activation function $\mathrm{z} = g(W\mathrm{u} + \mathrm{b})$ where u is the layer input, the weight matrix W and bias vector b are the layer parameters to be learned, and $g(x) = \frac{1}{1+\exp(-x)}$. As $|x|$ increases, $g'(x)$ tends to zero. This means that for all dimensions of $\mathrm{x} = W\mathrm{u}+\mathrm{b}$ except those with small absolute values, the gradient flowing down to u will vanish and the model will train slowly. However, since x is affected by W, b and the parameters of all the layers below, changes to those parameters during training will likely move many dimensions of x into the saturated regime of the nonlinearity and slow down the convergence. This effect is amplified as the network depth increases. In practice, the saturation problem and the resulting vanishing gradients are usually addressed by using Rectified Linear Units (Nair & Hinton, 2010) $ReLU(x) = \max(x, 0)$, careful initialization (Bengio & Glorot, 2010; Saxe et al., 2013), and small learning rates. If, however, we could ensure that the distribution of nonlinearity inputs remains more stable as the network trains, then the optimizer would be less likely to get stuck in the saturated regime, and the training would accelerate.

We refer to the change in the distributions of internal nodes of a deep network, in the course of training, as *Internal Covariate Shift*. Eliminating it offers a promise of faster training. We propose a new mechanism, which we call *Batch Normalization*, that takes a step towards reducing internal covariate shift, and in doing so dramatically accelerates the training of deep neural nets. It accomplishes this via a normalization step that fixes the means and variances of layer inputs. Batch Normalization also has a beneficial effect on the gradient flow through the network, by reducing the dependence of gradients on the scale of the parameters or of their initial values. This allows us to use much higher learning rates without the risk of divergence. Furthermore, batch normalization regularizes the model and reduces the need for Dropout (Srivastava et al., 2014). Finally, Batch Normalization makes it possible to use saturating nonlinearities by preventing the network from getting stuck in the saturated modes.

In Sec. 4.2, we apply Batch Normalization to the best-performing ImageNet classification network, and show that we can match its performance using only 7% of the training steps, and can further exceed its accuracy by a substantial margin. Using an ensemble of such networks trained with Batch Normalization, we achieve the top-5 error rate that improves upon the best known results on ImageNet classification.

2 Towards Reducing Internal Covariate Shift

We define *Internal Covariate Shift* as the change in the distribution of network activations due to the change in network parameters during training. To improve the training, we seek to reduce the internal covariate shift. By fixing the distribution of the layer inputs x as the training progresses, we expect to improve the training speed. It has been long known (LeCun et al., 1998b; Wiesler & Ney, 2011) that the network training converges faster if its inputs are whitened – i.e., linearly transformed to have zero means and unit variances, and decorrelated. As each layer observes the inputs produced by the layers below, it would be advantageous to achieve the same whitening of the inputs of each layer. By whitening the inputs to each layer, we would take a step towards achieving the fixed distributions of inputs that would remove the ill effects of the internal covariate shift.

We could consider whitening activations at every training step or at some interval, either by modifying the network directly or by changing the parameters of the optimization algorithm to depend on the network activation values (Wiesler et al., 2014; Raiko et al., 2012; Povey et al., 2014; Desjardins & Kavukcuoglu). However, if these modifications are interspersed with the optimization steps, then the gradient descent step may attempt to update the parameters in a way that requires the normalization to be updated, which reduces the effect of the gradient step. For example, consider a layer with the input u that adds the learned bias b, and normalizes the result by subtracting the mean of the activation computed over the training data: $\widehat{x} = x - E[x]$ where $x = u + b$, $\mathcal{X} = \{x_{1\ldots N}\}$ is the set of values of x over the training set, and $\mathrm{E}[x] = \frac{1}{N}\sum_{i=1}^{N} x_i$. If a gradient descent step ignores the dependence of $\mathrm{E}[x]$ on b, then it will update $b \leftarrow b + \Delta b$, where $\Delta b \propto -\partial\ell/\partial\widehat{x}$. Then $u + (b + \Delta b) - \mathrm{E}[u + (b + \Delta b)] = u + b - \mathrm{E}[u + b]$. Thus, the combination of the update to b and subsequent change in normalization led to no change in the output of the layer nor, consequently, the loss. As the training continues, b will grow indefinitely while the loss remains fixed. This problem can get worse if the normalization not only centers but also scales the activations. We have observed this empirically in initial experiments, where the model blows up when the normalization parameters are computed outside the gradient descent step.

The issue with the above approach is that the gradient descent optimization does not take into account the fact that the normalization takes place. To address this issue, we would like to ensure that, for any parameter values, the network *always* produces activations with the desired distribution. Doing so would allow the gradient of the loss with respect to the model parameters to account for the normalization, and for its dependence on the model parameters Θ. Let again x be a layer input, treated as a

vector, and $\mathcal{X}$ be the set of these inputs over the training data set. The normalization can then be written as a transformation

$$\hat{\mathrm{x}} = \mathrm{Norm}(\mathrm{x}, \mathcal{X})$$

which depends not only on the given training example x but on all examples $\mathcal{X}$ – each of which depends on Θ if x is generated by another layer. For backpropagation, we would need to compute the Jacobians

$$\frac{\partial \mathrm{Norm}(\mathrm{x}, \mathcal{X})}{\partial \mathrm{x}} \text{ and } \frac{\partial \mathrm{Norm}(\mathrm{x}, \mathcal{X})}{\partial \mathcal{X}};$$

ignoring the latter term would lead to the explosion described above. Within this framework, whitening the layer inputs is expensive, as it requires computing the covariance matrix $\mathrm{Cov}[\mathrm{x}] = \mathrm{E}_{\mathrm{x}\in\mathcal{X}}[\mathrm{x}\mathrm{x}^T] - \mathrm{E}[\mathrm{x}]\mathrm{E}[\mathrm{x}]^T$ and its inverse square root, to produce the whitened activations $\mathrm{Cov}[\mathrm{x}]^{-1/2}(\mathrm{x} - \mathrm{E}[\mathrm{x}])$, as well as the derivatives of these transforms for backpropagation. This motivates us to seek an alternative that performs input normalization in a way that is differentiable and does not require the analysis of the entire training set after every parameter update.

Some of the previous approaches (e.g. (Lyu & Simoncelli, 2008)) use statistics computed over a single training example, or, in the case of image networks, over different feature maps at a given location. However, this changes the representation ability of a network by discarding the absolute scale of activations. We want to a preserve the information in the network, by normalizing the activations in a training example relative to the statistics of the entire training data.

3 Normalization via Mini-Batch Statistics

Since the full whitening of each layer's inputs is costly and not everywhere differentiable, we make two necessary simplifications. The first is that instead of whitening the features in layer inputs and outputs jointly, we will normalize each scalar feature independently, by making it have the mean of zero and the variance of 1. For a layer with d-dimensional input $\mathrm{x} = (x^{(1)} \ldots x^{(d)})$, we will normalize each dimension

$$\hat{x}^{(k)} = \frac{x^{(k)} - \mathrm{E}[x^{(k)}]}{\sqrt{\mathrm{Var}[x^{(k)}]}}$$

where the expectation and variance are computed over the training data set. As shown in (LeCun et al., 1998b), such normalization speeds up convergence, even when the features are not decorrelated.

Note that simply normalizing each input of a layer may change what the layer can represent. For instance, normalizing the inputs of a sigmoid would constrain them to the linear regime of the nonlinearity. To address this, we make sure that *the transformation inserted in the network can represent the identity transform*. To accomplish this, we introduce, for each activation $x^{(k)}$, a pair of parameters $\gamma^{(k)}, \beta^{(k)}$, which scale and shift the normalized value:

$$y^{(k)} = \gamma^{(k)}\hat{x}^{(k)} + \beta^{(k)}.$$

These parameters are learned along with the original model parameters, and restore the representation power of the network. Indeed, by setting $\gamma^{(k)} = \sqrt{\mathrm{Var}[x^{(k)}]}$ and $\beta^{(k)} = \mathrm{E}[x^{(k)}]$, we could recover the original activations, if that were the optimal thing to do.

In the batch setting where each training step is based on the entire training set, we would use the whole set to normalize activations. However, this is impractical when using stochastic optimization. Therefore, we make the second simplification: since we use mini-batches in stochastic gradient training, *each mini-batch produces estimates of the mean and variance* of each activation. This way, the statistics used for normalization can fully participate in the gradient backpropagation. Note that the use of mini-batches is enabled by computation of per-dimension variances rather than joint covariances; in the joint case, regularization would be required since the mini-batch size is likely to be smaller than the number of activations being whitened, resulting in singular covariance matrices.

Consider a mini-batch $\mathcal{B}$ of size m. Since the normalization is applied to each activation independently, let us focus on a particular activation $x^{(k)}$ and omit k for clarity. We have m values of this activation in the mini-batch,

$$\mathcal{B} = \{x_{1\ldots m}\}.$$

Let the normalized values be $\hat{x}_{1\ldots m}$, and their linear transformations be $y_{1\ldots m}$. We refer to the transform

$$\mathrm{BN}_{\gamma,\beta} : x_{1\ldots m} \to y_{1\ldots m}$$

as the *Batch Normalizing Transform*. We present the BN Transform in Algorithm 1. In the algorithm, ϵ is a constant added to the mini-batch variance for numerical stability.

Input: Values of x over a mini-batch: $\mathcal{B} = \{x_{1\ldots m}\}$;
Parameters to be learned: γ, β
Output: $\{y_i = \mathrm{BN}_{\gamma,\beta}(x_i)\}$

$$\mu_{\mathcal{B}} \leftarrow \frac{1}{m}\sum_{i=1}^{m} x_i \qquad \text{// mini-batch mean}$$

$$\sigma^2_{\mathcal{B}} \leftarrow \frac{1}{m}\sum_{i=1}^{m}(x_i - \mu_{\mathcal{B}})^2 \qquad \text{// mini-batch variance}$$

$$\hat{x}_i \leftarrow \frac{x_i - \mu_{\mathcal{B}}}{\sqrt{\sigma^2_{\mathcal{B}} + \epsilon}} \qquad \text{// normalize}$$

$$y_i \leftarrow \gamma\hat{x}_i + \beta \equiv \mathrm{BN}_{\gamma,\beta}(x_i) \qquad \text{// scale and shift}$$

Algorithm 1: Batch Normalizing Transform, applied to activation x over a mini-batch.

The BN transform can be added to a network to manipulate any activation. In the notation $y = \mathrm{BN}_{\gamma,\beta}(x)$, we

indicate that the parameters γ and β are to be learned, but it should be noted that the BN transform does not independently process the activation in each training example. Rather, $\mathrm{BN}_{\gamma,\beta}(x)$ depends both on the training example *and the other examples in the mini-batch*. The scaled and shifted values y are passed to other network layers. The normalized activations $\widehat{x}$ are internal to our transformation, but their presence is crucial. The distributions of values of any $\widehat{x}$ has the expected value of 0 and the variance of 1, as long as the elements of each mini-batch are sampled from the same distribution, and if we neglect ϵ. This can be seen by observing that $\sum_{i=1}^{m} \widehat{x}_i = 0$ and $\frac{1}{m}\sum_{i=1}^{m} \widehat{x}_i^2 = 1$, and taking expectations. Each normalized activation $\widehat{x}^{(k)}$ can be viewed as an input to a sub-network composed of the linear transform $y^{(k)} = \gamma^{(k)}\widehat{x}^{(k)} + \beta^{(k)}$, followed by the other processing done by the original network. These sub-network inputs all have fixed means and variances, and although the joint distribution of these normalized $\widehat{x}^{(k)}$ can change over the course of training, we expect that the introduction of normalized inputs accelerates the training of the sub-network and, consequently, the network as a whole.

During training we need to backpropagate the gradient of loss ℓ through this transformation, as well as compute the gradients with respect to the parameters of the BN transform. We use chain rule, as follows (before simplification):

$$\begin{aligned}
\frac{\partial \ell}{\partial \widehat{x}_i} &= \frac{\partial \ell}{\partial y_i} \cdot \gamma \\
\frac{\partial \ell}{\partial \sigma_{\mathcal{B}}^2} &= \sum_{i=1}^{m} \frac{\partial \ell}{\partial \widehat{x}_i} \cdot (x_i - \mu_{\mathcal{B}}) \cdot \frac{-1}{2}(\sigma_{\mathcal{B}}^2 + \epsilon)^{-3/2} \\
\frac{\partial \ell}{\partial \mu_{\mathcal{B}}} &= \left(\sum_{i=1}^{m} \frac{\partial \ell}{\partial \widehat{x}_i} \cdot \frac{-1}{\sqrt{\sigma_{\mathcal{B}}^2 + \epsilon}} \right) + \frac{\partial \ell}{\partial \sigma_{\mathcal{B}}^2} \cdot \frac{\sum_{i=1}^{m} -2(x_i - \mu_{\mathcal{B}})}{m} \\
\frac{\partial \ell}{\partial x_i} &= \frac{\partial \ell}{\partial \widehat{x}_i} \cdot \frac{1}{\sqrt{\sigma_{\mathcal{B}}^2 + \epsilon}} + \frac{\partial \ell}{\partial \sigma_{\mathcal{B}}^2} \cdot \frac{2(x_i - \mu_{\mathcal{B}})}{m} + \frac{\partial \ell}{\partial \mu_{\mathcal{B}}} \cdot \frac{1}{m} \\
\frac{\partial \ell}{\partial \gamma} &= \sum_{i=1}^{m} \frac{\partial \ell}{\partial y_i} \cdot \widehat{x}_i \\
\frac{\partial \ell}{\partial \beta} &= \sum_{i=1}^{m} \frac{\partial \ell}{\partial y_i}
\end{aligned}$$

Thus, BN transform is a differentiable transformation that introduces normalized activations into the network. This ensures that as the model is training, layers can continue learning on input distributions that exhibit less internal covariate shift, thus accelerating the training. Furthermore, the learned affine transform applied to these normalized activations allows the BN transform to represent the identity transformation and preserves the network capacity.

3.1 Training and Inference with Batch-Normalized Networks

To *Batch-Normalize* a network, we specify a subset of activations and insert the BN transform for each of them, according to Alg. 1. Any layer that previously received x as the input, now receives $\mathrm{BN}(x)$. A model employing Batch Normalization can be trained using batch gradient descent, or Stochastic Gradient Descent with a mini-batch size $m > 1$, or with any of its variants such as Adagrad (Duchi et al., 2011). The normalization of activations that depends on the mini-batch allows efficient training, but is neither necessary nor desirable during inference; we want the output to depend only on the input, deterministically. For this, once the network has been trained, we use the normalization

$$\widehat{x} = \frac{x - \mathrm{E}[x]}{\sqrt{\mathrm{Var}[x] + \epsilon}}$$

using the population, rather than mini-batch, statistics. Neglecting ϵ, these normalized activations have the same mean 0 and variance 1 as during training. We use the unbiased variance estimate $\mathrm{Var}[x] = \frac{m}{m-1} \cdot \mathrm{E}_{\mathcal{B}}[\sigma_{\mathcal{B}}^2]$, where the expectation is over training mini-batches of size m and $\sigma_{\mathcal{B}}^2$ are their sample variances. Using moving averages instead, we can track the accuracy of a model as it trains. Since the means and variances are fixed during inference, the normalization is simply a linear transform applied to each activation. It may further be composed with the scaling by γ and shift by β, to yield a single linear transform that replaces $\mathrm{BN}(x)$. Algorithm 2 summarizes the procedure for training batch-normalized networks.

Input: Network N with trainable parameters Θ; subset of activations $\{x^{(k)}\}_{k=1}^{K}$

Output: Batch-normalized network for inference, $N_{\mathrm{BN}}^{\mathrm{inf}}$

1: $N_{\mathrm{BN}}^{\mathrm{tr}} \leftarrow N$ // Training BN network
2: **for** $k = 1 \ldots K$ **do**
3: Add transformation $y^{(k)} = \mathrm{BN}_{\gamma^{(k)},\beta^{(k)}}(x^{(k)})$ to $N_{\mathrm{BN}}^{\mathrm{tr}}$ (Alg. 1)
4: Modify each layer in $N_{\mathrm{BN}}^{\mathrm{tr}}$ with input $x^{(k)}$ to take $y^{(k)}$ instead
5: **end for**
6: Train $N_{\mathrm{BN}}^{\mathrm{tr}}$ to optimize the parameters $\Theta \cup \{\gamma^{(k)}, \beta^{(k)}\}_{k=1}^{K}$
7: $N_{\mathrm{BN}}^{\mathrm{inf}} \leftarrow N_{\mathrm{BN}}^{\mathrm{tr}}$ // Inference BN network with frozen // parameters
8: **for** $k = 1 \ldots K$ **do**
9: // For clarity, $x \equiv x^{(k)}, \gamma \equiv \gamma^{(k)}, \mu_{\mathcal{B}} \equiv \mu_{\mathcal{B}}^{(k)}$, etc.
10: Process multiple training mini-batches $\mathcal{B}$, each of size m, and average over them:
$$\mathrm{E}[x] \leftarrow \mathrm{E}_{\mathcal{B}}[\mu_{\mathcal{B}}]$$
$$\mathrm{Var}[x] \leftarrow \frac{m}{m-1}\mathrm{E}_{\mathcal{B}}[\sigma_{\mathcal{B}}^2]$$
11: In $N_{\mathrm{BN}}^{\mathrm{inf}}$, replace the transform $y = \mathrm{BN}_{\gamma,\beta}(x)$ with $y = \frac{\gamma}{\sqrt{\mathrm{Var}[x] + \epsilon}} \cdot x + \left(\beta - \frac{\gamma\,\mathrm{E}[x]}{\sqrt{\mathrm{Var}[x] + \epsilon}}\right)$
12: **end for**

Algorithm 2: Training a Batch-Normalized Network

3.2 Batch-Normalized Convolutional Networks

Batch Normalization can be applied to any set of activations in the network. Here, we focus on transforms

that consist of an affine transformation followed by an element-wise nonlinearity:

$$\mathrm{z} = g(W\mathrm{u} + \mathrm{b})$$

where W and b are learned parameters of the model, and $g(\cdot)$ is the nonlinearity such as sigmoid or ReLU. This formulation covers both fully-connected and convolutional layers. We add the BN transform immediately before the nonlinearity, by normalizing $\mathrm{x} = W\mathrm{u}+\mathrm{b}$. We could have also normalized the layer inputs u, but since u is likely the output of another nonlinearity, the shape of its distribution is likely to change during training, and constraining its first and second moments would not eliminate the covariate shift. In contrast, $W\mathrm{u} + \mathrm{b}$ is more likely to have a symmetric, non-sparse distribution, that is "more Gaussian" (Hyvärinen & Oja, 2000); normalizing it is likely to produce activations with a stable distribution.

Note that, since we normalize $W\mathrm{u}+\mathrm{b}$, the bias b can be ignored since its effect will be canceled by the subsequent mean subtraction (the role of the bias is subsumed by β in Alg. 1). Thus, $\mathrm{z} = g(W\mathrm{u} + \mathrm{b})$ is replaced with

$$\mathrm{z} = g(\mathrm{BN}(W\mathrm{u}))$$

where the BN transform is applied independently to each dimension of $\mathrm{x} = W\mathrm{u}$, with a separate pair of learned parameters $\gamma^{(k)}$, $\beta^{(k)}$ per dimension.

For convolutional layers, we additionally want the normalization to obey the convolutional property – so that different elements of the same feature map, at different locations, are normalized in the same way. To achieve this, we jointly normalize all the activations in a mini-batch, over all locations. In Alg. 1, we let $\mathcal{B}$ be the set of all values in a feature map across both the elements of a mini-batch and spatial locations – so for a mini-batch of size m and feature maps of size $p \times q$, we use the effective mini-batch of size $m' = |\mathcal{B}| = m \cdot p\,q$. We learn a pair of parameters $\gamma^{(k)}$ and $\beta^{(k)}$ per feature map, rather than per activation. Alg. 2 is modified similarly, so that during inference the BN transform applies the same linear transformation to each activation in a given feature map.

3.3 Batch Normalization enables higher learning rates

In traditional deep networks, too-high learning rate may result in the gradients that explode or vanish, as well as getting stuck in poor local minima. Batch Normalization helps address these issues. By normalizing activations throughout the network, it prevents small changes to the parameters from amplifying into larger and suboptimal changes in activations in gradients; for instance, it prevents the training from getting stuck in the saturated regimes of nonlinearities.

Batch Normalization also makes training more resilient to the parameter scale. Normally, large learning rates may increase the scale of layer parameters, which then amplify the gradient during backpropagation and lead to the model explosion. However, with Batch Normalization, backpropagation through a layer is unaffected by the scale of its parameters. Indeed, for a scalar a,

$$\mathrm{BN}(W\mathrm{u}) = \mathrm{BN}((aW)\mathrm{u})$$

and we can show that

$$\frac{\partial \mathrm{BN}((aW)\mathrm{u})}{\partial \mathrm{u}} = \frac{\partial \mathrm{BN}(W\mathrm{u})}{\partial \mathrm{u}}$$
$$\frac{\partial \mathrm{BN}((aW)\mathrm{u})}{\partial (aW)} = \frac{1}{a} \cdot \frac{\partial \mathrm{BN}(W\mathrm{u})}{\partial W}$$

The scale does not affect the layer Jacobian nor, consequently, the gradient propagation. Moreover, larger weights lead to *smaller* gradients, and Batch Normalization will stabilize the parameter growth.

We further conjecture that Batch Normalization may lead the layer Jacobians to have singular values close to 1, which is known to be beneficial for training (Saxe et al., 2013). Consider two consecutive layers with normalized inputs, and the transformation between these normalized vectors: $\hat{\mathrm{z}} = F(\hat{\mathrm{x}})$. If we assume that $\hat{\mathrm{x}}$ and $\hat{\mathrm{z}}$ are Gaussian and uncorrelated, and that $F(\hat{\mathrm{x}}) \approx J\hat{\mathrm{x}}$ is a linear transformation for the given model parameters, then both $\hat{\mathrm{x}}$ and $\hat{\mathrm{z}}$ have unit covariances, and $I = \mathrm{Cov}[\hat{\mathrm{z}}] = J\mathrm{Cov}[\hat{\mathrm{x}}]J^T = JJ^T$. Thus, $JJ^T = I$, and so all singular values of J are equal to 1, which preserves the gradient magnitudes during backpropagation. In reality, the transformation is not linear, and the normalized values are not guaranteed to be Gaussian nor independent, but we nevertheless expect Batch Normalization to help make gradient propagation better behaved. The precise effect of Batch Normalization on gradient propagation remains an area of further study.

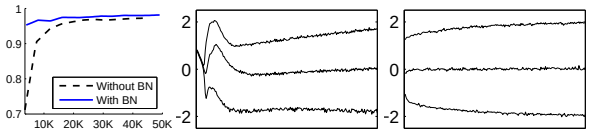
3.4 Batch Normalization regularizes the model

When training with Batch Normalization, a training example is seen in conjunction with other examples in the mini-batch, and the training network no longer producing deterministic values for a given training example. In our experiments, we found this effect to be advantageous to the generalization of the network. Whereas Dropout (Srivastava et al., 2014) is typically used to reduce overfitting, in a batch-normalized network we found that it can be either removed or reduced in strength.

4 Experiments

4.1 Activations over time

To verify the effects of internal covariate shift on training, and the ability of Batch Normalization to combat it, we considered the problem of predicting the digit class on the MNIST dataset (LeCun et al., 1998a). We used a very simple network, with a 28x28 binary image as input, and



(a) (b) Without BN (c) With BN

Figure 1: (a) *The test accuracy of the MNIST network trained with and without Batch Normalization, vs. the number of training steps. Batch Normalization helps the network train faster and achieve higher accuracy.* (b, c) *The evolution of input distributions to a typical sigmoid, over the course of training, shown as* $\{15, 50, 85\}$*th percentiles. Batch Normalization makes the distribution more stable and reduces the internal covariate shift.*

3 fully-connected hidden layers with 100 activations each. Each hidden layer computes $\mathrm{y} = g(W\mathrm{u}+\mathrm{b})$ with sigmoid nonlinearity, and the weights W initialized to small random Gaussian values. The last hidden layer is followed by a fully-connected layer with 10 activations (one per class) and cross-entropy loss. We trained the network for 50000 steps, with 60 examples per mini-batch. We added Batch Normalization to each hidden layer of the network, as in Sec. 3.1. We were interested in the comparison between the baseline and batch-normalized networks, rather than achieving the state of the art performance on MNIST (which the described architecture does not).

Figure 1(a) shows the fraction of correct predictions by the two networks on held-out test data, as training progresses. The batch-normalized network enjoys the higher test accuracy. To investigate why, we studied inputs to the sigmoid, in the original network N and batch-normalized network $N^{\mathrm{tr}}_{\mathrm{BN}}$ (Alg. 2) over the course of training. In Fig. 1(b,c) we show, for one typical activation from the last hidden layer of each network, how its distribution evolves. The distributions in the original network change significantly over time, both in their mean and the variance, which complicates the training of the subsequent layers. In contrast, the distributions in the batch-normalized network are much more stable as training progresses, which aids the training.

4.2 ImageNet classification

We applied Batch Normalization to a new variant of the Inception network (Szegedy et al., 2014), trained on the ImageNet classification task (Russakovsky et al., 2014). The network has a large number of convolutional and pooling layers, with a softmax layer to predict the image class, out of 1000 possibilities. Convolutional layers use ReLU as the nonlinearity. The main difference to the network described in (Szegedy et al., 2014) is that the 5×5 convolutional layers are replaced by two consecutive layers of 3×3 convolutions with up to 128 filters. The network contains $13.6 \cdot 10^6$ parameters, and, other than the top softmax layer, has no fully-connected layers. More details are given in the Appendix. We refer to this model as *Inception* in the rest of the text. The model was trained using a version of Stochastic Gradient Descent with momentum (Sutskever et al., 2013), using the mini-batch size of 32. The training was performed using a large-scale, distributed architecture (similar to (Dean et al., 2012)). All networks are evaluated as training progresses by computing the validation accuracy $@1$, i.e. the probability of predicting the correct label out of 1000 possibilities, on a held-out set, using a single crop per image.

In our experiments, we evaluated several modifications of Inception with Batch Normalization. In all cases, Batch Normalization was applied to the input of each nonlinearity, in a convolutional way, as described in section 3.2, while keeping the rest of the architecture constant.

4.2.1 Accelerating BN Networks

Simply adding Batch Normalization to a network does not take full advantage of our method. To do so, we further changed the network and its training parameters, as follows:

Increase learning rate. In a batch-normalized model, we have been able to achieve a training speedup from higher learning rates, with no ill side effects (Sec. 3.3).

Remove Dropout. As described in Sec. 3.4, Batch Normalization fulfills some of the same goals as Dropout. Removing Dropout from Modified BN-Inception speeds up training, without increasing overfitting.

Reduce the L_2 weight regularization. While in Inception an L_2 loss on the model parameters controls overfitting, in Modified BN-Inception the weight of this loss is reduced by a factor of 5. We find that this improves the accuracy on the held-out validation data.

Accelerate the learning rate decay. In training Inception, learning rate was decayed exponentially. Because our network trains faster than Inception, we lower the learning rate 6 times faster.

Remove Local Response Normalization While Inception and other networks (Srivastava et al., 2014) benefit from it, we found that with Batch Normalization it is not necessary.

Shuffle training examples more thoroughly. We enabled within-shard shuffling of the training data, which prevents the same examples from always appearing in a mini-batch together. This led to about 1% improvements in the validation accuracy, which is consistent with the view of Batch Normalization as a regularizer (Sec. 3.4): the randomization inherent in our method should be most beneficial when it affects an example differently each time it is seen.

Reduce the photometric distortions. Because batch-normalized networks train faster and observe each training example fewer times, we let the trainer focus on more "real" images by distorting them less.

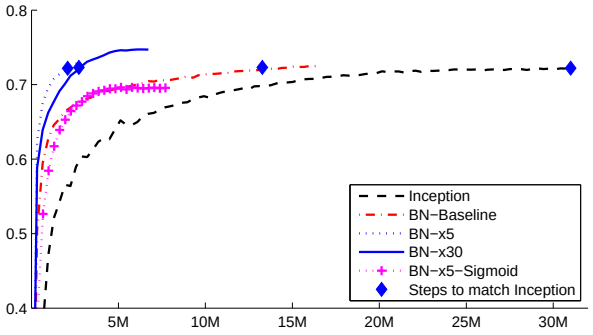


Figure 2: *Single crop validation accuracy of Inception and its batch-normalized variants, vs. the number of training steps.*

Model	Steps to 72.2%	Max accuracy
Inception	$31.0 \cdot 10^6$	72.2%
BN-Baseline	$13.3 \cdot 10^6$	72.7%
BN-x5	$2.1 \cdot 10^6$	73.0%
BN-x30	$2.7 \cdot 10^6$	74.8%
BN-x5-Sigmoid		69.8%

Figure 3: *For Inception and the batch-normalized variants, the number of training steps required to reach the maximum accuracy of Inception (72.2%), and the maximum accuracy achieved by the network.*

4.2.2 Single-Network Classification

We evaluated the following networks, all trained on the LSVRC2012 training data, and tested on the validation data:

Inception: the network described at the beginning of Section 4.2, trained with the initial learning rate of 0.0015.

BN-Baseline: Same as Inception with Batch Normalization before each nonlinearity.

BN-x5: Inception with Batch Normalization and the modifications in Sec. 4.2.1. The initial learning rate was increased by a factor of 5, to 0.0075. The same learning rate increase with original Inception caused the model parameters to reach machine infinity.

BN-x30: Like *BN-x5*, but with the initial learning rate 0.045 (30 times that of Inception).

BN-x5-Sigmoid: Like *BN-x5*, but with sigmoid nonlinearity $g(t) = \frac{1}{1+\exp(-x)}$ instead of ReLU. We also attempted to train the original Inception with sigmoid, but the model remained at the accuracy equivalent to chance.

In Figure 2, we show the validation accuracy of the networks, as a function of the number of training steps. Inception reached the accuracy of 72.2% after $31 \cdot 10^6$ training steps. The Figure 3 shows, for each network, the number of training steps required to reach the same 72.2% accuracy, as well as the maximum validation accuracy reached by the network and the number of steps to reach it.

By only using Batch Normalization (*BN-Baseline*), we match the accuracy of Inception in less than half the number of training steps. By applying the modifications in Sec. 4.2.1, we significantly increase the training speed of the network. *BN-x5* needs 14 times fewer steps than Inception to reach the 72.2% accuracy. Interestingly, increasing the learning rate further (*BN-x30*) causes the model to train somewhat *slower* initially, but allows it to reach a higher final accuracy. It reaches 74.8% after $6 \cdot 10^6$ steps, i.e. 5 times fewer steps than required by Inception to reach 72.2%.

We also verified that the reduction in internal covariate shift allows deep networks with Batch Normalization to be trained when sigmoid is used as the nonlinearity, despite the well-known difficulty of training such networks. Indeed, *BN-x5-Sigmoid* achieves the accuracy of 69.8%. Without Batch Normalization, Inception with sigmoid never achieves better than $1/1000$ accuracy.

4.2.3 Ensemble Classification

The current reported best results on the ImageNet Large Scale Visual Recognition Competition are reached by the Deep Image ensemble of traditional models (Wu et al., 2015) and the ensemble model of (He et al., 2015). The latter reports the top-5 error of 4.94%, as evaluated by the ILSVRC server. Here we report a top-5 validation error of 4.9%, and test error of 4.82% (according to the ILSVRC server). This improves upon the previous best result, and exceeds the estimated accuracy of human raters according to (Russakovsky et al., 2014).

For our ensemble, we used 6 networks. Each was based on *BN-x30*, modified via some of the following: increased initial weights in the convolutional layers; using Dropout (with the Dropout probability of 5% or 10%, vs. 40% for the original Inception); and using non-convolutional, per-activation Batch Normalization with last hidden layers of the model. Each network achieved its maximum accuracy after about $6 \cdot 10^6$ training steps. The ensemble prediction was based on the arithmetic average of class probabilities predicted by the constituent networks. The details of ensemble and multicrop inference are similar to (Szegedy et al., 2014).

We demonstrate in Fig. 4 that batch normalization allows us to set new state-of-the-art by a healthy margin on the ImageNet classification challenge benchmarks.

5 Conclusion

We have presented a novel mechanism for dramatically accelerating the training of deep networks. It is based on the premise that covariate shift, which is known to complicate the training of machine learning systems, also ap-

Model	Resolution	Crops	Models	Top-1 error	Top-5 error
GoogLeNet ensemble	224	144	7	-	6.67%
Deep Image low-res	256	-	1	-	7.96%
Deep Image high-res	512	-	1	24.88	7.42%
Deep Image ensemble	variable	-	-	-	5.98%
BN-Inception single crop	224	1	1	25.2%	7.82%
BN-Inception multicrop	224	144	1	21.99%	5.82%
BN-Inception ensemble	224	144	6	20.1%	**4.9%***

Figure 4: *Batch-Normalized Inception comparison with previous state of the art on the provided validation set comprising 50000 images. *BN-Inception ensemble has reached 4.82% top-5 error on the 100000 images of the test set of the ImageNet as reported by the test server.*

plies to sub-networks and layers, and removing it from internal activations of the network may aid in training. Our proposed method draws its power from normalizing activations, and from incorporating this normalization in the network architecture itself. This ensures that the normalization is appropriately handled by any optimization method that is being used to train the network. To enable stochastic optimization methods commonly used in deep network training, we perform the normalization for each mini-batch, and backpropagate the gradients through the normalization parameters. Batch Normalization adds only two extra parameters per activation, and in doing so preserves the representation ability of the network. We presented an algorithm for constructing, training, and performing inference with batch-normalized networks. The resulting networks can be trained with saturating nonlinearities, are more tolerant to increased training rates, and often do not require Dropout for regularization.

Merely adding Batch Normalization to a state-of-the-art image classification model yields a substantial speedup in training. By further increasing the learning rates, removing Dropout, and applying other modifications afforded by Batch Normalization, we reach the previous state of the art with only a small fraction of training steps – and then beat the state of the art in single-network image classification. Furthermore, by combining multiple models trained with Batch Normalization, we perform better than the best known system on ImageNet, by a significant margin.

Interestingly, our method bears similarity to the standardization layer of (Gülçehre & Bengio, 2013), though the two methods stem from very different goals, and perform different tasks. The goal of Batch Normalization is to achieve a stable distribution of activation values throughout training, and in our experiments we apply it before the nonlinearity since that is where matching the first and second moments is more likely to result in a stable distribution. On the contrary, (Gülçehre & Bengio, 2013) apply the standardization layer to the *output* of the nonlinearity, which results in sparser activations. In our large-scale image classification experiments, we have not observed the nonlinearity *inputs* to be sparse, neither with nor without Batch Normalization. Other notable differentiating characteristics of Batch Normalization include the learned scale and shift that allow the BN transform to represent identity (the standardization layer did not require this since it was followed by the learned linear transform that, conceptually, absorbs the necessary scale and shift), handling of convolutional layers, deterministic inference that does not depend on the mini-batch, and batch-normalizing each convolutional layer in the network.

In this work, we have not explored the full range of possibilities that Batch Normalization potentially enables. Our future work includes applications of our method to Recurrent Neural Networks (Pascanu et al., 2013), where the internal covariate shift and the vanishing or exploding gradients may be especially severe, and which would allow us to more thoroughly test the hypothesis that normalization improves gradient propagation (Sec. 3.3). We plan to investigate whether Batch Normalization can help with domain adaptation, in its traditional sense – i.e. whether the normalization performed by the network would allow it to more easily generalize to new data distributions, perhaps with just a recomputation of the population means and variances (Alg. 2). Finally, we believe that further theoretical analysis of the algorithm would allow still more improvements and applications.

References

Bengio, Yoshua and Glorot, Xavier. Understanding the difficulty of training deep feedforward neural networks. In *Proceedings of AISTATS 2010*, volume 9, pp. 249–256, May 2010.

Dean, Jeffrey, Corrado, Greg S., Monga, Rajat, Chen, Kai, Devin, Matthieu, Le, Quoc V., Mao, Mark Z., Ranzato, Marc'Aurelio, Senior, Andrew, Tucker, Paul, Yang, Ke, and Ng, Andrew Y. Large scale distributed deep networks. In *NIPS*, 2012.

Desjardins, Guillaume and Kavukcuoglu, Koray. Natural neural networks. (unpublished).

Duchi, John, Hazan, Elad, and Singer, Yoram. Adaptive subgradient methods for online learning and stochastic

type	**patch size/ stride**	**output size**	**depth**	#1×1	**#3×3 reduce**	#3×3	**double #3×3 reduce**	**double #3×3**	**Pool +proj**
convolution*	7×7/2	112×112×64	1						
max pool	3×3/2	56×56×64	0						
convolution	3×3/1	56×56×192	1		64	192			
max pool	3×3/2	28×28×192	0						
inception (3a)		28×28×256	3	64	64	64	64	96	avg + 32
inception (3b)		28×28×320	3	64	64	96	64	96	avg + 64
inception (3c)	stride 2	28×28×576	3	0	128	160	64	96	max + pass through
inception (4a)		14×14×576	3	224	64	96	96	128	avg + 128
inception (4b)		14×14×576	3	192	96	128	96	128	avg + 128
inception (4c)		14×14×576	3	160	128	160	128	160	avg + 128
inception (4d)		14×14×576	3	96	128	192	160	192	avg + 128
inception (4e)	stride 2	14×14×1024	3	0	128	192	192	256	max + pass through
inception (5a)		7×7×1024	3	352	192	320	160	224	avg + 128
inception (5b)		7×7×1024	3	352	192	320	192	224	max + 128
avg pool	7×7/1	1×1×1024	0						

Figure 5: Inception architecture

Layer Normalization

Jimmy Lei Ba
University of Toronto
jimmy@psi.toronto.edu

Jamie Ryan Kiros
University of Toronto
rkiros@cs.toronto.edu

Geoffrey E. Hinton
University of Toronto
and Google Inc.
hinton@cs.toronto.edu

Abstract

Training state-of-the-art, deep neural networks is computationally expensive. One way to reduce the training time is to normalize the activities of the neurons. A recently introduced technique called batch normalization uses the distribution of the summed input to a neuron over a mini-batch of training cases to compute a mean and variance which are then used to normalize the summed input to that neuron on each training case. This significantly reduces the training time in feed-forward neural networks. However, the effect of batch normalization is dependent on the mini-batch size and it is not obvious how to apply it to recurrent neural networks. In this paper, we transpose batch normalization into layer normalization by computing the mean and variance used for normalization from all of the summed inputs to the neurons in a layer on a *single* training case. Like batch normalization, we also give each neuron its own adaptive bias and gain which are applied after the normalization but before the non-linearity. Unlike batch normalization, layer normalization performs exactly the same computation at training and test times. It is also straightforward to apply to recurrent neural networks by computing the normalization statistics separately at each time step. Layer normalization is very effective at stabilizing the hidden state dynamics in recurrent networks. Empirically, we show that layer normalization can substantially reduce the training time compared with previously published techniques.

1 Introduction

Deep neural networks trained with some version of Stochastic Gradient Descent have been shown to substantially outperform previous approaches on various supervised learning tasks in computer vision [Krizhevsky et al., 2012] and speech processing [Hinton et al., 2012]. But state-of-the-art deep neural networks often require many days of training. It is possible to speed-up the learning by computing gradients for different subsets of the training cases on different machines or splitting the neural network itself over many machines [Dean et al., 2012], but this can require a lot of communication and complex software. It also tends to lead to rapidly diminishing returns as the degree of parallelization increases. An orthogonal approach is to modify the computations performed in the forward pass of the neural net to make learning easier. Recently, batch normalization [Ioffe and Szegedy, 2015] has been proposed to reduce training time by including additional normalization stages in deep neural networks. The normalization standardizes each summed input using its mean and its standard deviation across the training data. Feedforward neural networks trained using batch normalization converge faster even with simple SGD. In addition to training time improvement, the stochasticity from the batch statistics serves as a regularizer during training.

Despite its simplicity, batch normalization requires running averages of the summed input statistics. In feed-forward networks with fixed depth, it is straightforward to store the statistics separately for each hidden layer. However, the summed inputs to the recurrent neurons in a recurrent neural network (RNN) often vary with the length of the sequence so applying batch normalization to RNNs appears to require different statistics for different time-steps. Furthermore, batch normaliza-

tion cannot be applied to online learning tasks or to extremely large distributed models where the minibatches have to be small.

This paper introduces layer normalization, a simple normalization method to improve the training speed for various neural network models. Unlike batch normalization, the proposed method directly estimates the normalization statistics from the summed inputs to the neurons within a hidden layer so the normalization does not introduce any new dependencies between training cases. We show that layer normalization works well for RNNs and improves both the training time and the generalization performance of several existing RNN models.

2 Background

A feed-forward neural network is a non-linear mapping from a input pattern $\mathbf{x}$ to an output vector y. Consider the l^{th} hidden layer in a deep feed-forward, neural network, and let a^l be the vector representation of the summed inputs to the neurons in that layer. The summed inputs are computed through a linear projection with the weight matrix W^l and the bottom-up inputs h^l given as follows:

$$a_i^l = {w_i^l}^\top h^l \qquad h_i^{l+1} = f(a_i^l + b_i^l) \tag{1}$$

where $f(\cdot)$ is an element-wise non-linear function and w_i^l is the incoming weights to the i^{th} hidden units and b_i^l is the scalar bias parameter. The parameters in the neural network are learnt using gradient-based optimization algorithms with the gradients being computed by back-propagation.

One of the challenges of deep learning is that the gradients with respect to the weights in one layer are highly dependent on the outputs of the neurons in the previous layer especially if these outputs change in a highly correlated way. Batch normalization [Ioffe and Szegedy, 2015] was proposed to reduce such undesirable "covariate shift". The method normalizes the summed inputs to each hidden unit over the training cases. Specifically, for the i^{th} summed input in the l^{th} layer, the batch normalization method rescales the summed inputs according to their variances under the distribution of the data

$$\bar{a}_i^l = \frac{g_i^l}{\sigma_i^l}\left(a_i^l - \mu_i^l\right) \qquad \mu_i^l = \mathop{\mathbb{E}}_{\mathbf{x}\sim P(\mathbf{x})}\left[a_i^l\right] \qquad \sigma_i^l = \sqrt{\mathop{\mathbb{E}}_{\mathbf{x}\sim P(\mathbf{x})}\left[\left(a_i^l - \mu_i^l\right)^2\right]} \tag{2}$$

where $\bar{a}_i^l$ is normalized summed inputs to the i^{th} hidden unit in the l^{th} layer and g_i is a gain parameter scaling the normalized activation before the non-linear activation function. Note the expectation is under the whole training data distribution. It is typically impractical to compute the expectations in Eq. (2) exactly, since it would require forward passes through the whole training dataset with the current set of weights. Instead, μ and σ are estimated using the empirical samples from the current mini-batch. This puts constraints on the size of a mini-batch and it is hard to apply to recurrent neural networks.

3 Layer normalization

We now consider the layer normalization method which is designed to overcome the drawbacks of batch normalization.

Notice that changes in the output of one layer will tend to cause highly correlated changes in the summed inputs to the next layer, especially with ReLU units whose outputs can change by a lot. This suggests the "covariate shift" problem can be reduced by fixing the mean and the variance of the summed inputs within each layer. We, thus, compute the layer normalization statistics over all the hidden units in the same layer as follows:

$$\mu^l = \frac{1}{H}\sum_{i=1}^{H} a_i^l \qquad \sigma^l = \sqrt{\frac{1}{H}\sum_{i=1}^{H}\left(a_i^l - \mu^l\right)^2} \tag{3}$$

where H denotes the number of hidden units in a layer. The difference between Eq. (2) and Eq. (3) is that under layer normalization, all the hidden units in a layer share the same normalization terms μ and σ, but different training cases have different normalization terms. Unlike batch normalization, layer normaliztion does not impose any constraint on the size of a mini-batch and it can be used in the pure online regime with batch size 1.

3.1 Layer normalized recurrent neural networks

The recent sequence to sequence models [Sutskever et al., 2014] utilize compact recurrent neural networks to solve sequential prediction problems in natural language processing. It is common among the NLP tasks to have different sentence lengths for different training cases. This is easy to deal with in an RNN because the same weights are used at every time-step. But when we apply batch normalization to an RNN in the obvious way, we need to to compute and store separate statistics for each time step in a sequence. This is problematic if a test sequence is longer than any of the training sequences. Layer normalization does not have such problem because its normalization terms depend only on the summed inputs to a layer at the current time-step. It also has only one set of gain and bias parameters shared over all time-steps.

In a standard RNN, the summed inputs in the recurrent layer are computed from the current input $\mathbf{x}^t$ and previous vector of hidden states $\mathbf{h}^{t-1}$ which are computed as $\mathbf{a}^t = W_{hh}h^{t-1} + W_{xh}\mathbf{x}^t$. The layer normalized recurrent layer re-centers and re-scales its activations using the extra normalization terms similar to Eq. (3):

$$\mathbf{h}^t = f\left[\frac{\mathbf{g}}{\sigma^t} \odot \left(\mathbf{a}^t - \mu^t\right) + \mathbf{b}\right] \qquad \mu^t = \frac{1}{H}\sum_{i=1}^{H} a_i^t \qquad \sigma^t = \sqrt{\frac{1}{H}\sum_{i=1}^{H}\left(a_i^t - \mu^t\right)^2} \tag{4}$$

where W_{hh} is the recurrent hidden to hidden weights and W_{xh} are the bottom up input to hidden weights. $\odot$ is the element-wise multiplication between two vectors. $\mathbf{b}$ and $\mathbf{g}$ are defined as the bias and gain parameters of the same dimension as $\mathbf{h}^t$.

In a standard RNN, there is a tendency for the average magnitude of the summed inputs to the recurrent units to either grow or shrink at every time-step, leading to exploding or vanishing gradients. In a layer normalized RNN, the normalization terms make it invariant to re-scaling all of the summed inputs to a layer, which results in much more stable hidden-to-hidden dynamics.

4 Related work

Batch normalization has been previously extended to recurrent neural networks [Laurent et al., 2015, Amodei et al., 2015, Cooijmans et al., 2016]. The previous work [Cooijmans et al., 2016] suggests the best performance of recurrent batch normalization is obtained by keeping independent normalization statistics for each time-step. The authors show that initializing the gain parameter in the recurrent batch normalization layer to 0.1 makes significant difference in the final performance of the model. Our work is also related to weight normalization [Salimans and Kingma, 2016]. In weight normalization, instead of the variance, the L2 norm of the incoming weights is used to normalize the summed inputs to a neuron. Applying either weight normalization or batch normalization using expected statistics is equivalent to have a different parameterization of the original feed-forward neural network. Re-parameterization in the ReLU network was studied in the Path-normalized SGD [Neyshabur et al., 2015]. Our proposed layer normalization method, however, is not a re-parameterization of the original neural network. The layer normalized model, thus, has different invariance properties than the other methods, that we will study in the following section.

5 Analysis

In this section, we investigate the invariance properties of different normalization schemes.

5.1 Invariance under weights and data transformations

The proposed layer normalization is related to batch normalization and weight normalization. Although, their normalization scalars are computed differently, these methods can be summarized as normalizing the summed inputs a_i to a neuron through the two scalars μ and σ. They also learn an adaptive bias b and gain g for each neuron after the normalization.

$$h_i = f(\frac{g_i}{\sigma_i}\left(a_i - \mu_i\right) + b_i) \tag{5}$$

Note that for layer normalization and batch normalization, μ and σ is computed according to Eq. 2 and 3. In weight normalization, μ is 0, and $\sigma = \|w\|_2$.

	Weight matrix re-scaling	Weight matrix re-centering	Weight vector re-scaling	Dataset re-scaling	Dataset re-centering	Single training case re-scaling
Batch norm	Invariant	No	Invariant	Invariant	Invariant	No
Weight norm	Invariant	No	Invariant	No	No	No
Layer norm	Invariant	Invariant	No	Invariant	No	Invariant

Table 1: Invariance properties under the normalization methods.

Table 1 highlights the following invariance results for three normalization methods.

Weight re-scaling and re-centering: First, observe that under batch normalization and weight normalization, any re-scaling to the incoming weights w_i of a single neuron has no effect on the normalized summed inputs to a neuron. To be precise, under batch and weight normalization, if the weight vector is scaled by δ, the two scalar μ and σ will also be scaled by δ. The normalized summed inputs stays the same before and after scaling. So the batch and weight normalization are invariant to the re-scaling of the weights. Layer normalization, on the other hand, is not invariant to the individual scaling of the single weight vectors. Instead, layer normalization is invariant to scaling of the entire weight matrix and invariant to a shift to all of the incoming weights in the weight matrix. Let there be two sets of model parameters θ, θ' whose weight matrices W and W' differ by a scaling factor δ and all of the incoming weights in W' are also shifted by a constant vector $\boldsymbol{\gamma}$, that is $W' = \delta W + \mathbf{1}\boldsymbol{\gamma}^\top$. Under layer normalization, the two models effectively compute the same output:

$$\begin{aligned}\mathbf{h}' =& f(\frac{\mathbf{g}}{\sigma'}(W'\mathbf{x} - \mu') + \mathbf{b}) = f(\frac{\mathbf{g}}{\sigma'}\left((\delta W + \mathbf{1}\boldsymbol{\gamma}^\top)\mathbf{x} - \mu'\right) + \mathbf{b}) \\ =& f(\frac{\mathbf{g}}{\sigma}(W\mathbf{x} - \mu) + \mathbf{b}) = \mathbf{h}. \end{aligned} \tag{6}$$

Notice that if normalization is only applied to the input before the weights, the model will not be invariant to re-scaling and re-centering of the weights.

Data re-scaling and re-centering: We can show that all the normalization methods are invariant to re-scaling the dataset by verifying that the summed inputs of neurons stays constant under the changes. Furthermore, layer normalization is invariant to re-scaling of individual training cases, because the normalization scalars μ and σ in Eq. (3) only depend on the current input data. Let $\mathbf{x}'$ be a new data point obtained by re-scaling $\mathbf{x}$ by δ. Then we have,

$$h_i' = f(\frac{g_i}{\sigma'}\left(w_i^\top \mathbf{x}' - \mu'\right) + b_i) = f(\frac{g_i}{\delta\sigma}\left(\delta w_i^\top \mathbf{x} - \delta\mu\right) + b_i) = h_i. \tag{7}$$

It is easy to see re-scaling individual data points does not change the model's prediction under layer normalization. Similar to the re-centering of the weight matrix in layer normalization, we can also show that batch normalization is invariant to re-centering of the dataset.

5.2 Geometry of parameter space during learning

We have investigated the invariance of the model's prediction under re-centering and re-scaling of the parameters. Learning, however, can behave very differently under different parameterizations, even though the models express the same underlying function. In this section, we analyze learning behavior through the geometry and the manifold of the parameter space. We show that the normalization scalar σ can implicitly reduce learning rate and makes learning more stable.

5.2.1 Riemannian metric

The learnable parameters in a statistical model form a smooth manifold that consists of all possible input-output relations of the model. For models whose output is a probability distribution, a natural way to measure the separation of two points on this manifold is the Kullback-Leibler divergence between their model output distributions. Under the KL divergence metric, the parameter space is a Riemannian manifold.

The curvature of a Riemannian manifold is entirely captured by its Riemannian metric, whose quadratic form is denoted as ds^2. That is the infinitesimal distance in the tangent space at a point in the parameter space. Intuitively, it measures the changes in the model output from the parameter space along a tangent direction. The Riemannian metric under KL was previously studied [Amari, 1998] and was shown to be well approximated under second order Taylor expansion using the Fisher

information matrix:

$$ds^2 = \mathrm{D}_{\mathrm{KL}}\big[P(y \,|\, \mathbf{x};\ \theta) \| P(y \,|\, \mathbf{x};\ \theta + \delta)\big] \approx \frac{1}{2}\delta^\top F(\theta)\delta, \tag{8}$$

$$F(\theta) = \mathop{\mathbb{E}}_{\mathbf{x}\sim P(\mathbf{x}), y\sim P(y\,|\,\mathbf{x})}\left[\frac{\partial \log P(y \,|\, \mathbf{x};\ \theta)}{\partial \theta}\frac{\partial \log P(y \,|\, \mathbf{x};\ \theta)}{\partial \theta}^\top\right], \tag{9}$$

where, δ is a small change to the parameters. The Riemannian metric above presents a geometric view of parameter spaces. The following analysis of the Riemannian metric provides some insight into how normalization methods could help in training neural networks.

5.2.2 The geometry of normalized generalized linear models

We focus our geometric analysis on the generalized linear model. The results from the following analysis can be easily applied to understand deep neural networks with block-diagonal approximation to the Fisher information matrix, where each block corresponds to the parameters for a single neuron.

A generalized linear model (GLM) can be regarded as parameterizing an output distribution from the exponential family using a weight vector w and bias scalar b. To be consistent with the previous sections, the log likelihood of the GLM can be written using the summed inputs a as the following:

$$\log P(y \,|\, \mathbf{x};\ w, b) = \frac{(a+b)y - \eta(a+b)}{\phi} + c(y, \phi), \tag{10}$$

$$\mathbb{E}[y \,|\, \mathbf{x}] = f(a+b) = f(w^\top \mathbf{x} + b),\ \ \mathrm{Var}[y \,|\, \mathbf{x}] = \phi f'(a+b), \tag{11}$$

where, $f(\cdot)$ is the transfer function that is the analog of the non-linearity in neural networks, $f'(\cdot)$ is the derivative of the transfer function, $\eta(\cdot)$ is a real valued function and $c(\cdot)$ is the log partition function. ϕ is a constant that scales the output variance. Assume a H-dimensional output vector $\mathbf{y} = [y_1, y_2, \cdots, y_H]$ is modeled using H independent GLMs and $\log P(\mathbf{y} \,|\, \mathbf{x};\ W, \mathbf{b}) = \sum_{i=1}^{H} \log P(y_i \,|\, \mathbf{x};\ w_i, b_i)$. Let W be the weight matrix whose rows are the weight vectors of the individual GLMs, $\mathbf{b}$ denote the bias vector of length H and $\mathrm{vec}(\cdot)$ denote the Kronecker vector operator. The Fisher information matrix for the multi-dimensional GLM with respect to its parameters $\theta = [w_1^\top, b_1, \cdots, w_H^\top, b_H]^\top = \mathrm{vec}([W, \mathbf{b}]^\top)$ is simply the expected Kronecker product of the data features and the output covariance matrix:

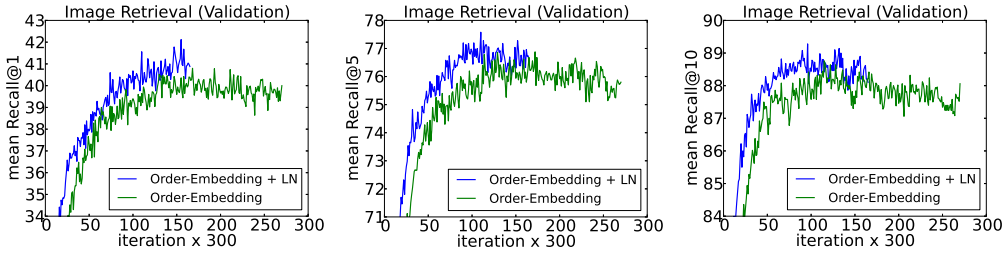
$$F(\theta) = \mathop{\mathbb{E}}_{\mathbf{x}\sim P(\mathbf{x})}\left[\frac{\mathrm{Cov}[\mathbf{y} \,|\, \mathbf{x}]}{\phi^2} \otimes \begin{bmatrix}\mathbf{x}\mathbf{x}^\top & \mathbf{x} \\ \mathbf{x}^\top & 1\end{bmatrix}\right]. \tag{12}$$

We obtain normalized GLMs by applying the normalization methods to the summed inputs a in the original model through μ and σ. Without loss of generality, we denote $\bar{F}$ as the Fisher information matrix under the normalized multi-dimensional GLM with the additional gain parameters $\theta = \mathrm{vec}([W, \mathbf{b}, \mathbf{g}]^\top)$:

$$\bar{F}(\theta) = \begin{bmatrix}\bar{F}_{11} & \cdots & \bar{F}_{1H} \\ \vdots & \ddots & \vdots \\ \bar{F}_{H1} & \cdots & \bar{F}_{HH}\end{bmatrix}, \quad \bar{F}_{ij} = \mathop{\mathbb{E}}_{\mathbf{x}\sim P(\mathbf{x})}\left[\frac{\mathrm{Cov}[y_i,\ y_j\ \,|\, \mathbf{x}]}{\phi^2}\begin{bmatrix}\frac{g_i g_j}{\sigma_i\sigma_j}\chi_i\chi_j^\top & \chi_i\frac{g_i}{\sigma_i} & \chi_i\frac{g_i(a_j-\mu_j)}{\sigma_i\sigma_j} \\ \chi_j^\top\frac{g_j}{\sigma_j} & 1 & \frac{a_j-\mu_j}{\sigma_j} \\ \chi_j^\top\frac{g_j(a_i-\mu_i)}{\sigma_i\sigma_j} & \frac{a_i-\mu_i}{\sigma_i} & \frac{(a_i-\mu_i)(a_j-\mu_j)}{\sigma_i\sigma_j}\end{bmatrix}\right] \tag{13}$$

$$\chi_i = \mathbf{x} - \frac{\partial \mu_i}{\partial w_i} - \frac{a_i - \mu_i}{\sigma_i}\frac{\partial \sigma_i}{\partial w_i}. \tag{14}$$

Implicit learning rate reduction through the growth of the weight vector: Notice that, comparing to standard GLM, the block $\bar{F}_{ij}$ along the weight vector w_i direction is scaled by the gain parameters and the normalization scalar σ_i. If the norm of the weight vector w_i grows twice as large, even though the model's output remains the same, the Fisher information matrix will be different. The curvature along the w_i direction will change by a factor of $\frac{1}{2}$ because the σ_i will also be twice as large. As a result, for the same parameter update in the normalized model, the norm of the weight vector effectively controls the learning rate for the weight vector. During learning, it is harder to change the orientation of the weight vector with large norm. The normalization methods, therefore,



(a) Recall@1 (b) Recall@5 (c) Recall@10

Figure 1: Recall@K curves using order-embeddings with and without layer normalization.

	MSCOCO							
	Caption Retrieval				Image Retrieval			
Model	**R@1**	**R@5**	**R@10**	**Mean *r***	**R@1**	**R@5**	**R@10**	**Mean *r***
Sym [Vendrov et al., 2016]	45.4		88.7	5.8	36.3		85.8	9.0
OE [Vendrov et al., 2016]	46.7		88.9	5.7	37.9		85.9	8.1
OE (ours)	46.6	79.3	89.1	5.2	37.8	73.6	85.7	7.9
OE + LN	**48.5**	**80.6**	**89.8**	**5.1**	**38.9**	**74.3**	**86.3**	**7.6**

Table 2: Average results across 5 test splits for caption and image retrieval. **R@K** is Recall@K (high is good). **Mean *r*** is the mean rank (low is good). Sym corresponds to the symmetric baseline while OE indicates order-embeddings.

have an implicit "early stopping" effect on the weight vectors and help to stabilize learning towards convergence.

Learning the magnitude of incoming weights: In normalized models, the magnitude of the incoming weights is explicitly parameterized by the gain parameters. We compare how the model output changes between updating the gain parameters in the normalized GLM and updating the magnitude of the equivalent weights under original parameterization during learning. The direction along the gain parameters in $\bar{F}$ captures the geometry for the magnitude of the incoming weights. We show that Riemannian metric along the magnitude of the incoming weights for the standard GLM is scaled by the norm of its input, whereas learning the gain parameters for the batch normalized and layer normalized models depends only on the magnitude of the prediction error. Learning the magnitude of incoming weights in the normalized model is therefore, more robust to the scaling of the input and its parameters than in the standard model. See Appendix for detailed derivations.

6 Experimental results

We perform experiments with layer normalization on 6 tasks, with a focus on recurrent neural networks: image-sentence ranking, question-answering, contextual language modelling, generative modelling, handwriting sequence generation and MNIST classification. Unless otherwise noted, the default initialization of layer normalization is to set the adaptive gains to 1 and the biases to 0 in the experiments.

6.1 Order embeddings of images and language

In this experiment, we apply layer normalization to the recently proposed order-embeddings model of Vendrov et al. [2016] for learning a joint embedding space of images and sentences. We follow the same experimental protocol as Vendrov et al. [2016] and modify their publicly available code to incorporate layer normalization [1] which utilizes Theano [Team et al., 2016]. Images and sentences from the Microsoft COCO dataset [Lin et al., 2014] are embedded into a common vector space, where a GRU [Cho et al., 2014] is used to encode sentences and the outputs of a pre-trained VGG ConvNet [Simonyan and Zisserman, 2015] (10-crop) are used to encode images. The order-embedding model represents images and sentences as a 2-level partial ordering and replaces the cosine similarity scoring function used in Kiros et al. [2014] with an asymmetric one.

[1] https://github.com/ivendrov/order-embedding

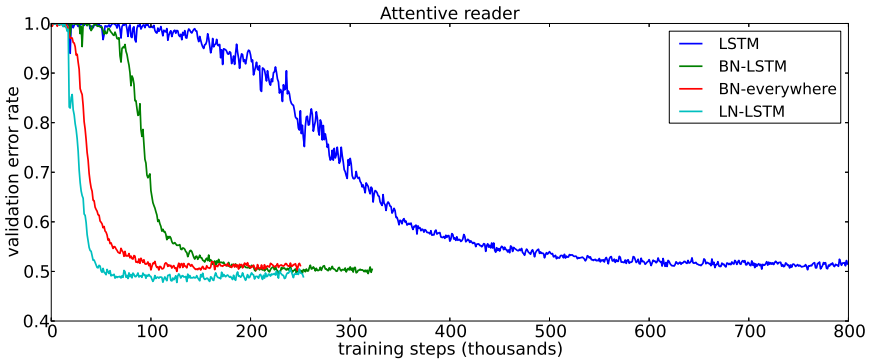


Figure 2: Validation curves for the attentive reader model. BN results are taken from [Cooijmans et al., 2016].

We trained two models: the baseline order-embedding model as well as the same model with layer normalization applied to the GRU. After every 300 iterations, we compute Recall@K (R@K) values on a held out validation set and save the model whenever R@K improves. The best performing models are then evaluated on 5 separate test sets, each containing 1000 images and 5000 captions, for which the mean results are reported. Both models use Adam [Kingma and Ba, 2014] with the same initial hyperparameters and both models are trained using the same architectural choices as used in Vendrov et al. [2016]. We refer the reader to the appendix for a description of how layer normalization is applied to GRU.

Figure 1 illustrates the validation curves of the models, with and without layer normalization. We plot R@1, R@5 and R@10 for the image retrieval task. We observe that layer normalization offers a per-iteration speedup across all metrics and converges to its best validation model in 60% of the time it takes the baseline model to do so. In Table 2, the test set results are reported from which we observe that layer normalization also results in improved generalization over the original model. The results we report are state-of-the-art for RNN embedding models, with only the structure-preserving model of Wang et al. [2016] reporting better results on this task. However, they evaluate under different conditions (1 test set instead of the mean over 5) and are thus not directly comparable.

6.2 Teaching machines to read and comprehend

In order to compare layer normalization to the recently proposed recurrent batch normalization [Cooijmans et al., 2016], we train an unidirectional attentive reader model on the CNN corpus both introduced by Hermann et al. [2015]. This is a question-answering task where a query description about a passage must be answered by filling in a blank. The data is anonymized such that entities are given randomized tokens to prevent degenerate solutions, which are consistently permuted during training and evaluation. We follow the same experimental protocol as Cooijmans et al. [2016] and modify their public code to incorporate layer normalization [2] which uses Theano [Team et al., 2016]. We obtained the pre-processed dataset used by Cooijmans et al. [2016] which differs from the original experiments of Hermann et al. [2015] in that each passage is limited to 4 sentences. In Cooijmans et al. [2016], two variants of recurrent batch normalization are used: one where BN is only applied to the LSTM while the other applies BN everywhere throughout the model. In our experiment, we only apply layer normalization within the LSTM.

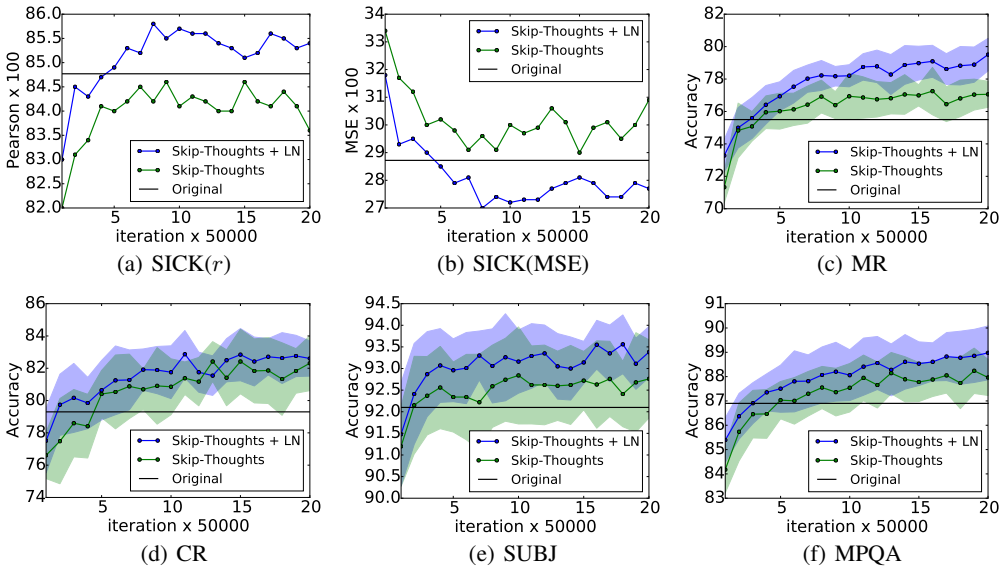
The results of this experiment are shown in Figure 2. We observe that layer normalization not only trains faster but converges to a better validation result over both the baseline and BN variants. In Cooijmans et al. [2016], it is argued that the scale parameter in BN must be carefully chosen and is set to 0.1 in their experiments. We experimented with layer normalization for both 1.0 and 0.1 scale initialization and found that the former model performed significantly better. This demonstrates that layer normalization is not sensitive to the initial scale in the same way that recurrent BN is. [3]

6.3 Skip-thought vectors

Skip-thoughts [Kiros et al., 2015] is a generalization of the skip-gram model [Mikolov et al., 2013] for learning unsupervised distributed sentence representations. Given contiguous text, a sentence is

[2] https://github.com/cooijmanstim/Attentive_reader/tree/bn

[3] We only produce results on the validation set, as in the case of Cooijmans et al. [2016]



(a) SICK(r) (b) SICK(MSE) (c) MR

(d) CR (e) SUBJ (f) MPQA

Figure 3: Performance of skip-thought vectors with and without layer normalization on downstream tasks as a function of training iterations. The original lines are the reported results in [Kiros et al., 2015]. Plots with error use 10-fold cross validation. Best seen in color.

Method	SICK(r)	SICK(ρ)	SICK(MSE)	MR	CR	SUBJ	MPQA
Original [Kiros et al., 2015]	0.848	0.778	0.287	75.5	79.3	92.1	86.9
Ours	0.842	0.767	0.298	77.3	81.8	92.6	87.9
Ours + LN	0.854	0.785	0.277	**79.5**	82.6	93.4	89.0
Ours + LN †	**0.858**	**0.788**	**0.270**	79.4	**83.1**	**93.7**	**89.3**

Table 3: Skip-thoughts results. The first two evaluation columns indicate Pearson and Spearman correlation, the third is mean squared error and the remaining indicate classification accuracy. Higher is better for all evaluations except MSE. Our models were trained for 1M iterations with the exception of (†) which was trained for 1 month (approximately 1.7M iterations)

encoded with a encoder RNN and decoder RNNs are used to predict the surrounding sentences. Kiros et al. [2015] showed that this model could produce generic sentence representations that perform well on several tasks without being fine-tuned. However, training this model is time-consuming, requiring several days of training in order to produce meaningful results.

In this experiment we determine to what effect layer normalization can speed up training. Using the publicly available code of Kiros et al. [2015] [4], we train two models on the BookCorpus dataset [Zhu et al., 2015]: one with and one without layer normalization. These experiments are performed with Theano [Team et al., 2016]. We adhere to the experimental setup used in Kiros et al. [2015], training a 2400-dimensional sentence encoder with the same hyperparameters. Given the size of the states used, it is conceivable layer normalization would produce slower per-iteration updates than without. However, we found that provided CNMeM [5] is used, there was no significant difference between the two models. We checkpoint both models after every 50,000 iterations and evaluate their performance on five tasks: semantic-relatedness (SICK) [Marelli et al., 2014], movie review sentiment (MR) [Pang and Lee, 2005], customer product reviews (CR) [Hu and Liu, 2004], subjectivity/objectivity classification (SUBJ) [Pang and Lee, 2004] and opinion polarity (MPQA) [Wiebe et al., 2005]. We plot the performance of both models for each checkpoint on all tasks to determine whether the performance rate can be improved with LN.

The experimental results are illustrated in Figure 3. We observe that applying layer normalization results both in speedup over the baseline as well as better final results after 1M iterations are performed as shown in Table 3. We also let the model with layer normalization train for a total of a month, resulting in further performance gains across all but one task. We note that the performance

[4] https://github.com/ryankiros/skip-thoughts

[5] https://github.com/NVIDIA/cnmem

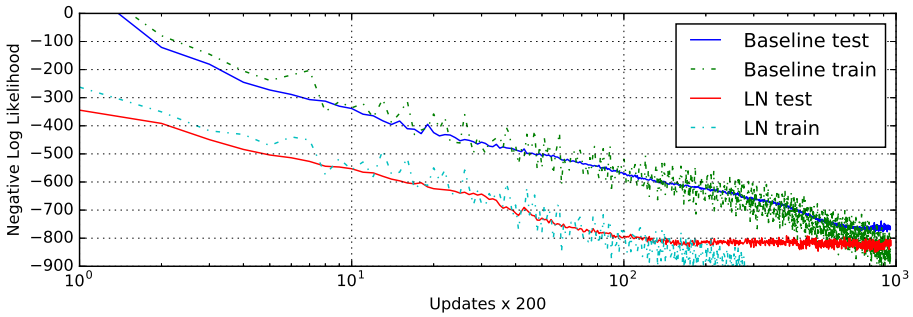


Figure 5: Handwriting sequence generation model negative log likelihood with and without layer normalization. The models are trained with mini-batch size of 8 and sequence length of 500,

differences between the original reported results and ours are likely due to the fact that the publicly available code does not condition at each timestep of the decoder, where the original model does.

6.4 Modeling binarized MNIST using DRAW

We also experimented with the generative modeling on the MNIST dataset. Deep Recurrent Attention Writer (DRAW) [Gregor et al., 2015] has previously achieved the state-of-the-art performance on modeling the distribution of MNIST digits. The model uses a differential attention mechanism and a recurrent neural network to sequentially generate pieces of an image. We evaluate the effect of layer normalization on a DRAW model using 64 glimpses and 256 LSTM hidden units. The model is trained with the default setting of Adam [Kingma and Ba, 2014] optimizer and the minibatch size of 128. Previous publications on binarized MNIST have used various training protocols to generate their datasets. In this experiment, we used the fixed binarization from Larochelle and Murray [2011]. The dataset has been split into 50,000 training, 10,000 validation and 10,000 test images.

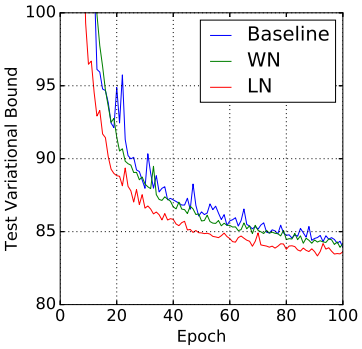


Figure 4: DRAW model test negative log likelihood with and without layer normalization.

Figure 4 shows the test variational bound for the first 100 epoch. It highlights the speedup benefit of applying layer normalization that the layer normalized DRAW converges almost twice as fast than the baseline model. After 200 epoches, the baseline model converges to a variational log likelihood of 82.36 nats on the test data and the layer normalization model obtains 82.09 nats.

6.5 Handwriting sequence generation

The previous experiments mostly examine RNNs on NLP tasks whose lengths are in the range of 10 to 40. To show the effectiveness of layer normalization on longer sequences, we performed handwriting generation tasks using the IAM Online Handwriting Database [Liwicki and Bunke, 2005]. IAM-OnDB consists of handwritten lines collected from 221 different writers. When given the input character string, the goal is to predict a sequence of x and y pen co-ordinates of the corresponding handwriting line on the whiteboard. There are, in total, 12179 handwriting line sequences. The input string is typically more than 25 characters and the average handwriting line has a length around 700.

We used the same model architecture as in Section (5.2) of Graves [2013]. The model architecture consists of three hidden layers of 400 LSTM cells, which produce 20 bivariate Gaussian mixture components at the output layer, and a size 3 input layer. The character sequence was encoded with one-hot vectors, and hence the window vectors were size 57. A mixture of 10 Gaussian functions was used for the window parameters, requiring a size 30 parameter vector. The total number of weights was increased to approximately 3.7M. The model is trained using mini-batches of size 8 and the Adam [Kingma and Ba, 2014] optimizer.

The combination of small mini-batch size and very long sequences makes it important to have very stable hidden dynamics. Figure 5 shows that layer normalization converges to a comparable log likelihood as the baseline model but is much faster.

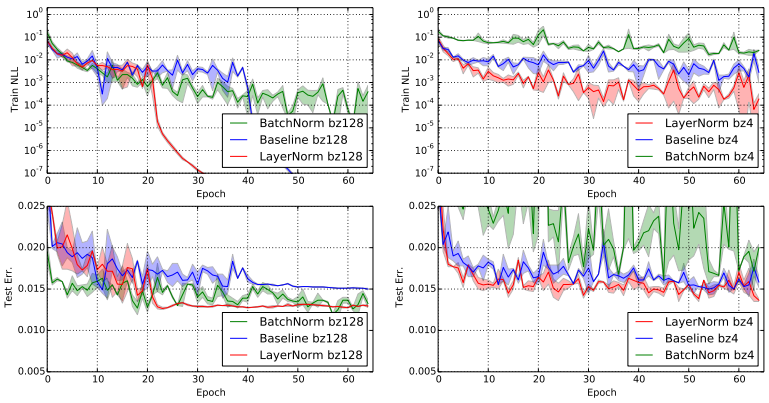


Figure 6: Permutation invariant MNIST 784-1000-1000-10 model negative log likelihood and test error with layer normalization and batch normalization. (Left) The models are trained with batch-size of 128. (Right) The models are trained with batch-size of 4.

6.6 Permutation invariant MNIST

In addition to RNNs, we investigated layer normalization in feed-forward networks. We show how layer normalization compares with batch normalization on the well-studied permutation invariant MNIST classification problem. From the previous analysis, layer normalization is invariant to input re-scaling which is desirable for the internal hidden layers. But this is unnecessary for the logit outputs where the prediction confidence is determined by the scale of the logits. We only apply layer normalization to the fully-connected hidden layers that excludes the last softmax layer.

All the models were trained using 55000 training data points and the Adam [Kingma and Ba, 2014] optimizer. For the smaller batch-size, the variance term for batch normalization is computed using the unbiased estimator. The experimental results from Figure 6 highlight that layer normalization is robust to the batch-sizes and exhibits a faster training convergence comparing to batch normalization that is applied to all layers.

6.7 Convolutional Networks

We have also experimented with convolutional neural networks. In our preliminary experiments, we observed that layer normalization offers a speedup over the baseline model without normalization, but batch normalization outperforms the other methods. With fully connected layers, all the hidden units in a layer tend to make similar contributions to the final prediction and re-centering and re-scaling the summed inputs to a layer works well. However, the assumption of similar contributions is no longer true for convolutional neural networks. The large number of the hidden units whose receptive fields lie near the boundary of the image are rarely turned on and thus have very different statistics from the rest of the hidden units within the same layer. We think further research is needed to make layer normalization work well in ConvNets.

7 Conclusion

In this paper, we introduced layer normalization to speed-up the training of neural networks. We provided a theoretical analysis that compared the invariance properties of layer normalization with batch normalization and weight normalization. We showed that layer normalization is invariant to per training-case feature shifting and scaling.

Empirically, we showed that recurrent neural networks benefit the most from the proposed method especially for long sequences and small mini-batches.

Acknowledgments

This research was funded by grants from NSERC, CFI, and Google.

Supplementary Material

Application of layer normalization to each experiment

This section describes how layer normalization is applied to each of the papers' experiments. For notation convenience, we define layer normalization as a function mapping $LN : \mathbb{R}^D \rightarrow \mathbb{R}^D$ with two set of adaptive parameters, gains $\boldsymbol{\alpha}$ and biases $\boldsymbol{\beta}$:

$$LN(\mathbf{z}; \boldsymbol{\alpha}, \boldsymbol{\beta}) = \frac{(\mathbf{z} - \mu)}{\sigma} \odot \boldsymbol{\alpha} + \boldsymbol{\beta}, \tag{15}$$

$$\mu = \frac{1}{D}\sum_{i=1}^{D} z_i, \quad \sigma = \sqrt{\frac{1}{D}\sum_{i=1}^{D}(z_i - \mu)^2}, \tag{16}$$

where, z_i is the i^{th} element of the vector $\mathbf{z}$.

Teaching machines to read and comprehend and handwriting sequence generation

The basic LSTM equations used for these experiment are given by:

$$\begin{pmatrix} \mathbf{f}_t \\ \mathbf{i}_t \\ \mathbf{o}_t \\ \mathbf{g}_t \end{pmatrix} = \mathbf{W}_h \mathbf{h}_{t-1} + \mathbf{W}_x \mathbf{x}_t + b \tag{17}$$

$$\mathbf{c}_t = \sigma(\mathbf{f}_t) \odot \mathbf{c}_{t-1} + \sigma(\mathbf{i}_t) \odot \tanh(\mathbf{g}_t) \tag{18}$$

$$\mathbf{h}_t = \sigma(\mathbf{o}_t) \odot \tanh(\mathbf{c}_t) \tag{19}$$

The version that incorporates layer normalization is modified as follows:

$$\begin{pmatrix} \mathbf{f}_t \\ \mathbf{i}_t \\ \mathbf{o}_t \\ \mathbf{g}_t \end{pmatrix} = LN(\mathbf{W}_h \mathbf{h}_{t-1}; \boldsymbol{\alpha}_1, \boldsymbol{\beta}_1) + LN(\mathbf{W}_x \mathbf{x}_t; \boldsymbol{\alpha}_2, \boldsymbol{\beta}_2) + b \tag{20}$$

$$\mathbf{c}_t = \sigma(\mathbf{f}_t) \odot \mathbf{c}_{t-1} + \sigma(\mathbf{i}_t) \odot \tanh(\mathbf{g}_t) \tag{21}$$

$$\mathbf{h}_t = \sigma(\mathbf{o}_t) \odot \tanh(LN(\mathbf{c}_t; \boldsymbol{\alpha}_3, \boldsymbol{\beta}_3)) \tag{22}$$

where $\boldsymbol{\alpha}_i, \boldsymbol{\beta}_i$ are the additive and multiplicative parameters, respectively. Each $\boldsymbol{\alpha}_i$ is initialized to a vector of zeros and each $\boldsymbol{\beta}_i$ is initialized to a vector of ones.

Order embeddings and skip-thoughts

These experiments utilize a variant of gated recurrent unit which is defined as follows:

$$\begin{pmatrix} \mathbf{z}_t \\ \mathbf{r}_t \end{pmatrix} = \mathbf{W}_h \mathbf{h}_{t-1} + \mathbf{W}_x \mathbf{x}_t \tag{23}$$

$$\hat{\mathbf{h}}_t = \tanh(\mathbf{W}\mathbf{x}_t + \sigma(\mathbf{r}_t) \odot (\mathbf{U}\mathbf{h}_{t-1})) \tag{24}$$

$$\mathbf{h}_t = (1 - \sigma(\mathbf{z}_t))\mathbf{h}_{t-1} + \sigma(\mathbf{z}_t)\hat{\mathbf{h}}_t \tag{25}$$

Layer normalization is applied as follows:

$$\begin{pmatrix} \mathbf{z}_t \\ \mathbf{r}_t \end{pmatrix} = LN(\mathbf{W}_h \mathbf{h}_{t-1}; \boldsymbol{\alpha}_1, \boldsymbol{\beta}_1) + LN(\mathbf{W}_x \mathbf{x}_t; \boldsymbol{\alpha}_2, \boldsymbol{\beta}_2) \tag{26}$$

$$\hat{\mathbf{h}}_t = \tanh(LN(\mathbf{W}\mathbf{x}_t; \boldsymbol{\alpha}_3, \boldsymbol{\beta}_3) + \sigma(\mathbf{r}_t) \odot LN(\mathbf{U}\mathbf{h}_{t-1}; \boldsymbol{\alpha}_4, \boldsymbol{\beta}_4)) \tag{27}$$

$$\mathbf{h}_t = (1 - \sigma(\mathbf{z}_t))\mathbf{h}_{t-1} + \sigma(\mathbf{z}_t)\hat{\mathbf{h}}_t \tag{28}$$

just as before, $\boldsymbol{\alpha}_i$ is initialized to a vector of zeros and each $\boldsymbol{\beta}_i$ is initialized to a vector of ones.

Modeling binarized MNIST using DRAW

The layer norm is only applied to the output of the LSTM hidden states in this experiment:

The version that incorporates layer normalization is modified as follows:

$$\begin{pmatrix} \mathbf{f}_t \\ \mathbf{i}_t \\ \mathbf{o}_t \\ \mathbf{g}_t \end{pmatrix} = \mathbf{W}_h \mathbf{h}_{t-1} + \mathbf{W}_x \mathbf{x}_t + b \tag{29}$$

$$\mathbf{c}_t = \sigma(\mathbf{f}_t) \odot \mathbf{c}_{t-1} + \sigma(\mathbf{i}_t) \odot \tanh(\mathbf{g}_t) \tag{30}$$

$$\mathbf{h}_t = \sigma(\mathbf{o}_t) \odot \tanh(LN(\mathbf{c}_t; \boldsymbol{\alpha}, \boldsymbol{\beta})) \tag{31}$$

where $\boldsymbol{\alpha}, \boldsymbol{\beta}$ are the additive and multiplicative parameters, respectively. $\boldsymbol{\alpha}$ is initialized to a vector of zeros and $\boldsymbol{\beta}$ is initialized to a vector of ones.

Learning the magnitude of incoming weights

We now compare how gradient descent updates changing magnitude of the equivalent weights between the normalized GLM and original parameterization. The magnitude of the weights are explicitly parameterized using the gain parameter in the normalized model. Assume there is a gradient update that changes norm of the weight vectors by δ_g. We can project the gradient updates to the weight vector for the normal GLM. The KL metric, ie how much the gradient update changes the model prediction, for the normalized model depends only on the magnitude of the prediction error. Specifically,

under batch normalization:

$$ds^2 = \frac{1}{2} \operatorname{vec}([0, 0, \delta_g]^\top)^\top \bar{F} (\operatorname{vec}([W, \mathbf{b}, \mathbf{g}]^\top) \operatorname{vec}([0, 0, \delta_g]^\top) = \frac{1}{2} \delta_g^\top \mathop{\mathbb{E}}_{\mathbf{x} \sim P(\mathbf{x})} \left[\frac{\operatorname{Cov}[\mathbf{y} \mid \mathbf{x}]}{\phi^2} \right] \delta_g. \tag{32}$$

Under layer normalization:

$$\begin{aligned} ds^2 =& \frac{1}{2} \operatorname{vec}([0, 0, \delta_g]^\top)^\top \bar{F} (\operatorname{vec}([W, \mathbf{b}, \mathbf{g}]^\top) \operatorname{vec}([0, 0, \delta_g]^\top) \\ =& \frac{1}{2} \delta_g^\top \frac{1}{\phi^2} \mathop{\mathbb{E}}_{\mathbf{x} \sim P(\mathbf{x})} \begin{bmatrix} \operatorname{Cov}(y_1, y_1 \mid \mathbf{x}) \frac{(a_1 - \mu)^2}{\sigma^2} & \cdots & \operatorname{Cov}(y_1, y_H \mid \mathbf{x}) \frac{(a_1 - \mu)(a_H - \mu)}{\sigma^2} \\ \vdots & \ddots & \vdots \\ \operatorname{Cov}(y_H, y_1 \mid \mathbf{x}) \frac{(a_H - \mu)(a_1 - \mu)}{\sigma^2} & \cdots & \operatorname{Cov}(y_H, y_H \mid \mathbf{x}) \frac{(a_H - \mu)^2}{\sigma^2} \end{bmatrix} \delta_g \end{aligned} \tag{33}$$

Under weight normalization:

$$\begin{aligned} ds^2 =& \frac{1}{2} \operatorname{vec}([0, 0, \delta_g]^\top)^\top \bar{F} (\operatorname{vec}([W, \mathbf{b}, \mathbf{g}]^\top) \operatorname{vec}([0, 0, \delta_g]^\top) \\ =& \frac{1}{2} \delta_g^\top \frac{1}{\phi^2} \mathop{\mathbb{E}}_{\mathbf{x} \sim P(\mathbf{x})} \begin{bmatrix} \operatorname{Cov}(y_1, y_1 \mid \mathbf{x}) \frac{a_1^2}{\|w_1\|_2^2} & \cdots & \operatorname{Cov}(y_1, y_H \mid \mathbf{x}) \frac{a_1 a_H}{\|w_1\|_2 \|w_H\|_2} \\ \vdots & \ddots & \vdots \\ \operatorname{Cov}(y_H, y_1 \mid \mathbf{x}) \frac{a_H a_1}{\|w_H\|_2 \|w_1\|_2} & \cdots & \operatorname{Cov}(y_H, y_H \mid \mathbf{x}) \frac{a_H^2}{\|w_H\|_2^2} \end{bmatrix} \delta_g. \end{aligned} \tag{34}$$

Whereas, the KL metric in the standard GLM is related to its activities $a_i = w_i^\top \mathbf{x}$, that is depended on both its current weights and input data. We project the gradient updates to the gain parameter $\delta_g i$ of the i^{th} neuron to its weight vector as $\delta_{gi} \frac{w_i}{\|w_i\|_2}$ in the standard GLM model:

$$\begin{aligned} & \frac{1}{2} \operatorname{vec}([\delta_{gi} \frac{w_i}{\|w_i\|_2}, 0, \delta_{gj} \frac{w_j}{\|w_i\|_2}, 0]^\top)^\top F([w_i^\top, b_i, w_j^\top, b_j]^\top) \operatorname{vec}([\delta_{gi} \frac{w_i}{\|w_i\|_2}, 0, \delta_{gj} \frac{w_j}{\|w_j\|_2}, 0]^\top) \\ =& \frac{\delta_{gi} \delta_{gj}}{2\phi^2} \mathop{\mathbb{E}}_{\mathbf{x} \sim P(\mathbf{x})} \left[\operatorname{Cov}(y_i, y_j \mid \mathbf{x}) \frac{a_i a_j}{\|w_i\|_2 \|w_j\|_2} \right] \end{aligned} \tag{35}$$

The batch normalized and layer normalized models are therefore more robust to the scaling of the input and its parameters than the standard model.

Instance Normalization: The Missing Ingredient for Fast Stylization

Dmitry Ulyanov
Computer Vision Group
Skoltech & Yandex
Russia
dmitry.ulyanov@skoltech.ru

Andrea Vedaldi
Visual Geometry Group
University of Oxford
United Kingdom
vedaldi@robots.ox.ac.uk

Victor Lempitsky
Computer Vision Group
Skoltech
Russia
lempitsky@skoltech.ru

Abstract

It this paper we revisit the fast stylization method introduced in Ulyanov et al. (2016). We show how a small change in the stylization architecture results in a significant qualitative improvement in the generated images. The change is limited to swapping batch normalization with instance normalization, and to apply the latter both at training and testing times. The resulting method can be used to train high-performance architectures for real-time image generation. The code is available at https://github.com/DmitryUlyanov/texture_nets. Full paper can be found at https://arxiv.org/abs/1701.02096.

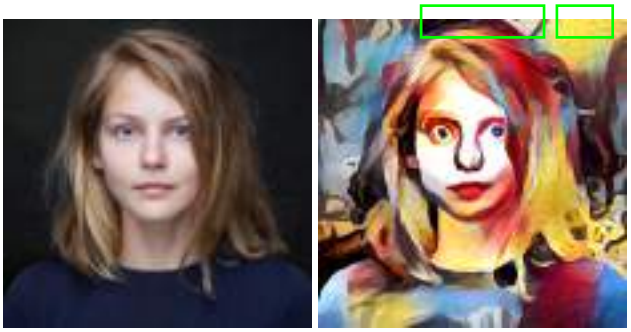
1 Introduction

The recent work of Gatys et al. (2016) introduced a method for transferring a style from an image onto another one, as demonstrated in fig. 1. The stylized image matches simultaneously selected statistics of the style image and of the content image. Both style and content statistics are obtained from a deep convolutional network pre-trained for image classification. The style statistics are extracted from shallower layers and averaged across spatial locations whereas the content statistics are extracted form deeper layers and preserve spatial information. In this manner, the style statistics capture the "texture" of the style image whereas the content statistics capture the "structure" of the content image.

Although the method of Gatys et. al produces remarkably good results, it is computationally inefficient. The stylized image is, in fact, obtained by iterative optimization until it matches the desired statistics. In practice, it takes several minutes to stylize an image of size 512×512. Two recent works, Ulyanov et al. (2016) Johnson et al. (2016), sought to address this problem by *learning equivalent feed-forward generator networks* that can generate the stylized image in a single pass. These two methods differ mainly by the details of the generator architecture and produce results of a comparable quality; however, neither achieved as good results as the slower optimization-based method of Gatys et. al.

In this paper we revisit the method for feed-forward stylization of Ulyanov et al. (2016) and show that a small change in a generator architecture leads to much improved results. The results are in fact of comparable quality as the slow optimization method of Gatys et al. but can be obtained in real time on standard GPU hardware. The key idea (section 2) is to replace batch normalization

Figure 1: Artistic style transfer example of Gatys et al. (2016) method.



(a) Content image. (b) Stylized image.

(c) Low contrast content image. (d) Stylized low contrast image.

Figure 2: A contrast of a stylized image is mostly determined by a contrast of a style image and almost independent of a content image contrast. The stylization is performed with method of Gatys et al. (2016).

layers in the generator architecture with instance normalization layers, and to keep them at test time (as opposed to freeze and simplify them out as done for batch normalization). Intuitively, the normalization process allows to remove instance-specific contrast information from the content image, which simplifies generation. In practice, this results in vastly improved images (section 3).

2 Method

The work of Ulyanov et al. (2016) showed that it is possible to learn a generator network $g(\mathbf{x}, \mathbf{z})$ that can apply to a given input image $\mathbf{x}$ the style of another $\mathbf{x}_0$, reproducing to some extent the results of the optimization method of Gatys et al. Here, the style image $\mathbf{x}_0$ is fixed and the generator g is learned to apply the style to any input image $\mathbf{x}$. The variable $\mathbf{z}$ is a random seed that can be used to obtain sample stylization results.

The function g is a convolutional neural network learned from examples. Here an example is just a content image $\mathbf{x}_t, t = 1, \ldots, n$ and learning solves the problem

$$\min_g \frac{1}{n} \sum_{t=1}^{n} \mathcal{L}(\mathbf{x}_0, \mathbf{x}_t, g(\mathbf{x}_t, \mathbf{z}_t))$$

Figure 3: Row 1: content image (left), style image (middle) and style transfer using method of Gatys et. al (right). Row 2: typical stylization results when trained for a large number of iterations using fast stylization method from Ulyanov et al. (2016): with zero padding (left), with a better padding technique (middle), with zero padding and instance normalization (right).

where $\mathbf{z}_t \sim \mathcal{N}(0, 1)$ are i.i.d. samples from a Gaussian distribution. The loss $\mathcal{L}$ uses a pre-trained CNN (not shown) to extracts features from the style $\mathbf{x}_0$ image, the content image $\mathbf{x}_t$, and the stylized image $g(\mathbf{x}_t, \mathbf{z}_t)$, and compares their statistics as explained before.

While the generator network g is fast, the authors of Ulyanov et al. (2016) observed that learning it from too many training examples yield poorer *qualitative* results. In particular, a network trained on just 16 example images produced better results than one trained from thousands of those. The most serious artifacts were found along the border of the image due to the zero padding added before every convolution operation (see fig. 3). Even by using more complex padding techniques it was not possible to solve this issue. Ultimately, the best results presented in Ulyanov et al. (2016) were obtained using a small number of training images and stopping the learning process early. We conjectured that the training objective was too hard to learn for a standard neural network architecture.

A simple observation is that the result of stylization should not, in general, depend on the contrast of the content image (see fig. 2). In fact, the *style loss* is designed to transfer elements from a style image to the content image such that the contrast of the stylized image is similar to the contrast of the style image. Thus, the generator network should discard contrast information in the content image. The question is whether contrast normalization can be implemented efficiently by combining standard CNN building blocks or whether, instead, is best implemented directly in the architecture.

The generators used in Ulyanov et al. (2016) and Johnson et al. (2016) use convolution, pooling, upsampling, and batch normalization. In practice, it may be difficult to learn a highly nonlinear contrast normalization function as a combination of such layers. To see why, let $x \in \mathbb{R}^{T \times C \times W \times H}$ be an input tensor containing a batch of T images. Let x_{tijk} denote its $tijk$-th element, where k and j span spatial dimensions, i is the feature channel (color channel if the input is an RGB image), and t is the index of the image in the batch. Then a simple version of contrast normalization is given by:

$$y_{tijk} = \frac{x_{tijk}}{\sum_{l=1}^{W} \sum_{m=1}^{H} x_{tilm}}. \tag{1}$$

It is unclear how such as function could be implemented as a sequence of ReLU and convolution operator.

On the other hand, the generator network of Ulyanov et al. (2016) does contain a normalization layers, and precisely *batch normalization* ones. The key difference between eq. (1) and batch normalization is that the latter applies the normalization to a whole batch of images instead for single ones:

$$y_{tijk} = \frac{x_{tijk} - \mu_i}{\sqrt{\sigma_i^2 + \epsilon}}, \quad \mu_i = \frac{1}{HWT}\sum_{t=1}^{T}\sum_{l=1}^{W}\sum_{m=1}^{H} x_{tilm}, \quad \sigma_i^2 = \frac{1}{HWT}\sum_{t=1}^{T}\sum_{l=1}^{W}\sum_{m=1}^{H}(x_{tilm} - mu_i)^2. \tag{2}$$

In order to combine the effects of instance-specific normalization and batch normalization, we propose to replace the latter by the *instance normalization* (also known as "contrast normalization") layer:

$$y_{tijk} = \frac{x_{tijk} - \mu_{ti}}{\sqrt{\sigma_{ti}^2 + \epsilon}}, \quad \mu_{ti} = \frac{1}{HW}\sum_{l=1}^{W}\sum_{m=1}^{H} x_{tilm}, \quad \sigma_{ti}^2 = \frac{1}{HW}\sum_{l=1}^{W}\sum_{m=1}^{H}(x_{tilm} - mu_{ti})^2. \tag{3}$$

We replace batch normalization with instance normalization everywhere in the generator network g. This prevents instance-specific mean and covariance shift simplifying the learning process. Differently from batch normalization, furthermore, the instance normalization layer is applied at test time as well.

3 Experiments

In this section, we evaluate the effect of the modification proposed in section 2 and replace batch normalization with instance normalization. We tested both generator architectures described in Ulyanov et al. (2016) and Johnson et al. (2016) in order too see whether the modification applies to different architectures. While we did not have access to the original network by Johnson et al. (2016), we carefully reproduced their model from the description in the paper. Ultimately, we found that both generator networks have similar performance and shortcomings (fig. 5 first row).

Next, the replaced batch normalization with instance normalization and retrained the generators using the same hyperparameters. We found that both architectures significantly improved by the use of instance normalization (fig. 5 second row). The quality of both generators is similar, but we found the residuals architecture of Johnson et al. (2016) to be somewhat more efficient and easy to use, so we adopted it for the results shown in fig. 4.

4 Conclusion

In this short note, we demonstrate that by replacing batch normalization with instance normalization it is possible to dramatically improve the performance of certain deep neural networks for image generation. The result is suggestive, and we are currently experimenting with similar ideas for image discrimination tasks as well.

References

Gatys, L. A., Ecker, A. S., and Bethge, M. (2016). Image style transfer using convolutional neural networks. In *The IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*.

Johnson, J., Alahi, A., and Li, F. (2016). Perceptual losses for real-time style transfer and super-resolution. *CoRR*, abs/1603.08155.

Ulyanov, D., Lebedev, V., Vedaldi, A., and Lempitsky, V. S. (2016). Texture networks: Feed-forward synthesis of textures and stylized images. In *Proceedings of the 33nd International Conference on Machine Learning, ICML 2016, New York City, NY, USA, June 19-24, 2016*, pages 1349–1357.

Figure 4: Stylization examples using proposed method. First row: style images; second row: original image and its stylized versions.

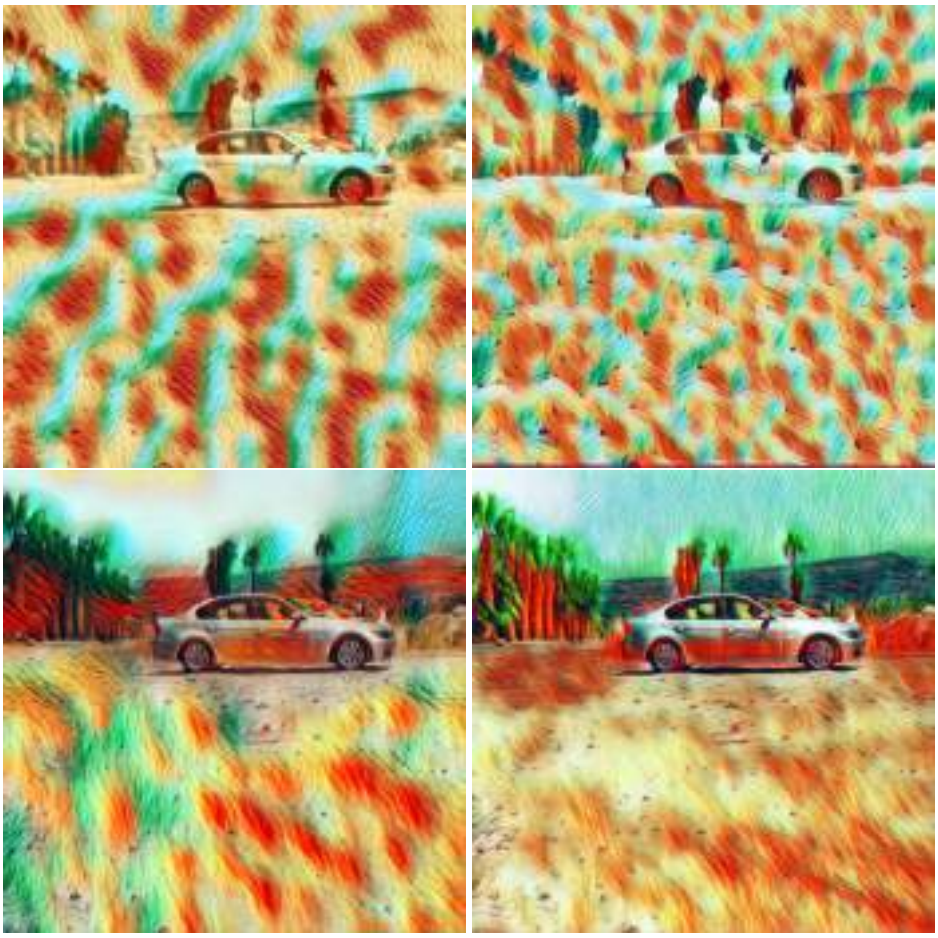


Figure 5: Qualitative comparison of generators proposed in Ulyanov et al. (2016) (left), Johnson et al. (2016) (right) with batch normalization (first row) and instance normalization (second row). Both architectures benefit from instance normalization.

Figure 6: Processing a content image from fig. 4 with Delaunay style at different resolutions: 512 (left) and 1080 (right).

Improved Texture Networks: Maximizing Quality and Diversity in Feed-forward Stylization and Texture Synthesis

Dmitry Ulyanov
Skolkovo Institute of Science and Technology & Yandex
dmitry.ulyanov@skoltech.ru

Andrea Vedaldi
University of Oxford
vedaldi@robots.ox.ac.uk

Victor Lempitsky
Skolkovo Institute of Science and Technology
lempitsky@skoltech.ru

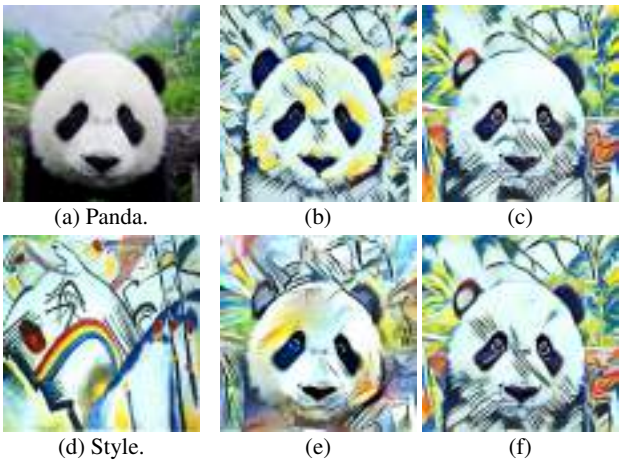
Abstract

The recent work of Gatys et al.*, who characterized the style of an image by the statistics of convolutional neural network filters, ignited a renewed interest in the texture generation and image stylization problems. While their image generation technique uses a slow optimization process, recently several authors have proposed to learn generator neural networks that can produce similar outputs in one quick forward pass. While generator networks are promising, they are still inferior in visual quality and diversity compared to generation-by-optimization. In this work, we advance them in two significant ways. First, we introduce an instance normalization module to replace batch normalization with significant improvements to the quality of image stylization. Second, we improve diversity by introducing a new learning formulation that encourages generators to sample unbiasedly from the Julesz texture ensemble, which is the equivalence class of all images characterized by certain filter responses. Together, these two improvements take feed forward texture synthesis and image stylization much closer to the quality of generation-via-optimization, while retaining the speed advantage.*

1. Introduction

The recent work of Gatys *et al*. [4, 5], which used deep neural networks for texture synthesis and image stylization to a great effect, has created a surge of interest in this area. Following an earlier work by Portilla and Simoncelli [15], they generate an image by matching the second order moments of the response of certain filters applied to a reference texture image. The innovation of Gatys *et al*. is to use non-linear convolutional neural network filters for this purpose. Despite the excellent results, however, the matching process is based on local optimization, and generally requires a considerable amount of time (tens of seconds to minutes) in order to generate a single textures or stylized image.

In order to address this shortcoming, Ulyanov *et al*. [19] and Johnson *et al*. [8] suggested to replace the optimization process with feed-forward generative convolutional networks. In particular, [19] introduced *texture networks*



(a) Panda. (b) (c) (d) Style. (e) (f)

Figure 1: Which panda stylization seems the best to you? Definitely not the variant (b), which has been produced by a state-of-the-art algorithm among methods that take no longer than a second. The (e) picture took several minutes to generate using an optimization process, but the quality is worth it, isn't it? We would be particularly happy if you chose one from the rightmost two examples, which are computed with our new method that aspires to combine the quality of the optimization-based method and the speed of the fast one. Moreover, our method is able to produce diverse stylizations using a single network.

The source code is available at https://github.com/DmitryUlyanov/texture_nets

to generate textures of a certain kind, as in [4], or to apply a certain texture style to an arbitrary image, as in [5]. Once trained, such texture networks operate in a feed-forward manner, three orders of magnitude faster than the optimization methods of [4, 5].

The price to pay for such speed is a reduced performance. For texture synthesis, the neural network of [19] generates good-quality samples, but these are not as diverse as the ones obtained from the iterative optimization method of [4]. For image stylization, the feed-forward results of [19, 8] are qualitatively and quantitatively worse than iterative optimization. In this work, we address both limitations by means of two contributions, both of which extend beyond the applications considered in this paper.

Our first contribution (section 4) is an architectural change that significantly improves the generator networks. The change is the introduction of an **instance-normalization layer** which, particularly for the stylization problem, greatly improves the performance of the deep network generators. This advance significantly reduces the gap in stylization quality between the feed-forward models and the original iterative optimization method of Gatys *et al*., both quantitatively and qualitatively.

Our second contribution (section 3) addresses the limited diversity of the samples generated by texture networks. In order to do so, we introduce a new formulation that learns generators that **uniformly sample the Julesz ensemble** [20]. The latter is the equivalence class of images that match certain filter statistics. Uniformly sampling this set guarantees diverse results, but traditionally doing so required slow Monte Carlo methods [20]; Portilla and Simoncelli, and hence Gatys *et al*., cannot sample from this set, but only find individual points in it, and possibly just one point. Our formulation minimizes the Kullback-Leibler divergence between the generated distribution and a quasi-uniform distribution on the Julesz ensemble. The learning objective decomposes into a loss term similar to Gatys *et al*. minus the entropy of the generated texture samples, which we estimate in a differentiable manner using a non-parametric estimator [12].

We validate our contributions by means of extensive quantitative and qualitative experiments, including comparing the feed-forward results with the gold-standard optimization-based ones (section 5). We show that, combined, these ideas dramatically improve the quality of feed-forward texture synthesis and image stylization, bringing them to a level comparable to the optimization-based approaches.

2. Background and related work

Julesz ensemble. Informally, a *texture* is a family of visual patterns, such as checkerboards or slabs of concrete, that share certain local statistical regularities. The concept was first studied by Julesz [9], who suggested that the visual system discriminates between different textures based on the average responses of certain image filters.

The work of [20] formalized Julesz' ideas by introducing the concept of *Julesz ensemble*. There, an image is a real function $x : \Omega \to \mathbb{R}^3$ defined on a discrete lattice $\Omega = \{1, \dots, H\} \times \{1, \dots, W\}$ and a texture is a distribution $p(x)$ over such images. The local statistics of an image are captured by a bank of (non-linear) filters $F_l : \mathcal{X} \times \Omega \to \mathbb{R}$, $l = 1, \dots, L$, where $F_l(x, u)$ denotes the response of filter F_l at location u on image x. The image x is characterized by the spatial average of the filter responses $\mu_l(x) = \sum_{u \in \Omega} F_l(x, u)/|\Omega|$. The image is perceived as a particular texture if these responses match certain characteristic values $\bar{\mu}_l$. Formally, given the loss function,

$$\mathcal{L}(x) = \sum_{l=1}^{L} (\mu_l(x) - \bar{\mu}_l)^2 \tag{1}$$

the Julesz ensemble is the set of all texture images

$$\mathcal{T}_\epsilon = \{x \in \mathcal{X} : \mathcal{L}(x) \leq \epsilon\}$$

that approximately satisfy such constraints. Since all textures in the Julesz ensemble are perceptually equivalent, it is natural to require the texture distribution $p(x)$ to be uniform over this set. In practice, it is more convenient to consider the exponential distribution

$$p(x) = \frac{e^{-\mathcal{L}(x)/T}}{\int e^{-\mathcal{L}(y)/T}\, dy}, \tag{2}$$

where $T > 0$ is a temperature parameter. This choice is motivated as follows [20]: since statistics are computed from spatial averages of filter responses, one can show that, in the limit of infinitely large lattices, the distribution $p(x)$ is zero outside the Julesz ensemble and uniform inside. In this manner, eq. (2) can be though as a uniform distribution over images that have a certain characteristic filter responses $\bar{\mu} = (\bar{\mu}_1, \dots, \bar{\mu}_L)$.

Note also that the texture is completely described by the filter bank $F = (F_1, \dots, F_L)$ and their characteristic responses $\bar{\mu}$. As discussed below, the filter bank is generally fixed, so in this framework different textures are given by different characteristics $\bar{\mu}$.

Generation-by-minimization. For any interesting choice of the filter bank F, sampling from eq. (2) is rather challenging and classically addressed by Monte Carlo methods [20]. In order to make this framework more practical, Portilla and Simoncelli [16] proposed instead to heuristically sample from the Julesz ensemble by the optimization process

$$x^* = \operatorname*{argmin}_{x \in \mathcal{X}} \mathcal{L}(x). \tag{3}$$

If this optimization problem can be solved, the minimizer x^* is by definition a texture image. However, there is no reason why this process should generate fair samples from the distribution $p(x)$. In fact, the only reason why eq. (3) may not simply return *always the same* image is that the optimization algorithm is randomly initialized, the loss function is highly non-convex, and search is local. Only because of this eq. (3) may land on different samples x^* on different runs.

Deep filter banks. Constructing a Julesz ensemble requires choosing a filter bank F. Originally, researchers considered the obvious candidates: Gaussian derivative filters, Gabor filters, wavelets, histograms, and similar [20, 16, 21]. More recently, the work of Gatys *et al*. [4, 5] demonstrated that much superior filters are automatically learned by deep convolutional neural networks (CNNs) even when trained for apparently unrelated problems, such as image classification. In this paper, in particular, we choose for $\mathcal{L}(x)$ the *style loss* proposed by [4]. The latter is the distance between the empirical correlation matrices of deep filter responses in a CNN.[1]

Stylization. The texture generation method of Gatys *et al*. [4] can be considered as a direct extension of the texture generation-by-minimization technique (3) of Portilla and Simoncelli [16]. Later, Gatys *et al*. [5] demonstrated that the same technique can be used to generate an image that mixes the statistics of two other images, one used as a texture template and one used as a content template. Content is captured by introducing a second loss $\mathcal{L}_{\text{cont.}}(x, x_0)$ that compares the responses of deep CNN filters extracted from the generated image x and a content image x_0. Minimizing the combined loss $\mathcal{L}(x) + \alpha\mathcal{L}_{\text{cont.}}(x, x_0)$ yields impressive *artistic images*, where a texture $\bar{\mu}$, defining the artistic style, is fused with the content image x_0.

Feed-forward generator networks. For all its simplicity and efficiency compared to Markov sampling techniques, generation-by-optimization (3) is still relatively slow, and certainly too slow for real-time applications. Therefore, in the past few months several authors [8, 19] have proposed to *learn generator neural networks* $g(z)$ that can directly map random noise samples $z \sim p_z = \mathcal{N}(0, I)$ to a local minimizer of eq. (3). Learning the neural network g amounts to minimizing the objective

$$g^* = \operatorname*{argmin}_g \mathop{\mathbb{E}}_{p_z} \mathcal{L}(g(z)). \tag{4}$$

[1]Note that such matrices are obtained by averaging local non-linear filters: these are the outer products of filters in a certain layer of the neural network. Hence, the style loss of Gatys *et al*. is in the same form as eq. (1).

While this approach works well in practice, it shares the same important limitation as the original work of Portilla and Simoncelli: there is no guarantee that samples generated by g^* would be fair samples of the texture distribution (2). In practice, as we show in the paper, such samples tend in fact to be not diverse enough.

Both [8, 19] have also shown that similar generator networks work also for stylization. In this case, the generator $g(x_0, z)$ is a function of the content image x_0 and of the random noise z. The network g is learned to minimize the sum of texture loss and the content loss:

$$g^* = \operatorname*{argmin}_g \mathop{\mathbb{E}}_{p_{x_0}, p_z} [\mathcal{L}(g(x_0, z)) + \alpha\mathcal{L}_{\text{cont.}}(g(x_0, z), x_0)]. \tag{5}$$

Alternative neural generator methods. There are many other techniques for image generation using deep neural networks.

The Julesz distribution is closely related to the FRAME maximum entropy model of [21], as well as to the concept of *Maximum Mean Discrepancy* (MMD) introduced in [7]. Both FRAME and MMD make the observation that a probability distribution $p(x)$ can be described by the expected values $\mu_\alpha = \mathbb{E}_{x\sim p(x)}[\phi_\alpha(x)]$ of a sufficiently rich set of statistics $\phi_\alpha(x)$. Building on these ideas, [14, 3] construct generator neural networks g with the goal of minimizing the discrepancy between the statistics averaged over a batch of generated images $\sum_{i=1}^N \phi_\alpha(g(z_i))/N$ and the statistics averaged over a training set $\sum_{i=1}^M \phi_\alpha(x_i)/M$. The resulting networks g are called *Moment Matching Networks* (MMN).

An important alternative methodology is based on the concept of Generative Adversarial Networks (GAN; [6]). This approach trains, together with the generator network $g(z)$, a second *adversarial* network $f(\cdot)$ that attempts to distinguish between generated samples $g(z), z \sim \mathcal{N}(0, I)$ and real samples $x \sim p_{\text{data}}(x)$. The adversarial model f can be used as a measure of quality of the generated samples and used to learn a better generator g. GAN are powerful but notoriously difficult to train. A lot of research is has recently focused on improving GAN or extending it. For instance, LAPGAN [2] combines GAN with a Laplacian pyramid and DCGAN [17] optimizes GAN for large datasets.

3. Julesz generator networks

This section describes our first contribution, namely a method to learn networks that draw samples from the Julesz ensemble modeling a texture (section 2), which is an intractable problem usually addressed by slow Monte Carlo methods [21, 20]. Generation-by-optimization, popularized by Portilla and Simoncelli and Gatys *et al*., is faster, but can only find one point in the ensemble, not sample from it,

with scarce sample diversity, particularly when used to train feed-forward generator networks [8, 19].

Here, we propose a new formulation that allows to train generator networks that *sample the Julesz ensemble*, generating images with high visual fidelity as well as high diversity.

A generator network [6] maps an i.i.d. noise vector $z \sim \mathcal{N}(0, I)$ to an image $x = g(z)$ in such a way that x is ideally a sample from the desired distribution $p(x)$. Such generators have been adopted for texture synthesis in [19], but without guarantees that the learned generator $g(z)$ would indeed sample a particular distribution.

Here, we would like to sample from the Gibbs distribution (2) defined over the Julesz ensemble. This distribution can be written compactly as $p(x) = Z^{-1}e^{-\mathcal{L}(x)/T}$, where $Z = \int e^{-\mathcal{L}(x)/T}\,dx$ is an intractable normalization constant.

Denote by $q(x)$ the distribution induced by a generator network g. The goal is to make the target distribution p and the generator distribution q as close as possible by minimizing their Kullback-Leibler (KL) divergence:

$$\begin{aligned} KL(q||p) &= \int q(x) \ln \frac{q(x)Z}{p(x)}\,dx \\ &= \frac{1}{T} \mathop{\mathbb{E}}_{x\sim q(x)} \mathcal{L}(x) + \mathop{\mathbb{E}}_{x\sim q(x)} \ln q(x) + \ln(Z) \quad (6) \\ &= \frac{1}{T} \mathop{\mathbb{E}}_{x\sim q(x)} \mathcal{L}(x) - H(q) + \text{const.} \end{aligned}$$

Hence, the KL divergence is the sum of the expected value of the style loss $\mathcal{L}$ and the negative entropy of the generated distribution q.

The first term can be estimated by taking the expectation over generated samples:

$$\mathop{\mathbb{E}}_{x\sim q(x)} \mathcal{L}(x) = \mathop{\mathbb{E}}_{z\sim\mathcal{N}(0,I)} \mathcal{L}(g(z)). \quad (7)$$

This is similar to the reparametrization trick of [11] and is also used in [8, 19] to construct their learning objectives.

The second term, the negative entropy, is harder to estimate accurately, but simple estimators exist. One which is particularly appealing in our scenario is the Kozachenko-Leonenko estimator [12]. This estimator considers a *batch* of N samples $x_1, \dots, x_n \sim q(x)$. Then, for each sample x_i, it computes the distance ρ_i to its nearest neighbour in the batch:

$$\rho_i = \min_{j\neq i} \|x_i - x_j\|. \quad (8)$$

The distances ρ_i can be used to approximate the entropy as follows:

$$H(q) \approx \frac{D}{N} \sum_{i=1}^{N} \ln \rho_i + \text{const.} \quad (9)$$

where $D = 3WH$ is the number of components of the images $x \in \mathbb{R}^{3\times W\times H}$.

An energy term similar to (6) was recently proposed in [10] for improving the diversity of a generator network in a adversarial learning scheme. While the idea is superficially similar, the application (sampling the Julesz ensemble) and instantiation (the way the entropy term is implemented) are very different.

Learning objective. We are now ready to define an objective function $E(g)$ to learn the generator network g. This is given by substituting the expected loss (7) and the entropy estimator (9), computed over a batch of N generated images, in the KL divergence (6):

$$\begin{aligned} E(g) = \frac{1}{N} \sum_{i=1}^{N} \Big[&\frac{1}{T}\mathcal{L}(g(z_i)) \\ &- \lambda \ln \min_{j\neq i} \|g(z_i) - g(z_j)\| \Big] \end{aligned} \quad (10)$$

The batch itself is obtained by drawing N samples $z_1, \dots, z_n \sim \mathcal{N}(0, I)$ from the noise distribution of the generator. The first term in eq. (10) measures how closely the generated images $g(z_i)$ are to the Julesz ensemble. The second term quantifies the lack of diversity in the batch by mutually comparing the generated images.

Learning. The loss function (10) is in a form that allows optimization by means of Stochastic Gradient Descent (SGD). The algorithm samples a batch $z_1, \dots, z_n$ at a time and then descends the gradient:

$$\begin{aligned} \frac{1}{N} \sum_{i=1}^{N} \Big[&\frac{d\mathcal{L}}{dx^\top} \frac{dg(z_i)}{d\theta^\top} \\ &- \frac{\lambda}{\rho_i} (g(z_i) - g(z_{j_i^*}))^\top \left(\frac{dg(z_i)}{d\theta^\top} - \frac{dg(z_{j_i^*})}{d\theta^\top} \right) \Big] \end{aligned} \quad (11)$$

where θ is the vector of parameters of the neural network g, the tensor image x has been implicitly vectorized and j_i^* is the index of the nearest neighbour of image i in the batch.

4. Stylization with instance normalization

The work of [19] showed that it is possible to learn high-quality texture networks $g(z)$ that generate images in a Julesz ensemble. They also showed that it is possible to learn good quality stylization networks $g(x_0, z)$ that apply the style of a fixed texture to an arbitrary content image x_0.

Nevertheless, the stylization problem was found to be harder than the texture generation one. For the stylization task, they found that learning the model from too many example content images x_0, say more than 16, yielded poorer *qualitative* results than using a smaller number of such examples. Some of the most significant errors appeared along

the border of the generated images, probably due to padding and other boundary effects in the generator network. We conjectured that these are symptoms of a learning problem too difficult for their choice of neural network architecture.

A simple observation that may make learning simpler is that the result of stylization should not, in general, depend on the contrast of the content image but rather should match the contrast of the texture that is being applied to it. Thus, the generator network should discard contrast information in the content image x_0. We argue that learning to discard contrast information by using standard CNN building block is unnecessarily difficult, and is best done by adding a suitable layer to the architecture.

To see why, let $x \in \mathbb{R}^{N\times C\times W\times H}$ be an input tensor containing a batch of N images. Let x_{nijk} denote its $nijk$-th element, where k and j span spatial dimensions, i is the feature channel (i.e. the color channel if the tensor is an RGB image), and n is the index of the image in the batch. Then, contrast normalization is given by:

$$\begin{aligned} y_{nijk} &= \frac{x_{nijk} - \mu_{ni}}{\sqrt{\sigma_{ni}^2 + \epsilon}}, \\ \mu_{ni} &= \frac{1}{HW}\sum_{l=1}^{W}\sum_{m=1}^{H} x_{nilm}, \\ \sigma_{ni}^2 &= \frac{1}{HW}\sum_{l=1}^{W}\sum_{m=1}^{H}(x_{nilm} - \mu_{ni})^2. \end{aligned} \tag{12}$$

It is unclear how such as function could be implemented as a sequence of standard operators such as ReLU and convolution.

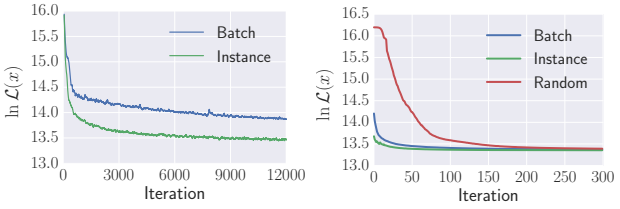
On the other hand, the generator network of [19] does contain a normalization layers, and precisely *batch normalization* (BN) ones. The key difference between eq. (12) and batch normalization is that the latter applies the normalization to a whole batch of images instead of single ones:

$$\begin{aligned} y_{nijk} &= \frac{x_{nijk} - \mu_{i}}{\sqrt{\sigma_{i}^2 + \epsilon}}, \\ \mu_{i} &= \frac{1}{HWN}\sum_{n=1}^{N}\sum_{l=1}^{W}\sum_{m=1}^{H} x_{nilm}, \\ \sigma_{i}^2 &= \frac{1}{HWN}\sum_{n=1}^{N}\sum_{l=1}^{W}\sum_{m=1}^{H}(x_{nilm} - \mu_{i})^2. \end{aligned} \tag{13}$$

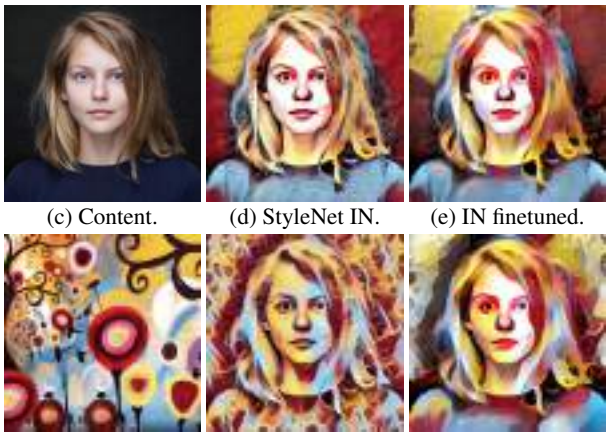
We argue that, for the purpose of stylization, the normalization operator of eq. (12) is preferable as it can normalize each individual content image x_0.

While some authors call layer eq. (12) contrast normalization, here we refer to it as *instance normalization* (IN) since we use it as a drop-in replacement for batch normalization operating on individual instances instead of the batch as a whole. Note in particular that this means that instance normalization is applied throughout the architecture, not just at the input image—fig. 2 shows the benefit of doing so.

Figure 2: Comparison of normalization techniques in image stylization. From left to right: BN, cross-channel LRN at the first layer, IN at the first layer, IN throughout.



(a) Feed-forward history. (b) Finetuning history.



(c) Content. (d) StyleNet IN. (e) IN finetuned.

(f) Style. (g) StyleNet BN. (h) BN finetuned.

Figure 3: (a) learning objective as a function of SGD iterations for StyleNet IN and BN. (b) Direct optimization of the Gatys *et al*. for this example image starting from the result of StyleNet IN and BN. (d,g) Result of StyleNet with instance (d) and batch normalization (g). (e,h) Result of finetuing the Gatys *et al*. energy.

Another similarity with BN is that each IN layer is followed by a scaling and bias operator $s \odot \mathbf{x} + b$. A difference is that the IN layer is applied at test time as well, unchanged, whereas BN is usually switched to use accumulated mean and variance instead of computing them over the batch.

IN appears to be similar to the layer normalization method introduced in [1] for recurrent networks, although it is not clear how they handle spatial data. Like theirs, IN is a generic layer, so we tested it in classification problems as well. In such cases, it still work surprisingly well, but not as well as batch normalization (e.g. AlexNet [13] IN has 2-

3% worse top-1 accuracy on ILSVRC [18] than AlexNet BN).

5. Experiments

In this section, after discussing the technical details of the method, we evaluate our new texture network architectures using instance normalization, and then investigate the ability of the new formulation to learn diverse generators.

5.1. Technical details

Network architecture. Among two generator network architectures, proposed previously in [19, 8], we choose the residual architecture from [8] for all our style transfer experiments. We also experimented with architecture from [19] and observed a similar improvement with our method, but use the one from [8] for convenience. We call it *StyleNet* with a postfix BN if it is equipped with batch normalization or IN for instance normalization.

For texture synthesis we compare two architectures: the multi-scale fully-convolutional architecture from [19] (TextureNetV1) and the one we design to have a very large receptive field (TextureNetV2). TextureNetV2 takes a noise vector of size 256 and first transforms it with two fully-connected layers. The output is then reshaped to a 4×4 image and repeatedly upsampled with fractionally-strided convolutions similar to [17]. More details can be found in the supplementary material.

Weight parameters. In practice, for the case of $\lambda > 0$, entropy loss and texture loss in eq. (10) should be weighted properly. As only the value of $T\lambda$ is important for optimization we assume $\lambda = 1$ and choose T from the set of three values $(5, 10, 20)$ for texture synthesis (we pick the higher value among those not leading to artifacts – see our discussion below). We fix $T = 10000$ for style transfer experiments. For texture synthesis, similarly to [19], we found useful to normalize gradient of the texture loss as it passes back through the VGG-19 network. This allows rapid convergence for stochastic optimization but implicitly alters the objective function and requires temperature to be adjusted. We observe that for textures with flat lightning high entropy weight results in brightness variations over the image fig. 7. We hypothesize this issue can be solved if either more clever distance for entropy estimation is used or an image prior introduced.

5.2. Effect of instance normalization

In order to evaluate the impact of replacing batch normalization with instance normalization, we consider first the problem of *stylization*, where the goal is to learn a generator $x = g(x_0, z)$ that applies a certain texture style to the content image x_0 using noise z as "random seed". We set $\lambda = 0$ for which generator is most likely to discard the noise.

The StyleNet IN and StyleNet BN are compared in fig. 3. Panel fig. 3.a shows the training objective (5) of the networks as a function of the SGD training iteration. The objective function is the same, but StyleNet IN converges much faster, suggesting that it can solve the stylization problem more easily. This is confirmed by the stark difference in the qualitative results in panels (d) end (g). Since the StyleNets are trained to minimize in one shot the same objective as the iterative optimization of Gatys *et al*., they can be used to *initialize* the latter algorithm. Panel (b) shows the result of applying the Gatys *et al*. optimization starting from their random initialization and the output of the two StyleNets. Clearly both networks start much closer to an optimum than random noise, and IN closer than BN. The difference is qualitatively large: panels (e) and (h) show the change in the StyleNets output after finetuning by iterative optimization of the loss, which has a small effect for the IN variant, and a much larger one for the BN one.

Similar results apply in general. Other examples are shown in fig. 4, where the IN variant is far superior to BN and much closer to the results obtained by the much slower iterative method of Gatys *et al*. StyleNets are trained on images of a fixed sized, but since they are convolutional, they can be applied to arbitrary sizes. In the figure, the top tree images are processed at 512×512 resolution and the bottom two at 1024×1024. In general, we found that higher resolution images yield visually better stylization results.

While instance normalization works much better than batch normalization for stylization, for texture synthesis the two normalization methods perform equally well. This is consistent with our intuition that IN helps in normalizing the information coming from content image x_0, which is highly variable, whereas it is not important to normalize the texture information, as each model learns only one texture style.

5.3. Effect of the diversity term

Having validated the IN-based architecture, we evaluate now the effect of the entropy-based diversity term in the objective function (10).

The experiment in fig. 5 starts by considering the problem of texture generation. We compare the new high-capacity TextureNetV2 and the low-capacity TextureNetsV1 texture synthesis networks. The low-capacity model is the same as [19]. This network was used there in order to force the network to learn a non-trivial dependency on the input noise, thus generating diverse outputs even though the learning objective of [19], which is the same as eq. (10) with diversity coefficient $\lambda = 0$, tends to suppress diversity. The results in fig. 5 are indeed diverse, but sometimes of low quality. This should be contrasted

Content | StyleNet IN (ours) | StyleNet BN | Gatys *et al.* | Style

Figure 4: Stylization results obtained by applying different textures (rightmost column) to different content images (leftmost column). Three methods are compared: StyleNet IN, StyleNet BN, and iterative optimization.

with TextureNetV2, the high-capacity model: its visual fidelity is much higher, but, by using the same objective function [19], the network learns to generate a single image, as expected. TextureNetV2 with the new diversity-inducing objective ($\lambda > 0$) is the best of both worlds, being both high-quality and diverse.

The experiment in fig. 6 assesses the effect of the diversity term in the stylization problem. The results are similar to the ones for texture synthesis and the diversity term effectively encourages the network to learn to produce different results based on the input noise.

One difficultly with texture and stylization networks is that the entropy loss weight λ must be tuned for each learned texture model. Choosing λ too small may fail to learn a diverse generator, and setting it too high may create artifacts, as shown in fig. 7.

6. Summary

This paper advances feed-forward texture synthesis and stylization networks in two significant ways. It introduces instance normalization, an architectural change that makes training stylization networks easier and allows the training process to achieve much lower loss levels. It also introduces a new learning formulation for training generator networks

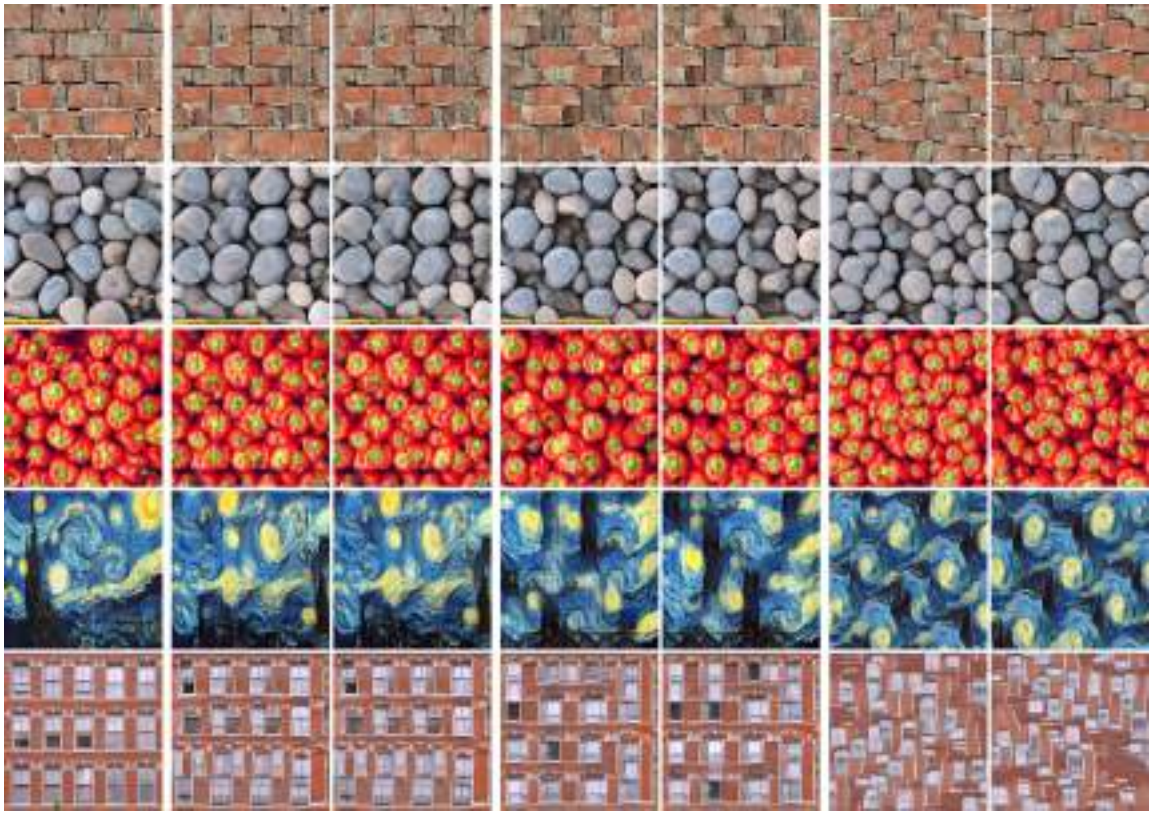


Figure 5: The textures generated by the high capacity Texture Net V2 without diversity term ($\lambda = 0$ in eq. (10)) are nearly identical. The low capacity TextureNet V1 of [19] achieves diversity, but has sometimes poor results. TextureNet V2 with diversity is the best of both worlds.

Figure 6: The StyleNetV2 $g(x_0, z)$, trained with diversity $\lambda > 0$, generates substantially different stylizations for different values of the input noise z. In this case, the lack of stylization diversity is visible in uniform regions such as the sky.

to sample uniformly from the Julesz ensemble, thus explicitly encouraging diversity in the generated outputs. We show that both improvements lead to noticeable improvements of the generated stylized images and textures, while keeping the generation run-times intact.

References

[1] L. J. Ba, R. Kiros, and G. E. Hinton. Layer normalization. *CoRR*, abs/1607.06450, 2016. 5

[2] E. L. Denton, S. Chintala, A. Szlam, and R. Fergus. Deep generative image models using a laplacian pyramid of adversarial networks. In *NIPS*, pages 1486–1494, 2015. 3

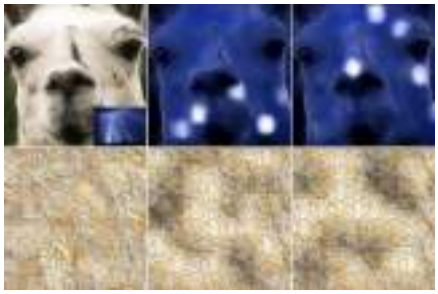


Figure 7: Negative examples. If the diversity term λ is too high for the learned style, the generator tends to generate artifacts in which brightness is changed locally (spotting) instead of (or as well as) changing the structure.

[3] G. K. Dziugaite, D. M. Roy, and Z. Ghahramani. Training generative neural networks via maximum mean discrepancy optimization. In *UAI*, pages 258–267. AUAI Press, 2015. 3

[4] L. Gatys, A. S. Ecker, and M. Bethge. Texture synthesis using convolutional neural networks. In *Advances in Neural Information Processing Systems, NIPS*, pages 262–270, 2015. 1, 2, 3

[5] L. A. Gatys, A. S. Ecker, and M. Bethge. A neural algorithm of artistic style. *CoRR*, abs/1508.06576, 2015. 1, 2, 3

[6] I. J. Goodfellow, J. Pouget-Abadie, M. Mirza, B. Xu, D. Warde-Farley, S. Ozair, A. C. Courville, and Y. Bengio. Generative adversarial nets. In *Advances in Neural Information Processing Systems (NIPS)*, pages 2672–2680, 2014. 3, 4

[7] A. Gretton, K. M. Borgwardt, M. Rasch, B. Schölkopf, and A. J. Smola. A kernel method for the two-sample-problem. In *Advances in neural information processing systems,NIPS*, pages 513–520, 2006. 3

[8] J. Johnson, A. Alahi, and L. Fei-Fei. Perceptual losses for real-time style transfer and super-resolution. In *Computer Vision - ECCV 2016 - 14th European Conference, Amsterdam, The Netherlands, October 11-14, 2016, Proceedings, Part II*, pages 694–711, 2016. 1, 2, 3, 4, 6

[9] B. Julesz. Textons, the elements of texture perception, and their interactions. *Nature*, 290(5802):91–97, 1981. 2

[10] T. Kim and Y. Bengio. Deep directed generative models with energy-based probability estimation. *arXiv preprint arXiv:1606.03439*, 2016. 4

[11] D. P. Kingma and M. Welling. Auto-encoding variational bayes. *CoRR*, abs/1312.6114, 2013. 4

[12] L. F. Kozachenko and N. N. Leonenko. Sample estimate of the entropy of a random vector. *Probl. Inf. Transm.*, 23(1-2):95–101, 1987. 2, 4

[13] A. Krizhevsky, I. Sutskever, and G. E. Hinton. Imagenet classification with deep convolutional neural networks. In *NIPS*, pages 1106–1114, 2012. 5

[14] Y. Li, K. Swersky, and R. S. Zemel. Generative moment matching networks. In *Proc. International Conference on Machine Learning, ICML*, pages 1718–1727, 2015. 3

[15] J. Portilla and E. P. Simoncelli. A parametric texture model based on joint statistics of complex wavelet coefficients. *IJCV*, 40(1):49–70, 2000. 1

[16] J. Portilla and E. P. Simoncelli. A parametric texture model based on joint statistics of complex wavelet coefficients. *IJCV*, 2000. 2, 3

[17] A. Radford, L. Metz, and S. Chintala. Unsupervised representation learning with deep convolutional generative adversarial networks. *CoRR*, abs/1511.06434, 2015. 3, 6

[18] O. Russakovsky, J. Deng, H. Su, J. Krause, S. Satheesh, S. Ma, Z. Huang, A. Karpathy, A. Khosla, M. Bernstein, A. C. Berg, and L. Fei-Fei. ImageNet Large Scale Visual Recognition Challenge. *International Journal of Computer Vision (IJCV)*, 115(3):211–252, 2015. 5

[19] D. Ulyanov, V. Lebedev, A. Vedaldi, and V. S. Lempitsky. Texture networks: Feed-forward synthesis of textures and stylized images. In *Proceedings of the 33nd International Conference on Machine Learning, ICML 2016, New York City, NY, USA, June 19-24, 2016*, pages 1349–1357, 2016. 1, 2, 3, 4, 5, 6, 8

[20] S. C. Zhu, X. W. Liu, and Y. N. Wu. Exploring texture ensembles by efficient markov chain monte carlotoward a atrichromacyo theory of texture. *PAMI*, 2000. 2, 3

[21] S. C. Zhu, Y. Wu, and D. Mumford. Filters, random fields and maximum entropy (FRAME): Towards a unified theory for texture modeling. *IJCV*, 27(2), 1998. 3

ELSEVIER

Journal of Statistical Planning and Inference 90 (2000) 227–244

journal of statistical planning and inference

www.elsevier.com/locate/jspi

Improving predictive inference under covariate shift by weighting the log-likelihood function

Hidetoshi Shimodaira *

The Institute of Statistical Mathematics, 4-6-7 Minami-Azabu, Minato-ku, Tokyo 106-8569, Japan

Received 17 December 1998; received in revised form 21 January; accepted 25 February 2000

Abstract

A class of predictive densities is derived by weighting the observed samples in maximizing the log-likelihood function. This approach is effective in cases such as sample surveys or design of experiments, where the observed covariate follows a different distribution than that in the whole population. Under misspecification of the parametric model, the optimal choice of the weight function is asymptotically shown to be the ratio of the density function of the covariate in the population to that in the observations. This is the pseudo-maximum likelihood estimation of sample surveys. The optimality is defined by the expected Kullback–Leibler loss, and the optimal weight is obtained by considering the importance sampling identity. Under correct specification of the model, however, the ordinary maximum likelihood estimate (i.e. the uniform weight) is shown to be optimal asymptotically. For moderate sample size, the situation is in between the two extreme cases, and the weight function is selected by minimizing a variant of the information criterion derived as an estimate of the expected loss. The method is also applied to a weighted version of the Bayesian predictive density. Numerical examples as well as Monte-Carlo simulations are shown for polynomial regression. A connection with the robust parametric estimation is discussed.

MSC: 62B10; 62D05

Keywords: Akaike information criterion; Design of experiments; Importance sampling; Kullback–Leibler divergence; Misspecification; Sample surveys; Weighted least squares

1. Introduction

Let x be the explanatory variable or the covariate, and y be the response variable. In predictive inference with the regression analysis, we are interested in estimating the conditional density $q(y|x)$ of y given x, using a parametric model. Let $p(y|x,\theta)$ be the model of the conditional density which is parameterized by $\theta=(\theta^1,\ldots,\theta^m)' \in \Theta \subset \mathbb{R}^m$.

* Correspondence address: Department of Statistics, Sequoia Hall, 390 Serra Mall, Stanford University, Stanford, CA 94305-4065, USA.

E-mail address: shimo@ism.ac.jp (H. Shimodaira).

0378-3758/00/$ - see front matter
PII: S0378-3758(00)00115-4

Having observed i.i.d. samples of size n, denoted by $(x^{(n)}, y^{(n)}) = ((x_t, y_t): t = 1, \ldots, n)$, we obtain a predictive density $p(y|x, \hat{\theta})$ by giving an estimate $\hat{\theta} = \hat{\theta}(x^{(n)}, y^{(n)})$. In this paper, we discuss improvement of the maximum likelihood estimate (MLE) under both (i) *covariate shift* in distribution and (ii) *misspecification* of the model as explained below.

Let $q_1(x)$ be the density of x for evaluation of the predictive performance, while $q_0(x)$ be the density of x in the observed data. We consider the Kullback–Leibler loss function

$$\mathrm{loss}_i(\theta) := -\int q_i(x) \int q(y|x) \log p(y|x, \theta)\, \mathrm{d}y\, \mathrm{d}x$$

for $i = 0, 1$, and then employ $\mathrm{loss}_1(\hat{\theta})$ for evaluation of $\hat{\theta}$, rather than the usual $\mathrm{loss}_0(\hat{\theta})$. The situation $q_0(x) \neq q_1(x)$ will be called covariate shift in distribution, which is one of the premises of this paper.

This situation is not so odd as it might look at first. In fact, it is seen in various fields as follows. In sample surveys, $q_0(x)$ is determined by the sampling scheme, while $q_1(x)$ is determined by the population. In regression analysis, covariate shift often happens because of the limitation of resources, or the design of experiments. In artificial neural networks literature, "active learning" is the typical situation where we control $q_0(x)$ for better prediction. We could say that the distribution of x in future observations is different from that of the past observations; x is not necessarily distributed as $q_1(x)$ in future, but we can give imaginary $q_1(x)$ to specify the region of x where the prediction accuracy should be controlled. Note that $q_0(x)$ and/or $q_1(x)$ are often estimated from data, but we assume they are known or estimated reasonably in advance.

The second premise of this paper is misspecification of the model. Let $\hat{\theta}_0$ be the MLE of θ, and θ_0^* be the asymptotic limit of $\hat{\theta}_0$ as $n \to \infty$. Under certain regularity conditions, MLE is consistent and $p(y|x, \theta_0^*) = q(y|x)$ provided that the model is correctly specified. In practice, however, $p(y|x, \theta_0^*)$ deviates more or less from $q(y|x)$.

Under both the covariate shift and the misspecification, MLE does not necessarily provide a good inference. We will show that MLE is improved by giving a weight function $w(x)$ of the covariate in the log-likelihood function

$$L_w(\theta|x^{(n)}, y^{(n)}) := -\sum_{t=1}^{n} l_w(x_t, y_t|\theta), \tag{1.1}$$

where $l_w(x, y|\theta) = -w(x) \log p(y|x, \theta)$. Then the maximum weighted log-likelihood estimate (MWLE), denoted by $\hat{\theta}_w$, is obtained by maximizing (1.1) over Θ. It will be seen that the weight function $w(x) = q_1(x)/q_0(x)$ is the optimal choice for sufficiently large n in terms of the expected loss with respect to $q_1(x)$. We denote MWLE with this weight function by $\hat{\theta}_1$. A comparison between $\hat{\theta}_0$ and $\hat{\theta}_1$ is made in the numerical example of polynomial regression of Section 2, and the asymptotic optimality of $\hat{\theta}_1$ is shown in Section 3. Note that MWLE turns out to be downweighting the observed samples which are not important in fitting the model with respect to the population. An interpretation of MWLE as one of the robust estimation techniques is given in Section 9.

This type of estimation is not new in statistics. Actually, $\hat{\theta}_1$ is regarded as a generalization of the pseudo-maximum likelihood estimation in sample surveys (Skinner et al., 1989, p. 80; Pfeffermann et al., 1998); the log likelihood is weighted inversely proportional to $q_0(x)$, the probability of selecting unit x, while $q_1(x)$ is equal probability for all possible values of x. The same idea is also seen in Rao (1991), where weighted maximum likelihood estimation is considered for unequally spaced time-series data.

The local likelihoods or the weighted likelihoods formally similar to (1.1) are found in the literature for semi-parametric inference. However, $\hat{\theta}_w$ is estimated using a weight function concentrated locally around each x or (x, y) in the semi-parametric approach; thus $\hat{\theta}_w$ in $p(y|x, \hat{\theta}_w)$ will depend on (x, y) as well as the data $(x^{(n)}, y^{(n)})$. On the other hand, we restrict our attention to a rather conventional parametric modeling approach here, and $\hat{\theta}_w$ depends only on the data.

In spite of the asymptotic optimality of $w(x)=q_1(x)/q_0(x)$ mentioned above, another choice of the weight function can improve the expected loss for moderate sample size by compromising the bias and the variance of $\hat{\theta}_w$. We develop a practical method for this improvement in Sections 4–7. The asymptotic expansion of the expected loss is given in Section 4, and a variant of the information criterion is derived as an estimate of the expected loss in Section 5. This new criterion is used to find a good $w(x)$ as well as a good form of $p(y|x, \theta)$. The numerical example is revisited in Section 6, and a simulation study is given in Section 7.

In Section 8, we show the Bayesian predictive density is also improved by considering the weight function. Finally, concluding remarks are given in Section 9. All the proofs are deferred to the appendix.

2. Illustrative example in regression

Here we consider the normal regression to predict the response $y \in \mathscr{R}$ using a polynomial function of $x \in \mathscr{R}$. Let the model $p(y|x, \theta)$ be the polynomial regression

$$y = \beta_0 + \beta_1 x + \cdots + \beta_d x^d + \varepsilon, \quad \varepsilon \sim \mathrm{N}(0, \sigma^2), \tag{2.1}$$

where $\theta = (\beta_0, \ldots, \beta_d, \sigma)$ and $\mathrm{N}(a, b)$ denotes the normal distribution with mean a and variance b. In the numerical example below, we assume the true $q(y|x)$ is also given by (2.1) with $d = 3$:

$$y = -x + x^3 + \varepsilon, \quad \varepsilon \sim \mathrm{N}(0, 0.3^2). \tag{2.2}$$

The density $q_0(x)$ of the covariate x is

$$x \sim \mathrm{N}(\mu_0, \tau_0^2), \tag{2.3}$$

where $\mu_0 = 0.5$, $\tau_0^2 = 0.5^2$. This corresponds to the sampling scheme of x or the design of experiments. A dataset $(x^{(n)}, y^{(n)})$ of size $n = 100$ is generated from (2.2) and (2.3), and plotted by circles in Fig. 1a. MLE $\hat{\theta}_0$ is obtained by the ordinary least squares (OLS) for the normal regression; we consider a model of the form (2.1) with $d = 1$, and the regression line fitted by OLS is drawn in solid line in Fig. 1a.

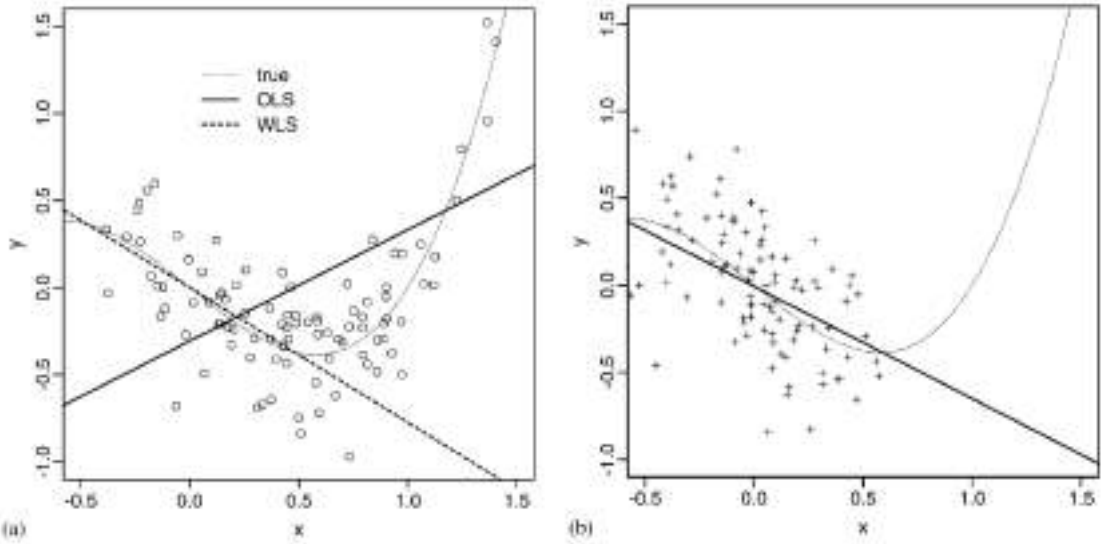


Fig. 1. Fitting of polynomial regression with degree $d = 1$. (a) Samples (x_i, y_i) of size $n = 100$ are generated from $q(y|x)q_0(x)$ and plotted as circles, where the underlying true curve is indicated by the thin dotted line. The solid line is obtained by OLS, and the dotted line is WLS with weight $q_1(x)/q_0(x)$. (b) Samples of $n = 100$ are generated from $q(y|x)q_1(x)$, and the regression line is obtained by OLS.

On the other hand, MWLE $\hat{\theta}_w$ is obtained by weighted least squares (WLS) with weights $w(x_i)$ for the normal regression. We again consider the model with $d = 1$, and the regression line fitted by WLS with $w(x) = q_1(x)/q_0(x)$ is drawn in dotted line in Fig. 1a. Here, the density $q_1(x)$ for imaginary "future" observations or that for the whole population in sample surveys is specified in advance by

$$x \sim \mathrm{N}(\mu_1, \tau_1^2), \tag{2.4}$$

where $\mu_1 = 0.0$, $\tau_1^2 = 0.3^2$. The ratio of $q_1(x)$ to $q_0(x)$ is

$$\frac{q_1(x)}{q_0(x)} = \frac{\exp(-(x-\mu_1)^2/2\tau_1^2)/\tau_1}{\exp(-(x-\mu_0)^2/2\tau_0^2)/\tau_0} \propto \exp\left(-\frac{(x-\bar{\mu})^2}{2\bar{\tau}^2}\right), \tag{2.5}$$

where $\bar{\tau}^2 = (\tau_1^{-2} - \tau_0^{-2})^{-1} = 0.38^2$, and $\bar{\mu} = \bar{\tau}^2(\tau_1^{-2}\mu_1 - \tau_0^{-2}\mu_0) = -0.28$.

The obtained lines in Fig. 1a are very different for OLS and WLS. The question is: which is better than the other? It is known that OLS is the best linear unbiased estimate and makes small mean squared error of prediction in terms of $q(y|x)q_0(x)$ which generated the data. On the other hand, WLS with weight (2.5) makes small prediction error in terms of $q(y|x)q_1(x)$ which will generate future observations, and thus WLS is better than OLS here. To confirm this, a dataset of size $n = 100$ is generated from $q(y|x)q_1(x)$ specified by (2.2) and (2.4). The regression line of $d = 1$ fitted by OLS is shown in Fig. 1b, which is considered to have small prediction error for the "future" data. The regression line of WLS fitted to the past data in Fig. 1a is quite similar to the line of OLS fitted to the future data in Fig. 1b. In practice, only the past data is available. The WLS gave almost the equivalent result to the future OLS by using only the past data.

The underlying true curve is the polynomial with $d = 3$, and thus the regression line of $d = 1$ cannot be fitted to it nicely over all the region of x. However, the true curve is almost linear in the region of $\mu_1 \pm 2\tau_1$, and the nice fit of the WLS in this region is obtained by throwing away the observed samples which are outside of this region. The "effective sample size" may be defined in terms of the entropy by $n_e = \exp(-\sum_{t=1}^{n} p_t \log p_t)$, where $p_t = w(x_t)/\sum_{t'=1}^{n} w(x_{t'})$. In the WLS above, $n_e = 49.3$, which is about the half of the original sample size $n = 100$, and then increases the variance of the WLS. This is discussed later in detail.

3. Asymptotic properties of MWLE

Let $E_i(\cdot)$ denote the expectation with respect to $q(y|x)q_i(x)$ for $i = 0, 1$. Considering $-L_w(\theta)$ as the summation of i.i.d. random variables $l_w(x_t, y_t|\theta)$, it follows from the law of large numbers that $-L_w(\theta)/n \to E_0(l_w(x, y|\theta))$ as n grows to infinity. Then we have $\hat{\theta}_w \to \theta_w^*$ in probability as $n \to \infty$, where θ_w^* is the minimizer of $E_0(l_w(x, y|\theta))$ over $\theta \in \Theta$. Hereafter, we restrict our attention to proper $w(x)$ such that $E_0(l_w(x, y|\theta))$ exists for all $\theta \in \Theta$ and that the Hessian of $E_0(l_w(x, y|\theta))$ is non-singular at θ_w^*, which is uniquely determined and interior to Θ.

If the above result is applied to $w(x) = q_1(x)/q_0(x)$, we find that $\hat{\theta}_1$ converges in probability to the minimizer of $\mathrm{loss}_1(\theta)$ over $\theta \in \Theta$, which we denote θ_1^*. Here the key idea is the importance sampling identity:

$$E_0\left\{\frac{q_1(x)}{q_0(x)}\log p(y|x,\theta)\right\} = \int q(y|x)q_0(x)\frac{q_1(x)}{q_0(x)}\log p(y|x,\theta)\,\mathrm{d}x\,\mathrm{d}y$$

$$= E_1(\log p(y|x,\theta)), \tag{3.1}$$

which implies $E_0(l_w(x, y|\theta)) \equiv \mathrm{loss}_1(\theta)$ and $\theta_w^* = \theta_1^*$ when $w(x) = q_1(x)/q_0(x)$.

Except for the equivalent weight $w(x) \propto q_1(x)/q_0(x)$, we have $\theta_w^* \neq \theta_1^*$ under misspecification in general. From the definition of θ_1^*, therefore, $\mathrm{loss}_1(\theta_w^*) > \mathrm{loss}_1(\theta_1^*)$. This immediately implies the asymptotic optimality of the weight $w(x) = q_1(x)/q_0(x)$, because $\hat{\theta}_w \to \theta_w^*$ and $\hat{\theta}_1 \to \theta_1^*$ and thus $\mathrm{loss}_1(\hat{\theta}_w) > \mathrm{loss}_1(\hat{\theta}_1)$ for sufficiently large n.

$\hat{\theta}_1$ has consistency in a sense that it converges to the optimal parameter value. However, $\hat{\theta}_0$ is more efficient than $\hat{\theta}_1$ in terms of the asymptotic variance. This will be important for moderate sample size, where n is large enough for the asymptotic expansions to be allowed, but not enough for the optimality of $\hat{\theta}_1$ to hold. The following lemma, which is used in the subsequent sections, gives the asymptotic distribution of $\hat{\theta}_w$. The derivation is parallel to that of MLE under misspecification given in White (1982); we replace $\log p(y|x,\theta)q_0(x)$ of MLE with $w(x)\log p(y|x,\theta)q_0(x)$ of MWLE.

Lemma 1. *Assume the regularity conditions similar to those of White* (1982), *i.e., the model is sufficiently smooth and the support of* $p(y|x,\theta)$ *is the same as that of* $q(y|x)$ *for all* $\theta \in \Theta$. *Also assume* θ_w^* *is an interior point of* Θ. *Then,* $n^{1/2}(\hat{\theta}_w - \theta_w^*)$

For sufficiently large n, $\mathrm{loss}_1(\theta_w^*)$ is the dominant term on the right-hand side of (4.1), and the optimal weight is $w(x) = q_1(x)/q_0(x)$ as seen in Section 3. If n is not large enough compared with the extent of the misspecification, the $O(n^{-1})$ terms related to the first and second moments of $\hat{\theta}_w - \theta_w^*$ cannot be ignored in (4.1), and the optimal weight changes. In an extreme case where the model is correctly specified, we only have to look at the $O(n^{-1})$ terms as shown in the following lemma.

Lemma 4. *Assume there exists $\theta^* \in \Theta$ such that $q(y|x) = p(y|x, \theta^*)$. Then, $\theta_w^* = \theta^*$ and $q(y|x) = p(y|x, \theta_w^*)$ for all proper $w(x)$. The expected loss $E_0^{(n)}(\mathrm{loss}_1(\hat{\theta}_w))$ is minimized when $w(x) \equiv 1$ for sufficiently large n.*

5. Information criterion

The performance of MWLE for a specified $w(x)$ is given by (4.1). However, we cannot calculate the value of the expected loss from it in practice, because $q(y|x)$ is unknown. We provide a variant of the information criterion as an estimate of (4.1).

Theorem 1. *Let the information criterion for MWLE be*

$$\mathrm{IC}_w := -2L_1(\hat{\theta}_w) + 2\,\mathrm{tr}(J_w H_w^{-1}), \tag{5.1}$$

where

$$L_1(\theta) = \sum_{t=1}^{n} \frac{q_1(x_t)}{q_0(x_t)} \log p(y_t|x_t, \theta),$$

$$J_w = -E_0\left\{ \frac{q_1(x)}{q_0(x)} \frac{\partial \log p(y|x,\theta)}{\partial \theta}\bigg|_{\theta_w^*} \frac{\partial l_w(x,y|\theta)}{\partial \theta'}\bigg|_{\theta_w^*} \right\}.$$

The matrices J_w and H_w may be replaced by their consistent estimates

$$\hat{J}_w = -\frac{1}{n}\sum_{t=1}^{n} \frac{q_1(x_t)}{q_0(x_t)} \frac{\partial \log p(y_t|x_t,\theta)}{\partial \theta}\bigg|_{\hat{\theta}_w} \frac{\partial l_w(x_t,y_t|\theta)}{\partial \theta'}\bigg|_{\hat{\theta}_w},$$

$$\hat{H}_w = \frac{1}{n}\sum_{t=1}^{n} \frac{\partial^2 l_w(x_t,y_t|\theta)}{\partial\theta\partial\theta'}\bigg|_{\hat{\theta}_w}.$$

Then, $\mathrm{IC}_w/2n$ is an estimate of the expected loss unbiased up to $O(n^{-1})$ term:

$$E_0^{(n)}(\mathrm{IC}_w/2n) = E_0^{(n)}(\mathrm{loss}_1(\hat{\theta}_w)) + o(n^{-1}). \tag{5.2}$$

Fortunately, expression (5.1) turns out to be rather simpler than that of (4.1), and we do not have to worry about the calculation of the third derivatives appeared in (4.2). The explicit form of (5.1) for the normal regression is given in (6.1) and (6.2).

Given the model $p(y|x,\theta)$ and the data $(x^{(n)}, y^{(n)})$, we choose a weight function $w(x)$ which attains the minimum of IC_w over a certain class of weights. This is selection of the weight rather than model selection. Searching the optimal weight over all the

possible forms of $w(x)$ is equivalent to n-dimensional optimization problem with respect to $(w(x_t): t=1,\ldots,n)$. But we do not take this line here, because of the computational cost as well as a conceptual difficulty which will be mentioned in Section 9. Rather than the global search, we shall pick a better one from the two extreme cases of $w(x) \equiv 1$ and $w(x)=q_1(x)/q_0(x)$, or consider a class of weights by connecting the two extremes continuously:

$$w(x) = \left(\frac{q_1(x)}{q_0(x)}\right)^{\lambda}, \quad \lambda \in [0,1], \tag{5.3}$$

where $\lambda = 0$ corresponds to $\hat{\theta}_0$ and $\lambda = 1$ corresponds to $\hat{\theta}_1$. In the next section, we numerically find $\hat{\lambda}$ which minimizes IC_w by searching over $\lambda \in [0,1]$. Note that (5.3) is proportional to $\mathrm{N}(\bar{\mu}, \bar{\tau}^2/\lambda)$ in the case of (2.5), and $\lambda^{-1/2}$ is the window scale parameter.

When we have several candidate forms of $p(y|x,\theta)$, the model and the weight are selected simultaneously by minimizing IC_w. A similar idea of the simultaneous selection is found in Shibata (1989), where an information criterion RIC is derived for the penalized likelihood. A crucial distinction, however, is that the weight for θ is selected in RIC, whereas the weight for x is selected in IC_w. Another distinction is that the weight is additive to the log likelihood in RIC, while it is multiplicative in IC_w.

Akaike (1974) gave an information criterion

$$\mathrm{AIC} = -2L_0(\hat{\theta}_0) + 2\dim\theta, \tag{5.4}$$

where $L_0(\theta)$ is the log-likelihood function. AIC is intended for MLE, and it is obtained as a special case of IC_w. When $q_1(x) = q_0(x)$ and $w(x) \equiv 1$, IC_w reduces to

$$\mathrm{TIC} = -2L_0(\hat{\theta}_0) + 2\,\mathrm{tr}(G_0 H_0^{-1}),$$

where $G_0 = G_w$ and $H_0 = H_w$ when $w(x) \equiv 1$. TIC is derived by Takeuchi (1976) as a precise version of AIC, and it is equivalent to the criterion of Linhart and Zucchini (1986). If $p(y|x,\theta_0^*)$ is sufficiently close to $q(y|x)$, $\mathrm{tr}(G_0 H_0^{-1}) \approx \dim\theta$ and TIC reduces to AIC.

6. Numerical example revisited

For the normal linear regression, such as the polynomial regression given in (2.1), β-components of $\hat{\theta}_w$ are obtained by WLS with weights $w(x_t)$. σ-component of $\hat{\theta}_w$ is then given by $\hat{\sigma}^2 = \sum_{t=1}^n w(x_t)\hat{\varepsilon}_t^2/\hat{c}_w$, where $\hat{c}_w = \sum_{t=1}^n w(x_t)$ and $\hat{\varepsilon}_t$ is the residual. Letting $\hat{h}_t$, $t=1,\ldots,n$ be the diagonal elements of the hat matrix used in the WLS, the information criterion (5.1) is calculated from

$$-L_1(\hat{\theta}_w) = \frac{1}{2}\sum_{t=1}^{n} \frac{q_1(x_t)}{q_0(x_t)} \left\{ \frac{\hat{\varepsilon}_t^2}{\hat{\sigma}^2} + \log(2\pi\hat{\sigma}^2) \right\}, \tag{6.1}$$

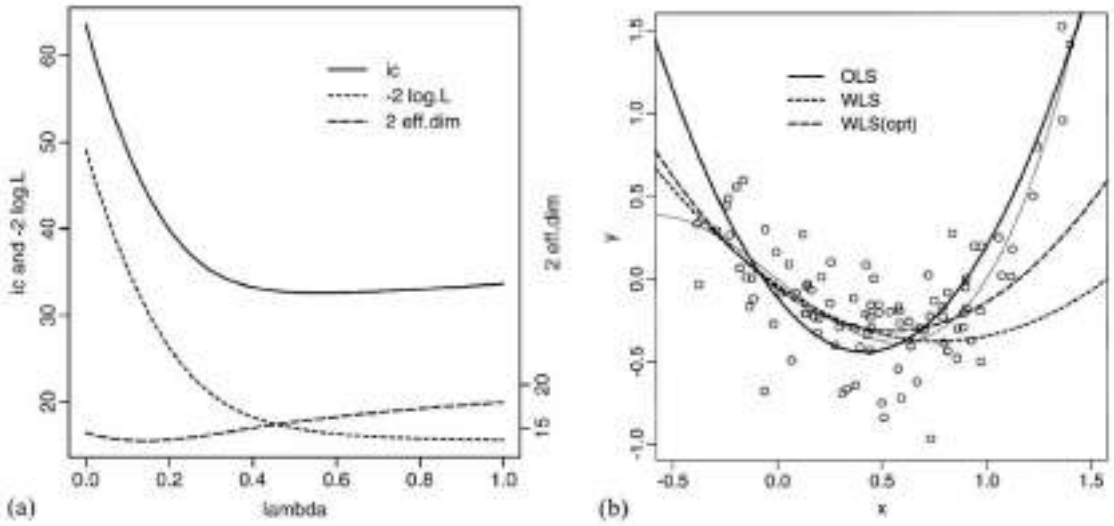


Fig. 2. (a) Curve of IC_w versus $\lambda \in [0,1]$ for the model of Section 2 with $d=2$. The weight function (5.3) connecting from $w(x) \equiv 1$ (i.e. $\lambda=0$) to $w(x)=q_1(x)/q_0(x)$ (i.e. $\lambda=1$) was used. Also shown are $-2L_1(\hat{\theta}_w)$ in dotted lines, and $2\,\mathrm{tr}(J_w H_w^{-1})$ in broken lines. (b) The regression curves for $d=2$. The WLS curve with the optimal $\hat{\lambda}$ as well as those for OLS ($\lambda=0$) and WLS ($\lambda=1$) are drawn.

Table 1
IC_w values with weight (5.3) for $\lambda=0$, $\lambda=1$, and $\lambda=\hat{\lambda}$. Also shown is $\hat{\lambda}$ value. Calculated for the polynomial regression example of Section 2 with $d=0,\ldots,4$

	$d=0$	$d=1$	$d=2$	$d=3$	$d=4$
$\lambda=0$	138.72	174.02	63.59	28.97	31.75
$\lambda=1$	73.96	33.23	33.64	34.80	34.98
$\lambda=\hat{\lambda}$	73.92	32.68	32.62	28.96	31.75
$\hat{\lambda}$	0.95	0.77	0.56	0.01	0.00

$$\mathrm{tr}(\hat{J}_w \hat{H}_w^{-1}) = \sum_{t=1}^{n} \frac{q_1(x_t)}{q_0(x_t)} \left\{ \frac{\hat{\varepsilon}_t^2}{\hat{\sigma}^2} \hat{h}_t + \frac{w(x_t)}{2\hat{c}_w} \left(\frac{\hat{\varepsilon}_t^2}{\hat{\sigma}^2} - 1 \right)^2 \right\}. \tag{6.2}$$

We apply the above formulas to the data generated from (2.2) and (2.3) in Section 2. Fig. 2a shows the plot of the information criterion and its two components for $d=2$. By increasing λ from 0 to 1, the first term of (5.1) decreases while the second term increases in general. We numerically find $\hat{\lambda}$ so that the two terms balance. For $d=2$, IC_w takes the minimum 32.62 at $\hat{\lambda}=0.56$. The regression curves obtained by this method are shown in Fig. 2b.

Table 1 shows IC_w values for $d=0,\ldots,4$. For each d, IC_w is minimized at $\lambda=\hat{\lambda}$. Then, IC_w of $\hat{\lambda}$ is minimized at the model $d=3$. By minimizing IC_w, λ and d are simultaneously selected. For $d=3$, it turns out that $\hat{\lambda}=0.01 \approx 0$. In fact, the model of $d=3$ is correctly specified in this dataset, and it follows from Lemma 4 that $\hat{\theta}_0$ is optimal for $d \geqslant 3$. Even in such a situation, the appropriate $\hat{\lambda}$ is selected by minimizing IC_w.

Table 2
Asymptotic convergence of the second term of (4.1). $2n$ times of $\mathrm{loss}_1(\hat{\theta}_w) - \mathrm{loss}_1(\theta^*_w)$ is calculated for the Monte-Carlo replicates, and its average is tabulated. For every simulation ($d = 0, 1, 2$ and $\lambda = 0, 1$), the values are showing a good convergence as $n \to \infty$

n	$d=0$		$d=1$		$d=2$	
	$\lambda=0$	$\lambda=1$	$\lambda=0$	$\lambda=1$	$\lambda=0$	$\lambda=1$
50	−3.9	5.5	−2.0	8.9	19.0	15.9
100	−5.1	4.8	−3.1	7.6	17.7	11.3
300	−7.0	4.5	−4.5	6.8	17.7	9.0
1000	−8.0	4.4	−4.9	6.6	19.3	8.5

Table 3
Asymptotic convergence of (5.2). $2n\,\mathrm{loss}_1(\hat{\theta}_w) + 2L_1(\hat{\theta}_w)$ is calculated for the Monte-Carlo replicates, and its average is tabulated in the left columns. For $n \geqslant 300$, this agrees very well with the average of $2\,\mathrm{tr}(\hat{J}_w \hat{H}_w^{-1})$ tabulated in the right columns. $2\,\mathrm{tr}(J_w H_w^{-1})$ is shown in the $n = \infty$ row

n	$d=0$				$d=1$				$d=2$			
	$\lambda=0$		$\lambda=1$		$\lambda=0$		$\lambda=1$		$\lambda=0$		$\lambda=1$	
50	1.8	1.7	9.9	7.7	6.8	6.1	15.7	11.1	26.4	13.1	24.8	13.4
100	1.5	1.4	9.2	8.1	6.0	5.7	14.2	12.0	22.6	14.9	19.8	14.9
300	1.3	1.3	8.8	8.4	5.4	5.3	13.3	12.6	18.5	15.7	17.3	16.0
1000	1.2	1.2	8.7	8.6	5.1	5.1	13.1	12.9	16.2	15.4	16.7	16.3
∞		1.2		8.6		5.0		13.0		14.9		16.4

In practical data analysis, it would be rare to have correctly specified models at hand. Therefore, we exclude $d \geqslant 3$ from the above example, and restrict the candidates to $d < 3$. Then, $d = 2$ is selected, and $d = 1$ has almost the same IC_w value, while $d = 0$ has significantly larger IC_w value. This agrees with the asymptotic result verified by extensive Monte-Carlo simulations in Shimodaira (1997) that (4.1) is minimized when $\lambda = 1$ and $d = 1$ over $d \in \{0, 1, 2\}$, for sufficiently large n.

7. Simulation study

First we show simulation results in Tables 1–3 which confirm the theory of Sections 4 and 5. A large number N of replicates of the dataset of size n are generated from (2.2) and (2.3). We used (2.4) as $q_1(x)$. Four simulations of $n = 50$, 100, 300, and 1000 are done with $N = 10^5$ for $n = 50$–300 and $N = 10^6$ for $n = 1000$. For each replicate of the dataset, $\hat{\theta}_w$ is calculated for $\lambda = 0, 1$ and $d = 0, 1, 2$. Then, $\mathrm{loss}_1(\hat{\theta}_w)$, $L_1(\hat{\theta}_w)$, and $\mathrm{tr}(\hat{J}_w \hat{H}_w^{-1})$ are calculated, and their averages over the N replicates are obtained for each simulation. The tables show nice agreement between the simulations and the theory.

Next, we show the results of another set of simulations in Table 4 which confirms the practical performance of the weight selection procedure. We used (2.3) and (2.4) as before, but (2.2) is replaced by $y = -x + \beta_3 x^3 + \varepsilon$ for generating replicates of the

Table 4
The expected loss for the selected weight and the selected model. $2n$ times of $\mathrm{loss}_1(\hat{\theta}_w) + E_1(\log q(y|x))$ is calculated for the replicates, and its average is tabulated in the columns of $\lambda = 0, 1$ for $d = 0, 1, 2$. The average of the loss of the selected weight is shown in the columns of $\hat{\lambda}$. The right most columns show the average of the loss of the selected model

β_3	$d=0$			$d=1$			$d=2$			$\hat{d}$		
	0	1	$\hat{\lambda}$	0	1	$\hat{\lambda}$	0	1	$\hat{\lambda}$	0	1	$\hat{\lambda}$
1.0	98.0	50.7	50.9	123.8	12.3	11.9	71.7	16.0	16.7	73.2	15.0	15.3
0.5	96.0	61.1	61.1	49.9	9.0	8.2	25.6	11.8	11.5	26.9	11.0	10.8
0.2	142.2	68.4	68.4	12.6	7.8	6.4	8.5	10.4	8.1	10.1	9.6	7.9
0.1	152.8	71.1	71.0	6.0	7.8	5.7	5.9	10.4	7.2	6.3	9.6	6.9
0.0	162.2	73.8	73.7	3.6	7.7	4.8	5.0	10.2	6.6	4.4	9.5	6.0

dataset of size $n=100$. Five simulations of $\beta_3=1.0$, 0.5, 0.2, 0.1, and 0.0 are done with $N=10^4$. In the table, $E_1(-\log q(y|x))$ is subtracted from the loss to make comparisons easier.

The expected loss of MLE ($\lambda=0$) is 123.8 for $\beta_3=1.0$, $d=1$, while that of MWLE ($\lambda=1$) is 12.3, showing a great improvement as we have observed in the numerical example. The expected loss of MWLE with the selected weight ($\hat{\lambda}$) is 11.9, which is not significantly different from that of $\lambda=1$. This is a consequence of the large sample size $n=100$, where the first term of (4.1) or (5.1) is dominant. The same observation holds for $\beta_3=1.0$–0.0, $d=0$, and $\beta_3=1.0, 0.5$, $d=1,2$.

On the other hand, $\lambda=0$ is significantly better than $\lambda=1$ for $\beta_3=0.0$, $d=1,2$, where the model is correctly specified. In this case the second term of (4.1) is dominant and $\lambda=0$ is the optimal choice as mentioned in Lemma 4. The selected weight $\hat{\lambda}$ performs close to $\lambda=0$, but with slightly larger expected loss. This difference is the price we pay for the weight selection using (5.1) which is an estimate of (4.1), not the true value of (4.1).

For all the cases of $\beta_3=1.0$–0.0, $d=0,1,2$, the weight selection procedure gives the expected loss close to the optimal choice. Fixing λ to either of 0 or 1 can lead to very poor performance. The same observation holds for the model selection as shown in the columns of $\hat{d}$.

8. Bayesian inference

We have been working on the predictive density

$$p(y|x,\hat{\theta}_w), \tag{8.1}$$

which is based on MWLE $\hat{\theta}_w$. This type of predictive density is occasionally called as an estimative density in the literature. Another possibility is the Bayesian predictive density. Here we consider a weighted version of it, and examine its performance in prediction.

Let $p(\theta)$ be the prior density of θ. Given the data $(x^{(n)}, y^{(n)})$, we shall define the weighted posterior density by

$$p_w(\theta|x^{(n)}, y^{(n)}) \propto p(\theta)\exp L_w(\theta|x^{(n)}, y^{(n)}). \tag{8.2}$$

Then the predictive density will be

$$p_w(y|x, x^{(n)}, y^{(n)}) = \int p(y|x, \theta)\, p_w(\theta|x^{(n)}, y^{(n)})\, \mathrm{d}\theta. \tag{8.3}$$

In the case of $w(x) \equiv 1$, (8.2) reduces to the ordinary posterior density, and (8.3) reduces to the ordinary Bayesian predictive density.

The Kullback–Leibler loss of (8.3) with respect to $q(y|x)q_1(x)$ is

$$-\int q_1(x) \int q(y|x) \log\, p_w(y|x, x^{(n)}, y^{(n)})\, \mathrm{d}y\, \mathrm{d}x$$

and thus the expected loss is given by

$$E_0^{(n)}(E_1(-\log\, p_w(y|x, x^{(n)}, y^{(n)}))). \tag{8.4}$$

Lemma 5. *For sufficiently large n,* (8.4) *is asymptotically expanded as*

$$E_0^{(n)}(\mathrm{loss}_1(\hat{\theta}_w)) + \frac{1}{n}\left\{ K_w^{[1]\prime} a_w - \frac{1}{2}\mathrm{tr}((K_w^{[1\cdot 1]} - K_w^{[2]})H_w^{-1}) \right\} + \mathrm{o}(n^{-1}), \tag{8.5}$$

where

$$K_w^{[1\cdot 1]} = E_0\left\{ \frac{q_1(x)}{q_0(x)} \left.\frac{\partial \log\, p(y|x,\theta)}{\partial \theta}\right|_{\theta_w^*} \left.\frac{\partial \log\, p(y|x,\theta)}{\partial \theta'}\right|_{\theta_w^*} \right\}$$

and $a_w = \mathrm{plim}_{n\to\infty} \hat{a}_w$ *is the probability limit of*

$$\hat{a}_w = n \int (\theta - \hat{\theta}_w)\, p_w(\theta|x^{(n)}, y^{(n)})\, \mathrm{d}\theta.$$

Furthermore, (8.4) *is estimated by an information criterion*

$$-2\sum_{t=1}^{n} \frac{q_1(x_t)}{q_0(x_t)} \log\, p_w(y_t|x_t, x^{(n)}, y^{(n)}) + 2\,\mathrm{tr}(J_w H_w^{-1}). \tag{8.6}$$

In fact, the expectation of (8.6)*, if divided by* $2n$*, is equal to* (8.5) *up to* $\mathrm{O}(n^{-1})$ *terms.*

When $q_1(x) = q_0(x)$ and $w(x) \equiv 1$, (8.6) reduces to the information criterion for the Bayesian predictive density given in Konishi and Kitagawa (1996). Selection of $w(x)$ as well as selection of $p(\theta)$ and $p(y|x, \theta)$ becomes possible by minimizing (8.6). Comparing the values of (5.1) and (8.6), we can also choose which to use from (8.1) and (8.3).

The decrease of the expected loss of (8.3) from that of (8.1) is of order $\mathrm{O}(n^{-1})$ as shown in (8.5), which can be positive or negative depending on $q(y|x)$. For brevity sake, we assume $q_1(x) = q_0(x)$ and $w(x) \equiv 1$ below. Then the decrease in the expected loss is $\Delta/2n + \mathrm{o}(n^{-1})$, where $\Delta = (\mathrm{tr}(G_0 H_0^{-1}) - \dim\theta)$. $G_0 - H_0 \neq 0$ and $\Delta \neq 0$ under

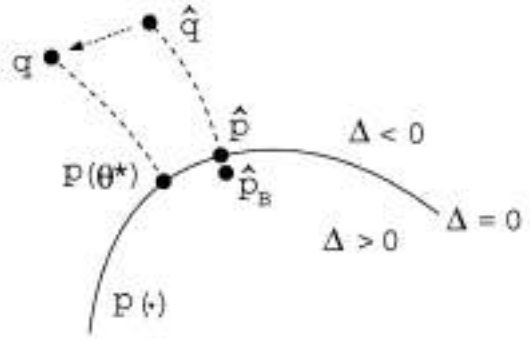


Fig. 3. The curvature of the model in relation to the location of the true density q. On the parametric model denoted by $p(\cdot)$, we have $\Delta = 0$, and $|\Delta|$ increases as q deviates from $p(\cdot)$. The region of $\Delta > 0$ is in the inside direction of the model, and $\Delta < 0$ is in the outside direction of the model. $\hat{q}$ denotes the empirical distribution, and the projection of $\hat{q}$ to the model is the estimative density $\hat{p} = p(y|x, \hat{\theta}_0)$. $\hat{p}_B$ denotes the Bayesian predictive density.

misspecification in general, which is utilized in the information matrix tests (White, 1982) for detecting the misspecification.

An enlightening interpretation of Δ may be possible by the following geometric argument using the terminology of Efron (1978) and Amari (1985). Fig. 3 shows the space of all conditional densities. Δ is determined by the extent of the misspecification multiplied by the "embedding mixture curvature" of the model (S. Amari, personal communication). Bayesian predictive density $\hat{p}_B$ is a mixture of $p(y|x, \theta)$ around $\hat{\theta}_0$, and thus it is located in the inside of the model because of the curvature; $\hat{p}_B$ deviates from $\hat{p}$ of order $O(n^{-1})$ as shown in Davison (1986). Therefore, $\hat{p}_B$ has larger expected loss than $\hat{p}$ if $q(y|x)$ is located in the outside of the model (i.e. $\Delta < 0$), because $\hat{p}_B$ is located in the opposite side of q. This does not contradict the classical result that the expected loss of $\hat{p}_B$ is asymptotically smaller than that of $\hat{p}$ for some prior. In Bayesian literature, the case of correct specification (i.e. $\Delta = 0$) is discussed and the difference of the expected loss is of order $O(n^{-2})$ as seen in Komaki (1996). Note that the quadratic form of Δ is relevant when the curvature vanishes, and $\Delta \geqslant 0$ if all the eigenvalues are non-negative; see Shimodaira (1997) for details.

9. Concluding remarks

Although the ratio $q_1(x)/q_0(x)$ has been assumed to be known, it is often estimated from data in practice. Assuming $q_1(x)$ is known, we tried three possibilities in the numerical example of Section 2: (i) $q_0(x)$ is specified correctly without unknown parameters. (ii) Assuming the normality of $q_0(x)$, the unknown μ_0 and τ_0 are estimated. (iii) Non-parametric kernel density estimation is applied to $q_0(x)$. Then, it turns out that MWLE is robust against the estimation of $q_1(x)/q_0(x)$ and the results are almost identical in the three cases. This may be because the form of $q_0(x)$ is quite simple and the sample size $n = 100$ is rather large.

A parametric approach to take account of estimation of $q_1(x)/q_0(x)$ is considered as follows. Let the observed data z_t, $t=1,\ldots,n$, follow $p_0(z|\theta)$, while future observations will follow $p_1(z|\theta)$. Then a possible estimating equation will be

$$\sum_{t=1}^{n} w(z_t|\theta)\frac{\partial \log p_1(z_t|\theta)}{\partial\theta}=0, \tag{9.1}$$

where $w(z|\theta)=(p_1(z|\theta)/p_0(z|\theta))^{\lambda}$. The solution of (9.1) reduces to the MWLE discussed in this paper by letting $z=(x,y)$, $p_0(z|\theta)=p(y|x,\theta)q_0(x)$, and $p_1(z|\theta)=p(y|x,\theta)q_1(x)$.

The estimating equation (9.1) is often seen in the literature of the robust parametric estimation; e.g., Green (1984), Hampel et al. (1986), Lindsay (1994), Basu and Lindsay (1994), Field and Smith (1994) and Windham (1995). In this context, the samples which are not concordant to the model $p_1(z|\theta)$ will be regarded as "outliers" and downweighted to reduce the impact on the parameter estimation. The specification of the weight function is thus the focal point of the argument. Although the covariate shift is a mechanism different from the outliers, an interpretation of MWLE in terms of the robust estimation can be given as follows. For simplicity, let z be a discrete random variable and $\hat{p}_0(z)$ be the observed relative frequency which estimates the contaminated distribution $p_0(z)$ consistently. The weight $(p_1(z|\theta)/\hat{p}_0(z))^{\lambda}$ in (9.1) then leads to the minimum disparity estimator obtained by minimizing the power divergence

$$2nI^{-\lambda}=\frac{2}{\lambda(\lambda-1)}\sum_{z}\hat{p}_0(z)\left\{\left(\frac{\hat{p}_0(z)}{p_1(z|\theta)}\right)^{-\lambda}-1\right\}$$

(Cressie and Read, 1984; Lindsay, 1994). The uniform weight $\lambda=0$ is sensitive to outliers, and a positive value $\lambda=0.5$, say, improves the robustness. In the regression analysis, the deviation of $p_0(z)$ from $p_1(z)$ is decomposed into two parts: the misspecification of $p(y|x,\theta)$ and the covariate shift. MWLE is obtained by applying the power weighting scheme to the second part where $\lambda=1$ is asymptotically optimal. It may be interesting to consider a robust version of MWLE by applying the weight, say $(p_1(y|x,\theta)/\hat{p}_0(y|x))^{\nu}$ with $\nu=0.5$, to the first part as well.

A numerical example of simultaneous selection of the weight and the model by the information criterion is shown in Section 6. The information criterion takes account of the selection bias caused by estimation of the parameter, but it does not take account the bias caused by the selection of the weight and the model. It is important to evaluate the expected loss of the predictive density obtained after these selection. The simulation study of Section 7 indicates that the method presented in this paper is effective, and the final expected loss of the selected weight and/or model is reasonably small.

Although we have employed a specific type of one-parameter connection in (5.3), other types of connection may work similarly. However, increasing the number of connection parameters will increase the final expected loss because of the sampling error of (5.1) as an estimator of (4.1). We observed the slight increase of the expected loss of $\hat{\lambda}$ in Table 4, and this increase can be much larger for multi-parameter connections.

We derived a variant of AIC for MWLE under covariate shift. On the other hand, Shimodaira (1994) and Cavanaugh and Shumway (1998) discussed variants of AIC for MLE in the presence of incomplete data. Information criteria have to be tailored for different styles of sampling scheme, and the unified approach for them is left as a future work.

A software for calculating IC_w and $\hat{\lambda}$ of normal regression will be available in S language at `http://www.ism.ac.jp/~shimo/`.

Acknowledgements

I would like to thank John Copas, Tony Hayter, Motoaki Kawanabe, Shinto Eguchi, and the reviewers for helpful comments and suggestions.

Appendix A.

Proof of Lemma 1. Since θ^*_w is interior to Θ, so is $\hat{\theta}_w$ for sufficiently large n. Then, $\hat{\theta}_w$ is obtained as a solution of the estimating equation

$$\sum_{t=1}^{n} \left. \frac{\partial I_w(x_t, y_t|\theta)}{\partial \theta} \right|_{\hat{\theta}_w} = 0. \tag{A.1}$$

The Taylor expansion of (A.1) leads to

$$n^{-1} \sum_{t=1}^{n} \left. \frac{\partial^2 I_w(x_t, y_t|\theta)}{\partial \theta \partial \theta'} \right|_{\theta^*_w} n^{1/2}(\hat{\theta}_w - \theta^*_w) = -n^{-1/2} \sum_{t=1}^{n} \left. \frac{\partial I_w(x_t, y_t|\theta)}{\partial \theta} \right|_{\theta^*_w} + \mathrm{O_p}(n^{-1/2}). \tag{A.2}$$

It follows from the central limit theorem that the right-hand side is asymptotically distributed as $\mathrm{N}(0, G_w)$, while the left-hand side converges to $H_w\, n^{1/2}(\hat{\theta}_w - \theta^*_w)$. Thus we obtained the desired result. □

Proof of Lemma 2. The Taylor expansion of $\mathrm{loss}_1(\hat{\theta}_w)$ around θ^*_w is

$$\mathrm{loss}_1(\theta^*_w) + \frac{1}{n}\left\{ (K^{[1]}_w)_i\, n^{1/2} \dot{\theta}^i_w + \frac{1}{2}(K^{[2]}_w)_{ij} \dot{\theta}^i_w \dot{\theta}^j_w \right\} + \mathrm{O_p}(n^{-3/2}), \tag{A.3}$$

where $\dot{\theta}_w = n^{1/2}(\hat{\theta}_w - \theta^*_w)$, and the summation convention $A_i B^i = \sum_{i=1}^{m} A_i B^i$ is used. Considering Lemma 1, the expectation of (A.3) gives (4.1). By taking expectation of (A.2), we observe $b_w = \mathrm{O}(1)$. □

Proof of Lemma 3. Considering the $\mathrm{O_p}(n^{-1/2})$ term in (A.2) explicitly, the Taylor expansion of (A.1) gives

$$\hat{H}^*_w \dot{\theta}_w = -n^{-1/2} \sum_{t=1}^{n} \left. \frac{\partial I_w(x_t, y_t|\theta)}{\partial \theta} \right|_{\theta^*_w} + n^{-1/2} e_w,$$

A Literature Survey on Domain Adaptation of Statistical Classifiers

Jing Jiang
jiang4@cs.uiuc.edu

Last modified in March 2008

Contents

1 Introduction **1**

2 Notations and Overview **2**
2.1 Notations 2
2.2 Overview 3

3 Instance Weighting **3**
3.1 Class Imbalance 4
3.2 Covariate Shift 5
3.3 Change of Functional Relations 6

4 Semi-Supervised Learning **6**

5 Change of Representation **6**

6 Bayesian Priors **7**

7 Multi-Task Learning **8**

8 Ensemble Methods **9**

1 Introduction

Domain adaptation of statistical classifiers is the problem that arises when the data distribution in our test domain is different from that in our training domain. The need for domain adaptation is prevalent in many real-world classification problems. For example, spam filters can be trained on some public collection of spam and ham emails. But when applied to an individual person's inbox, we may want to "personalize" the spam filter, i.e. to adapt the spam filter to fit the person's own distribution of emails in order to achieve better performance.

Although the domain adaptation problem is a fundamental problem in machine learning, it only started gaining much attention very recently (Daumé III and Marcu, 2006; Blitzer et al., 2006; Ben-David et al., 2007; Daumé III, 2007; Jiang and Zhai, 2007a; Satpal and Sarawagi, 2007; Jiang and Zhai, 2007b; Blitzer

et al., 2008). However, some special kinds of domain adaptation problems have been studied before under different names including class imbalance (Japkowicz and Stephen, 2002), covariate shift (Shimodaira, 2000), and sample selection bias (Heckman, 1979; Zadrozny, 2004). There are also some closely-related but not equivalent machine learning problems that have been studied extensively, including multi-task learning (Caruana, 1997) and semi-supervised learning (Zhu, 2005; Chapelle et al., 2006).

In this literature survey, we review some existing work in both the machine learning and the natural language processing communities related to domain adaptation. The goal of this survey is twofold. First, there have been a number of methods proposed to address domain adaptation, but it is not clear how these methods are related to each other. This survey thus tries to organize the existing work and lay out an overall picture of the domain adaptation problem with its possible solutions. Second, a systematic literature survey naturally reveals the limitations of current work and points out promising directions that should be explored in the future.

Because domain adaptation is a relatively new topic that is still constantly attracting attention, our survey is necessarily incomplete. Nevertheless, we try to cover the major lines of work that we are aware of up to the date this survey is written. This survey will also be updated periodically.

2 Notations and Overview

2.1 Notations

We first introduce some notations that are needed in the discussion in this survey. We refer to the training domain where labeled data is abundant as the *source* domain, and the test domain where labeled data is not available or very little as the *target* domain. Let X denote the input variable (i.e. an observation) and Y the output variable (i.e. a class label). We use $P(X, Y)$ to denote the true underlying joint distribution of X and Y, which is unknown. In domain adaptation, this joint distribution in the target domain differs from that in the source domain. We therefore use $P_t(X, Y)$ to denote the true underlying joint distribution in the target domain, and $P_s(X, Y)$ to denote that in the source domain. We use $P_t(Y)$, $P_s(Y)$, $P_t(X)$ and $P_s(X)$ to denote the true marginal distributions of Y and X in the target and the source domains, respectively. Similarly, we use $P_t(X|Y)$, $P_s(X|Y)$, $P_t(Y|X)$ and $P_s(Y|X)$ to denote the true conditional distributions in the two domains. We use lowercase x to denote a specific value of X, and lowercase y to denote a specific class label. A specific x is also referred to as an observation, an unlabeled instance or simply an instance. A pair (x, y) is referred to as a labeled instance. Here, $x \in \mathcal{X}$, where $\mathcal{X}$ is the input space, i.e. the set of all possible observations. Similarly, $y \in \mathcal{Y}$, where $\mathcal{Y}$ is the class label set. Without any ambiguity, $P(X = x, Y = y)$ or simply $P(x, y)$ should refer to the joint probability of $X = x$ and $Y = y$. Similarly, $P(X = x)$ (or $P(x)$), $P(Y = y)$ (or $P(y)$), $P(X = x|Y = y)$ (or $P(x|y)$) and $P(Y = y|X = x)$ (or $P(y|x)$) also refer to probabilities rather than distributions.

We assume that there is always a relatively large amount of *labeled* data available in the source domain. We use $D_s = \{(x_i^s, y_i^s)\}_{i=1}^{N_s}$ to denote this set of labeled instances in the source domain. In the target domain, we assume that we always have access to a large amount of *unlabeled* data, and we use $D_{t,u} = \{x_i^{t,u}\}_{i=1}^{N_{t,u}}$ to denote this set of unlabeled instances. Sometimes, we may also have a small amount of labeled data from the target domain, which is denoted as $D_{t,l} = \{(x_i^{t,l}, y_i^{t,l})\}_{i=1}^{N_{t,l}}$. In the case when $D_{t,l}$ is not available, we refer to the problem as *unsupervised domain adaptation*, while when $D_{t,l}$ is available, we refer to the problem as *supervised domain adaptation*.

2.2 Overview

Recently, there have been a number of studies related to domain adaptation. However, the motivating ideas behind these methods are different. To connect the existing work and hence to better understand the problem, in the following sections, we organize the existing work into several categories from our own viewpoint. First, in Section 3, we consider a line of work that is based on instance weighting. In Section 4, we look at some work that bears strong resemblance to semi-supervised learning. In Section 5, we review another line of work that is based on changing the representation of X. Section 6 reviews work using Bayesian priors, and Section 7 reviews work related to multi-task learning. In Section 8, ensemble methods for domain adaptation are considered.

The categories are ordered in this way so that methods in Section 3, Section 4 and Section 5 are generally applicable to unsupervised domain adaptation problems, while methods in Section 6 and Section 7 can only handle supervised domain adaptation problems.

3 Instance Weighting

One general approach to addressing the domain adaptation problem is to assign instance-dependent weights to the loss function when minimizing the expected loss over the distribution of data. To see why instance weighting may help, let us first briefly review the empirical risk minimization framework for standard supervised learning (Vapnik, 1999), and then informally derive an instance weighting solution to domain adaptation. Let Θ be a model family from which we want to select an optimal model θ^* for our classification task. Let $l(x, y, \theta)$ be a loss function. Strictly speaking, we want to minimize the following objective function in order to obtain the optimal model θ^* for the distribution $P(X, Y)$:

$$\theta^* = \arg\min_{\theta\in\Theta} \sum_{(x,y)\in\mathcal{X}\times\mathcal{Y}} P(x, y)l(x, y, \theta).$$

Because $P(X, Y)$ is unknown, we can use the empirical distribution $\tilde{P}(X, Y)$ to approximate $P(X, Y)$. Let $\{(x_i, y_i)\}_{i=1}^N$ be a set of training instances randomly sampled from $P(X, Y)$. We then minimize the following empirical risk in order to find a good model $\hat{\theta}$:

$$\begin{aligned}\hat{\theta} &= \arg\min_{\theta\in\Theta} \sum_{(x,y)\in\mathcal{X}\times\mathcal{Y}} \tilde{P}(x, y)l(x, y, \theta) \\ &= \arg\min_{\theta\in\Theta} \sum_{i=1}^{N} l(x_i, y_i, \theta).\end{aligned}$$

Now consider the setting of domain adaptation. Ideally, we want to find an optimal model for the target domain that minimizes the expected loss over the target distribution:

$$\theta_t^* = \arg\min_{\theta\in\Theta} \sum_{(x,y)\in\mathcal{X}\times\mathcal{Y}} P_t(x, y)l(x, y, \theta).$$

However, our training instances, $D_s = \{(x_i^s, y_i^s\}_{i=1}^{N_s}$, are randomly sampled from the source distribution

$P_s(X, Y)$. We can rewrite the equation above as follows:

$$\begin{aligned}
\theta_t^* &= \arg\min_{\theta\in\Theta} \sum_{(x,y)\in\mathcal{X}\times\mathcal{Y}} \frac{P_t(x,y)}{P_s(x,y)} P_s(x,y) l(x,y,\theta) \\
&\approx \arg\min_{\theta\in\Theta} \sum_{(x,y)\in\mathcal{X}\times\mathcal{Y}} \frac{P_t(x,y)}{P_s(x,y)} \tilde{P}_s(x,y) l(x,y,\theta) \\
&= \arg\min_{\theta\in\Theta} \sum_{i=1}^{N_s} \frac{P_t(x_i^s, y_i^s)}{P_s(x_i^s, y_i^s)} l(x_i^s, y_i^s, \theta). \qquad (1)
\end{aligned}$$

As we can see, weighting the loss for the instance (x_i^s, y_i^s) with $\frac{P_t(x_i^s,y_i^s)}{P_s(x_i^s,y_i^s)}$ provides a well-justified solution to the domain adaptation problem.

It is not possible to compute the exact value of $\frac{P_t(x,y)}{P_s(x,y)}$ for a pair (x, y), especially because we do not have enough labeled instances in the target domain. Section 3.1 reviews one line of work in which $P_t(X|Y) = P_s(X|Y)$ is assumed, while Section 3.2 reviews another line of work in which $P_t(Y|X) = P_s(Y|X)$ is assumed.

3.1 Class Imbalance

One simple assumption we can make about the connection between the distributions of the source and the target domains is that given the same class label, the conditional distributions of X are the same in the two domains. However, the class distributions may be different in the source and the target domains. Formally, we assume that $P_s(X|Y = y) = P_t(X|Y = y)$ for all $y \in \mathcal{Y}$, but $P_s(Y) \neq P_t(Y)$. This difference is referred to as the *class imbalance* problem in some work (Japkowicz and Stephen, 2002).

When this class imbalance assumption is made, the ratio $\frac{P_t(x,y)}{P_s(x,y)}$ that we derived in Equation (1) can be rewritten as follows:

$$\begin{aligned}
\frac{P_t(x,y)}{P_s(x,y)} &= \frac{P_t(y)}{P_s(y)} \frac{P_t(x|y)}{P_s(x|y)} \\
&= \frac{P_t(y)}{P_s(y)}.
\end{aligned}$$

Therefore, we only need to use $\frac{P_t(y)}{P_s(y)}$ to weight the instances. This approach has been explored in (Lin et al., 2002). Alternatively, we can re-sample the training instances from the source domain so that the re-sampled data roughly has the same class distribution as the target domain. In re-sampling methods, under-represented classes are over-sampled, and over-represented classes are under-sampled (Kubat and Matwin, 1997; Chawla et al., 2002; Zhu and Hovy, 2007).

For classification algorithms that directly model the probability distribution $P(Y|X)$ such as logistic regression classifiers, it can be shown theoretically that the estimated probability $P_s(y|x)$ can be transformed into $P_t(y|x)$ in the following way (Lin et al., 2002; Chan and Ng, 2005):

$$P_t(y|x) = \frac{r(y) P_s(y|x)}{\sum_{y'\in\mathcal{Y}} r(y') P_s(y'|x)},$$

where $r(y)$ is defined as

$$r(y) = \frac{P_t(y)}{P_s(y)}.$$

Now we can first estimate $P_s(y|x)$ from the source domain, and then derive $P_t(y|x)$ using $P_s(Y)$ and $P_t(Y)$.

For other classification algorithms that do not directly model $P(Y|X)$, such as naive Bayes classifiers and support vector machines, if $P(Y|X)$ can be obtained through careful calibration, the same trick can be applied. Chan and Ng (2006) applied this method to the domain adaptation problem in word sense disambiguation (WSD) using naive Bayes classifiers.

In practice, one needs to know the class distribution in the target domain in order to apply the methods described above. In some studies, it is assumed that this distribution is known a priori (Lin et al., 2002). However, in reality, we may not have this information. Chan and Ng (2005) proposed to use the EM algorithm to estimate the class distribution in the target domain.

3.2 Covariate Shift

Another assumption one can make about the connection between the source and the target domains is that given the same observation $X = x$, the conditional distributions of Y are the same in the two domains. However, the marginal distributions of X may be different in the source and the target domains. Formally, we assume that $P_s(Y|X = x) = P_t(Y|X = x)$ for all $x \in \mathcal{X}$, but $P_s(X) \neq P_t(X)$. This difference between the two domains is called *covariate shift* (Shimodaira, 2000).

At first glance, it may appear that covariate shift is not a problem. For classification, we are only interested in $P(Y|X)$. If $P_s(Y|X) = P_t(Y|X)$, why would the classifier learned from the source domain not perform well on the target domain even if $P_s(X) \neq P_t(X)$? Shimodaira (2000) showed that this covariate shift becomes a problem when *misspecified* models are used. Suppose we consider a parametric model family $\{P(Y|X, \theta)\}_{\theta \in \Theta}$ from which a model $P(Y|X, \theta^*)$ is selected to minimize the expected classification error. If none of the models in the model family can exactly match the true relation between X and Y, that is, there does not exist any $\theta \in \Theta$ such that $P(Y|X = x, \theta) = P(Y|X = x)$ for all $x \in \mathcal{X}$, then we say that we have a misspecified model family. The intuition of why covariate shift under model misspecification becomes a problem is as follows. With a misspecified model family, the optimal model we select depends on $P(X)$, and if $P_t(X) \neq P_s(X)$, then the optimal model for the target domain will differ from that for the source domain. The intuitive is that the optimal model performs better in dense regions of X than in sparse regions of X, because the dense regions dominate the average classification error, which is what we want to minimize. If the dense regions of X are different in the source and the target domains, the optimal model for the source domain will no longer be optimal for the target domain.

Under covariate shift, the ratio $\frac{P_t(x,y)}{P_s(x,y)}$ that we derived in Equation (1) can be rewritten as follows:

$$\begin{aligned} \frac{P_t(x,y)}{P_s(x,y)} &= \frac{P_t(x)}{P_s(x)} \frac{P_t(y|x)}{P_s(y|x)} \\ &= \frac{P_t(x)}{P_s(x)}. \end{aligned}$$

We therefore want to weight each training instance with $\frac{P_t(x)}{P_s(x)}$.

Shimodaira (2000) first proposed to re-weight the log likelihood of each training instance (x, y) using $\frac{P_t(x)}{P_s(x)}$ in maximum likelihood estimation for covariate shift. It can be shown theoretically that if the support of $P_t(X)$ (the set of x's for which $P_t(X = x) > 0$) is contained in the support of $P_s(X)$, then the optimal model that maximizes this re-weighted log likelihood function asymptotically converges to the optimal model for the target domain.

A major challenge is how to estimate the ratio $\frac{P_t(x)}{P_s(x)}$ for each x in the training set. In some work, a principled method of using non-parametric kernel density estimation is explored (Shimodaira, 2000; Sugiyama

and Müller, 2005). In some other work, it is proposed to transform this density ratio estimation into a problem of predicting whether an instance is from the source domain or from the target domain (Zadrozny, 2004; Bickel and Scheffer, 2007). Huang et al. (2007) transformed the problem into a kernel mean matching problem in a reproducing kernel Hilbert space. Bickel et al. (2007) proposed to learn this ratio together with the classification model parameters.

3.3 Change of Functional Relations

Both class imbalance and covariate shift simplify the difference between $P_s(X, Y)$ and $P_t(X, Y)$. It is still possible that $P_t(X|Y)$ differs from $P_s(X|Y)$ or $P_t(Y|X)$ differs from $P_s(Y|X)$.

Jiang and Zhai (2007a) considered the case when $P_t(Y|X)$ differs from $P_s(Y|X)$, and proposed a heuristic method to remove "misleading" training instances from the source domain, where $P_s(y|x)$ is very different from $P_t(y|x)$. To discover these "misleading" training instances, some labeled data from the target domain is needed. This method therefore is only suitable for *supervised* domain adaptation.

4 Semi-Supervised Learning

If we ignore the domain difference, and treat the labeled source domain instances as labeled data and the unlabeled target domain instances as unlabeled data, then we are facing a semi-supervised learning (SSL) problem. We can then apply any SSL algorithms (Zhu, 2005; Chapelle et al., 2006) to the domain adaptation problem. The subtle difference between SSL and domain adaptation is that (1) the amount of labeled data in SSL is small but large in domain adaptation, and (2) the labeled data may be noisy in domain adaptation if we do not assume $P_s(Y|X = x) = P_t(Y|X = x)$ for all x, whereas in SSL the labeled data is all reliable.

There has been some work extending semi-supervised learning methods for domain adaptation. Dai et al. (2007a) proposed an EM-based algorithm for domain adaptation, which can be shown to be equivalent to a semi-supervised EM algorithm (Nigam et al., 2000) except that Dai et al. proposed to estimate the trade-off parameter between the labeled and the unlabeled data using the KL-divergence between the two domains. Jiang and Zhai (2007a) proposed to not only include weighted source domain instances but also weighted unlabeled target domain instances in training, which essentially combines instance weighting with bootstrapping. Xing et al. (2007) proposed a bridged refinement method for domain adaptation using label propagation on a nearest neighbor graph, which has resemblance to graph-based semi-supervised learning algorithms (Zhu, 2005; Chapelle et al., 2006).

5 Change of Representation

As has been pointed out, the cause of the domain adaptation problem is the difference between $P_t(X, Y)$ and $P_s(X, Y)$. Note that while the representation of Y is fixed, the representation of X can change if we use different features. Such a change of representation of X can affect both the marginal distribution $P(X)$ and the conditional distribution $P(Y|X)$. One can assume that under some change of representation of X, $P_t(X, Y)$ and $P_s(X, Y)$ will become the same.

Formally, let $g : \mathcal{X} \rightarrow \mathcal{Z}$ denote a transformation function that transforms an observation x represented in the original form into another form $z = g(x) \in \mathcal{Z}$. Define variable Z and an induced distribution of Z

that satisfies $P(z) = \sum_{x \in \mathcal{X}, g(x)=z} P(x)$. The joint distribution of Z and Y is then

$$P(z, y) = \sum_{x \in \mathcal{X}, g(x)=z} P(x, y).$$

If we can find a transformation function g so that under this transformation, we have $P_t(Z, Y) = P_s(Z, Y)$, then we no longer have the domain adaptation problem because the two domains have the same joint distribution of the observation and the class label. The optimal model $P(Y|Z, \theta^*)$ we learn to approximate $P_s(Y|Z)$ is still optimal for $P_t(Y|Z)$.

Note that with a change of representation, the entropy of Y conditional on Z is likely to increase from the entropy of Y conditional on X, because Z is usually a simpler representation of the observation than X, and thus encodes less information. In another word, the Bayes error rate usually increases under a change of representation. Therefore, the criteria for good transformation functions include not only the distance between the induced distributions $P_t(Z, Y)$ and $P_s(Z, Y)$ but also the amount of increment of the Bayes error rate.

Ben-David et al. (2007) first formally analyzed the effect of representation change for domain adaptation. They proved a generalization bound for domain adaptation that is dependent on the distance between the induced $P_s(Z, Y)$ and $P_t(Z, Y)$.

A special and simple kind of transformation is feature subset selection. Satpal and Sarawagi (2007) proposed a feature subset selection method for domain adaptation, where the criterion for selecting features is to minimize an approximated distance function between the distributions in the two domains. Note that to measure the distance between $P_s(Z, Y)$ and $P_t(Z, Y)$, we still need class labels in the target domain. To solve this problem, in (Satpal and Sarawagi, 2007), predicted labels for the target domain instances are used.

Blitzer et al. (2006) proposed a structural correspondence learning (SCL) algorithm that makes use of the unlabeled data from the target domain to find a low-rank representation that is suitable for domain adaptation. It is empirically shown in (Ben-David et al., 2007) that the low-rank representation found by SCL indeed decreases the distance between the distributions in the two domains. However, SCL does not directly try to find a representation Z that minimizes the distance between $P_s(Z, Y)$ and $P_t(Z, Y)$. Instead, SCL tries to find a representation that works well for many related classification tasks for which labels are available in both the source and the target domains. The assumption is that if a representation Z gives good performance for the many related classification tasks in both domains, then Z is also a good representation for the main classification task we are interested in in both domains. The core algorithm in SCL is from (Ando and Zhang, 2005).

6 Bayesian Priors

Most of the work reviewed in the previous sections does not require labeled data from the target domain. In this section and the next section, we review two kinds of methods that work for supervised domain adaptation, i.e. when a small amount of labeled data from the target domain is available.

When we use the maximum a posterior (MAP) estimation approach for supervised learning, we can encode some prior knowledge about the classification model into a Bayesian prior distribution $P(\theta)$, where θ is the model parameter. More specifically, instead of maximizing

$$\prod_{i=1}^{N} P(y_i|x_i; \theta),$$

we maximize

$$P(\theta)\prod_{i=1}^{N} P(y_i|x_i;\theta).$$

In domain adaptation, the prior knowledge can be drawn from the source domain. More specifically, we first construct a Bayesian prior $P(\theta|D_s)$, which is dependent on the labeled instances from the source domain. We then maximize the following objective function:

$$P(\theta|D_s)P(D_{t,l}|\theta) \quad = \quad P(\theta|D_s)\prod_{i=1}^{N_{t,l}} P(y_i^t|x_i^t;\theta).$$

Li and Bilmes (2007) proposed a general Bayesian divergence prior framework for domain adaptation. They then showed how this general prior can be instantiated for generative classifiers and discriminative classifiers. Chelba and Acero (2004) applied this kind of a Bayesian prior for the task of adapting a maximum entropy capitalizer across domains.

7 Multi-Task Learning

Multi-task learning, sometimes known as transfer learning, is a machine learning topic highly related to domain adaptation. The original definition of multi-task learning considers a different setting than domain adaptation. In multi-task learning, there is a single distribution of the observation, i.e. a single $P(X)$. There are, however, a number of different output variables $Y_1, Y_2, \ldots, Y_M$, corresponding to M different tasks. Therefore, there are M different joint distributions $\{P(X, Y_k)\}_{k=1}^{M}$. Note that the class label sets are different for these M different tasks. We assume that these different tasks are related. When learning M conditional models $\{P(Y_k|X, \theta_k)\}_{k=1}^{M}$ for the M tasks, we impose a common component shared by $\{\theta_k\}_{k=1}^{M}$. There have been a number of studies on multi-task learning (Caruana, 1997; Ben-David and Schuller, 2003; Micchelli and Pontil, 2005; Xue et al., 2007).

Strictly speaking, domain adaptation is a different problem than multi-task learning because we have only a single task but different domains. However, domain adaptation can be treated as a special case of multi-task learning, where we have two tasks, one on the source domain and the other on the target domain, and the class label sets of these two tasks are the same. If we have some labeled data from the target domain, we can then directly apply some existing multi-task learning algorithm.

Indeed, some domain adaptation methods proposed recently are essentially multi-task learning algorithms. Daumé III (2007) proposed a simple method for domain adaptation based on feature duplications. The idea is to make a domain-specific copy of the original features for each domain. An instance from domain k is then represented by both the original features and the features specific to domain k. It can be shown that when linear classification algorithms are used, this feature duplication based method is equivalent to decomposing the model parameter θ_k for domain k into $\theta_c + \theta'_k$, where θ_c is shared by all domains. This formulation then is very similar to the regularized multi-task learning method proposed by Evgeniou and Pontil (2004). Similarly, Jiang and Zhai (2007b) proposed a two-stage domain adaptation method, where in the first generalization stage, labeled instances from K different source training domains are used together to train K different models, but these models share a common component, and this common model component only applies to a subset of features that are considered generalizable across domains.

8 Ensemble Methods

In previous sections, only learning algorithms that return single classification models are considered. Ensemble methods are a type of learning algorithms that combine a set of models to construct a complex classifier for a classification problem. Ensemble methods include bagging, boosting, mixture of experts, etc. There has been some work using ensemble methods for domain adaptation.

One line of work uses mixture models. It can be assumed that there are a number of different component distributions $\{P^{(k)}(X,Y)\}_{k=1}^{K}$, each of which modeled by a simple model. The distribution of X and Y in either the source domain or the target domain is then a mixture of these component distributions. The source and the target domains are related because they share some of these component distributions. However, the mixture coefficients are different in the two domains, making the overall distributions different.

Daumé III and Marcu (2006) proposed a mixture model for domain adaptation, in which three mixture components are assumed, one shared by both the source and the target domains, one specific to the source domain, and one specific to the target domain. Labeled data from both the source and the target domains is needed to learn this three-component mixture model using the conditional expectation maximization (CEM) algorithm. Storkey and Sugiyama (2007) considered a more general mixture model in which the source and the target domains share more than one mixture components. However, they did not assume any target domain specific component, and as a result, no labeled data from the target domain is needed. The mixture model is learned using the expectation maximization (EM) algorithm.

Boosting is a general ensemble method that combines multiple weak learners to form a complex and effective classifier. Dai et al. (2007b) proposed to modify the widely-used *AdaBoost* algorithm to address the domain adaptation problem. With some labeled data from the target domain, the idea here is to put more weight on mistakenly classified target domain instances but less weight on mistakenly classified source domain instances in each iteration, because the goal is to improve the performance of the final classifier on the target domain only.

References

Rie Ando and Tong Zhang. A framework for learning predictive structure from multiple tasks and unlabeled data. *Journal of Machine Learning Research*, 6:1817–1853, November 2005.

Shai Ben-David and Reba Schuller. Exploiting task relatedness for multiple task learning. In *Proceedings of the 16th Annual Conference on Learning Theory*, Washington D.C., USA, August 2003.

Shai Ben-David, John Blitzer, Koby Crammer, and Fernando Pereira. Analysis of representations for domain adaptation. In B. Schölkopf, J. Platt, and T. Hoffman, editors, *Advances in Neural Information Processing Systems 19*, pages 137–144. MIT Press, Cambridge, Massachusetts, USA, 2007.

Steffen Bickel and Tobias Scheffer. Dirichlet-enhanced spam filtering based on biased samples. In B. Schölkopf, J. Platt, and T. Hoffman, editors, *Advances in Neural Information Processing Systems 19*, pages 161–168. MIT Press, Cambridge, Massachusetts, USA, 2007.

Steffen Bickel, Michael Brückner, and Tobias Scheffer. Discriminative learning for differing training and test distributions. In *Proceedings of the 24th Annual International Conference on Machine Learning*, pages 81–88, Corvallis, Oregon, USA, June 2007.

On the importance of initialization and momentum in deep learning

Ilya Sutskever[1] ILYASU@GOOGLE.COM
James Martens JMARTENS@CS.TORONTO.EDU
George Dahl GDAHL@CS.TORONTO.EDU
Geoffrey Hinton HINTON@CS.TORONTO.EDU

Abstract

Deep and recurrent neural networks (DNNs and RNNs respectively) are powerful models that were considered to be almost impossible to train using stochastic gradient descent with momentum. In this paper, we show that when stochastic gradient descent with momentum uses a well-designed random initialization and a particular type of slowly increasing schedule for the momentum parameter, it can train both DNNs and RNNs (on datasets with long-term dependencies) to levels of performance that were previously achievable only with Hessian-Free optimization. We find that both the initialization and the momentum are crucial since poorly initialized networks cannot be trained with momentum and well-initialized networks perform markedly worse when the momentum is absent or poorly tuned.

Our success training these models suggests that previous attempts to train deep and recurrent neural networks from random initializations have likely failed due to poor initialization schemes. Furthermore, carefully tuned momentum methods suffice for dealing with the curvature issues in deep and recurrent network training objectives without the need for sophisticated second-order methods.

1. Introduction

Deep and recurrent neural networks (DNNs and RNNs, respectively) are powerful models that achieve high performance on difficult pattern recognition problems in vision, and speech (Krizhevsky et al., 2012; Hinton et al., 2012; Dahl et al., 2012; Graves, 2012).

Although their representational power is appealing, the difficulty of training DNNs has prevented their widepread use until fairly recently. DNNs became the subject of renewed attention following the work of Hinton et al. (2006) who introduced the idea of greedy layerwise pre-training. This approach has since branched into a family of methods (Bengio et al., 2007), all of which train the layers of the DNN in a sequence using an auxiliary objective and then "fine-tune" the entire network with standard optimization methods such as stochastic gradient descent (SGD). More recently, Martens (2010) attracted considerable attention by showing that a type of truncated-Newton method called Hessian-free Optimization (HF) is capable of training DNNs from certain random initializations without the use of pre-training, and can achieve lower errors for the various auto-encoding tasks considered by Hinton & Salakhutdinov (2006).

Recurrent neural networks (RNNs), the temporal analogue of DNNs, are highly expressive sequence models that can model complex sequence relationships. They can be viewed as very deep neural networks that have a "layer" for each time-step with parameter sharing across the layers and, for this reason, they are considered to be even harder to train than DNNs. Recently, Martens & Sutskever (2011) showed that the HF method of Martens (2010) could effectively train RNNs on artificial problems that exhibit very long-range dependencies (Hochreiter & Schmidhuber, 1997). Without resorting to special types of memory units, these problems were considered to be impossibly difficult for first-order optimization methods due to the well known vanishing gradient problem (Bengio et al., 1994). Sutskever et al. (2011) and later Mikolov et al. (2012) then applied HF to train RNNs to perform character-level language modeling and achieved excellent results.

Recently, several results have appeared to challenge the commonly held belief that simpler first-order methods are incapable of learning deep models from random initializations. The work of Glorot & Bengio (2010), Mohamed et al. (2012), and Krizhevsky et al. (2012) reported little difficulty training neural networks with depths up to 8 from certain well-chosen

Proceedings of the 30^{th} International Conference on Machine Learning, Atlanta, Georgia, USA, 2013. JMLR: W&CP volume 28.

[1]Work was done while the author was at the University of Toronto.

random initializations. Notably, Chapelle & Erhan (2011) used the random initialization of Glorot & Bengio (2010) and SGD to train the 11-layer autoencoder of Hinton & Salakhutdinov (2006), and were able to surpass the results reported by Hinton & Salakhutdinov (2006). While these results still fall short of those reported in Martens (2010) for the same tasks, they indicate that learning deep networks is not nearly as hard as was previously believed.

The first contribution of this paper is a much more thorough investigation of the difficulty of training deep and temporal networks than has been previously done. In particular, we study the effectiveness of SGD when combined with well-chosen initialization schemes and various forms of momentum-based acceleration. We show that while a definite performance gap seems to exist between plain SGD and HF on certain deep and temporal learning problems, this gap can be eliminated or nearly eliminated (depending on the problem) by careful use of classical momentum methods or Nesterov's accelerated gradient. In particular, we show how certain carefully designed schedules for the constant of momentum μ, which are inspired by various theoretical convergence-rate theorems (Nesterov, 1983; 2003), produce results that even surpass those reported by Martens (2010) on certain deep-autencoder training tasks. For the long-term dependency RNN tasks examined in Martens & Sutskever (2011), which first appeared in Hochreiter & Schmidhuber (1997), we obtain results that fall just short of those reported in that work, where a considerably more complex approach was used.

Our results are particularly surprising given that momentum and its use within neural network optimization has been studied extensively before, such as in the work of Orr (1996), and it was never found to have such an important role in deep learning. One explanation is that previous theoretical analyses and practical benchmarking focused on local convergence in the stochastic setting, which is more of an estimation problem than an optimization one (Bottou & LeCun, 2004). In deep learning problems this final phase of learning is not nearly as long or important as the initial "transient phase" (Darken & Moody, 1993), where a better argument can be made for the beneficial effects of momentum.

In addition to the inappropriate focus on purely local convergence rates, we believe that the use of poorly designed standard random initializations, such as those in Hinton & Salakhutdinov (2006), and suboptimal meta-parameter schedules (for the momentum constant in particular) has hampered the discovery of the true effectiveness of first-order momentum methods in deep learning. We carefully avoid both of these pitfalls in our experiments and provide a simple to understand and easy to use framework for deep learning that is surprisingly effective and can be naturally combined with techniques such as those in Raiko et al. (2011).

We will also discuss the links between classical momentum and Nesterov's accelerated gradient method (which has been the subject of much recent study in convex optimization theory), arguing that the latter can be viewed as a simple modification of the former which increases stability, and can sometimes provide a distinct improvement in performance we demonstrated in our experiments. We perform a theoretical analysis which makes clear the precise difference in local behavior of these two algorithms. Additionally, we show how HF employs what can be viewed as a type of "momentum" through its use of special initializations to conjugate gradient that are computed from the update at the previous time-step. We use this property to develop a more momentum-like version of HF which combines some of the advantages of both methods to further improve on the results of Martens (2010).

2. Momentum and Nesterov's Accelerated Gradient

The momentum method (Polyak, 1964), which we refer to as classical momentum (CM), is a technique for accelerating gradient descent that accumulates a velocity vector in directions of persistent reduction in the objective across iterations. Given an objective function $f(\theta)$ to be minimized, classical momentum is given by:

$$v_{t+1} = \mu v_t - \varepsilon \nabla f(\theta_t) \quad (1)$$

$$\theta_{t+1} = \theta_t + v_{t+1} \quad (2)$$

where $\varepsilon > 0$ is the learning rate, $\mu \in [0, 1]$ is the momentum coefficient, and $\nabla f(\theta_t)$ is the gradient at θ_t.

Since directions d of low-curvature have, by definition, slower local change in their rate of reduction (i.e., $d^\top \nabla f$), they will tend to persist across iterations and be amplified by CM. Second-order methods also amplify steps in low-curvature directions, but instead of accumulating changes they reweight the update along each eigen-direction of the curvature matrix by the inverse of the associated curvature. And just as second-order methods enjoy improved local convergence rates, Polyak (1964) showed that CM can considerably accelerate convergence to a local minimum, requiring $\sqrt{R}$-times fewer iterations than steepest descent to reach the same level of accuracy, where R is the condition number of the curvature at the minimum and μ is set to $(\sqrt{R} - 1)/(\sqrt{R} + 1)$.

Nesterov's Accelerated Gradient (abbrv. NAG; Nesterov, 1983) has been the subject of much recent attention by the convex optimization community (e.g., Cotter et al., 2011; Lan, 2010). Like momentum, NAG is a first-order optimization method with better convergence rate guarantee than gradient descent in

certain situations. In particular, for general smooth (non-strongly) convex functions and a deterministic gradient, NAG achieves a global convergence rate of $O(1/T^2)$ (versus the $O(1/T)$ of gradient descent), with constant proportional to the Lipschitz coefficient of the derivative and the squared Euclidean distance to the solution. While NAG is not typically thought of as a type of momentum, it indeed turns out to be closely related to classical momentum, differing only in the precise update of the velocity vector v, the significance of which we will discuss in the next sub-section. Specifically, as shown in the appendix, the NAG update may be rewritten as:

$$v_{t+1} = \mu v_t - \varepsilon \nabla f(\theta_t + \mu v_t) \quad (3)$$
$$\theta_{t+1} = \theta_t + v_{t+1} \quad (4)$$

While the classical convergence theories for both methods rely on noiseless gradient estimates (i.e., not stochastic), with some care in practice they are both applicable to the stochastic setting. However, the theory predicts that any advantages in terms of asymptotic local rate of convergence will be lost (Orr, 1996; Wiegerinck et al., 1999), a result also confirmed in experiments (LeCun et al., 1998). For these reasons, interest in momentum methods diminished after they had received substantial attention in the 90's. And because of this apparent incompatibility with stochastic optimization, some authors even discourage using momentum or downplay its potential advantages (LeCun et al., 1998).

However, while local convergence is all that matters in terms of asymptotic convergence rates (and on certain very simple/shallow neural network optimization problems it may even dominate the total learning time), in practice, the "transient phase" of convergence (Darken & Moody, 1993), which occurs before fine local convergence sets in, seems to matter a lot more for optimizing deep neural networks. In this transient phase of learning, directions of reduction in the objective tend to persist across many successive gradient estimates and are not completely swamped by noise.

Although the transient phase of learning is most noticeable in training deep learning models, it is still noticeable in convex objectives. The convergence rate of stochastic gradient descent on smooth convex functions is given by $O(L/T + \sigma/\sqrt{T})$, where σ is the variance in the gradient estimate and L is the Lipshits coefficient of ∇f. In contrast, the convergence rate of an accelerated gradient method of Lan (2010) (which is related to but different from NAG, in that it combines Nesterov style momentum with dual averaging) is $O(L/T^2 + \sigma/\sqrt{T})$. Thus, for convex objectives, momentum-based methods will outperform SGD in the early or transient stages of the optimization where L/T is the dominant term. However, the two methods will be equally effective during the final stages of the optimization where $\sigma/\sqrt{T}$ is the dominant term (i.e., when the optimization problem resembles an estimation one).

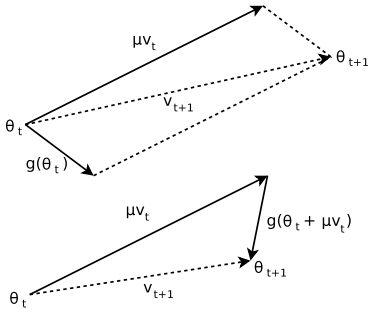


Figure 1. **(Top)** Classical Momentum **(Bottom)** Nesterov Accelerated Gradient

2.1. The Relationship between CM and NAG

From Eqs. 1-4 we see that both CM and NAG compute the new velocity by applying a gradient-based correction to the previous velocity vector (which is decayed), and then add the velocity to θ_t. But while CM computes the gradient update from the current position θ_t, NAG first performs a partial update to θ_t, computing $\theta_t + \mu v_t$, which is similar to θ_{t+1}, but missing the as yet unknown correction. This benign-looking difference seems to allow NAG to change v in a quicker and more responsive way, letting it behave more stably than CM in many situations, especially for higher values of μ.

Indeed, consider the situation where the addition of μv_t results in an immediate undesirable increase in the objective f. The gradient correction to the velocity v_t is computed at position $\theta_t + \mu v_t$ and if μv_t is indeed a poor update, then $\nabla f(\theta_t + \mu v_t)$ will point back towards θ_t more strongly than $\nabla f(\theta_t)$ does, thus providing a larger and more timely correction to v_t than CM. See fig. 1 for a diagram which illustrates this phenomenon geometrically. While each iteration of NAG may only be slightly more effective than CM at correcting a large and inappropriate velocity, this difference in effectiveness may compound as the algorithms iterate. To demonstrate this compounding, we applied both NAG and CM to a two-dimensional oblong quadratic objective, both with the same momentum and learning rate constants (see fig. 2 in the appendix). While the optimization path taken by CM exhibits large oscillations along the high-curvature vertical direction, NAG is able to avoid these oscillations almost entirely, confirming the intuition that it is much more effective than CM at decelerating over the course of multiple iterations, thus making NAG more tolerant of large values of μ compared to CM.

In order to make these intuitions more rigorous and

help quantify precisely the way in which CM and NAG differ, we analyzed the behavior of each method when applied to a positive definite quadratic objective $q(x) = x^\top Ax/2 + b^\top x$. We can think of CM and NAG as operating independently over the different eigendirections of A. NAG operates along any one of these directions equivalently to CM, except with an effective value of μ that is given by $\mu(1 - \lambda\varepsilon)$, where λ is the associated eigenvalue/curvature.

The first step of this argument is to reparameterize $q(x)$ in terms of the coefficients of x under the basis of eigenvectors of A. Note that since $A = U^\top DU$ for a diagonal D and orthonormal U (as A is symmetric), we can reparameterize $q(x)$ by the matrix transform U and optimize $y = Ux$ using the objective $p(y) \equiv q(x) = q(U^\top y) = y^\top U \left(U^\top DU\right) U^\top y/2 + b^\top U^\top y = y^\top Dy/2 + c^\top y$, where $c = Ub$. We can further rewrite p as $p(y) = \sum_{i=1}^n [p]_i([y]_i)$, where $[p]_i(t) = \lambda_i t^2/2 + [c]_i t$ and $\lambda_i > 0$ are the diagonal entries of D (and thus the eigenvalues of A) and correspond to the curvature along the associated eigenvector directions. As shown in the appendix (Proposition 6.1), both CM and NAG, being first-order methods, are "invariant" to these kinds of reparameterizations by orthonormal transformations such as U. Thus when analyzing the behavior of either algorithm applied to $q(x)$, we can instead apply them to $p(y)$, and transform the resulting sequence of iterates back to the default parameterization (via multiplication by $U^{-1} = U^\top$).

Theorem 2.1. *Let $p(y) = \sum_{i=1}^n [p]_i([y]_i)$ such that $[p]_i(t) = \lambda_i t^2/2 + c_i t$. Let ε be arbitrary and fixed. Denote by $CM_{\mathrm{x}}(\mu, p, y, v)$ and $CM_{\mathrm{v}}(\mu, p, y, v)$ the parameter vector and the velocity vector respectively, obtained by applying one step of CM (i.e., Eq. 1 and then Eq. 2) to the function p at point y, with velocity v, momentum coefficient μ, and learning rate ε. Define NAG_{x} and NAG_{v} analogously. Then the following holds for $\mathrm{z} \in \{\mathrm{x}, \mathrm{v}\}$:*

$$CM_{\mathrm{z}}(\mu, p, y, v) = \begin{bmatrix} CM_{\mathrm{z}}(\mu, [p]_1, [y]_1, [v]_1) \\ \vdots \\ CM_{\mathrm{z}}(\mu, [p]_n, [y]_n, [v]_n) \end{bmatrix}$$

$$NAG_{\mathrm{z}}(\mu, p, y, v) = \begin{bmatrix} CM_{\mathrm{z}}(\mu(1 - \lambda_1\varepsilon), [p]_1, [y]_1, [v]_1) \\ \vdots \\ CM_{\mathrm{z}}(\mu(1 - \lambda_n\varepsilon), [p]_n, [y]_n, [v]_n) \end{bmatrix}$$

Proof. See the appendix. □

The theorem has several implications. First, CM and NAG become equivalent when ε is small (when $\varepsilon\lambda \ll 1$ for every eigenvalue λ of A), so NAG and CM are distinct only when ε is reasonably large. When ε is relatively large, NAG uses smaller effective momentum for the high-curvature eigen-directions, which prevents oscillations (or divergence) and thus allows the use of a larger μ than is possible with CM for a given ε.

3. Deep Autoencoders

The aim of our experiments is three-fold. First, to investigate the attainable performance of stochastic momentum methods on deep autoencoders starting from well-designed random initializations; second, to explore the importance and effect of the schedule for the momentum parameter μ assuming an optimal fixed choice of the learning rate ε; and third, to compare the performance of NAG versus CM.

For our experiments with feed-forward nets, we focused on training the three deep autoencoder problems described in Hinton & Salakhutdinov (2006) (see sec. A.2 for details). The task of the neural network autoencoder is to reconstruct its own input subject to the constraint that one of its hidden layers is of low-dimension. This "bottleneck" layer acts as a low-dimensional code for the original input, similar to other dimensionality reduction techniques like Principle Component Analysis (PCA). These autoencoders are some of the deepest neural networks with published results, ranging between 7 and 11 layers, and have become a standard benchmarking problem (e.g., Martens, 2010; Glorot & Bengio, 2010; Chapelle & Erhan, 2011; Raiko et al., 2011). See the appendix for more details.

Because the focus of this study is on optimization, we only report training errors in our experiments. Test error depends strongly on the amount of overfitting in these problems, which in turn depends on the type and amount of regularization used during training. While regularization is an issue of vital importance when designing systems of practical utility, it is outside the scope of our discussion. And while it could be objected that the gains achieved using better optimization methods are only due to more exact fitting of the training set in a manner that does not generalize, this is simply not the case in these problems, where undertrained solutions are known to perform poorly on both the training and test sets (underfitting).

The networks we trained used the standard sigmoid nonlinearity and were initialized using the "sparse initialization" technique (SI) of Martens (2010) that is described in sec. 3.1. Each trial consists of 750,000 parameter updates on minibatches of size 200. No regularization is used. The schedule for μ was given by the following formula:

$$\mu_t = \min(1 - 2^{-1-\log_2(\lfloor t/250 \rfloor + 1)}, \mu_{\max}) \qquad (5)$$

where $\mu_{\max}$ was chosen from $\{0.999, 0.995, 0.99, 0.9, 0\}$. This schedule was motivated by Nesterov (1983) who advocates using what amounts to $\mu_t = 1 - 3/(t+5)$ after some manipulation

task	$0_{(\text{SGD})}$	0.9N	0.99N	0.995N	0.999N	0.9M	0.99M	0.995M	0.999M	SGD_C	$\text{HF}^\dagger$	HF^*
Curves	0.48	0.16	0.096	0.091	**0.074**	0.15	0.10	0.10	0.10	0.16	0.058	0.11
Mnist	2.1	1.0	**0.73**	0.75	0.80	1.0	0.77	0.84	0.90	0.9	0.69	1.40
Faces	36.4	14.2	8.5	7.8	**7.7**	15.3	8.7	8.3	9.3	NA	7.5	12.0

Table 1. The table reports the squared errors on the problems for each combination of $\mu_{\max}$ and a momentum type (NAG, CM). When $\mu_{\max}$ is 0 the choice of NAG vs CM is of no consequence so the training errors are presented in a single column. For each choice of $\mu_{\max}$, the highest-performing learning rate is used. The column SGD_C lists the results of Chapelle & Erhan (2011) who used 1.7M SGD steps and tanh networks. The column $HF^\dagger$ lists the results of HF without L2 regularization, as described in sec. 5; and the column HF^* lists the results of Martens (2010).

problem	before	after
Curves	0.096	0.074
Mnist	1.20	0.73
Faces	10.83	7.7

Table 2. The effect of low-momentum finetuning for NAG. The table shows the training squared errors before and after the momentum coefficient is reduced. During the primary ("transient") phase of learning we used the optimal momentum and learning rates.

(see appendix), and by Nesterov (2003) who advocates a constant μ_t that depends on (essentially) the condition number. The constant μ_t achieves exponential convergence on strongly convex functions, while the $1-3/(t+5)$ schedule is appropriate when the function is not strongly convex. The schedule of Eq. 5 blends these proposals. For each choice of $\mu_{\max}$, we report the learning rate that achieved the best training error. Given the schedule for μ, the learning rate ε was chosen from $\{0.05, 0.01, 0.005, 0.001, 0.0005, 0.0001\}$ in order to achieve the lowest final error training error after our fixed number of updates.

Table 1 summarizes the results of these experiments. It shows that NAG achieves the lowest published results on this set of problems, including those of Martens (2010). It also shows that larger values of $\mu_{\max}$ tend to achieve better performance and that NAG usually outperforms CM, especially when $\mu_{\max}$ is 0.995 and 0.999. Most surprising and importantly, the results demonstrate that NAG can achieve results that are comparable with some of the best HF results for training deep autoencoders. Note that the previously published results on HF used L2 regularization, so they cannot be directly compared. However, the table also includes experiments we performed with an improved version of HF (see sec. 2.1) where weight decay was removed towards the end of training.

We found it beneficial to reduce μ to 0.9 (unless μ is 0, in which case it is unchanged) during the final 1000 parameter updates of the optimization without reducing the learning rate, as shown in Table 2. It appears that reducing the momentum coefficient allows for finer convergence to take place whereas otherwise the overly aggressive nature of CM or NAG would prevent this. This phase shift between optimization that favors fast accelerated motion along the error surface (the "transient phase") followed by more careful optimization-as-estimation phase seems consistent with the picture presented by Darken & Moody (1993). However, while asymptotically it is the second phase which must eventually dominate computation time, in practice it seems that for deeper networks in particular, the first phase dominates overall computation time as long as the second phase is cut off before the remaining potential gains become either insignificant or entirely dominated by overfitting (or both).

It may be tempting then to use lower values of μ from the outset, or to reduce it immediately when progress in reducing the error appears to slow down. However, in our experiments we found that doing this was detrimental in terms of the final errors we could achieve, and that despite appearing to not make much progress, or even becoming significantly non-monotonic, the optimizers were doing something apparently useful over these extended periods of time at higher values of μ.

A speculative explanation as to why we see this behavior is as follows. While a large value of μ allows the momentum methods to make useful progress along slowly-changing directions of low-curvature, this may not immediately result in a significant reduction in error, due to the failure of these methods to converge in the more turbulent high-curvature directions (which is especially hard when μ is large). Nevertheless, this progress in low-curvature directions takes the optimizers to new regions of the parameter space that are characterized by closer proximity to the optimum (in the case of a convex objective), or just higher-quality local minimia (in the case of non-convex optimization). Thus, while it is important to adopt a more careful scheme that allows fine convergence to take place along the high-curvature directions, this must be done with care. Reducing μ and moving to this fine convergence regime too early may make it difficult for the optimization to make significant progress along the low-curvature directions, since without the benefit of momentum-based acceleration, first-order methods are notoriously bad at this (which is what motivated the use of second-order methods like HF for deep learning).

SI scale multiplier	0.25	0.5	1	2	4
error	16	16	0.074	0.083	0.35

Table 3. The table reports the training squared error that is attained by changing the scale of the initialization.

3.1. Random Initializations

The results in the previous section were obtained with standard logistic sigmoid neural networks that were initialized with the sparse initialization technique (SI) described in Martens (2010). In this scheme, each random unit is connected to 15 randomly chosen units in the previous layer, whose weights are drawn from a unit Gaussian, and the biases are set to zero. The intuitive justification is that the total amount of input to each unit will not depend on the size of the previous layer and hence they will not as easily saturate. Meanwhile, because the inputs to each unit are not all randomly weighted blends of the outputs of many 100s or 1000s of units in the previous layer, they will tend to be qualitatively more "diverse" in their response to inputs. When using tanh units, we transform the weights to simulate sigmoid units by setting the biases to 0.5 and rescaling the weights by 0.25.

We investigated the performance of the optimization as a function of the scale constant used in SI (which defaults to 1 for sigmoid units). We found that SI works reasonably well if it is rescaled by a factor of 2, but leads to noticeable (but not severe) slow down when scaled by a factor of 3. When we used the factor 1/2 or 5 we did not achieve sensible results.

4. Recurrent Neural Networks

Echo-State Networks (ESNs) is a family of RNNs with an unusually simple training method: their hidden-to-output connections are learned from data, but their recurrent connections are fixed to a random draw from a specific distribution and are not learned. Despite their simplicity, ESNs with many hidden units (or with units with explicit temporal integration, like the LSTM) have achieved high performance on tasks with long range dependencies (**?**). In this section, we investigate the effectiveness of momentum-based methods with ESN-inspired initialization at training RNNs with conventional size and standard (i.e., non-integrating) neurons. We find that momentum-accelerated SGD can successfully train such RNNs on various artificial datasets exhibiting considerable long-range temporal dependencies. This is unexpected because RNNs were believed to be almost impossible to successfully train on such datasets with first-order methods, due to various difficulties such as vanishing/exploding gradients (Bengio et al., 1994). While we found that the use of momentum significantly improved performance and robustness, we obtained nontrivial results even with standard SGD, provided that the learning rate was set low enough.

connection type	sparsity	scale
in-to-hid (add,mul)	dense	$0.001 \cdot N(0, 1)$
in-to-hid (mem)	dense	$0.1 \cdot N(0, 1)$
hid-to-hid	15 fan-in	spectral radius of 1.1
hid-to-out	dense	$0.1 \cdot N(0, 1)$
hid-bias	dense	0
out-bias	dense	average of outputs

Table 4. The RNN initialization used in the experiments. The scale of the vis-hid connections is problem dependent.

Each task involved optimizing the parameters of a randomly initialized RNN with 100 standard tanh hidden units (the same model used by Martens & Sutskever (2011)). The tasks were designed by Hochreiter & Schmidhuber (1997), and are referred to as training "problems". See sec. A.3 of the appendix for details.

4.1. ESN-based Initialization

As argued by Jaeger & Haas (2004), the spectral radius of the hidden-to-hidden matrix has a profound effect on the dynamics of the RNN's hidden state (with a tanh nonlinearity). When it is smaller than 1, the dynamics will have a tendency to quickly "forget" whatever input signal they may have been exposed to. When it is much larger than 1, the dynamics become oscillatory and chaotic, allowing it to generate responses that are varied for different input histories. While this allows information to be retained over many time steps, it can also lead to severe exploding gradients that make gradient-based learning much more difficult. However, when the spectral radius is only slightly greater than 1, the dynamics remain oscillatory and chaotic while the gradient are no longer exploding (and if they do explode, then only "slightly so"), so learning may be possible with a spectral radius of this order. This suggests that a spectral radius of around 1.1 may be effective.

To achieve robust results, we also found it is essential to carefully set the initial scale of the input-to-hidden connections. When training RNNs to solve those tasks that possess many random and irrelevant distractor inputs, we found that having the scale of these connections set too high at the start led to relevant information in the hidden state being too easily "overwritten" by the many irrelevant signals, which ultimately led the optimizer to converge towards an extremely poor local minimum where useful information was never relayed over long distances. Conversely, we found that if this scale was set too low, it led to significantly slower learning. Having experimented with multiple scales we found that a Gaussian draw with a standard deviation of 0.001 achieved a good balance between these concerns. However, unlike the value of 1.1 for the spectral radius of the dynamics matrix, which worked well on all tasks, we found that good choices for initial scale of the input-to-hidden weights depended a lot on the particular characteristics of the particular task (such

as its dimensionality or the input variance). Indeed, for tasks that do not have many irrelevant inputs, a larger scale of the input-to-hidden weights (namely, 0.1) worked better, because the aforementioned disadvantage of large input-to-hidden weights does not apply. See table 4 for a summary of the initializations used in the experiments. Finally, we found centering (mean subtraction) of both the inputs and the outputs to be important to reliably solve all of the training problems. See the appendix for more details.

4.2. Experimental Results

We conducted experiments to determine the efficacy of our initializations, the effect of momentum, and to compare NAG with CM. Every learning trial used the aforementioned initialization, 50,000 parameter updates and on minibatches of 100 sequences, and the following schedule for the momentum coefficient μ: $\mu = 0.9$ for the first 1000 parameter, after which $\mu = \mu_0$, where μ_0 can take the following values $\{0, 0.9, 0.98, 0.995\}$. For each μ_0, we use the empirically best learning rate chosen from $\{10^{-3}, 10^{-4}, 10^{-5}, 10^{-6}\}$.

The results are presented in Table 5, which are the average loss over 4 different random seeds. Instead of reporting the loss being minimized (which is the squared error or cross entropy), we use a more interpretable zero-one loss, as is standard practice with these problems. For the bit memorization, we report the fraction of timesteps that are computed incorrectly. And for the addition and the multiplication problems, we report the fraction of cases where the RNN the error in the final output prediction exceeded 0.04.

Our results show that despite the considerable long-range dependencies present in training data for these problems, RNNs can be successfully and robustly trained to solve them, through the use of the initialization discussed in sec. 4.1, momentum of the NAG type, a large μ_0, and a particularly small learning rate (as compared with feedforward networks). Our results also suggest that with larger values of μ_0 achieve better results with NAG but not with CM, possibly due to NAG's tolerance of larger μ_0's (as discussed in sec. 2).

Although we were able to achieve surprisingly good training performance on these problems using a sufficiently strong momentum, the results of Martens & Sutskever (2011) appear to be moderately better and more robust. They achieved lower error rates and their initialization was chosen with less care, although the initializations are in many ways similar to ours. Notably, Martens & Sutskever (2011) were able to solve these problems without centering, while we had to use centering to solve the multiplication problem (the other problems are already centered). This suggests that the initialization proposed here, together with the method of Martens & Sutskever (2011), could achieve even better performance. But the main achievement of these results is a demonstration of the ability of momentum methods to cope with long-range temporal dependency training tasks to a level which seems sufficient for most practical purposes. Moreover, our approach seems to be more tolerant of smaller minibatches, and is considerably simpler than the particular version of HF proposed in Martens & Sutskever (2011), which used a specialized update damping technique whose benefits seemed mostly limited to training RNNs to solve these kinds of extreme temporal dependency problems.

5. Momentum and HF

Truncated Newton methods, that include the HF method of Martens (2010) as a particular example, work by optimizing a local quadratic model of the objective via the linear conjugate gradient algorithm (CG), which is a first-order method. While HF, like all truncated-Newton methods, takes steps computed using partially converged calls to CG, it is naturally accelerated along at least some directions of lower curvature compared to the gradient. It can even be shown (Martens & Sutskever, 2012) that CG will tend to favor convergence to the exact solution to the quadratic sub-problem first along higher curvature directions (with a bias towards those which are more clustered together in their curvature-scalars/eigenvalues).

While CG accumulates information as it iterates which allows it to be optimal in a much stronger sense than any other first-order method (like NAG), once it is terminated, this information is lost. Thus, standard truncated Newton methods can be thought of as persisting information which accelerates convergence (of the current quadratic) only over the number of iterations CG performs. By contrast, momentum methods persist information that can inform new updates across an arbitrary number of iterations.

One key difference between standard truncated Newton methods and HF is the use of "hot-started" calls to CG, which use as their initial solution the one found at the previous call to CG. While this solution was computed using old gradient and curvature information from a previous point in parameter space and possibly a different set of training data, it may be well-converged along certain eigen-directions of the new quadratic, despite being very poorly converged along others (perhaps worse than the default initial solution of $\vec{0}$). However, to the extent to which the new local quadratic model resembles the old one, and in particular in the more difficult to optimize directions of low-curvature (which will arguably be more likely to persist across nearby locations in parameter space), the previous solution will be a preferable starting point to 0, and may even allow for gradually increasing levels of convergence along certain directions which persist

problem	biases	0	0.9N	0.98N	0.995N	0.9M	0.98M	0.995M
add $T = 80$	0.82	0.39	0.02	0.21	**0.00025**	0.43	0.62	0.036
mul $T = 80$	0.84	0.48	0.36	0.22	**0.0013**	0.029	0.025	0.37
mem-5 $T = 200$	2.5	1.27	1.02	0.96	**0.63**	1.12	1.09	0.92
mem-20 $T = 80$	8.0	5.37	2.77	0.0144	**0.00005**	1.75	0.0017	0.053

Table 5. Each column reports the errors (zero-one losses; sec. 4.2) on different problems for each combination of μ_0 and momentum type (NAG, CM), averaged over 4 different random seeds. The "biases" column lists the error attainable by learning the output biases and ignoring the hidden state. This is the error of an RNN that failed to "establish communication" between its inputs and targets. For each μ_0, we used the fixed learning rate that gave the best performance.

in the local quadratic models across many updates.

The connection between HF and momentum methods can be made more concrete by noticing that a single step of CG is effectively a gradient update taken from the current point, plus the previous update reapplied, just as with NAG, and that if CG terminated after just 1 step, HF becomes equivalent to NAG, except that it uses a special formula based on the curvature matrix for the learning rate instead of a fixed constant. The most effective implementations of HF even employ a "decay" constant (Martens & Sutskever, 2012) which acts analogously to the momentum constant μ. Thus, in this sense, the CG initializations used by HF allow us to view it as a hybrid of NAG and an exact second-order method, with the number of CG iterations used to compute each update effectively acting as a dial between the two extremes.

Inspired by the surprising success of momentum-based methods for deep learning problems, we experimented with making HF behave even more like NAG than it already does. The resulting approach performed surprisingly well (see Table 1). For a more detailed account of these experiments, see sec. A.6 of the appendix.

If viewed on the basis of each CG step (instead of each update to parameters θ), HF can be thought of as a peculiar type of first-order method which approximates the objective as a series of quadratics only so that it can make use of the powerful first-order CG method. So apart from any potential benefit to global convergence from its tendency to prefer certain directions of movement in parameter space over others, perhaps the main theoretical benefit to using HF over a first-order method like NAG is its use of CG, which, while itself a first-order method, is well known to have strongly optimal convergence properties for quadratics, and can take advantage of clustered eigenvalues to accelerate convergence (see Martens & Sutskever (2012) for a detailed account of this well-known phenomenon). However, it is known that in the worst case that CG, when run in batch mode, will converge asymptotically no faster than NAG (also run in batch mode) for certain specially designed quadratics with very evenly distributed eigenvalues/curvatures. Thus it is worth asking whether the quadratics which arise during the optimization of neural networks by HF are such that CG has a distinct advantage in optimizing them over NAG, or if they are closer to the aforementioned worst-case examples. To examine this question we took a quadratic generated during the middle of a typical run of HF on the curves dataset and compared the convergence rate of CG, initialized from zero, to NAG (also initialized from zero). Figure 5 in the appendix presents the results of this experiment. While this experiment indicates some potential advantages to HF, the closeness of the performance of NAG and HF suggests that these results might be explained by the solutions leaving the area of trust in the quadratics before any extra speed kicks in, or more subtly, that the faithfulness of approximation goes down just enough as CG iterates to offset the benefit of the acceleration it provides.

6. Discussion

Martens (2010) and Martens & Sutskever (2011) demonstrated the effectiveness of the HF method as a tool for performing optimizations for which previous attempts to apply simpler first-order methods had failed. While some recent work (Chapelle & Erhan, 2011; Glorot & Bengio, 2010) suggested that first-order methods can actually achieve some success on these kinds of problems when used in conjunction with good initializations, their results still fell short of those reported for HF. In this paper we have completed this picture and demonstrated conclusively that a large part of the remaining performance gap that is not addressed by using a well-designed random initialization is in fact addressed by careful use of momentum-based acceleration (possibly of the Nesterov type). We showed that careful attention must be paid to the momentum constant μ, as predicted by the theory for local and convex optimization.

Momentum-accelerated SGD, despite being a first-order approach, is capable of accelerating directions of low-curvature just like an approximate Newton method such as HF. Our experiments support the idea that this is important, as we observed that the use of stronger momentum (as determined by μ) had a dramatic effect on optimization performance, particularly for the RNNs. Moreover, we showed that HF can be viewed as a first-order method, and as a generalization of NAG in particular, and that it already derives some of its benefits through a momentum-like mechanism.

ADAM: A METHOD FOR STOCHASTIC OPTIMIZATION

Diederik P. Kingma*
University of Amsterdam, OpenAI
dpkingma@openai.com

Jimmy Lei Ba*
University of Toronto
jimmy@psi.utoronto.ca

ABSTRACT

We introduce *Adam*, an algorithm for first-order gradient-based optimization of stochastic objective functions, based on adaptive estimates of lower-order moments. The method is straightforward to implement, is computationally efficient, has little memory requirements, is invariant to diagonal rescaling of the gradients, and is well suited for problems that are large in terms of data and/or parameters. The method is also appropriate for non-stationary objectives and problems with very noisy and/or sparse gradients. The hyper-parameters have intuitive interpretations and typically require little tuning. Some connections to related algorithms, on which *Adam* was inspired, are discussed. We also analyze the theoretical convergence properties of the algorithm and provide a regret bound on the convergence rate that is comparable to the best known results under the online convex optimization framework. Empirical results demonstrate that Adam works well in practice and compares favorably to other stochastic optimization methods. Finally, we discuss *AdaMax*, a variant of *Adam* based on the infinity norm.

1 INTRODUCTION

Stochastic gradient-based optimization is of core practical importance in many fields of science and engineering. Many problems in these fields can be cast as the optimization of some scalar parameterized objective function requiring maximization or minimization with respect to its parameters. If the function is differentiable w.r.t. its parameters, gradient descent is a relatively efficient optimization method, since the computation of first-order partial derivatives w.r.t. all the parameters is of the same computational complexity as just evaluating the function. Often, objective functions are stochastic. For example, many objective functions are composed of a sum of subfunctions evaluated at different subsamples of data; in this case optimization can be made more efficient by taking gradient steps w.r.t. individual subfunctions, i.e. stochastic gradient descent (SGD) or ascent. SGD proved itself as an efficient and effective optimization method that was central in many machine learning success stories, such as recent advances in deep learning (Deng et al., 2013; Krizhevsky et al., 2012; Hinton & Salakhutdinov, 2006; Hinton et al., 2012a; Graves et al., 2013). Objectives may also have other sources of noise than data subsampling, such as dropout (Hinton et al., 2012b) regularization. For all such noisy objectives, efficient stochastic optimization techniques are required. The focus of this paper is on the optimization of stochastic objectives with high-dimensional parameters spaces. In these cases, higher-order optimization methods are ill-suited, and discussion in this paper will be restricted to first-order methods.

We propose *Adam*, a method for efficient stochastic optimization that only requires first-order gradients with little memory requirement. The method computes individual adaptive learning rates for different parameters from estimates of first and second moments of the gradients; the name *Adam* is derived from adaptive moment estimation. Our method is designed to combine the advantages of two recently popular methods: AdaGrad (Duchi et al., 2011), which works well with sparse gradients, and RMSProp (Tieleman & Hinton, 2012), which works well in on-line and non-stationary settings; important connections to these and other stochastic optimization methods are clarified in section 5. Some of Adam's advantages are that the magnitudes of parameter updates are invariant to rescaling of the gradient, its stepsizes are approximately bounded by the stepsize hyperparameter, it does not require a stationary objective, it works with sparse gradients, and it naturally performs a form of step size annealing.

*Equal contribution. Author ordering determined by coin flip over a Google Hangout.

Algorithm 1: *Adam*, our proposed algorithm for stochastic optimization. See section 2 for details, and for a slightly more efficient (but less clear) order of computation. g_t^2 indicates the elementwise square $g_t \odot g_t$. Good default settings for the tested machine learning problems are $\alpha = 0.001$, $\beta_1 = 0.9$, $\beta_2 = 0.999$ and $\epsilon = 10^{-8}$. All operations on vectors are element-wise. With β_1^t and β_2^t we denote β_1 and β_2 to the power t.

Require: α: Stepsize
Require: $\beta_1, \beta_2 \in [0, 1)$: Exponential decay rates for the moment estimates
Require: $f(\theta)$: Stochastic objective function with parameters θ
Require: θ_0: Initial parameter vector
  $m_0 \leftarrow 0$ (Initialize 1st moment vector)
  $v_0 \leftarrow 0$ (Initialize 2nd moment vector)
  $t \leftarrow 0$ (Initialize timestep)
  while θ_t not converged **do**
    $t \leftarrow t + 1$
    $g_t \leftarrow \nabla_\theta f_t(\theta_{t-1})$ (Get gradients w.r.t. stochastic objective at timestep t)
    $m_t \leftarrow \beta_1 \cdot m_{t-1} + (1 - \beta_1) \cdot g_t$ (Update biased first moment estimate)
    $v_t \leftarrow \beta_2 \cdot v_{t-1} + (1 - \beta_2) \cdot g_t^2$ (Update biased second raw moment estimate)
    $\widehat{m}_t \leftarrow m_t / (1 - \beta_1^t)$ (Compute bias-corrected first moment estimate)
    $\widehat{v}_t \leftarrow v_t / (1 - \beta_2^t)$ (Compute bias-corrected second raw moment estimate)
    $\theta_t \leftarrow \theta_{t-1} - \alpha \cdot \widehat{m}_t / (\sqrt{\widehat{v}_t} + \epsilon)$ (Update parameters)
  end while
  return θ_t (Resulting parameters)

In section 2 we describe the algorithm and the properties of its update rule. Section 3 explains our initialization bias correction technique, and section 4 provides a theoretical analysis of Adam's convergence in online convex programming. Empirically, our method consistently outperforms other methods for a variety of models and datasets, as shown in section 6. Overall, we show that Adam is a versatile algorithm that scales to large-scale high-dimensional machine learning problems.

2 ALGORITHM

See algorithm 1 for pseudo-code of our proposed algorithm *Adam*. Let $f(\theta)$ be a noisy objective function: a stochastic scalar function that is differentiable w.r.t. parameters θ. We are interested in minimizing the expected value of this function, $\mathbb{E}[f(\theta)]$ w.r.t. its parameters θ. With $f_1(\theta), ..., f_T(\theta)$ we denote the realisations of the stochastic function at subsequent timesteps $1, ..., T$. The stochasticity might come from the evaluation at random subsamples (minibatches) of datapoints, or arise from inherent function noise. With $g_t = \nabla_\theta f_t(\theta)$ we denote the gradient, i.e. the vector of partial derivatives of f_t, w.r.t θ evaluated at timestep t.

The algorithm updates exponential moving averages of the gradient (m_t) and the squared gradient (v_t) where the hyper-parameters $\beta_1, \beta_2 \in [0, 1)$ control the exponential decay rates of these moving averages. The moving averages themselves are estimates of the 1st moment (the mean) and the 2nd raw moment (the uncentered variance) of the gradient. However, these moving averages are initialized as (vectors of) 0's, leading to moment estimates that are biased towards zero, especially during the initial timesteps, and especially when the decay rates are small (i.e. the βs are close to 1). The good news is that this initialization bias can be easily counteracted, resulting in bias-corrected estimates $\widehat{m}_t$ and $\widehat{v}_t$. See section 3 for more details.

Note that the efficiency of algorithm 1 can, at the expense of clarity, be improved upon by changing the order of computation, e.g. by replacing the last three lines in the loop with the following lines: $\alpha_t = \alpha \cdot \sqrt{1 - \beta_2^t}/(1 - \beta_1^t)$ and $\theta_t \leftarrow \theta_{t-1} - \alpha_t \cdot m_t / (\sqrt{v_t} + \hat{\epsilon})$.

2.1 ADAM'S UPDATE RULE

An important property of Adam's update rule is its careful choice of stepsizes. Assuming $\epsilon = 0$, the effective step taken in parameter space at timestep t is $\Delta_t = \alpha \cdot \widehat{m}_t / \sqrt{\widehat{v}_t}$. The effective stepsize has two upper bounds: $|\Delta_t| \leq \alpha \cdot (1 - \beta_1)/\sqrt{1 - \beta_2}$ in the case $(1 - \beta_1) > \sqrt{1 - \beta_2}$, and $|\Delta_t| \leq \alpha$

otherwise. The first case only happens in the most severe case of sparsity: when a gradient has been zero at all timesteps except at the current timestep. For less sparse cases, the effective stepsize will be smaller. When $(1-\beta_1) = \sqrt{1-\beta_2}$ we have that $|\widehat{m}_t/\sqrt{\widehat{v}_t}| < 1$ therefore $|\Delta_t| < \alpha$. In more common scenarios, we will have that $\widehat{m}_t/\sqrt{\widehat{v}_t} \approx \pm 1$ since $|\mathbb{E}[g]/\sqrt{\mathbb{E}[g^2]}| \leq 1$. The effective magnitude of the steps taken in parameter space at each timestep are approximately bounded by the stepsize setting α, i.e., $|\Delta_t| \lessapprox \alpha$. This can be understood as establishing a *trust region* around the current parameter value, beyond which the current gradient estimate does not provide sufficient information. This typically makes it relatively easy to know the right scale of α in advance. For many machine learning models, for instance, we often know in advance that good optima are with high probability within some set region in parameter space; it is not uncommon, for example, to have a prior distribution over the parameters. Since α sets (an upper bound of) the magnitude of steps in parameter space, we can often deduce the right order of magnitude of α such that optima can be reached from θ_0 within some number of iterations. With a slight abuse of terminology, we will call the ratio $\widehat{m}_t/\sqrt{\widehat{v}_t}$ the *signal-to-noise* ratio (SNR). With a smaller SNR the effective stepsize Δ_t will be closer to zero. This is a desirable property, since a smaller SNR means that there is greater uncertainty about whether the direction of $\widehat{m}_t$ corresponds to the direction of the true gradient. For example, the SNR value typically becomes closer to 0 towards an optimum, leading to smaller effective steps in parameter space: a form of automatic annealing. The effective stepsize Δ_t is also invariant to the scale of the gradients; rescaling the gradients g with factor c will scale $\widehat{m}_t$ with a factor c and $\widehat{v}_t$ with a factor c^2, which cancel out: $(c \cdot \widehat{m}_t)/(\sqrt{c^2 \cdot \widehat{v}_t}) = \widehat{m}_t/\sqrt{\widehat{v}_t}$.

3 INITIALIZATION BIAS CORRECTION

As explained in section 2, Adam utilizes initialization bias correction terms. We will here derive the term for the second moment estimate; the derivation for the first moment estimate is completely analogous. Let g be the gradient of the stochastic objective f, and we wish to estimate its second raw moment (uncentered variance) using an exponential moving average of the squared gradient, with decay rate β_2. Let $g_1, ..., g_T$ be the gradients at subsequent timesteps, each a draw from an underlying gradient distribution $g_t \sim p(g_t)$. Let us initialize the exponential moving average as $v_0 = 0$ (a vector of zeros). First note that the update at timestep t of the exponential moving average $v_t = \beta_2 \cdot v_{t-1} + (1-\beta_2) \cdot g_t^2$ (where g_t^2 indicates the elementwise square $g_t \odot g_t$) can be written as a function of the gradients at all previous timesteps:

$$v_t = (1-\beta_2) \sum_{i=1}^{t} \beta_2^{t-i} \cdot g_i^2 \tag{1}$$

We wish to know how $\mathbb{E}[v_t]$, the expected value of the exponential moving average at timestep t, relates to the true second moment $\mathbb{E}[g_t^2]$, so we can correct for the discrepancy between the two. Taking expectations of the left-hand and right-hand sides of eq. (1):

$$\mathbb{E}[v_t] = \mathbb{E}\left[(1-\beta_2) \sum_{i=1}^{t} \beta_2^{t-i} \cdot g_i^2\right] \tag{2}$$

$$= \mathbb{E}[g_t^2] \cdot (1-\beta_2) \sum_{i=1}^{t} \beta_2^{t-i} + \zeta \tag{3}$$

$$= \mathbb{E}[g_t^2] \cdot (1-\beta_2^t) + \zeta \tag{4}$$

where $\zeta = 0$ if the true second moment $\mathbb{E}[g_i^2]$ is stationary; otherwise ζ can be kept small since the exponential decay rate β_1 can (and should) be chosen such that the exponential moving average assigns small weights to gradients too far in the past. What is left is the term $(1-\beta_2^t)$ which is caused by initializing the running average with zeros. In algorithm 1 we therefore divide by this term to correct the initialization bias.

In case of sparse gradients, for a reliable estimate of the second moment one needs to average over many gradients by chosing a small value of β_2; however it is exactly this case of small β_2 where a lack of initialisation bias correction would lead to initial steps that are much larger.

4 CONVERGENCE ANALYSIS

We analyze the convergence of Adam using the online learning framework proposed in (Zinkevich, 2003). Given an arbitrary, unknown sequence of convex cost functions $f_1(\theta)$, $f_2(\theta)$,..., $f_T(\theta)$. At each time t, our goal is to predict the parameter θ_t and evaluate it on a previously unknown cost function f_t. Since the nature of the sequence is unknown in advance, we evaluate our algorithm using the regret, that is the sum of all the previous difference between the online prediction $f_t(\theta_t)$ and the best fixed point parameter $f_t(\theta^*)$ from a feasible set $\mathcal{X}$ for all the previous steps. Concretely, the regret is defined as:

$$R(T) = \sum_{t=1}^{T} [f_t(\theta_t) - f_t(\theta^*)] \tag{5}$$

where $\theta^* = \arg\min_{\theta \in \mathcal{X}} \sum_{t=1}^{T} f_t(\theta)$. We show Adam has $O(\sqrt{T})$ regret bound and a proof is given in the appendix. Our result is comparable to the best known bound for this general convex online learning problem. We also use some definitions simplify our notation, where $g_t \triangleq \nabla f_t(\theta_t)$ and $g_{t,i}$ as the i^{th} element. We define $g_{1:t,i} \in \mathbb{R}^t$ as a vector that contains the i^{th} dimension of the gradients over all iterations till t, $g_{1:t,i} = [g_{1,i}, g_{2,i}, \cdots, g_{t,i}]$. Also, we define $\gamma \triangleq \frac{\beta_1^2}{\sqrt{\beta_2}}$. Our following theorem holds when the learning rate α_t is decaying at a rate of $t^{-\frac{1}{2}}$ and first moment running average coefficient $\beta_{1,t}$ decay exponentially with λ, that is typically close to 1, e.g. $1 - 10^{-8}$.

Theorem 4.1. *Assume that the function f_t has bounded gradients, $\|\nabla f_t(\theta)\|_2 \leq G$, $\|\nabla f_t(\theta)\|_\infty \leq G_\infty$ for all $\theta \in R^d$ and distance between any θ_t generated by Adam is bounded, $\|\theta_n - \theta_m\|_2 \leq D$, $\|\theta_m - \theta_n\|_\infty \leq D_\infty$ for any $m, n \in \{1, ..., T\}$, and β_1, $\beta_2 \in [0, 1)$ satisfy $\frac{\beta_1^2}{\sqrt{\beta_2}} < 1$. Let $\alpha_t = \frac{\alpha}{\sqrt{t}}$ and $\beta_{1,t} = \beta_1 \lambda^{t-1}$, $\lambda \in (0, 1)$. Adam achieves the following guarantee, for all $T \geq 1$.*

$$R(T) \leq \frac{D^2}{2\alpha(1-\beta_1)} \sum_{i=1}^{d} \sqrt{T\widehat{v}_{T,i}} + \frac{\alpha(1+\beta_1)G_\infty}{(1-\beta_1)\sqrt{1-\beta_2}(1-\gamma)^2} \sum_{i=1}^{d} \|g_{1:T,i}\|_2 + \sum_{i=1}^{d} \frac{D_\infty^2 G_\infty \sqrt{1-\beta_2}}{2\alpha(1-\beta_1)(1-\lambda)^2}$$

Our Theorem 4.1 implies when the data features are sparse and bounded gradients, the summation term can be much smaller than its upper bound $\sum_{i=1}^{d} \|g_{1:T,i}\|_2 << dG_\infty\sqrt{T}$ and $\sum_{i=1}^{d} \sqrt{T\widehat{v}_{T,i}} << dG_\infty\sqrt{T}$, in particular if the class of function and data features are in the form of section 1.2 in (Duchi et al., 2011). Their results for the expected value $\mathbb{E}[\sum_{i=1}^{d} \|g_{1:T,i}\|_2]$ also apply to Adam. In particular, the adaptive method, such as Adam and Adagrad, can achieve $O(\log d\sqrt{T})$, an improvement over $O(\sqrt{dT})$ for the non-adaptive method. Decaying $\beta_{1,t}$ towards zero is important in our theoretical analysis and also matches previous empirical findings, e.g. (Sutskever et al., 2013) suggests reducing the momentum coefficient in the end of training can improve convergence.

Finally, we can show the average regret of Adam converges,

Corollary 4.2. *Assume that the function f_t has bounded gradients, $\|\nabla f_t(\theta)\|_2 \leq G$, $\|\nabla f_t(\theta)\|_\infty \leq G_\infty$ for all $\theta \in R^d$ and distance between any θ_t generated by Adam is bounded, $\|\theta_n - \theta_m\|_2 \leq D$, $\|\theta_m - \theta_n\|_\infty \leq D_\infty$ for any $m, n \in \{1, ..., T\}$. Adam achieves the following guarantee, for all $T \geq 1$.*

$$\frac{R(T)}{T} = O(\frac{1}{\sqrt{T}})$$

This result can be obtained by using Theorem 4.1 and $\sum_{i=1}^{d} \|g_{1:T,i}\|_2 \leq dG_\infty\sqrt{T}$. Thus, $\lim_{T\to\infty} \frac{R(T)}{T} = 0$.

5 RELATED WORK

Optimization methods bearing a direct relation to Adam are RMSProp (Tieleman & Hinton, 2012; Graves, 2013) and AdaGrad (Duchi et al., 2011); these relationships are discussed below. Other stochastic optimization methods include vSGD (Schaul et al., 2012), AdaDelta (Zeiler, 2012) and the natural Newton method from Roux & Fitzgibbon (2010), all setting stepsizes by estimating curvature

from first-order information. The Sum-of-Functions Optimizer (SFO) (Sohl-Dickstein et al., 2014) is a quasi-Newton method based on minibatches, but (unlike Adam) has memory requirements linear in the number of minibatch partitions of a dataset, which is often infeasible on memory-constrained systems such as a GPU. Like natural gradient descent (NGD) (Amari, 1998), Adam employs a preconditioner that adapts to the geometry of the data, since $\widehat{v}_t$ is an approximation to the diagonal of the Fisher information matrix (Pascanu & Bengio, 2013); however, Adam's preconditioner (like AdaGrad's) is more conservative in its adaption than vanilla NGD by preconditioning with the square root of the inverse of the diagonal Fisher information matrix approximation.

RMSProp: An optimization method closely related to Adam is RMSProp (Tieleman & Hinton, 2012). A version with momentum has sometimes been used (Graves, 2013). There are a few important differences between RMSProp with momentum and Adam: RMSProp with momentum generates its parameter updates using a momentum on the rescaled gradient, whereas Adam updates are directly estimated using a running average of first and second moment of the gradient. RMSProp also lacks a bias-correction term; this matters most in case of a value of β_2 close to 1 (required in case of sparse gradients), since in that case not correcting the bias leads to very large stepsizes and often divergence, as we also empirically demonstrate in section 6.4.

AdaGrad: An algorithm that works well for sparse gradients is AdaGrad (Duchi et al., 2011). Its basic version updates parameters as $\theta_{t+1} = \theta_t - \alpha \cdot g_t / \sqrt{\sum_{i=1}^{t} g_t^2}$. Note that if we choose β_2 to be infinitesimally close to 1 from below, then $\lim_{\beta_2 \to 1} \widehat{v}_t = t^{-1} \cdot \sum_{i=1}^{t} g_t^2$. AdaGrad corresponds to a version of Adam with $\beta_1 = 0$, infinitesimal $(1 - \beta_2)$ and a replacement of α by an annealed version $\alpha_t = \alpha \cdot t^{-1/2}$, namely $\theta_t - \alpha \cdot t^{-1/2} \cdot \widehat{m}_t / \sqrt{\lim_{\beta_2 \to 1} \widehat{v}_t} = \theta_t - \alpha \cdot t^{-1/2} \cdot g_t / \sqrt{t^{-1} \cdot \sum_{i=1}^{t} g_t^2} = \theta_t - \alpha \cdot g_t / \sqrt{\sum_{i=1}^{t} g_t^2}$. Note that this direct correspondence between Adam and Adagrad does not hold when removing the bias-correction terms; without bias correction, like in RMSProp, a β_2 infinitesimally close to 1 would lead to infinitely large bias, and infinitely large parameter updates.

6 EXPERIMENTS

To empirically evaluate the proposed method, we investigated different popular machine learning models, including logistic regression, multilayer fully connected neural networks and deep convolutional neural networks. Using large models and datasets, we demonstrate Adam can efficiently solve practical deep learning problems.

We use the same parameter initialization when comparing different optimization algorithms. The hyper-parameters, such as learning rate and momentum, are searched over a dense grid and the results are reported using the best hyper-parameter setting.

6.1 EXPERIMENT: LOGISTIC REGRESSION

We evaluate our proposed method on L2-regularized multi-class logistic regression using the MNIST dataset. Logistic regression has a well-studied convex objective, making it suitable for comparison of different optimizers without worrying about local minimum issues. The stepsize α in our logistic regression experiments is adjusted by $1/\sqrt{t}$ decay, namely $\alpha_t = \frac{\alpha}{\sqrt{t}}$ that matches with our theoratical prediction from section 4. The logistic regression classifies the class label directly on the 784 dimension image vectors. We compare Adam to accelerated SGD with Nesterov momentum and Adagrad using minibatch size of 128. According to Figure 1, we found that the Adam yields similar convergence as SGD with momentum and both converge faster than Adagrad.

As discussed in (Duchi et al., 2011), Adagrad can efficiently deal with sparse features and gradients as one of its main theoretical results whereas SGD is low at learning rare features. Adam with $1/\sqrt{t}$ decay on its stepsize should theoratically match the performance of Adagrad. We examine the sparse feature problem using IMDB movie review dataset from (Maas et al., 2011). We pre-process the IMDB movie reviews into bag-of-words (BoW) feature vectors including the first 10,000 most frequent words. The 10,000 dimension BoW feature vector for each review is highly sparse. As suggested in (Wang & Manning, 2013), 50% dropout noise can be applied to the BoW features during

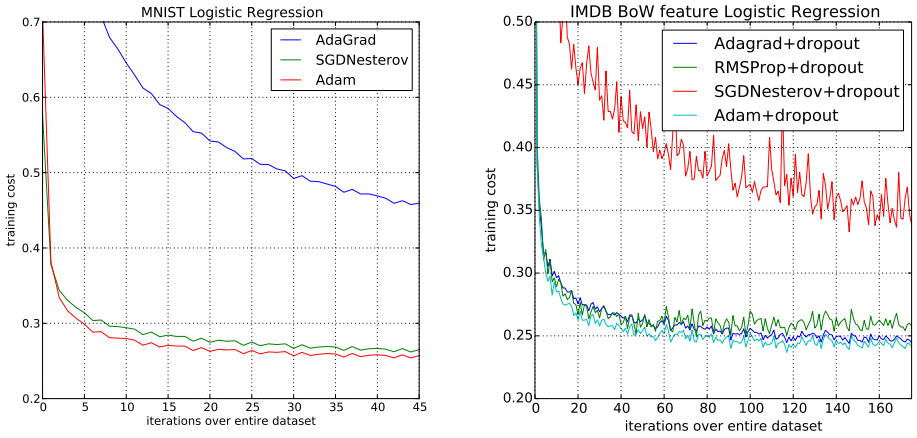


Figure 1: Logistic regression training negative log likelihood on MNIST images and IMDB movie reviews with 10,000 bag-of-words (BoW) feature vectors.

training to prevent over-fitting. In figure 1, Adagrad outperforms SGD with Nesterov momentum by a large margin both with and without dropout noise. Adam converges as fast as Adagrad. The empirical performance of Adam is consistent with our theoretical findings in sections 2 and 4. Similar to Adagrad, Adam can take advantage of sparse features and obtain faster convergence rate than normal SGD with momentum.

6.2 Experiment: Multi-layer Neural Networks

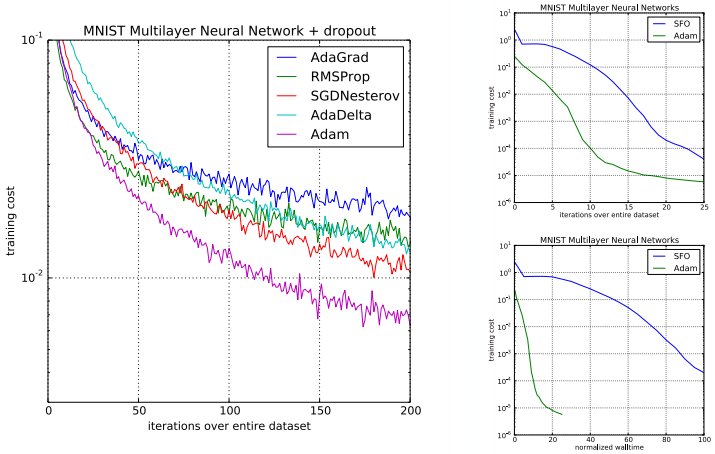
Multi-layer neural network are powerful models with non-convex objective functions. Although our convergence analysis does not apply to non-convex problems, we empirically found that Adam often outperforms other methods in such cases. In our experiments, we made model choices that are consistent with previous publications in the area; a neural network model with two fully connected hidden layers with 1000 hidden units each and ReLU activation are used for this experiment with minibatch size of 128.

First, we study different optimizers using the standard deterministic cross-entropy objective function with L_2 weight decay on the parameters to prevent over-fitting. The sum-of-functions (SFO) method (Sohl-Dickstein et al., 2014) is a recently proposed quasi-Newton method that works with minibatches of data and has shown good performance on optimization of multi-layer neural networks. We used their implementation and compared with Adam to train such models. Figure 2 shows that Adam makes faster progress in terms of both the number of iterations and wall-clock time. Due to the cost of updating curvature information, SFO is 5-10x slower per iteration compared to Adam, and has a memory requirement that is linear in the number minibatches.

Stochastic regularization methods, such as dropout, are an effective way to prevent over-fitting and often used in practice due to their simplicity. SFO assumes deterministic subfunctions, and indeed failed to converge on cost functions with stochastic regularization. We compare the effectiveness of Adam to other stochastic first order methods on multi-layer neural networks trained with dropout noise. Figure 2 shows our results; Adam shows better convergence than other methods.

6.3 Experiment: Convolutional Neural Networks

Convolutional neural networks (CNNs) with several layers of convolution, pooling and non-linear units have shown considerable success in computer vision tasks. Unlike most fully connected neural nets, weight sharing in CNNs results in vastly different gradients in different layers. A smaller learning rate for the convolution layers is often used in practice when applying SGD. We show the effectiveness of Adam in deep CNNs. Our CNN architecture has three alternating stages of 5x5 convolution filters and 3x3 max pooling with stride of 2 that are followed by a fully connected layer of 1000 rectified linear hidden units (ReLU's). The input image are pre-processed by whitening, and



(a) (b)

Figure 2: Training of multilayer neural networks on MNIST images. (a) Neural networks using dropout stochastic regularization. (b) Neural networks with deterministic cost function. We compare with the sum-of-functions (SFO) optimizer (Sohl-Dickstein et al., 2014)

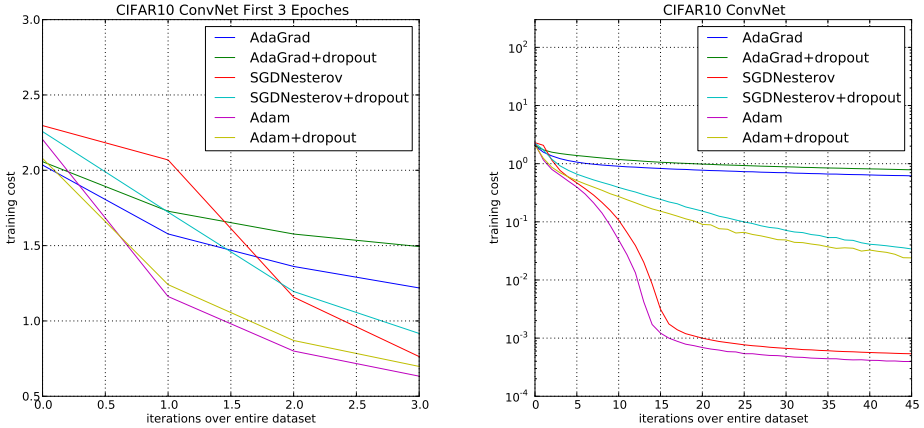
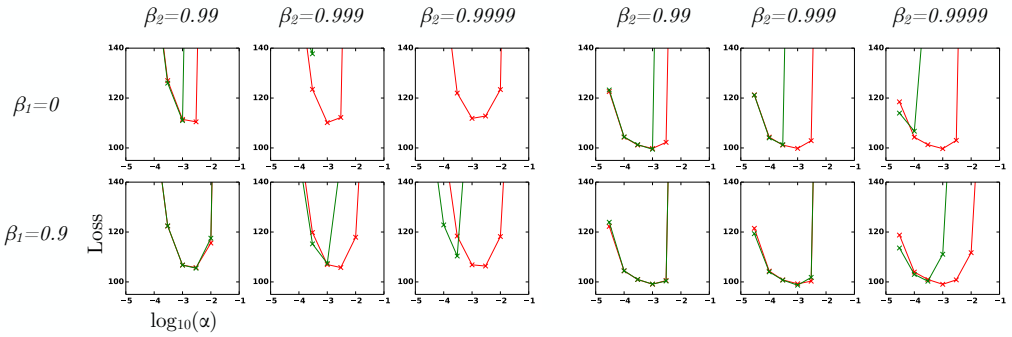


Figure 3: Convolutional neural networks training cost. (left) Training cost for the first three epochs. (right) Training cost over 45 epochs. CIFAR-10 with c64-c64-c128-1000 architecture.

dropout noise is applied to the input layer and fully connected layer. The minibatch size is also set to 128 similar to previous experiments.

Interestingly, although both Adam and Adagrad make rapid progress lowering the cost in the initial stage of the training, shown in Figure 3 (left), Adam and SGD eventually converge considerably faster than Adagrad for CNNs shown in Figure 3 (right). We notice the second moment estimate $\widehat{v}_t$ vanishes to zeros after a few epochs and is dominated by the ϵ in algorithm 1. The second moment estimate is therefore a poor approximation to the geometry of the cost function in CNNs comparing to fully connected network from Section 6.2. Whereas, reducing the minibatch variance through the first moment is more important in CNNs and contributes to the speed-up. As a result, Adagrad converges much slower than others in this particular experiment. Though Adam shows marginal improvement over SGD with momentum, it adapts learning rate scale for different layers instead of hand picking manually as in SGD.



(a) after 10 epochs (b) after 100 epochs

Figure 4: Effect of bias-correction terms (red line) versus no bias correction terms (green line) after 10 epochs (left) and 100 epochs (right) on the loss (y-axes) when learning a Variational Auto-Encoder (VAE) (Kingma & Welling, 2013), for different settings of stepsize α (x-axes) and hyper-parameters β_1 and β_2.

6.4 Experiment: bias-correction term

We also empirically evaluate the effect of the bias correction terms explained in sections 2 and 3. Discussed in section 5, removal of the bias correction terms results in a version of RMSProp (Tieleman & Hinton, 2012) with momentum. We vary the β_1 and β_2 when training a variational auto-encoder (VAE) with the same architecture as in (Kingma & Welling, 2013) with a single hidden layer with 500 hidden units with softplus nonlinearities and a 50-dimensional spherical Gaussian latent variable. We iterated over a broad range of hyper-parameter choices, i.e. $\beta_1 \in [0, 0.9]$ and $\beta_2 \in [0.99, 0.999, 0.9999]$, and $\log_{10}(\alpha) \in [-5, ..., -1]$. Values of β_2 close to 1, required for robustness to sparse gradients, results in larger initialization bias; therefore we expect the bias correction term is important in such cases of slow decay, preventing an adverse effect on optimization.

In Figure 4, values β_2 close to 1 indeed lead to instabilities in training when no bias correction term was present, especially at first few epochs of the training. The best results were achieved with small values of $(1-\beta_2)$ and bias correction; this was more apparent towards the end of optimization when gradients tends to become sparser as hidden units specialize to specific patterns. In summary, Adam performed equal or better than RMSProp, regardless of hyper-parameter setting.

7 Extensions

7.1 AdaMax

In Adam, the update rule for individual weights is to scale their gradients inversely proportional to a (scaled) L^2 norm of their individual current and past gradients. We can generalize the L^2 norm based update rule to a L^p norm based update rule. Such variants become numerically unstable for large p. However, in the special case where we let $p \to \infty$, a surprisingly simple and stable algorithm emerges; see algorithm 2. We'll now derive the algorithm. Let, in case of the L^p norm, the stepsize at time t be inversely proportional to $v_t^{1/p}$, where:

$$v_t = \beta_2^p v_{t-1} + (1-\beta_2^p)|g_t|^p \tag{6}$$

$$= (1-\beta_2^p)\sum_{i=1}^{t} \beta_2^{p(t-i)} \cdot |g_i|^p \tag{7}$$

Algorithm 2: *AdaMax*, a variant of Adam based on the infinity norm. See section 7.1 for details. Good default settings for the tested machine learning problems are $\alpha = 0.002$, $\beta_1 = 0.9$ and $\beta_2 = 0.999$. With β_1^t we denote β_1 to the power t. Here, $(\alpha/(1-\beta_1^t))$ is the learning rate with the bias-correction term for the first moment. All operations on vectors are element-wise.

Require: α: Stepsize
Require: $\beta_1, \beta_2 \in [0, 1)$: Exponential decay rates
Require: $f(\theta)$: Stochastic objective function with parameters θ
Require: θ_0: Initial parameter vector
  $m_0 \leftarrow 0$ (Initialize 1$^{\text{st}}$ moment vector)
  $u_0 \leftarrow 0$ (Initialize the exponentially weighted infinity norm)
  $t \leftarrow 0$ (Initialize timestep)
  while θ_t not converged **do**
    $t \leftarrow t + 1$
    $g_t \leftarrow \nabla_\theta f_t(\theta_{t-1})$ (Get gradients w.r.t. stochastic objective at timestep t)
    $m_t \leftarrow \beta_1 \cdot m_{t-1} + (1 - \beta_1) \cdot g_t$ (Update biased first moment estimate)
    $u_t \leftarrow \max(\beta_2 \cdot u_{t-1}, |g_t|)$ (Update the exponentially weighted infinity norm)
    $\theta_t \leftarrow \theta_{t-1} - (\alpha/(1-\beta_1^t)) \cdot m_t/u_t$ (Update parameters)
  end while
  return θ_t (Resulting parameters)

Note that the decay term is here equivalently parameterised as β_2^p instead of β_2. Now let $p \to \infty$, and define $u_t = \lim_{p\to\infty}(v_t)^{1/p}$, then:

$$u_t = \lim_{p\to\infty}(v_t)^{1/p} = \lim_{p\to\infty}\left((1-\beta_2^p)\sum_{i=1}^{t}\beta_2^{p(t-i)}\cdot|g_i|^p\right)^{1/p} \tag{8}$$

$$= \lim_{p\to\infty}(1-\beta_2^p)^{1/p}\left(\sum_{i=1}^{t}\beta_2^{p(t-i)}\cdot|g_i|^p\right)^{1/p} \tag{9}$$

$$= \lim_{p\to\infty}\left(\sum_{i=1}^{t}\left(\beta_2^{(t-i)}\cdot|g_i|\right)^p\right)^{1/p} \tag{10}$$

$$= \max\left(\beta_2^{t-1}|g_1|, \beta_2^{t-2}|g_2|, \ldots, \beta_2|g_{t-1}|, |g_t|\right) \tag{11}$$

Which corresponds to the remarkably simple recursive formula:

$$u_t = \max(\beta_2 \cdot u_{t-1}, |g_t|) \tag{12}$$

with initial value $u_0 = 0$. Note that, conveniently enough, we don't need to correct for initialization bias in this case. Also note that the magnitude of parameter updates has a simpler bound with AdaMax than Adam, namely: $|\Delta_t| \leq \alpha$.

7.2 Temporal averaging

Since the last iterate is noisy due to stochastic approximation, better generalization performance is often achieved by averaging. Previously in Moulines & Bach (2011), Polyak-Ruppert averaging (Polyak & Juditsky, 1992; Ruppert, 1988) has been shown to improve the convergence of standard SGD, where $\bar{\theta}_t = \frac{1}{t}\sum_{k=1}^{n}\theta_k$. Alternatively, an exponential moving average over the parameters can be used, giving higher weight to more recent parameter values. This can be trivially implemented by adding one line to the inner loop of algorithms 1 and 2: $\bar{\theta}_t \leftarrow \beta_2 \cdot \bar{\theta}_{t-1} + (1-\beta_2)\theta_t$, with $\bar{\theta}_0 = 0$. Initalization bias can again be corrected by the estimator $\widehat{\theta}_t = \bar{\theta}_t/(1-\beta_2^t)$.

8 Conclusion

We have introduced a simple and computationally efficient algorithm for gradient-based optimization of stochastic objective functions. Our method is aimed towards machine learning problems with

large datasets and/or high-dimensional parameter spaces. The method combines the advantages of two recently popular optimization methods: the ability of AdaGrad to deal with sparse gradients, and the ability of RMSProp to deal with non-stationary objectives. The method is straightforward to implement and requires little memory. The experiments confirm the analysis on the rate of convergence in convex problems. Overall, we found Adam to be robust and well-suited to a wide range of non-convex optimization problems in the field machine learning.

9 ACKNOWLEDGMENTS

This paper would probably not have existed without the support of Google Deepmind. We would like to give special thanks to Ivo Danihelka, and Tom Schaul for coining the name Adam. Thanks to Kai Fan from Duke University for spotting an error in the original AdaMax derivation. Experiments in this work were partly carried out on the Dutch national e-infrastructure with the support of SURF Foundation. Diederik Kingma is supported by the Google European Doctorate Fellowship in Deep Learning.

REFERENCES

Amari, Shun-Ichi. Natural gradient works efficiently in learning. *Neural computation*, 10(2):251–276, 1998.

Deng, Li, Li, Jinyu, Huang, Jui-Ting, Yao, Kaisheng, Yu, Dong, Seide, Frank, Seltzer, Michael, Zweig, Geoff, He, Xiaodong, Williams, Jason, et al. Recent advances in deep learning for speech research at microsoft. *ICASSP 2013*, 2013.

Duchi, John, Hazan, Elad, and Singer, Yoram. Adaptive subgradient methods for online learning and stochastic optimization. *The Journal of Machine Learning Research*, 12:2121–2159, 2011.

Graves, Alex. Generating sequences with recurrent neural networks. *arXiv preprint arXiv:1308.0850*, 2013.

Graves, Alex, Mohamed, Abdel-rahman, and Hinton, Geoffrey. Speech recognition with deep recurrent neural networks. In *Acoustics, Speech and Signal Processing (ICASSP), 2013 IEEE International Conference on*, pp. 6645–6649. IEEE, 2013.

Hinton, G.E. and Salakhutdinov, R.R. Reducing the dimensionality of data with neural networks. *Science*, 313 (5786):504–507, 2006.

Hinton, Geoffrey, Deng, Li, Yu, Dong, Dahl, George E, Mohamed, Abdel-rahman, Jaitly, Navdeep, Senior, Andrew, Vanhoucke, Vincent, Nguyen, Patrick, Sainath, Tara N, et al. Deep neural networks for acoustic modeling in speech recognition: The shared views of four research groups. *Signal Processing Magazine, IEEE*, 29(6):82–97, 2012a.

Hinton, Geoffrey E, Srivastava, Nitish, Krizhevsky, Alex, Sutskever, Ilya, and Salakhutdinov, Ruslan R. Improving neural networks by preventing co-adaptation of feature detectors. *arXiv preprint arXiv:1207.0580*, 2012b.

Kingma, Diederik P and Welling, Max. Auto-Encoding Variational Bayes. In *The 2nd International Conference on Learning Representations (ICLR)*, 2013.

Krizhevsky, Alex, Sutskever, Ilya, and Hinton, Geoffrey E. Imagenet classification with deep convolutional neural networks. In *Advances in neural information processing systems*, pp. 1097–1105, 2012.

Maas, Andrew L, Daly, Raymond E, Pham, Peter T, Huang, Dan, Ng, Andrew Y, and Potts, Christopher. Learning word vectors for sentiment analysis. In *Proceedings of the 49th Annual Meeting of the Association for Computational Linguistics: Human Language Technologies-Volume 1*, pp. 142–150. Association for Computational Linguistics, 2011.

Moulines, Eric and Bach, Francis R. Non-asymptotic analysis of stochastic approximation algorithms for machine learning. In *Advances in Neural Information Processing Systems*, pp. 451–459, 2011.

Pascanu, Razvan and Bengio, Yoshua. Revisiting natural gradient for deep networks. *arXiv preprint arXiv:1301.3584*, 2013.

Polyak, Boris T and Juditsky, Anatoli B. Acceleration of stochastic approximation by averaging. *SIAM Journal on Control and Optimization*, 30(4):838–855, 1992.

10 APPENDIX

10.1 CONVERGENCE PROOF

Definition 10.1. *A function $f : R^d \to R$ is convex if for all $x, y \in R^d$, for all $\lambda \in [0, 1]$,*

$$\lambda f(x) + (1 - \lambda) f(y) \geq f(\lambda x + (1 - \lambda) y)$$

Also, notice that a convex function can be lower bounded by a hyperplane at its tangent.

Lemma 10.2. *If a function $f : R^d \to R$ is convex, then for all $x, y \in R^d$,*

$$f(y) \geq f(x) + \nabla f(x)^T (y - x)$$

The above lemma can be used to upper bound the regret and our proof for the main theorem is constructed by substituting the hyperplane with the Adam update rules.

The following two lemmas are used to support our main theorem. We also use some definitions simplify our notation, where $g_t \triangleq \nabla f_t(\theta_t)$ and $g_{t,i}$ as the i^{th} element. We define $g_{1:t,i} \in \mathbb{R}^t$ as a vector that contains the i^{th} dimension of the gradients over all iterations till t, $g_{1:t,i} = [g_{1,i}, g_{2,i}, \cdots, g_{t,i}]$

Lemma 10.3. *Let $g_t = \nabla f_t(\theta_t)$ and $g_{1:t}$ be defined as above and bounded, $\|g_t\|_2 \leq G$, $\|g_t\|_\infty \leq G_\infty$. Then,*

$$\sum_{t=1}^{T} \sqrt{\frac{g_{t,i}^2}{t}} \leq 2 G_\infty \|g_{1:T,i}\|_2$$

Proof. We will prove the inequality using induction over T.

The base case for $T = 1$, we have $\sqrt{g_{1,i}^2} \leq 2 G_\infty \|g_{1,i}\|_2$.

For the inductive step,

$$\begin{aligned}
\sum_{t=1}^{T} \sqrt{\frac{g_{t,i}^2}{t}} &= \sum_{t=1}^{T-1} \sqrt{\frac{g_{t,i}^2}{t}} + \sqrt{\frac{g_{T,i}^2}{T}} \\
&\leq 2 G_\infty \|g_{1:T-1,i}\|_2 + \sqrt{\frac{g_{T,i}^2}{T}} \\
&= 2 G_\infty \sqrt{\|g_{1:T,i}\|_2^2 - g_T^2} + \sqrt{\frac{g_{T,i}^2}{T}}
\end{aligned}$$

From, $\|g_{1:T,i}\|_2^2 - g_{T,i}^2 + \frac{g_{T,i}^4}{4\|g_{1:T,i}\|_2^2} \geq \|g_{1:T,i}\|_2^2 - g_{T,i}^2$, we can take square root of both side and have,

$$\begin{aligned}
\sqrt{\|g_{1:T,i}\|_2^2 - g_{T,i}^2} &\leq \|g_{1:T,i}\|_2 - \frac{g_{T,i}^2}{2\|g_{1:T,i}\|_2} \\
&\leq \|g_{1:T,i}\|_2 - \frac{g_{T,i}^2}{2\sqrt{T G_\infty^2}}
\end{aligned}$$

Rearrange the inequality and substitute the $\sqrt{\|g_{1:T,i}\|_2^2 - g_{T,i}^2}$ term,

$$G_\infty \sqrt{\|g_{1:T,i}\|_2^2 - g_T^2} + \sqrt{\frac{g_{T,i}^2}{T}} \leq 2 G_\infty \|g_{1:T,i}\|_2$$

□

Lemma 10.4. *Let $\gamma \triangleq \frac{\beta_1^2}{\sqrt{\beta_2}}$. For β_1, $\beta_2 \in [0, 1)$ that satisfy $\frac{\beta_1^2}{\sqrt{\beta_2}} < 1$ and bounded g_t, $\|g_t\|_2 \leq G$, $\|g_t\|_\infty \leq G_\infty$, the following inequality holds*

$$\sum_{t=1}^{T} \frac{\widehat{m}_{t,i}^2}{\sqrt{t\widehat{v}_{t,i}}} \leq \frac{2}{1-\gamma} \frac{1}{\sqrt{1-\beta_2}} \|g_{1:T,i}\|_2$$

Proof. Under the assumption, $\frac{\sqrt{1-\beta_2^t}}{(1-\beta_1^t)^2} \leq \frac{1}{(1-\beta_1)^2}$. We can expand the last term in the summation using the update rules in Algorithm 1,

$$\begin{aligned}
\sum_{t=1}^{T} \frac{\widehat{m}_{t,i}^2}{\sqrt{t\widehat{v}_{t,i}}} &= \sum_{t=1}^{T-1} \frac{\widehat{m}_{t,i}^2}{\sqrt{t\widehat{v}_{t,i}}} + \frac{\sqrt{1-\beta_2^T}}{(1-\beta_1^T)^2} \frac{(\sum_{k=1}^{T}(1-\beta_1)\beta_1^{T-k} g_{k,i})^2}{\sqrt{T\sum_{j=1}^{T}(1-\beta_2)\beta_2^{T-j} g_{j,i}^2}} \\
&\leq \sum_{t=1}^{T-1} \frac{\widehat{m}_{t,i}^2}{\sqrt{t\widehat{v}_{t,i}}} + \frac{\sqrt{1-\beta_2^T}}{(1-\beta_1^T)^2} \sum_{k=1}^{T} \frac{T((1-\beta_1)\beta_1^{T-k} g_{k,i})^2}{\sqrt{T\sum_{j=1}^{T}(1-\beta_2)\beta_2^{T-j} g_{j,i}^2}} \\
&\leq \sum_{t=1}^{T-1} \frac{\widehat{m}_{t,i}^2}{\sqrt{t\widehat{v}_{t,i}}} + \frac{\sqrt{1-\beta_2^T}}{(1-\beta_1^T)^2} \sum_{k=1}^{T} \frac{T((1-\beta_1)\beta_1^{T-k} g_{k,i})^2}{\sqrt{T(1-\beta_2)\beta_2^{T-k} g_{k,i}^2}} \\
&\leq \sum_{t=1}^{T-1} \frac{\widehat{m}_{t,i}^2}{\sqrt{t\widehat{v}_{t,i}}} + \frac{\sqrt{1-\beta_2^T}}{(1-\beta_1^T)^2} \frac{(1-\beta_1)^2}{\sqrt{T(1-\beta_2)}} \sum_{k=1}^{T} T\left(\frac{\beta_1^2}{\sqrt{\beta_2}}\right)^{T-k} \|g_{k,i}\|_2 \\
&\leq \sum_{t=1}^{T-1} \frac{\widehat{m}_{t,i}^2}{\sqrt{t\widehat{v}_{t,i}}} + \frac{T}{\sqrt{T(1-\beta_2)}} \sum_{k=1}^{T} \gamma^{T-k} \|g_{k,i}\|_2
\end{aligned}$$

Similarly, we can upper bound the rest of the terms in the summation.

$$\begin{aligned}
\sum_{t=1}^{T} \frac{\widehat{m}_{t,i}^2}{\sqrt{t\widehat{v}_{t,i}}} &\leq \sum_{t=1}^{T} \frac{\|g_{t,i}\|_2}{\sqrt{t(1-\beta_2)}} \sum_{j=0}^{T-t} t\gamma^j \\
&\leq \sum_{t=1}^{T} \frac{\|g_{t,i}\|_2}{\sqrt{t(1-\beta_2)}} \sum_{j=0}^{T} t\gamma^j
\end{aligned}$$

For $\gamma < 1$, using the upper bound on the arithmetic-geometric series, $\sum_t t\gamma^t < \frac{1}{(1-\gamma)^2}$:

$$\sum_{t=1}^{T} \frac{\|g_{t,i}\|_2}{\sqrt{t(1-\beta_2)}} \sum_{j=0}^{T} t\gamma^j \leq \frac{1}{(1-\gamma)^2\sqrt{1-\beta_2}} \sum_{t=1}^{T} \frac{\|g_{t,i}\|_2}{\sqrt{t}}$$

Apply Lemma 10.3,

$$\sum_{t=1}^{T} \frac{\widehat{m}_{t,i}^2}{\sqrt{t\widehat{v}_{t,i}}} \leq \frac{2G_\infty}{(1-\gamma)^2\sqrt{1-\beta_2}} \|g_{1:T,i}\|_2$$

□

To simplify the notation, we define $\gamma \triangleq \frac{\beta_1^2}{\sqrt{\beta_2}}$. Intuitively, our following theorem holds when the learning rate α_t is decaying at a rate of $t^{-\frac{1}{2}}$ and first moment running average coefficient $\beta_{1,t}$ decay exponentially with λ, that is typically close to 1, e.g. $1 - 10^{-8}$.

Theorem 10.5. *Assume that the function f_t has bounded gradients, $\|\nabla f_t(\theta)\|_2 \leq G$, $\|\nabla f_t(\theta)\|_\infty \leq G_\infty$ for all $\theta \in R^d$ and distance between any θ_t generated by Adam is bounded, $\|\theta_n - \theta_m\|_2 \leq D$,*

$\|\theta_m - \theta_n\|_\infty \leq D_\infty$ *for any* $m, n \in \{1, ..., T\}$*, and* β_1*,* $\beta_2 \in [0, 1)$ *satisfy* $\frac{\beta_1^2}{\sqrt{\beta_2}} < 1$*. Let* $\alpha_t = \frac{\alpha}{\sqrt{t}}$ *and* $\beta_{1,t} = \beta_1 \lambda^{t-1}, \lambda \in (0, 1)$*. Adam achieves the following guarantee, for all* $T \geq 1$*.*

$$R(T) \leq \frac{D^2}{2\alpha(1-\beta_1)} \sum_{i=1}^{d} \sqrt{T\widehat{v}_{T,i}} + \frac{\alpha(\beta_1+1)G_\infty}{(1-\beta_1)\sqrt{1-\beta_2}(1-\gamma)^2} \sum_{i=1}^{d} \|g_{1:T,i}\|_2 + \sum_{i=1}^{d} \frac{D_\infty^2 G_\infty \sqrt{1-\beta_2}}{2\alpha(1-\beta_1)(1-\lambda)^2}$$

Proof. Using Lemma 10.2, we have,

$$f_t(\theta_t) - f_t(\theta^*) \leq g_t^T(\theta_t - \theta^*) = \sum_{i=1}^{d} g_{t,i}(\theta_{t,i} - \theta_{,i}^*)$$

From the update rules presented in algorithm 1,

$$\begin{aligned}\theta_{t+1} &= \theta_t - \alpha_t \widehat{m}_t / \sqrt{\widehat{v}_t} \\ &= \theta_t - \frac{\alpha_t}{1-\beta_1^t}\left(\frac{\beta_{1,t}}{\sqrt{\widehat{v}_t}} m_{t-1} + \frac{(1-\beta_{1,t})}{\sqrt{\widehat{v}_t}} g_t\right)\end{aligned}$$

We focus on the i^{th} dimension of the parameter vector $\theta_t \in R^d$. Subtract the scalar $\theta_{,i}^*$ and square both sides of the above update rule, we have,

$$(\theta_{t+1,i} - \theta_{,i}^*)^2 = (\theta_{t,i} - \theta_{,i}^*)^2 - \frac{2\alpha_t}{1-\beta_1^t}\left(\frac{\beta_{1,t}}{\sqrt{\widehat{v}_{t,i}}} m_{t-1,i} + \left(1 - \frac{\beta_{1,t})}{\sqrt{\widehat{v}_{t,i}}} g_{t,i}\right)(\theta_{t,i} - \theta_{,i}^*) + \alpha_t^2 \left(\frac{\widehat{m}_{t,i}}{\sqrt{\widehat{v}_{t,i}}}\right)^2$$

We can rearrange the above equation and use Young's inequality, $ab \leq a^2/2 + b^2/2$. Also, it can be shown that $\sqrt{\widehat{v}_{t,i}} = \sqrt{\sum_{j=1}^{t}(1-\beta_2)\beta_2^{t-j} g_{j,i}^2} / \sqrt{1-\beta_2^t} \leq \|g_{1:t,i}\|_2$ and $\beta_{1,t} \leq \beta_1$. Then

$$\begin{aligned} g_{t,i}(\theta_{t,i} - \theta_{,i}^*) =& \frac{(1-\beta_1^t)\sqrt{\widehat{v}_{t,i}}}{2\alpha_t(1-\beta_{1,t})}\left((\theta_{t,i} - \theta_{,t}^*)^2 - (\theta_{t+1,i} - \theta_{,i}^*)^2\right) \\ &+ \frac{\beta_{1,t}}{(1-\beta_{1,t})} \frac{\widehat{v}_{t-1,i}^{\frac{1}{4}}}{\sqrt{\alpha_{t-1}}} (\theta_{,i}^* - \theta_{t,i}) \sqrt{\alpha_{t-1}} \frac{m_{t-1,i}}{\widehat{v}_{t-1,i}^{\frac{1}{4}}} + \frac{\alpha_t(1-\beta_1^t)\sqrt{\widehat{v}_{t,i}}}{2(1-\beta_{1,t})} \left(\frac{\widehat{m}_{t,i}}{\sqrt{\widehat{v}_{t,i}}}\right)^2 \\ \leq& \frac{1}{2\alpha_t(1-\beta_1)}\left((\theta_{t,i} - \theta_{,t}^*)^2 - (\theta_{t+1,i} - \theta_{,i}^*)^2\right)\sqrt{\widehat{v}_{t,i}} + \frac{\beta_{1,t}}{2\alpha_{t-1}(1-\beta_{1,t})}(\theta_{,i}^* - \theta_{t,i})^2 \sqrt{\widehat{v}_{t-1,i}} \\ &+ \frac{\beta_1 \alpha_{t-1}}{2(1-\beta_1)} \frac{m_{t-1,i}^2}{\sqrt{\widehat{v}_{t-1,i}}} + \frac{\alpha_t}{2(1-\beta_1)} \frac{\widehat{m}_{t,i}^2}{\sqrt{\widehat{v}_{t,i}}} \end{aligned}$$

We apply Lemma 10.4 to the above inequality and derive the regret bound by summing across all the dimensions for $i \in 1, ..., d$ in the upper bound of $f_t(\theta_t) - f_t(\theta^*)$ and the sequence of convex functions for $t \in 1, ..., T$:

$$\begin{aligned} R(T) \leq& \sum_{i=1}^{d} \frac{1}{2\alpha_1(1-\beta_1)}(\theta_{1,i} - \theta_{,i}^*)^2 \sqrt{\widehat{v}_{1,i}} + \sum_{i=1}^{d}\sum_{t=2}^{T} \frac{1}{2(1-\beta_1)}(\theta_{t,i} - \theta_{,i}^*)^2 \left(\frac{\sqrt{\widehat{v}_{t,i}}}{\alpha_t} - \frac{\sqrt{\widehat{v}_{t-1,i}}}{\alpha_{t-1}}\right) \\ &+ \frac{\beta_1 \alpha G_\infty}{(1-\beta_1)\sqrt{1-\beta_2}(1-\gamma)^2} \sum_{i=1}^{d} \|g_{1:T,i}\|_2 + \frac{\alpha G_\infty}{(1-\beta_1)\sqrt{1-\beta_2}(1-\gamma)^2} \sum_{i=1}^{d} \|g_{1:T,i}\|_2 \\ &+ \sum_{i=1}^{d}\sum_{t=1}^{T} \frac{\beta_{1,t}}{2\alpha_t(1-\beta_{1,t})}(\theta_{,i}^* - \theta_{t,i})^2 \sqrt{\widehat{v}_{t,i}} \end{aligned}$$

From the assumption, $\|\theta_t - \theta^*\|_2 \leq D$, $\|\theta_m - \theta_n\|_\infty \leq D_\infty$, we have:

$$\begin{aligned}
R(T) \leq & \frac{D^2}{2\alpha(1-\beta_1)} \sum_{i=1}^{d} \sqrt{T\widehat{v}_{T,i}} + \frac{\alpha(1+\beta_1)G_\infty}{(1-\beta_1)\sqrt{1-\beta_2}(1-\gamma)^2} \sum_{i=1}^{d} \|g_{1:T,i}\|_2 + \frac{D_\infty^2}{2\alpha} \sum_{i=1}^{d} \sum_{t=1}^{t} \frac{\beta_{1,t}}{(1-\beta_{1,t})} \sqrt{t\widehat{v}_{t,i}} \\
\leq & \frac{D^2}{2\alpha(1-\beta_1)} \sum_{i=1}^{d} \sqrt{T\widehat{v}_{T,i}} + \frac{\alpha(1+\beta_1)G_\infty}{(1-\beta_1)\sqrt{1-\beta_2}(1-\gamma)^2} \sum_{i=1}^{d} \|g_{1:T,i}\|_2 \\
& + \frac{D_\infty^2 G_\infty \sqrt{1-\beta_2}}{2\alpha} \sum_{i=1}^{d} \sum_{t=1}^{t} \frac{\beta_{1,t}}{(1-\beta_{1,t})} \sqrt{t}
\end{aligned}$$

We can use arithmetic geometric series upper bound for the last term:

$$\begin{aligned}
\sum_{t=1}^{t} \frac{\beta_{1,t}}{(1-\beta_{1,t})} \sqrt{t} &\leq \sum_{t=1}^{t} \frac{1}{(1-\beta_1)} \lambda^{t-1} \sqrt{t} \\
&\leq \sum_{t=1}^{t} \frac{1}{(1-\beta_1)} \lambda^{t-1} t \\
&\leq \frac{1}{(1-\beta_1)(1-\lambda)^2}
\end{aligned}$$

Therefore, we have the following regret bound:

$$R(T) \leq \frac{D^2}{2\alpha(1-\beta_1)} \sum_{i=1}^{d} \sqrt{T\widehat{v}_{T,i}} + \frac{\alpha(1+\beta_1)G_\infty}{(1-\beta_1)\sqrt{1-\beta_2}(1-\gamma)^2} \sum_{i=1}^{d} \|g_{1:T,i}\|_2 + \sum_{i=1}^{d} \frac{D_\infty^2 G_\infty \sqrt{1-\beta_2}}{2\alpha\beta_1(1-\lambda)^2}$$

□

Journal of Machine Learning Research 12 (2011) 2121-2159 Submitted 3/10; Revised 3/11; Published 7/11

Adaptive Subgradient Methods for Online Learning and Stochastic Optimization*

John Duchi JDUCHI@CS.BERKELEY.EDU
Computer Science Division
University of California, Berkeley
Berkeley, CA 94720 USA

Elad Hazan EHAZAN@IE.TECHNION.AC.IL
Technion - Israel Institute of Technology
Technion City
Haifa, 32000, Israel

Yoram Singer SINGER@GOOGLE.COM
Google
1600 Amphitheatre Parkway
Mountain View, CA 94043 USA

Editor: Tong Zhang

Abstract

We present a new family of subgradient methods that dynamically incorporate knowledge of the geometry of the data observed in earlier iterations to perform more informative gradient-based learning. Metaphorically, the adaptation allows us to find needles in haystacks in the form of very predictive but rarely seen features. Our paradigm stems from recent advances in stochastic optimization and online learning which employ proximal functions to control the gradient steps of the algorithm. We describe and analyze an apparatus for adaptively modifying the proximal function, which significantly simplifies setting a learning rate and results in regret guarantees that are provably as good as the best proximal function that can be chosen in hindsight. We give several efficient algorithms for empirical risk minimization problems with common and important regularization functions and domain constraints. We experimentally study our theoretical analysis and show that adaptive subgradient methods outperform state-of-the-art, yet non-adaptive, subgradient algorithms.

Keywords: subgradient methods, adaptivity, online learning, stochastic convex optimization

1. Introduction

In many applications of online and stochastic learning, the input instances are of very high dimension, yet within any particular instance only a few features are non-zero. It is often the case, however, that infrequently occurring features are highly informative and discriminative. The informativeness of rare features has led practitioners to craft domain-specific feature weightings, such as TF-IDF (Salton and Buckley, 1988), which pre-emphasize infrequently occurring features. We use this old idea as a motivation for applying modern learning-theoretic techniques to the problem of online and stochastic learning, focusing concretely on (sub)gradient methods.

*. A preliminary version of this work was published in COLT 2010.

Standard stochastic subgradient methods largely follow a predetermined procedural scheme that is oblivious to the characteristics of the data being observed. In contrast, our algorithms dynamically incorporate knowledge of the geometry of the data observed in earlier iterations to perform more informative gradient-based learning. Informally, our procedures give frequently occurring features very low learning rates and infrequent features high learning rates, where the intuition is that each time an infrequent feature is seen, the learner should "take notice." Thus, the adaptation facilitates finding and identifying very predictive but comparatively rare features.

1.1 The Adaptive Gradient Algorithm

Before introducing our adaptive gradient algorithm, which we term ADAGRAD, we establish notation. Vectors and scalars are lower case italic letters, such as $x \in \mathcal{X}$. We denote a sequence of vectors by subscripts, that is, $x_t, x_{t+1}, \ldots$, and entries of each vector by an additional subscript, for example, $x_{t,j}$. The subdifferential set of a function f evaluated at x is denoted $\partial f(x)$, and a particular vector in the subdifferential set is denoted by $f'(x) \in \partial f(x)$ or $g_t \in \partial f_t(x_t)$. When a function is differentiable, we write $\nabla f(x)$. We use $\langle x, y \rangle$ to denote the inner product between x and y. The Bregman divergence associated with a strongly convex and differentiable function ψ is

$$B_\psi(x, y) = \psi(x) - \psi(y) - \langle \nabla\psi(y), x - y \rangle \ .$$

We also make frequent use of the following two matrices. Let $g_{1:t} = [g_1 \ \cdots \ g_t]$ denote the matrix obtained by concatenating the subgradient sequence. We denote the ith row of this matrix, which amounts to the concatenation of the ith component of each subgradient we observe, by $g_{1:t,i}$. We also define the outer product matrix $G_t = \sum_{\tau=1}^{t} g_\tau g_\tau^\top$.

Online learning and stochastic optimization are closely related and basically interchangeable (Cesa-Bianchi et al., 2004). In order to keep our presentation simple, we confine our discussion and algorithmic descriptions to the online setting with the regret bound model. In online learning, the learner repeatedly predicts a point $x_t \in \mathcal{X} \subseteq \mathbb{R}^d$, which often represents a weight vector assigning importance values to various features. The learner's goal is to achieve low regret with respect to a static predictor x^* in the (closed) convex set $\mathcal{X} \subseteq \mathbb{R}^d$ (possibly $\mathcal{X} = \mathbb{R}^d$) on a sequence of functions $f_t(x)$, measured as

$$R(T) = \sum_{t=1}^{T} f_t(x_t) - \inf_{x \in X} \sum_{t=1}^{T} f_t(x) \ .$$

At every timestep t, the learner receives the (sub)gradient information $g_t \in \partial f_t(x_t)$. Standard subgradient algorithms then move the predictor x_t in the opposite direction of g_t while maintaining $x_{t+1} \in \mathcal{X}$ via the projected gradient update (e.g., Zinkevich, 2003)

$$x_{t+1} = \Pi_{\mathcal{X}}\left(x_t - \eta g_t\right) = \operatorname*{argmin}_{x \in \mathcal{X}} \|x - (x_t - \eta g_t)\|_2^2 \ .$$

In contrast, let the Mahalanobis norm $\|\cdot\|_A = \sqrt{\langle \cdot, A\cdot \rangle}$ and denote the projection of a point y onto $\mathcal{X}$ according to A by $\Pi_{\mathcal{X}}^A(y) = \operatorname{argmin}_{x \in \mathcal{X}} \|x - y\|_A = \operatorname{argmin}_{x \in \mathcal{X}} \langle x - y, A(x - y) \rangle$. Using this notation, our generalization of standard gradient descent employs the update

$$x_{t+1} = \Pi_{\mathcal{X}}^{G_t^{1/2}}\left(x_t - \eta G_t^{-1/2} g_t\right) \ .$$

The above algorithm is computationally impractical in high dimensions since it requires computation of the root of the matrix G_t, the outer product matrix. Thus we specialize the update to

$$x_{t+1} = \Pi_{\mathcal{X}}^{\mathrm{diag}(G_t)^{1/2}}\left(x_t - \eta\,\mathrm{diag}(G_t)^{-1/2} g_t\right). \tag{1}$$

Both the inverse and root of $\mathrm{diag}(G_t)$ can be computed in linear time. Moreover, as we discuss later, when the gradient vectors are sparse the update above can often be performed in time proportional to the support of the gradient. We now elaborate and give a more formal discussion of our setting.

In this paper we consider several different online learning algorithms and their stochastic convex optimization counterparts. Formally, we consider online learning with a sequence of composite functions ϕ_t. Each function is of the form $\phi_t(x) = f_t(x) + \varphi(x)$ where f_t and φ are (closed) convex functions. In the learning settings we study, f_t is either an instantaneous loss or a stochastic estimate of the objective function in an optimization task. The function φ serves as a fixed regularization function and is typically used to control the complexity of x. At each round the algorithm makes a prediction $x_t \in \mathcal{X}$ and then receives the function f_t. We define the regret with respect to the fixed (optimal) predictor x^* as

$$R_\phi(T) \triangleq \sum_{t=1}^{T} \left[\phi_t(x_t) - \phi_t(x^*)\right] = \sum_{t=1}^{T} \left[f_t(x_t) + \varphi(x_t) - f_t(x^*) - \varphi(x^*)\right] . \tag{2}$$

Our goal is to devise algorithms which are guaranteed to suffer asymptotically sub-linear regret, namely, $R_\phi(T) = o(T)$.

Our analysis applies to related, yet different, methods for minimizing the regret (2). The first is Nesterov's primal-dual subgradient method (2009), and in particular Xiao's (2010) extension, regularized dual averaging, and the follow-the-regularized-leader (FTRL) family of algorithms (see for instance Kalai and Vempala, 2003; Hazan et al., 2006). In the primal-dual subgradient method the algorithm makes a prediction x_t on round t using the average gradient $\bar{g}_t = \frac{1}{t}\sum_{\tau=1}^{t} g_\tau$. The update encompasses a trade-off between a gradient-dependent linear term, the regularizer φ, and a strongly-convex term ψ_t for well-conditioned predictions. Here ψ_t is the *proximal* term. The update amounts to solving

$$x_{t+1} = \operatorname*{argmin}_{x\in\mathcal{X}} \left\{ \eta \left\langle \bar{g}_t, x \right\rangle + \eta\varphi(x) + \frac{1}{t}\psi_t(x) \right\} , \tag{3}$$

where η is a fixed step-size and $x_1 = \operatorname{argmin}_{x\in\mathcal{X}} \varphi(x)$. The second method similarly has numerous names, including proximal gradient, forward-backward splitting, and composite mirror descent (Tseng, 2008; Duchi et al., 2010). We use the term composite mirror descent. The composite mirror descent method employs a more immediate trade-off between the current gradient g_t, φ, and staying close to x_t using the proximal function ψ,

$$x_{t+1} = \operatorname*{argmin}_{x\in\mathcal{X}} \left\{ \eta \left\langle g_t, x \right\rangle + \eta\varphi(x) + B_{\psi_t}(x, x_t) \right\} . \tag{4}$$

Our work focuses on temporal adaptation of the proximal function in a data driven way, while previous work simply sets $\psi_t \equiv \psi$, $\psi_t(\cdot) = \sqrt{t}\psi(\cdot)$, or $\psi_t(\cdot) = t\psi(\cdot)$ for some fixed ψ.

We provide formal analyses equally applicable to the above two updates and show how to automatically choose the function ψ_t so as to achieve asymptotically small regret. We describe and analyze two algorithms. Both algorithms use squared Mahalanobis norms as their proximal functions, setting $\psi_t(x) = \langle x, H_t x\rangle$ for a symmetric matrix $H_t \succeq 0$. The first uses diagonal matrices while

the second constructs full dimensional matrices. Concretely, for some small fixed $\delta \geq 0$ (specified later, though in practice δ can be set to 0) we set

$$H_t = \delta I + \operatorname{diag}(G_t)^{1/2} \ \text{(Diagonal)} \quad \text{and} \quad H_t = \delta I + G_t^{1/2} \ \text{(Full)} \, . \tag{5}$$

Plugging the appropriate matrix from the above equation into ψ_t in (3) or (4) gives rise to our ADAGRAD family of algorithms. Informally, we obtain algorithms which are similar to second-order gradient descent by constructing approximations to the Hessian of the functions f_t, though we use roots of the matrices.

1.2 Outline of Results

We now outline our results, deferring formal statements of the theorems to later sections. Recall the definitions of $g_{1:t}$ as the matrix of concatenated subgradients and G_t as the outer product matrix in the prequel. The ADAGRAD algorithm with full matrix divergences entertains bounds of the form

$$R_\phi(T) = O\left(\|x^*\|_2 \operatorname{tr}(G_T^{1/2})\right) \quad \text{and} \quad R_\phi(T) = O\left(\max_{t \leq T} \|x_t - x^*\|_2 \operatorname{tr}(G_T^{1/2})\right).$$

We further show that

$$\operatorname{tr}\left(G_T^{1/2}\right) = d^{1/2} \sqrt{\inf_S \left\{ \sum_{t=1}^T \langle g_t, S^{-1} g_t \rangle \ : \ S \succeq 0, \operatorname{tr}(S) \leq d \right\}} \, .$$

These results are formally given in Theorem 7 and its corollaries. When our proximal function $\psi_t(x) = \left\langle x, \operatorname{diag}(G_t)^{1/2} x \right\rangle$ we have bounds attainable in time at most linear in the dimension d of our problems of the form

$$R_\phi(T) = O\left(\|x^*\|_\infty \sum_{i=1}^d \|g_{1:T,i}\|_2\right) \quad \text{and} \quad R_\phi(T) = O\left(\max_{t \leq T} \|x_t - x^*\|_\infty \sum_{i=1}^d \|g_{1:T,i}\|_2\right).$$

Similar to the above, we will show that

$$\sum_{i=1}^d \|g_{1:T,i}\|_2 = d^{1/2} \sqrt{\inf_s \left\{ \sum_{t=1}^T \langle g_t, \operatorname{diag}(s)^{-1} g_t \rangle \ : \ s \succeq 0, \langle 1, s \rangle \leq d \right\}} \, .$$

We formally state the above two regret bounds in Theorem 5 and its corollaries.

Following are a simple example and corollary to Theorem 5 to illustrate one regime in which we expect substantial improvements (see also the next subsection). Let $\varphi \equiv 0$ and consider Zinkevich's online gradient descent algorithm. Given a compact convex set $\mathcal{X} \subseteq \mathbb{R}^d$ and sequence of convex functions f_t, Zinkevich's algorithm makes the sequence of predictions $x_1, \ldots, x_T$ with $x_{t+1} = \Pi_{\mathcal{X}}(x_t - (\eta/\sqrt{t}) g_t)$. If the diameter of $\mathcal{X}$ is bounded, thus $\sup_{x,y \in \mathcal{X}} \|x - y\|_2 \leq D_2$, then online gradient descent, with the optimal choice in *hindsight* for the stepsize η (see the bound (7) in Section 1.4), achieves a regret bound of

$$\sum_{t=1}^T f_t(x_t) - \inf_{x \in \mathcal{X}} \sum_{t=1}^T f_t(x) \leq \sqrt{2} D_2 \sqrt{\sum_{t=1}^T \|g_t\|_2^2} \, . \tag{6}$$

When $\mathcal{X}$ is bounded via $\sup_{x,y \in \mathcal{X}} \|x - y\|_\infty \leq D_\infty$, the following corollary is a simple consequence of our Theorem 5.

Corollary 1 *Let the sequence* $\{x_t\} \subset \mathbb{R}^d$ *be generated by the update (4) and assume that* $\max_t \|x^* - x_t\|_\infty \leq D_\infty$. *Using stepsize* $\eta = D_\infty/\sqrt{2}$, *for any* x^*, *the following bound holds.*

$$R_\phi(T) \leq \sqrt{2d} D_\infty \sqrt{\inf_{s \succeq 0, \langle 1, s \rangle \leq d} \sum_{t=1}^T \|g_t\|^2_{\mathrm{diag}(s)^{-1}}} = \sqrt{2} D_\infty \sum_{i=1}^d \|g_{1:T,i}\|_2 \ .$$

The important feature of the bound above is the infimum under the square root, which allows us to perform better than simply using the identity matrix, and the fact that the stepsize is easy to set a priori. For example, if the set $\mathcal{X} = \{x : \|x\|_\infty \leq 1\}$, then $D_2 = 2\sqrt{d}$ while $D_\infty = 2$, which suggests that if we are learning a dense predictor over a box, the adaptive method should perform well. Indeed, in this case we are guaranteed that the bound in Corollary 1 is better than (6) as the identity matrix belongs to the set over which we take the infimum.

To conclude the outline of results, we would like to point to two relevant research papers. First, Zinkevich's regret bound is tight and cannot be improved in a minimax sense (Abernethy et al., 2008). Therefore, improving the regret bound requires further reasonable assumptions on the input space. Second, in a independent work, performed concurrently to the research presented in this paper, McMahan and Streeter (2010) study *competitive ratios*, showing guaranteed improvements of the above bounds relative to families of online algorithms.

1.3 Improvements and Motivating Example

As mentioned in the prequel, we expect our adaptive methods to outperform standard online learning methods when the gradient vectors are sparse. We give empirical evidence supporting the improved performance of the adaptive methods in Section 6. Here we give a few abstract examples that show that for sparse data (input sequences where g_t has many zeros) the adaptive methods herein have better performance than non-adaptive methods. In our examples we use the hinge loss, that is,

$$f_t(x) = [1 - y_t \langle z_t, x \rangle]_+ \ ,$$

where y_t is the label of example t and $z_t \in \mathbb{R}^d$ is the data vector.

For our first example, which was also given by McMahan and Streeter (2010), consider the following sparse random data scenario, where the vectors $z_t \in \{-1, 0, 1\}^d$. Assume that at in each round t, feature i appears with probability $p_i = \min\{1, ci^{-\alpha}\}$ for some $\alpha \in (1, \infty)$ and a dimension-independent constant c. Then taking the expectation of the gradient terms in the bound in Corollary 1, we have

$$\mathbb{E} \sum_{i=1}^d \|g_{1:T,i}\|_2 = \sum_{i=1}^d \mathbb{E}\left[\sqrt{|\{t : |g_{t,i}| = 1\}|}\right] \leq \sum_{i=1}^d \sqrt{\mathbb{E}|\{t : |g_{t,i}| = 1\}|} = \sum_{i=1}^d \sqrt{p_i T}$$

by Jensen's inequality. In the rightmost sum, we have $c\sum_{i=1}^d i^{-\alpha/2} = O(\log d)$ for $\alpha \geq 2$, and $\sum_{i=1}^d i^{-\alpha/2} = O(d^{1-\alpha/2})$ for $\alpha \in (1, 2)$. If the domain $\mathcal{X}$ is a hypercube, say $\mathcal{X} = \{x : \|x\|_\infty \leq 1\}$, then in Corollary 1 $D_\infty = 2$, and the regret of ADAGRAD is $O(\max\{\log d, d^{1-\alpha/2}\}\sqrt{T})$. For contrast, the standard regret bound (6) for online gradient descent has $D_2 = 2\sqrt{d}$ and $\|g_t\|_2^2 \geq 1$, yielding best case regret $O(\sqrt{dT})$. So we see that in this sparse yet heavy tailed feature setting, ADAGRAD's regret guarantee can be exponentially smaller in the dimension d than the non-adaptive regret bound.

Our remaining examples construct a sparse sequence for which there is a perfect predictor that the adaptive methods learn after d iterations, while standard online gradient descent (Zinkevich,

2003) suffers significantly higher loss. We assume the domain $\mathcal{X}$ is compact, so that for online gradient descent we set $\eta_t = \eta/\sqrt{t}$, which gives the optimal $O(\sqrt{T})$ regret (the setting of η does not matter to the adversary we construct).

1.3.1 DIAGONAL ADAPTATION

Consider the diagonal version of our proposed update (4) with $\mathcal{X} = \{x : \|x\|_\infty \leq 1\}$. Evidently, we can take $D_\infty = 2$, and this choice simply results in the update $x_{t+1} = x_t - \sqrt{2}\,\mathrm{diag}(G_t)^{-1/2} g_t$ followed by projection (1) onto $\mathcal{X}$ for ADAGRAD (we use a pseudo-inverse if the inverse does not exist). Let e_i denote the ith unit basis vector, and assume that for each t, $z_t = \pm e_i$ for some i. Also let $y_t = \mathrm{sign}(\langle 1, z_t \rangle)$ so that there exists a perfect classifier $x^* = 1 \in \mathcal{X} \subset \mathbb{R}^d$. We initialize x_1 to be the zero vector. Fix some $\varepsilon > 0$, and on rounds rounds $t = 1, \ldots, \eta^2/\varepsilon^2$, set $z_t = e_1$. After these rounds, simply choose $z_t = \pm e_i$ for index $i \in \{2, \ldots, d\}$ chosen at random. It is clear that the update to parameter x_i at these iterations is different, and amounts to

$$x_{t+1} = x_t + e_i \;\; \text{ADAGRAD} \qquad x_{t+1} = \left[x_t + \frac{\eta}{\sqrt{t}}\right]_{[-1,1]^d} \;\; \text{(Gradient Descent)} \, .$$

(Here $[\cdot]_{[-1,1]^d}$ denotes the truncation of the vector to $[-1,1]^d$). In particular, after suffering $d-1$ more losses, ADAGRAD has a perfect classifier. However, on the remaining iterations gradient descent has $\eta/\sqrt{t} \leq \varepsilon$ and thus evidently suffers loss at least $d/(2\varepsilon)$. Of course, for small ε, we have $d/(2\varepsilon) \gg d$. In short, ADAGRAD achieves constant regret per dimension while online gradient descent can suffer arbitrary loss (for unbounded t). It seems quite silly, then, to use a global learning rate rather than one for each feature.

Full Matrix Adaptation. We use a similar construction to the diagonal case to show a situation in which the full matrix update from (5) gives substantially lower regret than stochastic gradient descent. For full divergences we set $\mathcal{X} = \{x : \|x\|_2 \leq \sqrt{d}\}$. Let $V = [v_1 \;\; \ldots \;\; v_d] \in \mathbb{R}^{d \times d}$ be an orthonormal matrix. Instead of having z_t cycle through the unit vectors, we make z_t cycle through the v_i so that $z_t = \pm v_i$. We let the label $y_t = \mathrm{sign}(\langle 1, V^\top z_t \rangle) = \mathrm{sign}\left(\sum_{i=1}^d \langle v_i, z_t \rangle\right)$. We provide an elaborated explanation in Appendix A. Intuitively, with $\psi_t(x) = \langle x, H_t x \rangle$ and H_t set to be the full matrix from (5), ADAGRAD again needs to observe each orthonormal vector v_i only once while stochastic gradient descent's loss can be made $\Omega(d/\varepsilon)$ for any $\varepsilon > 0$.

1.4 Related Work

Many successful algorithms have been developed over the past few years to minimize regret in the online learning setting. A modern view of these algorithms casts the problem as the task of following the (regularized) leader (see Rakhlin, 2009, and the references therein) or FTRL in short. Informally, FTRL methods choose the best decision in hindsight at every iteration. Verbatim usage of the FTRL approach fails to achieve low regret, however, adding a proximal[1] term to the past predictions leads to numerous low regret algorithms (Kalai and Vempala, 2003; Hazan and Kale, 2008; Rakhlin, 2009). The proximal term strongly affects the performance of the learning algorithm. Therefore, adapting the proximal function to the characteristics of the problem at hand is desirable.

Our approach is thus motivated by two goals. The first is to generalize the agnostic online learning paradigm to the meta-task of specializing an algorithm to fit a particular data set. Specifically,

1. The proximal term is also referred to as regularization in the online learning literature. We use the phrase proximal term in order to avoid confusion with the statistical regularization function φ.

we change the proximal function to achieve performance guarantees which are competitive with the best proximal term found in hindsight. The second, as alluded to earlier, is to automatically adjust the learning rates for online learning and stochastic gradient descent on a per-feature basis. The latter can be very useful when our gradient vectors g_t are sparse, for example, in a classification setting where examples may have only a small number of non-zero features. As we demonstrated in the examples above, it is rather deficient to employ exactly the same learning rate for a feature seen hundreds of times and for a feature seen only once or twice.

Our techniques stem from a variety of research directions, and as a byproduct we also extend a few well-known algorithms. In particular, we consider variants of the follow-the-regularized leader (FTRL) algorithms mentioned above, which are kin to Zinkevich's lazy projection algorithm. We use Xiao's recently analyzed regularized dual averaging (RDA) algorithm (2010), which builds upon Nesterov's (2009) primal-dual subgradient method. We also consider forward-backward splitting (FOBOS) (Duchi and Singer, 2009) and its composite mirror-descent (proximal gradient) generalizations (Tseng, 2008; Duchi et al., 2010), which in turn include as special cases projected gradients (Zinkevich, 2003) and mirror descent (Nemirovski and Yudin, 1983; Beck and Teboulle, 2003). Recent work by several authors (Nemirovski et al., 2009; Juditsky et al., 2008; Lan, 2010; Xiao, 2010) considered efficient and robust methods for stochastic optimization, especially in the case when the expected objective f is smooth. It may be interesting to investigate adaptive metric approaches in smooth stochastic optimization.

The idea of adapting first order optimization methods is by no means new and can be traced back at least to the 1970s with the work on space dilation methods of Shor (1972) and variable metric methods, such as the BFGS family of algorithms (e.g., Fletcher, 1970). This prior work often assumed that the function to be minimized was differentiable and, to our knowledge, did not consider stochastic, online, or composite optimization. In her thesis, Nedić (2002) studied variable metric subgradient methods, though it seems difficult to derive explicit rates of convergence from the results there, and the algorithms apply only when the constraint set $\mathcal{X} = \mathbb{R}^d$. More recently, Bordes et al. (2009) proposed a Quasi-Newton stochastic gradient-descent procedure, which is similar in spirit to our methods. However, their convergence results assume a smooth objective with positive definite Hessian bounded away from 0. Our results apply more generally.

Prior to the analysis presented in this paper for online and stochastic optimization, the strongly convex function ψ in the update equations (3) and (4) either remained intact or was simply multiplied by a time-dependent scalar throughout the run of the algorithm. Zinkevich's projected gradient, for example, uses $\psi_t(x) = \|x\|_2^2$, while RDA (Xiao, 2010) employs $\psi_t(x) = \sqrt{t}\psi(x)$ where ψ is a strongly convex function. The bounds for both types of algorithms are similar, and both rely on the norm $\|\cdot\|$ (and its associated dual $\|\cdot\|_*$) with respect to which ψ is strongly convex. Mirror-descent type first order algorithms, such as projected gradient methods, attain regret bounds of the form (Zinkevich, 2003; Bartlett et al., 2007; Duchi et al., 2010)

$$R_\phi(T) \leq \frac{1}{\eta} B_\psi(x^*, x_1) + \frac{\eta}{2} \sum_{t=1}^{T} \left\| f_t'(x_t) \right\|_*^2 . \tag{7}$$

Choosing $\eta \propto 1/\sqrt{T}$ gives $R_\phi(T) = O(\sqrt{T})$. When $B_\psi(x, x^*)$ is bounded for all $x \in \mathcal{X}$, we choose step sizes $\eta_t \propto 1/\sqrt{t}$ which is equivalent to setting $\psi_t(x) = \sqrt{t}\psi(x)$. Therefore, no assumption on the time horizon is necessary. For RDA and follow-the-leader algorithms, the bounds are similar

(Xiao, 2010, Theorem 3):

$$R_\phi(T) \leq \sqrt{T}\psi(x^*) + \frac{1}{2\sqrt{T}} \sum_{t=1}^{T} \left\| f_t'(x_t) \right\|_*^2 . \tag{8}$$

The problem of adapting to data and obtaining tighter data-dependent bounds for algorithms such as those above is a natural one and has been studied in the mistake-bound setting for online learning in the past. A framework that is somewhat related to ours is the confidence weighted learning scheme by Crammer et al. (2008) and the adaptive regularization of weights algorithm (AROW) of Crammer et al. (2009). These papers provide mistake-bound analyses for second-order algorithms, which in turn are similar in spirit to the second-order Perceptron algorithm (Cesa-Bianchi et al., 2005). The analyses by Crammer and colleagues, however, yield mistake bounds dependent on the runs of the individual algorithms and are thus difficult to compare with our regret bounds.

AROW maintains a mean prediction vector $\mu_t \in \mathbb{R}^d$ and a covariance matrix $\Sigma_t \in \mathbb{R}^{d\times d}$ over μ_t as well. At every step of the algorithm, the learner receives a pair (z_t, y_t) where $z_t \in \mathbb{R}^d$ is the tth example and $y_t \in \{-1, +1\}$ is the label. Whenever the predictor μ_t attains a margin value smaller than 1, AROW performs the update

$$\begin{aligned} \beta_t &= \frac{1}{\langle z_t, \Sigma_t z_t \rangle + \lambda}, \quad \alpha_t = \left[1 - y_t \langle z_t, \mu_t \rangle\right]_+ , \\ \mu_{t+1} &= \mu_t + \alpha_t \Sigma_t y_t z_t, \quad \Sigma_{t+1} = \Sigma_t - \beta_t \Sigma_t x_t x_t^\top \Sigma_t . \end{aligned} \tag{9}$$

In the above scheme, one can force Σ_t to be diagonal, which reduces the run-time and storage requirements of the algorithm but still gives good performance (Crammer et al., 2009). In contrast to AROW, the ADAGRAD algorithm uses the *root* of the inverse covariance matrix, a consequence of our formal analysis. Crammer et al.'s algorithm and our algorithms have similar run times, generally linear in the dimension d, when using diagonal matrices. However, when using full matrices the runtime of AROW algorithm is $O(d^2)$, which is faster than ours as it requires computing the root of a matrix.

In concurrent work, McMahan and Streeter (2010) propose and analyze an algorithm which is very similar to some of the algorithms presented in this paper. Our analysis builds on recent advances in online learning and stochastic optimization (Duchi et al., 2010; Xiao, 2010), whereas McMahan and Streeter use first-principles to derive their regret bounds. As a consequence of our approach, we are able to apply our analysis to algorithms for composite minimization with a known additional objective term φ. We are also able to generalize and analyze both the mirror descent and dual-averaging family of algorithms. McMahan and Streeter focus on what they term the *competitive ratio*, which is the ratio of the worst case regret of the adaptive algorithm to the worst case regret of a non-adaptive algorithm with the best proximal term ψ chosen in hindsight. We touch on this issue briefly in the sequel, but refer the interested reader to McMahan and Streeter (2010) for this alternative elegant perspective. We believe that both analyses shed insights into the problems studied in this paper and complement each other.

There are also other lines of work on adaptive gradient methods that are not directly related to our work but nonetheless relevant. Tighter regret bounds using the variation of the cost functions f_t were proposed by Cesa-Bianchi et al. (2007) and derived by Hazan and Kale (2008). Bartlett et al. (2007) explore another adaptation technique for η_t where they adapt the step size to accommodate

both strongly and weakly convex functions. Our approach differs from previous approaches as it does not focus on a particular loss function or mistake bound. Instead, we view the problem of adapting the proximal function as a meta-learning problem. We then obtain a bound comparable to the bound obtained using the best proximal function chosen in hindsight.

2. Adaptive Proximal Functions

Examining the bounds (7) and (8), we see that most of the regret depends on dual norms of $f_t'(x_t)$, and the dual norms in turn depend on the choice of ψ. This naturally leads to the question of whether we can modify the proximal term ψ along the run of the algorithm in order to lower the contribution of the aforementioned norms. We achieve this goal by keeping second order information about the sequence f_t and allow ψ to vary on each round of the algorithms.

We begin by providing two corollaries based on previous work that give the regret of our base algorithms when the proximal function ψ_t is allowed to change. These corollaries are used in the sequel in our regret analysis. We assume that ψ_t is monotonically non-decreasing, that is, $\psi_{t+1}(x) \geq \psi_t(x)$. We also assume that ψ_t is 1-strongly convex with respect to a time-dependent semi-norm $\|\cdot\|_{\psi_t}$. Formally, ψ is 1-strongly convex with respect to $\|\cdot\|_{\psi}$ if

$$\psi(y) \geq \psi(x) + \langle \nabla\psi(x), y - x\rangle + \frac{1}{2}\|x-y\|_{\psi}^2 \ .$$

Strong convexity is guaranteed if and only if $B_{\psi_t}(x,y) \geq \frac{1}{2}\|x-y\|_{\psi_t}^2$. We also denote the dual norm of $\|\cdot\|_{\psi_t}$ by $\|\cdot\|_{\psi_t^*}$. For completeness, we provide the proofs of following two results in Appendix F, as they build straightforwardly on work by Duchi et al. (2010) and Xiao (2010). For the primal-dual subgradient update, the following bound holds.

Proposition 2 *Let the sequence $\{x_t\}$ be defined by the update (3). For any $x^* \in \mathcal{X}$,*

$$\sum_{t=1}^{T} f_t(x_t) + \varphi(x_t) - f_t(x^*) - \varphi(x^*) \leq \frac{1}{\eta}\psi_T(x^*) + \frac{\eta}{2}\sum_{t=1}^{T}\left\|f_t'(x_t)\right\|_{\psi_{t-1}^*}^2 \ . \tag{10}$$

For composite mirror descent algorithms a similar result holds.

Proposition 3 *Let the sequence $\{x_t\}$ be defined by the update (4). Assume w.l.o.g. that $\varphi(x_1) = 0$. For any $x^* \in \mathcal{X}$,*

$$\begin{aligned}&\sum_{t=1}^{T} f_t(x_t) + \varphi(x_t) - f_t(x^*) - \varphi(x^*) \\ &\leq \frac{1}{\eta}B_{\psi_1}(x^*, x_1) + \frac{1}{\eta}\sum_{t=1}^{T-1}\left[B_{\psi_{t+1}}(x^*, x_{t+1}) - B_{\psi_t}(x^*, x_{t+1})\right] + \frac{\eta}{2}\sum_{t=1}^{T}\left\|f_t'(x_t)\right\|_{\psi_t^*}^2 .\end{aligned} \tag{11}$$

The above corollaries allow us to prove regret bounds for a family of algorithms that iteratively modify the proximal functions ψ_t in attempt to lower the regret bounds.

INPUT: $\eta > 0$, $\delta \geq 0$
VARIABLES: $s \in \mathbb{R}^d, H \in \mathbb{R}^{d \times d}$, $g_{1:t,i} \in \mathbb{R}^t$ for $i \in \{1, \ldots, d\}$
INITIALIZE $x_1 = 0$, $g_{1:0} = []$
FOR $t = 1$ to T
Suffer loss $f_t(x_t)$
Receive subgradient $g_t \in \partial f_t(x_t)$ of f_t at x_t
UPDATE $g_{1:t} = [g_{1:t-1}\ g_t]$, $s_{t,i} = \|g_{1:t,i}\|_2$
SET $H_t = \delta I + \mathrm{diag}(s_t)$, $\psi_t(x) = \frac{1}{2}\langle x, H_t x\rangle$

Primal-Dual Subgradient Update (3):

$$x_{t+1} = \operatorname*{argmin}_{x \in \mathcal{X}} \left\{ \eta \left\langle \frac{1}{t} \sum_{\tau=1}^{t} g_\tau, x \right\rangle + \eta\varphi(x) + \frac{1}{t}\psi_t(x) \right\}.$$

Composite Mirror Descent Update (4):

$$x_{t+1} = \operatorname*{argmin}_{x \in \mathcal{X}} \left\{ \eta\langle g_t, x\rangle + \eta\varphi(x) + B_{\psi_t}(x, x_t) \right\}.$$

Figure 1: ADAGRAD with diagonal matrices

3. Diagonal Matrix Proximal Functions

We begin by restricting ourselves to using diagonal matrices to define matrix proximal functions and (semi)norms. This restriction serves a two-fold purpose. First, the analysis for the general case is somewhat complicated and thus the analysis of the diagonal restriction serves as a proxy for better understanding. Second, in problems with high dimension where we expect this type of modification to help, maintaining more complicated proximal functions is likely to be prohibitively expensive. Whereas earlier analysis requires a learning rate to slow changes between predictors x_t and x_{t+1}, we will instead automatically grow the proximal function we use to achieve asymptotically low regret. To remind the reader, $g_{1:t,i}$ is the ith row of the matrix obtained by concatenating the subgradients from iteration 1 through t in the online algorithm.

To provide some intuition for the algorithm we show in Algorithm 1, let us examine the problem

$$\min_s \sum_{t=1}^{T} \sum_{i=1}^{d} \frac{g_{t,i}^2}{s_i} \quad \text{s.t. } s \succeq 0,\ \langle 1, s\rangle \leq c\ .$$

This problem is solved by setting $s_i = \|g_{1:T,i}\|_2$ and scaling s so that $\langle s, 1\rangle = c$. To see this, we can write the Lagrangian of the minimization problem by introducing multipliers $\lambda \succeq 0$ and $\theta \geq 0$ to get

$$\mathcal{L}(s, \lambda, \theta) = \sum_{i=1}^{d} \frac{\|g_{1:T,i}\|_2^2}{s_i} - \langle \lambda, s\rangle + \theta(\langle 1, s\rangle - c).$$

Taking partial derivatives to find the infimum of $\mathcal{L}$, we see that $-\|g_{1:T,i}\|_2^2 / s_i^2 - \lambda_i + \theta = 0$, and complementarity conditions on $\lambda_i s_i$ (Boyd and Vandenberghe, 2004) imply that $\lambda_i = 0$. Thus we have $s_i = \theta^{-\frac{1}{2}} \|g_{1:T,i}\|_2$, and normalizing appropriately using θ gives that $s_i = c\,\|g_{1:T,i}\|_2 / \sum_{j=1}^{d} \|g_{1:T,j}\|_2$.

As a final note, we can plug s_i into the objective above to see

$$\inf_s \left\{ \sum_{t=1}^{T} \sum_{i=1}^{d} \frac{g_{t,i}^2}{s_i} \ : \ s \succeq 0, \langle 1, s \rangle \leq c \right\} = \frac{1}{c} \left(\sum_{i=1}^{d} \|g_{1:T,i}\|_2 \right)^2 . \tag{12}$$

Let $\mathrm{diag}(v)$ denote the diagonal matrix with diagonal v. It is natural to suspect that for s achieving the infimum in Equation (12), if we use a proximal function similar to $\psi(x) = \langle x, \mathrm{diag}(s)x \rangle$ with associated squared dual norm $\|x\|_{\psi^*}^2 = \langle x, \mathrm{diag}(s)^{-1} x \rangle$, we should do well lowering the gradient terms in the regret bounds (10) and (11).

To prove a regret bound for our Algorithm 1, we note that both types of updates suffer losses that include a term depending solely on the gradients obtained along their run. The following lemma is applicable to both updates, and was originally proved by Auer and Gentile (2000), though we provide a proof in Appendix C. McMahan and Streeter (2010) also give an identical lemma.

Lemma 4 *Let $g_t = f_t'(x_t)$ and $g_{1:t}$ and s_t be defined as in Algorithm 1. Then*

$$\sum_{t=1}^{T} \left\langle g_t, \mathrm{diag}(s_t)^{-1} g_t \right\rangle \leq 2 \sum_{i=1}^{d} \|g_{1:T,i}\|_2 \ .$$

To obtain a regret bound, we need to consider the terms consisting of the dual-norm of the subgradient in the regret bounds (10) and (11), which is $\|f_t'(x_t)\|_{\psi_t^*}^2$. When $\psi_t(x) = \langle x, (\delta I + \mathrm{diag}(s_t))x \rangle$, it is easy to see that the associated dual-norm is

$$\|g\|_{\psi_t^*}^2 = \left\langle g, (\delta I + \mathrm{diag}(s_t))^{-1} g \right\rangle .$$

From the definition of s_t in Algorithm 1, we clearly have $\|f_t'(x_t)\|_{\psi_t^*}^2 \leq \left\langle g_t, \mathrm{diag}(s_t)^{-1} g_t \right\rangle$. Note that if $s_{t,i} = 0$ then $g_{t,i} = 0$ by definition of $s_{t,i}$. Thus, for any $\delta \geq 0$, Lemma 4 implies

$$\sum_{t=1}^{T} \left\| f_t'(x_t) \right\|_{\psi_t^*}^2 \leq 2 \sum_{i=1}^{d} \|g_{1:T,i}\|_2 . \tag{13}$$

To obtain a bound for a primal-dual subgradient method, we set $\delta \geq \max_t \|g_t\|_\infty$, in which case $\|g_t\|_{\psi_{t-1}^*}^2 \leq \left\langle g_t, \mathrm{diag}(s_t)^{-1} g_t \right\rangle$, and we follow the same lines of reasoning to achieve the inequality (13).

It remains to bound the various Bregman divergence terms for Corollary 3 and the term $\psi_T(x^*)$ for Corollary 2. We focus first on the composite mirror-descent update. Examining the bound (11) and Algorithm 1, we notice that

$$\begin{aligned} B_{\psi_{t+1}}(x^*, x_{t+1}) - B_{\psi_t}(x^*, x_{t+1}) &= \frac{1}{2} \left\langle x^* - x_{t+1}, \mathrm{diag}(s_{t+1} - s_t)(x^* - x_{t+1}) \right\rangle \\ &\leq \frac{1}{2} \max_i (x_i^* - x_{t+1,i})^2 \left\| s_{t+1} - s_t \right\|_1 . \end{aligned}$$

Since $\|s_{t+1} - s_t\|_1 = \langle s_{t+1} - s_t, 1 \rangle$ and $\langle s_T, 1 \rangle = \sum_{i=1}^{d} \|g_{1:T,i}\|_2$, we have

$$\begin{aligned} \sum_{t=1}^{T-1} B_{\psi_{t+1}}(x^*, x_{t+1}) - B_{\psi_t}(x^*, x_{t+1}) \ &\leq \ \frac{1}{2} \sum_{t=1}^{T-1} \|x^* - x_{t+1}\|_\infty^2 \langle s_{t+1} - s_t, 1 \rangle \\ &\leq \ \frac{1}{2} \max_{t \leq T} \|x^* - x_t\|_\infty^2 \sum_{i=1}^{d} \|g_{1:T,i}\|_2 - \frac{1}{2} \|x^* - x_1\|_\infty^2 \langle s_1, 1 \rangle \ . \end{aligned} \tag{14}$$

We also have

$$\psi_T(x^*) = \delta \|x^*\|_2^2 + \langle x^*, \operatorname{diag}(s_T) x^* \rangle \le \delta \|x^*\|_2^2 + \|x^*\|_\infty^2 \sum_{i=1}^d \|g_{1:T,i}\|_2 .$$

Combining the above arguments with Corollaries 2 and 3, and using (14) with the fact that $B_{\psi_1}(x^*, x_1) \le \frac{1}{2}\|x^* - x_1\|_\infty^2 \langle 1, s_1 \rangle$, we have proved the following theorem.

Theorem 5 *Let the sequence $\{x_t\}$ be defined by Algorithm 1. For x_t generated using the primal-dual subgradient update (3) with $\delta \ge \max_t \|g_t\|_\infty$, for any $x^* \in \mathcal{X}$,*

$$R_\phi(T) \le \frac{\delta}{\eta}\|x^*\|_2^2 + \frac{1}{\eta}\|x^*\|_\infty^2 \sum_{i=1}^d \|g_{1:T,i}\|_2 + \eta \sum_{i=1}^d \|g_{1:T,i}\|_2 .$$

For x_t generated using the composite mirror-descent update (4), for any $x^* \in \mathcal{X}$

$$R_\phi(T) \le \frac{1}{2\eta} \max_{t \le T} \|x^* - x_t\|_\infty^2 \sum_{i=1}^d \|g_{1:T,i}\|_2 + \eta \sum_{i=1}^d \|g_{1:T,i}\|_2 .$$

The above theorem is a bit unwieldy. We thus perform a few algebraic simplifications to get the next corollary, which has a more intuitive form. Let us assume that $\mathcal{X}$ is compact and set $D_\infty = \sup_{x \in \mathcal{X}} \|x - x^*\|_\infty$. Furthermore, define

$$\gamma_T \triangleq \sum_{i=1}^d \|g_{1:T,i}\|_2 = \inf_s \left\{ \sum_{t=1}^T \left\langle g_t, \operatorname{diag}(s)^{-1} g_t \right\rangle : \langle 1, s \rangle \le \sum_{i=1}^d \|g_{1:T,i}\|_2 , \; s \succeq 0 \right\} .$$

Also w.l.o.g. let $0 \in \mathcal{X}$. The following corollary is immediate (this is equivalent to Corollary 1, though we have moved the $\sqrt{d}$ term in the earlier bound).

Corollary 6 *Assume that D_∞ and γ_T are defined as above. For $\{x_t\}$ generated by Algorithm 1 using the primal-dual subgradient update (3) with $\eta = \|x^*\|_\infty$, for any $x^* \in \mathcal{X}$ we have*

$$R_\phi(T) \le 2\|x^*\|_\infty \gamma_T + \delta \frac{\|x^*\|_2^2}{\|x^*\|_\infty} \le 2\|x^*\|_\infty \gamma_T + \delta \|x^*\|_1 .$$

Using the composite mirror descent update (4) to generate $\{x_t\}$ and setting $\eta = D_\infty/\sqrt{2}$, we have

$$R_\phi(T) \le \sqrt{2} D_\infty \sum_{i=1}^d \|g_{1:T,i}\|_2 = \sqrt{2} D_\infty \gamma_T .$$

We now give a short derivation of Corollary 1 from the introduction: use Theorem 5, Corollary 6, and the fact that

$$\inf_s \left\{ \sum_{t=1}^T \sum_{i=1}^d \frac{g_{t,i}^2}{s_i} \; : \; s \succeq 0, \langle 1, s \rangle \le d \right\} = \frac{1}{d} \left(\sum_{i=1}^d \|g_{1:T,i}\|_2 \right)^2 .$$

as in (12) in the beginning of Section 3. Plugging the γ_T term in from Corollary 6 and multiplying D_∞ by $\sqrt{d}$ completes the proof of the corollary.

As discussed in the introduction, Algorithm 1 should have lower regret than non-adaptive algorithms on sparse data, though this depends on the geometry of the underlying optimization space $\mathcal{X}$. For example, suppose that our learning problem is a logistic regression with 0/1-valued features. Then the gradient terms are likewise based on 0/1-valued features and sparse, so the gradient terms in the bound $\sum_{i=1}^{d} \|g_{1:T,i}\|_2$ should all be much smaller than $\sqrt{T}$. If some features appear much more frequently than others, then the infimal representation of γ_T and the infimal equality in Corollary 1 show that we have significantly lower regret by using higher learning rates for infrequent features and lower learning rates on commonly appearing features. Further, if the optimal predictor is relatively dense, as is often the case in predictions problems with sparse inputs, then $\|x^*\|_\infty$ is the best p-norm we can have in the regret.

More precisely, McMahan and Streeter (2010) show that if $\mathcal{X}$ is contained within an ℓ_∞ ball of radius R and contains an ℓ_∞ ball of radius r, then the bound in the above corollary is within a factor of $\sqrt{2}R/r$ of the regret of the best diagonal proximal matrix, chosen in hindsight. So, for example, if $\mathcal{X} = \{x \in \mathbb{R}^d : \|x\|_p \leq C\}$, then $R/r = d^{1/p}$, which shows that the domain $\mathcal{X}$ does effect the guarantees we can give on optimality of ADAGRAD.

4. Full Matrix Proximal Functions

In this section we derive and analyze new updates when we estimate a full matrix for the divergence ψ_t instead of a diagonal one. In this generalized case, we use the root of the matrix of outer products of the gradients that we have observed to update our parameters. As in the diagonal case, we build on intuition garnered from an optimization problem, and in particular, we seek a matrix S which is the solution to the following minimization problem:

$$\min_S \sum_{t=1}^{T} \left\langle g_t, S^{-1} g_t \right\rangle \quad \text{s.t. } S \succeq 0, \ \operatorname{tr}(S) \leq c \,. \tag{15}$$

The solution is obtained by defining $G_t = \sum_{\tau=1}^{t} g_\tau g_\tau^\top$ and setting S to be a normalized version of the root of G_T, that is, $S = c\, G_T^{1/2} / \operatorname{tr}(G_T^{1/2})$. For a proof, see Lemma 15 in Appendix E, which also shows that when G_T is not full rank we can instead use its pseudo-inverse. If we iteratively use divergences of the form $\psi_t(x) = \left\langle x, G_t^{1/2} x \right\rangle$, we might expect as in the diagonal case to attain low regret by collecting gradient information. We achieve our low regret goal by employing a similar doubling lemma to Lemma 4 and bounding the gradient norm terms. The resulting algorithm is given in Algorithm 2, and the next theorem provides a quantitative analysis of the brief motivation above.

Theorem 7 *Let G_t be the outer product matrix defined above and the sequence $\{x_t\}$ be defined by Algorithm 2. For x_t generated using the primal-dual subgradient update of (3) and $\delta \geq \max_t \|g_t\|_2$, for any $x^* \in \mathcal{X}$*

$$R_\phi(T) \leq \frac{\delta}{\eta} \|x^*\|_2^2 + \frac{1}{\eta} \|x^*\|_2^2 \operatorname{tr}(G_T^{1/2}) + \eta \operatorname{tr}(G_T^{1/2}).$$

For x_t generated with the composite mirror-descent update of (4), if $x^* \in \mathcal{X}$ and $\delta \geq 0$

$$R_\phi(T) \leq \frac{\delta}{\eta} \|x^*\|_2^2 + \frac{1}{2\eta} \max_{t \leq T} \|x^* - x_t\|_2^2 \operatorname{tr}(G_T^{1/2}) + \eta \operatorname{tr}(G_T^{1/2}).$$

INPUT: $\eta > 0$, $\delta \geq 0$
VARIABLES: $S_t \in \mathbb{R}^{d\times d}$, $H_t \in \mathbb{R}^{d\times d}$, $G_t \in \mathbb{R}^{d\times d}$
INITIALIZE $x_1 = 0$, $S_0 = 0$, $H_0 = 0$, $G_0 = 0$
FOR $t = 1$ to T
Suffer loss $f_t(x_t)$
Receive subgradient $g_t \in \partial f_t(x_t)$ of f_t at x_t
UPDATE $G_t = G_{t-1} + g_t g_t^\top$, $S_t = G_t^{\frac{1}{2}}$
SET $H_t = \delta I + S_t$, $\psi_t(x) = \frac{1}{2}\langle x, H_t x\rangle$

Primal-Dual Subgradient Update ((3)):

$$x_{t+1} = \operatorname*{argmin}_{x\in\mathcal{X}} \left\{ \eta \left\langle \frac{1}{t}\sum_{\tau=1}^{t} g_\tau, x \right\rangle + \eta\varphi(x) + \frac{1}{t}\psi_t(x) \right\}.$$

Composite Mirror Descent Update ((4)):

$$x_{t+1} = \operatorname*{argmin}_{x\in\mathcal{X}} \left\{ \eta\langle g_t, x\rangle + \eta\varphi(x) + B_{\psi_t}(x, x_t) \right\}.$$

Figure 2: ADAGRAD with full matrices

Proof To begin, we consider the difference between the divergence terms at time $t+1$ and time t from the regret (11) in Corollary 3. Let $\lambda_{\max}(M)$ denote the largest eigenvalue of a matrix M. We have

$$\begin{aligned} B_{\psi_{t+1}}(x^*, x_{t+1}) - B_{\psi_t}(x^*, x_{t+1}) &= \frac{1}{2}\left\langle x^* - x_{t+1}, ({G_{t+1}}^{1/2} - {G_t}^{1/2})(x^* - x_{t+1}) \right\rangle \\ &\leq \frac{1}{2}\|x^* - x_{t+1}\|_2^2 \lambda_{\max}(G_{t+1}^{1/2} - G_t^{1/2}) \leq \frac{1}{2}\|x^* - x_{t+1}\|_2^2 \operatorname{tr}(G_{t+1}^{1/2} - G_t^{1/2}) . \end{aligned}$$

For the last inequality we used the fact that the trace of a matrix is equal to the sum of its eigenvalues along with the property ${G_{t+1}}^{1/2} - {G_t}^{1/2} \succeq 0$ (see Lemma 13 in Appendix B) and therefore $\operatorname{tr}(G_{t+1}^{1/2} - G_t^{1/2}) \geq \lambda_{\max}(G_{t+1}^{1/2} - G_t^{1/2})$. Thus, we get

$$\sum_{t=1}^{T-1} B_{\psi_{t+1}}(x^*, x_{t+1}) - B_{\psi_t}(x^*, x_{t+1}) \leq \frac{1}{2}\sum_{t=1}^{T-1} \|x^* - x_{t+1}\|_2^2 \left(\operatorname{tr}(G_{t+1}^{1/2}) - \operatorname{tr}(G_t^{1/2}) \right) .$$

Now we use the fact that G_1 is a rank 1 PSD matrix with non-negative trace to see that

$$\begin{aligned} &\sum_{t=1}^{T-1} \|x^* - x_{t+1}\|_2^2 \left(\operatorname{tr}(G_{t+1}^{1/2}) - \operatorname{tr}(G_t^{1/2}) \right) \\ &\qquad \leq \max_{t\leq T} \|x^* - x_t\|_2^2 \operatorname{tr}({G_T}^{1/2}) - \|x^* - x_1\|_2^2 \operatorname{tr}(G_1^{1/2}) . \end{aligned} \tag{16}$$

It remains to bound the gradient terms common to all our bounds. We use the following three lemmas, which essentially directly applicable. We prove the first two in Appendix D.

Lemma 8 *Let $B \succeq 0$ and $B^{-1/2}$ denote the root of the inverse of B when $B \succ 0$ and the root of the pseudo-inverse of B otherwise. For any ν such that $B - \nu gg^\top \succeq 0$ the following inequality holds.*

$$2\operatorname{tr}((B - \nu gg^\top)^{1/2}) \leq 2\operatorname{tr}(B^{1/2}) - \nu \operatorname{tr}(B^{-1/2} gg^\top) .$$

Lemma 9 *Let $\delta \geq \|g\|_2$ and $A \succeq 0$, then $\left\langle g, (\delta I + A^{1/2})^{-1} g \right\rangle \leq \left\langle g, \left((A + gg^\top)^\dagger\right)^{1/2} g \right\rangle$.*

Lemma 10 *Let $S_t = {G_t}^{1/2}$ be as defined in Algorithm 2 and $A^\dagger$ denote the pseudo-inverse of A. Then*

$$\sum_{t=1}^{T} \left\langle g_t, S_t^\dagger g_t \right\rangle \leq 2 \sum_{t=1}^{T} \left\langle g_t, S_T^\dagger g_t \right\rangle = 2 \operatorname{tr}({G_T}^{1/2}) \, .$$

Proof We prove the lemma by induction. The base case is immediate, since we have

$$\left\langle g_1, (G_1^\dagger)^{1/2} g_1 \right\rangle = \frac{\langle g_1, g_1 \rangle}{\|g_1\|_2} = \|g_1\|_2 \leq 2 \|g_1\|_2 .$$

Now, assume the lemma is true for $T-1$, so from the inductive assumption we get

$$\sum_{t=1}^{T} \left\langle g_t, S_t^\dagger g_t \right\rangle \leq 2 \sum_{t=1}^{T-1} \left\langle g_t, S_{T-1}^\dagger g_t \right\rangle + \left\langle g_T, S_T^\dagger g_T \right\rangle \, .$$

Since S_{T-1} does not depend on t we can rewrite $\sum_{t=1}^{T-1} \left\langle g_t, S_{T-1}^\dagger g_t \right\rangle$ as

$$\operatorname{tr}\left(S_{T-1}^\dagger, \sum_{t=1}^{T-1} g_t g_t^\top \right) = \operatorname{tr}((G_{T-1}^\dagger)^{1/2} G_{T-1}) \, ,$$

where the right-most equality follows from the definitions of S_t and G_t. Therefore, we get

$$\begin{aligned} \sum_{t=1}^{T} \left\langle g_t, S_t^\dagger g_t \right\rangle &\leq 2 \operatorname{tr}((G_{T-1}^\dagger)^{1/2} G_{T-1}) + \left\langle g_T, (G_T^\dagger)^{1/2} g_T \right\rangle \\ &= 2 \operatorname{tr}(G_{T-1}^{1/2}) + \left\langle g_T, (G_T^\dagger)^{1/2} g_T \right\rangle \, . \end{aligned}$$

Using Lemma 8 with the substitution $B = G_T$, $\nu = 1$, and $g = g_t$ lets us exploit the concavity of the function $\operatorname{tr}(A^{1/2})$ to bound the above sum by $2 \operatorname{tr}(G_T^{1/2})$. ▲

We can now finalize our proof of the theorem. As in the diagonal case, we have that the squared dual norm (seminorm when $\delta = 0$) associated with ψ_t is

$$\|x\|_{\psi_t^*}^2 = \left\langle x, (\delta I + S_t)^{-1} x \right\rangle \, .$$

Thus it is clear that $\|g_t\|_{\psi_t^*}^2 \leq \left\langle g_t, S_t^\dagger g_t \right\rangle$. For the dual-averaging algorithms, we use Lemma 9 above show that $\|g_t\|_{\psi_{t-1}^*}^2 \leq \left\langle g_t, S_t^\dagger g_t \right\rangle$ so long as $\delta \geq \|g_t\|_2$. Lemma 10's doubling inequality then implies that

$$\sum_{t=1}^{T} \left\| f_t'(x_t) \right\|_{\psi_t^*}^2 \leq 2 \operatorname{tr}(G_T^{1/2}) \quad \text{and} \quad \sum_{t=1}^{T} \left\| f_t'(x_t) \right\|_{\psi_{t-1}^*}^2 \leq 2 \operatorname{tr}(G_T^{1/2}) \tag{17}$$

for the mirror-descent and primal-dual subgradient algorithm, respectively.

To finish the proof, Note that $B_{\psi_1}(x^*, x_1) \leq \frac{1}{2} \|x^* - x_1\|_2^2 \operatorname{tr}(G_1^{1/2})$ when $\delta = 0$. By combining this with the first of the bounds (17) and the bound (16) on $\sum_{t=1}^{T-1} B_{\psi_{t+1}}(x^*, x^{t+1}) - B_{\psi_t}(x^*, x^{t+1})$, Corollary 3 gives the theorem's statement for the mirror-descent family of algorithms. Combining the

fact that $\sum_{t=1}^{T} \|f_t'(x_t)\|_{\psi_{t-1}^*}^2 \leq 2\operatorname{tr}(G_T^{1/2})$ and the bound (16) with Corollary 2 gives the desired bound on $R_\phi(T)$ for the primal-dual subgradient algorithms, which completes the proof of the theorem. ■

As before, we can give a corollary that simplifies the bound implied by Theorem 7. The infimal equality in the corollary uses Lemma 15 in Appendix B. The corollary underscores that for learning problems in which there is a rotation U of the space for which the gradient vectors g_t have small inner products $\langle g_t, Ug_t \rangle$ (essentially a sparse basis for the g_t) then using full-matrix proximal functions can attain significantly lower regret.

Corollary 11 *Assume that $\varphi(x_1) = 0$. Then the regret of the sequence $\{x_t\}$ generated by Algorithm 2 when using the primal-dual subgradient update with $\eta = \|x^*\|_2$ is*

$$R_\phi(T) \leq 2\|x^*\|_2 \operatorname{tr}(G_T^{1/2}) + \delta\|x^*\|_2 \ .$$

Let $\mathcal{X}$ be compact set so that $\sup_{x\in\mathcal{X}} \|x - x^*\|_2 \leq D$. Taking $\eta = D/\sqrt{2}$ and using the composite mirror descent update with $\delta = 0$, we have

$$R_\phi(T) \leq \sqrt{2}D\operatorname{tr}(G_T^{1/2}) = \sqrt{2d}D\sqrt{\inf_S \left\{ \sum_{t=1}^{T} g_t^\top S^{-1} g_t \ : \ S \succeq 0, \operatorname{tr}(S) \leq d \right\}} \ .$$

5. Derived Algorithms

In this section, we derive updates using concrete regularization functions φ and settings of the domain $\mathcal{X}$ for the ADAGRAD framework. We focus on showing how to solve Equations (3) and (4) with the diagonal matrix version of the algorithms we have presented. We focus on the diagonal case for two reasons. First, the updates often take closed-form in this case and carry some intuition. Second, the diagonal case is feasible to implement in very high dimensions, whereas the full matrix version is likely to be confined to a few thousand dimensions. We also discuss how to efficiently compute the updates when the gradient vectors are sparse.

We begin by noting a simple but useful fact. Let G_t denote either the outer product matrix of gradients or its diagonal counterpart and let $H_t = \delta I + G_t^{1/2}$, as usual. Simple algebraic manipulations yield that each of the updates (3) and (4) in the prequel can be written in the following form (omitting the stepsize η):

$$x_{t+1} = \operatorname*{argmin}_{x\in\mathcal{X}} \left\{ \langle u, x \rangle + \varphi(x) + \frac{1}{2}\langle x, H_t x \rangle \right\} \ . \tag{18}$$

In particular, at time t for the RDA update, we have $u = \eta t \bar{g}_t$. For the composite gradient update (4),

$$\eta\langle g_t, x \rangle + \frac{1}{2}\langle x - x_t, H_t(x - x_t)\rangle = \langle \eta g_t - H_t x_t, x\rangle + \frac{1}{2}\langle x, H_t x\rangle + \frac{1}{2}\langle x_t, H_t x_t\rangle$$

so that $u = \eta g_t - H_t x_t$. We now derive algorithms for solving the general update (18). Since most of the derivations are known, we generally provide only the closed-form solutions or algorithms for the solutions in the remainder of the subsection, deferring detailed derivations to Appendix G for the interested reader.

5.1 ℓ_1-regularization

We begin by considering how to solve the minimization problems necessary for Algorithm 1 with diagonal matrix divergences and $\varphi(x) = \lambda \|x\|_1$. We consider the two updates we proposed and denote the ith diagonal element of the matrix $H_t = \delta I + \mathrm{diag}(s_t)$ from Algorithm 1 by $H_{t,ii} = \delta + \|g_{1:t,i}\|_2$. For the primal-dual subgradient update, the solution to (3) amounts to the following simple update for $x_{t+1,i}$:

$$x_{t+1,i} = \mathrm{sign}\left(-\bar{g}_{t,i}\right) \frac{\eta t}{H_{t,ii}} \left[|\bar{g}_{t,i}| - \lambda\right]_+ . \tag{19}$$

Comparing the update (19) to the standard dual averaging update (Xiao, 2010), which is

$$x_{t+1,i} = \mathrm{sign}\left(-\bar{g}_{t,i}\right) \eta \sqrt{t} \left[|\bar{g}_{t,i}| - \lambda\right]_+ ,$$

it is clear that the difference distills to the step size employed for each coordinate. Our generalization of RDA yields a dedicated step size for each coordinate inversely proportional to the time-based norm of the coordinate in the sequence of gradients. Due to the normalization by this term the step size scales *linearly* with t, so when $H_{t,ii}$ is small, gradient information on coordinate i is quickly incorporated.

The composite mirror-descent update (4) has a similar form that essentially amounts to iterative shrinkage and thresholding, where the shrinkage differs per coordinate:

$$x_{t+1,i} = \mathrm{sign}\left(x_{t,i} - \frac{\eta}{H_{t,ii}} g_{t,i}\right) \left[\left|x_{t,i} - \frac{\eta}{H_{t,ii}} g_{t,i}\right| - \frac{\lambda\eta}{H_{t,ii}}\right]_+ .$$

We compare the actual performance of the newly derived algorithms to previously studied versions in the next section.

For both updates it is clear that we can perform "lazy" computation when the gradient vectors are sparse, a frequently occurring setting when learning for instance from text corpora. Suppose that from time step t_0 through t, the ith component of the gradient is 0. Then we can evaluate the above updates on demand since $H_{t,ii}$ remains intact. For composite mirror-descent, at time t when $x_{t,i}$ is needed, we update

$$x_{t,i} = \mathrm{sign}(x_{t_0,i}) \left[|x_{t_0,i}| - \frac{\lambda\eta}{H_{t_0,ii}}(t - t_0)\right]_+ .$$

Even simpler just in time evaluation can be performed for the the primal-dual subgradient update. Here we need to keep an unnormalized version of the average $\bar{g}_t$. Concretely, we keep track of $u_t = t\bar{g}_t = \sum_{\tau=1}^{t} g_\tau = u_{t-1} + g_t$, then use the update (19):

$$x_{t,i} = \mathrm{sign}(-u_{t,i}) \frac{\eta t}{H_{t,ii}} \left[\frac{|u_{t,i}|}{t} - \lambda\right]_+ ,$$

where H_t can clearly be updated lazily in a similar fashion.

5.2 ℓ_1-ball Projections

We next consider the setting in which $\varphi \equiv 0$ and $\mathcal{X} = \{x : \|x\|_1 \leq c\}$, for which it is straightforward to adapt efficient solutions to continuous quadratic knapsack problems (Brucker, 1984). We

INPUT: $v \succeq 0$, $a \succeq 0$, $c \geq 0$.
IF $\sum_i v_i \leq c$ RETURN $z^* = v$
SORT v_i/a_i into $\mu = \left[v_{i_j}/a_{i_j}\right]$ s.t. $v_{i_j}/a_{i_j} \geq v_{i_{j+1}}/a_{i_{j+1}}$
SET $\rho := \max\left\{\rho : \sum_{j=1}^{\rho} a_{i_j} v_{i_j} - \frac{v_{i_\rho}}{a_{i_\rho}} \sum_{j=1}^{\rho} a_{i_j}^2 < c\right\}$
SET $\theta = \frac{\sum_{j=1}^{\rho} a_{i_j} v_{i_j} - c}{\sum_{j=1}^{\rho} a_{i_j}^2}$
RETURN z^* where $z_i^* = [v_i - \theta a_i]_+$.

Figure 3: Project $v \succeq 0$ to $\{z : \langle a, z\rangle \leq c, z \succeq 0\}$.

use the matrix $H_t = \delta I + \mathrm{diag}(G_t)^{1/2}$ from Algorithm 1. We provide a brief derivation sketch and an $O(d \log d)$ algorithm in this section. First, we convert the problem (18) into a projection problem onto a scaled ℓ_1-ball. By making the substitutions $z = H^{1/2}x$ and $A = H^{-1/2}$, it is clear that problem (18) is equivalent to

$$\min_z \left\|z + H^{-1/2}u\right\|_2^2 \quad \text{s.t.} \quad \|Az\|_1 \leq c \; .$$

Now, by appropriate choice of $v = -H^{-1/2}u = -\eta t H_t^{-1/2}\overline{g}_t$ for the primal-dual update (3) and $v = H_t^{1/2}x_t - \eta H_t^{-1/2}g_t$ for the mirror-descent update (4), we arrive at the problem

$$\min_z \frac{1}{2}\|z - v\|_2^2 \quad \text{s.t.} \quad \sum_{i=1}^{d} a_i |z_i| \leq c \; . \tag{20}$$

We can clearly recover x_{t+1} from the solution z^* to the projection (20) via $x_{t+1} = H_t^{-1/2}z^*$.

By the symmetry of the objective (20), we can assume without loss of generality that $v \succeq 0$ and constrain $z \succeq 0$, and a bit of manipulation with the Lagrangian (see Appendix G) for the problem shows that the solution z^* has the form

$$z_i^* = \begin{cases} v_i - \theta^* a_i & \text{if } v_i \geq \theta^* a_i \\ 0 & \text{otherwise} \end{cases}$$

for some $\theta^* \geq 0$. The algorithm in Figure 3 constructs the optimal θ and returns z^*.

5.3 ℓ_2 Regularization

We now turn to the case where $\varphi(x) = \lambda\|x\|_2$ while $\mathcal{X} = \mathbb{R}^d$. This type of regularization is useful for zeroing multiple weights in a group, for example in multi-task or multiclass learning (Obozinski et al., 2007). Recalling the general proximal step (18), we must solve

$$\min_x \langle u, x\rangle + \frac{1}{2}\langle x, Hx\rangle + \lambda\|x\|_2 \; . \tag{21}$$

There is no closed form solution for this problem, but we give an efficient bisection-based procedure for solving (21). We start by deriving the dual. Introducing a variable $z = x$, we get the equivalent problem of minimizing $\langle u, x\rangle + \frac{1}{2}\langle x, Hx\rangle + \lambda\|z\|_2$ subject to $x = z$. With Lagrange multipliers α for the equality constraint, we obtain the Lagrangian

$$\mathcal{L}(x, z, \alpha) = \langle u, x\rangle + \frac{1}{2}\langle x, Hx\rangle + \lambda\|z\|_2 + \langle \alpha, x - z\rangle \; .$$

INPUT: $u \in \mathbb{R}^d$, $H \succeq 0$, $\lambda > 0$.
IF $\|u\|_2 \leq \lambda$
  RETURN $x = 0$
SET $v = H^{-1}u$, $\theta_{\max} = \|v\|_2 / \lambda - 1/\sigma_{\min}(H)$
   $\theta_{\min} = \|v\|_2 / \lambda - 1/\sigma_{\max}(H)$
WHILE $\theta_{\max} - \theta_{\min} > \varepsilon$
  SET $\theta = (\theta_{\max} + \theta_{\min})/2$, $\alpha(\theta) = -(H^{-1} + \theta I)^{-1} v$
  IF $\|\alpha(\theta)\|_2 > \lambda$
    SET $\theta_{\min} = \theta$
  ELSE
    SET $\theta_{\max} = \theta$
RETURN $x = -H^{-1}(u + \alpha(\theta))$

Figure 4: Minimize $\langle u, x \rangle + \frac{1}{2} \langle x, Hx \rangle + \lambda \|x\|_2$

Taking the infimum of $\mathcal{L}$ with respect to the primal variables x and z, we see that the infimum is attained at $x = -H^{-1}(u+\alpha)$. Coupled with the fact that $\inf_z \lambda \|z\|_2 - \langle \alpha, z \rangle = -\infty$ unless $\|\alpha\|_2 \leq \lambda$, in which case the infimum is 0, we arrive at the dual form

$$\inf_{x,z} \mathcal{L}(x,z,\alpha) = \begin{cases} -\frac{1}{2} \left\langle u+\alpha, H^{-1}(u+\alpha) \right\rangle & \text{if } \|\alpha\|_2 \leq \lambda \\ -\infty & \text{otherwise.} \end{cases}$$

Setting $v = H^{-1}u$, we further distill the dual to

$$\min_{\alpha} \ \langle v, \alpha \rangle + \frac{1}{2} \left\langle \alpha, H^{-1}\alpha \right\rangle \ \text{ s.t. } \|\alpha\|_2 \leq \lambda \, . \tag{22}$$

We can solve problem (22) efficiently using a bisection search of its equivalent representation in Lagrange form,

$$\min_{\alpha} \ \langle v, \alpha \rangle + \frac{1}{2} \left\langle \alpha, H^{-1}\alpha \right\rangle + \frac{\theta}{2} \|\alpha\|_2^2 \, ,$$

where $\theta > 0$ is an unknown scalar. The solution to the latter as a function of θ is clearly $\alpha(\theta) = -(H^{-1}+\theta I)^{-1}v = -(H^{-1}+\theta I)^{-1}H^{-1}u$. Since $\|\alpha(\theta)\|_2$ is monotonically decreasing in θ (consider the the eigen-decomposition of the positive definite H^{-1}), we can simply perform a bisection search over θ, checking at each point whether $\|\alpha(\theta)\|_2 \gtrless \lambda$.

To find initial upper and lower bounds on θ, we note that

$$(1/\sigma_{\max}(H) + \theta)^{-1} \|v\|_2 \leq \|\alpha(\theta)\|_2 \leq (1/\sigma_{\min}(H) + \theta)^{-1} \|v\|_2$$

where $\sigma_{\max}(H)$ denotes the maximum singular value of H and $\sigma_{\min}(H)$ the minimum. To guarantee $\|\alpha(\theta_{\max})\|_2 \leq \lambda$, we thus set $\theta_{\max} = \|v\|_2 / \lambda - 1/\sigma_{\max}(H)$. Similarly, for $\theta_{\min}$ we see that so long as $\theta \geq \|v\|_2 / \lambda - 1/\sigma_{\min}(H)$ we have $\|\alpha(\theta)\|_2 \geq \lambda$. The fact that $\partial \|x\|_2 = \{z : \|z\|_2 \leq 1\}$ when $x = 0$ implies that the solution for the original problem (21) is $x = 0$ if and only if $\|u\|_2 \leq \lambda$. We provide pseudocode for solving (21) in Algorithm 4.

5.4 ℓ_∞ Regularization

We again let $\mathcal{X} = \mathbb{R}^d$ but now choose $\varphi(x) = \lambda \|x\|_\infty$. This type of update, similarly to ℓ_2, zeroes groups of variables, which is handy in finding structurally sparse solutions for multitask or multi-class problems. Solving the ℓ_∞ regularized problem amounts to

$$\min_x \ \langle u, x \rangle + \frac{1}{2} \langle x, Hx \rangle + \lambda \|x\|_\infty \ . \tag{23}$$

The dual of this problem is a modified ℓ_1-projection problem. As in the case of ℓ_2 regularization, we introduce an equality constrained variable $z = x$ with associated Lagrange multipliers $\alpha \in \mathbb{R}^d$ to obtain

$$\mathcal{L}(x, z, \alpha) = \langle u, x \rangle + \frac{1}{2} \langle x, Hx \rangle + \lambda \|z\|_\infty + \langle \alpha, x - z \rangle \ .$$

Performing identical manipulations to the ℓ_2 case, we take derivatives and get that $x = -H^{-1}(u + \alpha)$ and, similarly, unless $\|\alpha\|_1 \le \lambda$, $\inf_z \mathcal{L}(x, z, \alpha) = -\infty$. Thus the dual problem for (23) is

$$\max_\alpha \ -\frac{1}{2}(u + \alpha) H^{-1}(u + \alpha) \ \text{ s.t. } \|\alpha\|_1 \le \lambda \ .$$

When H is diagonal we can find the optimal α^* using the generalized ℓ_1-projection in Algorithm 3, then reconstruct the optimal x via $x = -H^{-1}(u + \alpha^*)$.

5.5 Mixed-norm Regularization

Finally, we combine the above results to show how to solve problems with matrix-valued inputs $X \in \mathbb{R}^{d \times k}$, where $X = [\bar{x}_1 \ \cdots \ \bar{x}_d]^\top$. We consider mixed-norm regularization, which is very useful for encouraging sparsity across several tasks (Obozinski et al., 2007). Now φ is an ℓ_1/ℓ_p norm, that is, $\varphi(X) = \lambda \sum_{i=1}^d \|\bar{x}_i\|_p$. By imposing an ℓ_1-norm over p-norms of the rows of X, entire rows are nulled at once.

When $p \in \{2, \infty\}$ and the proximal H in (18) is diagonal, the previous algorithms can be readily used to solve the mixed norm problems. We simply maintain diagonal matrix information for each of the rows $\bar{x}_i$ of X separately, then solve one of the previous updates for each row independently. We use this form of regularization in our experiments with multiclass prediction problems in the next section.

6. Experiments

We performed experiments with several real world data sets with different characteristics: the ImageNet image database (Deng et al., 2009), the Reuters RCV1 text classification data set (Lewis et al., 2004), the MNIST multiclass digit recognition problem, and the census income data set from the UCI repository (Asuncion and Newman, 2007). For uniformity across experiments, we focus on the completely online (fully stochastic) optimization setting, in which at each iteration the learning algorithm receives a single example. We measure performance using two metrics: the online loss or error and the test set performance of the predictor the learning algorithm outputs at the end of a single pass through the training data. We also give some results that show how imposing sparsity constraints (in the form of ℓ_1 and mixed-norm regularization) affects the learning algorithm's performance. One benefit of the ADAGRAD framework is its ability to straightforwardly generalize to

	RDA	FB	ADAGRAD-RDA	ADAGRAD-FB	PA	AROW
ECAT	.051 (.099)	.058 (.194)	**.044** (**.086**)	**.044** (.238)	.059	.049
CCAT	.064 (.123)	.111 (.226)	**.053** (**.105**)	**.053** (.276)	.107	.061
GCAT	.046 (.092)	.056 (.183)	**.040** (**.080**)	**.040** (.225)	.066	.044
MCAT	.037 (.074)	.056 (.146)	.035 (**.063**)	**.034** (.176)	.053	.039

Table 1: Test set error rates and proportion non-zero (in parenthesis) on Reuters RCV1.

domain constraints $\mathcal{X} \neq \mathbb{R}^d$ and arbitrary regularization functions φ, in contrast to previous adaptive online algorithms.

We experiment with RDA (Xiao, 2010), FOBOS(Duchi and Singer, 2009), adaptive RDA, adaptive FOBOS, the Passive-Aggressive (PA) algorithm (Crammer et al., 2006), and AROW (Crammer et al., 2009). To remind the reader, PA is an online learning procedure with the update

$$x_{t+1} = \operatorname*{argmin}_x \left[1 - y_t \left\langle z_t, x \right\rangle\right]_+ + \frac{\lambda}{2} \left\|x - x_t\right\|_2^2 ,$$

where λ is a regularization parameter. PA's update is similar to the update employed by AROW (see (9)), but the latter maintains second order information on x. By using a representer theorem it is also possible to derive efficient updates for PA and AROW when the loss is the logistic loss, $\log(1 + \exp(-y_t \left\langle z_t, x_t \right\rangle))$. We thus we compare the above six algorithms using both hinge and logistic loss.

6.1 Text Classification

The Reuters RCV1 data set consists of a collection of approximately 800,000 text articles, each of which is assigned multiple labels. There are 4 high-level categories, Economics, Commerce, Medical, and Government (ECAT, CCAT, MCAT, GCAT), and multiple more specific categories. We focus on training binary classifiers for each of the four major categories. The input features we use are 0/1 bigram features, which, post word stemming, give data of approximately 2 million dimensions. The feature vectors are very sparse, however, and most examples have fewer than 5000 non-zero features.

We compare the twelve different algorithms mentioned in the prequel as well as variants of FOBOS and RDA with ℓ_1-regularization. We summarize the results of the ℓ_1-regularized runs as well as AROW and PA in Table 1. The results for both hinge and logistic losses are qualitatively and quantitatively very similar, so we report results only for training with the hinge loss in Table 1. Each row in the table represents the average of four different experiments in which we hold out 25% of the data for a test set and perform an online pass on the remaining 75% of the data. For RDA and FOBOS, we cross-validate the stepsize parameter η by simply running multiple passes and then choosing the output of the learner that had the fewest mistakes during training. For PA and AROW we choose λ using the same approach. We use the same regularization multiplier on the ℓ_1 term for RDA and FOBOS, selected so that RDA achieved approximately 10% non-zero predictors.

It is evident from the results presented in Table 1 that the adaptive algorithms (AROW and ADAGRAD) are far superior to non-adaptive algorithms in terms of error rate on test data. The ADAGRAD algorithms naturally incorporate sparsity as well since they are run with ℓ_1-regularization, though RDA has significantly higher sparsity levels (PA and AROW do not have any sparsity). Furthermore, although omitted from the table to avoid clutter, in *every* test with the RCV1 corpus, the

Alg.	Avg. Prec.	P@1	P@3	P@5	P@10	Prop. nonzero
ADAGRAD RDA	**0.6022**	0.8502	0.8307	0.8130	0.7811	**0.7267**
AROW	0.5813	**0.8597**	**0.8369**	**0.8165**	**0.7816**	1.0000
PA	0.5581	0.8455	0.8184	0.7957	0.7576	1.0000
RDA	0.5042	0.7496	0.7185	0.6950	0.6545	0.8996

Table 2: Test set precision for ImageNet

adaptive algorithms outperformed the non-adaptive algorithms. Moreover, both ADAGRAD-RDA and ADAGRAD-Fobos outperform AROW on all the classification tasks. Unregularized RDA and FOBOS attained similar results as did the ℓ_1-regularized variants (of course without sparsity), but we omit the results to avoid clutter and because they do not give much more understanding.

6.2 Image Ranking

ImageNet (Deng et al., 2009) consists of images organized according to the nouns in the WordNet hierarchy, where each noun is associated on average with more than 500 images collected from the web. We selected 15,000 important nouns from the hierarchy and conducted a large scale image ranking task for *each* noun. This approach is identical to the task tackled by Grangier and Bengio (2008) using the Passive-Aggressive algorithm. To solve this problem, we train 15,000 ranking machines using Grangier and Bengio's visterms features, which represent patches in an image with 79-dimensional sparse vectors. There are approximately 120 patches per image, resulting in a 10,000-dimensional feature space.

Based on the results in the previous section, we focus on four algorithms for solving this task: AROW, ADAGRAD with RDA updates and ℓ_1-regularization, vanilla RDA with ℓ_1, and Passive-Aggressive. We use the ranking hinge loss, which is $[1 - \langle x, z_1 - z_2 \rangle]_+$ when z_1 is ranked above z_2. We train a ranker x_c for each of the image classes individually, cross-validating the choice of initial stepsize for each algorithm on a small held-out set. To train an individual ranker for class c, at each step of the algorithm we randomly sample a positive image z_1 for the category c and an image z_2 from the training set (which with high probability is a negative example for class c) and perform an update on the example $z_1 - z_2$. We let each algorithm take 100,000 such steps for each image category, we train four sets of rankers with each algorithm, and the training set includes approximately 2 million images.

For evaluation, we use a distinct test set of approximately 1 million images. To evaluate a set of rankers, we iterate through all 15,000 classes in the data set. For each class we take all the positive image examples in the test set and sample 10 times as many negative image examples. Following Grangier and Bengio, we then rank the set of positive and negative images and compute precision-at-k for $k = \{1, \ldots, 10\}$ and the average precision for each category. The precision-at-k is defined as the proportion of examples ranked in the top k for a category c that actually belong to c, and the average precision is the average of the precisions at each position in which a relevant picture appears. Letting $\mathrm{Pos}(c)$ denote the positive examples for category c and $p(i)$ denote the position of the ith returned picture in list of images sorted by inner product with x_c, the average precision is

$$\frac{1}{|\mathrm{Pos}(c)|} \sum_{i=1}^{|\mathrm{Pos}(c)|} \frac{i}{p(i)} .$$

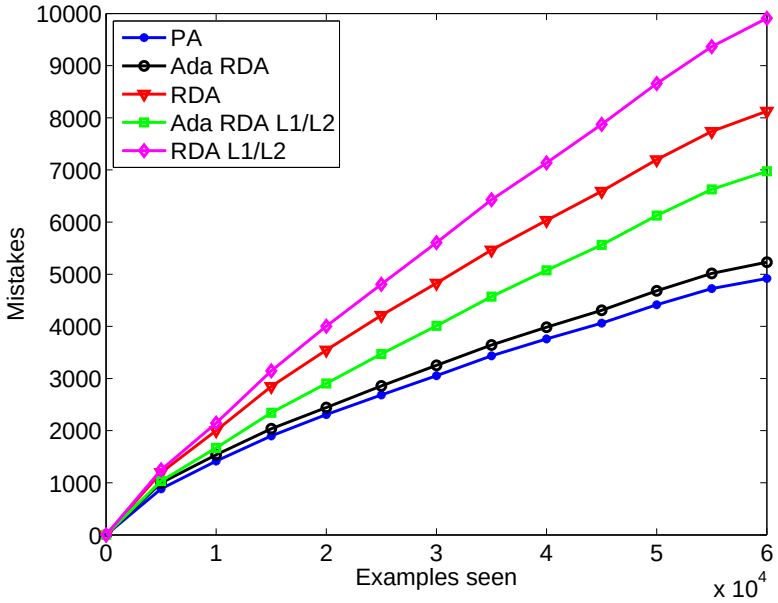


Figure 5: Learning curves on MNIST

We compute the mean of each measurement across all classes, performing this twelve times for each of the sets of rankers trained. Table 2 summarizes our results. We do not report variance as the variance was on the order of 10^{-5} for each algorithm. One apparent characteristic to note from the table is that ADAGRAD RDA achieves higher levels of sparsity than the other algorithms—using only 73% of the input features it achieves very high performance. Moreover, it outperforms all the algorithms in average precision. AROW has better results than the other algorithms in terms of precision-at-k for $k \leq 10$, though ADAGRAD's performance catches up to and eventually surpasses AROW's as k grows.

6.3 Multiclass Optical Character Recognition

In the well-known MNIST multiclass classification data set, we are given 28×28 pixel images a_i, and the learner's task is to classify each image as a digit in $\{0, \ldots, 9\}$. Linear classifiers do not work well on a simple pixel-based representation. Thus we learn classifiers built on top of a kernel machine with Gaussian kernels, as do Duchi and Singer (2009), which gives a different (and non-sparse) structure to the feature space in contrast to our previous experiments. In particular, for the ith example and jth feature, the feature value is $z_{ij} = K(a_i, a_j) \triangleq \exp\left(-\frac{1}{2\sigma^2}\left\|a_i - a_j\right\|_2^2\right)$. We use a support set of approximately 3000 images to compute the kernels and trained multiclass predictors, which consist of one vector $x_c \in \mathbb{R}^{3000}$ for each class c, giving a 30,000 dimensional problem. There is no known multiclass AROW algorithm. We therefore compare adaptive RDA with and without mixed-norm ℓ_1/ℓ_2 and ℓ_1/ℓ_∞ regularization (see Section 5.5), RDA, and multiclass Passive Aggressive to one another using the multiclass hinge loss (Crammer et al., 2006). For each algorithm we used the first 5000 of 60,000 training examples to choose the stepsize η (for RDA) and λ (for PA).

In Figure 5, we plot the learning curves (cumulative mistakes made) of multiclass PA, RDA, RDA with ℓ_1/ℓ_2 regularization, adaptive RDA, and adaptive RDA with ℓ_1/ℓ_2 regularization (ℓ_1/ℓ_∞

	Test error rate	Prop. nonzero
PA	0.062	1.000
Ada-RDA	0.066	1.000
RDA	0.108	1.000
Ada-RDA $\lambda = 5 \cdot 10^{-4}$	0.100	0.569
RDA $\lambda = 5 \cdot 10^{-4}$	0.138	0.878
Ada-RDA $\lambda = 10^{-3}$	0.137	0.144
RDA $\lambda = 10^{-3}$	0.192	0.532

Table 3: Test set error rates and sparsity proportions on MNIST. The scalar λ is the multiplier on the ℓ_1/ℓ_2 regularization term.

is similar). From the curves, we see that Adaptive RDA seems to have similar performance to PA, and the adaptive versions of RDA are vastly superior to their non-adaptive counterparts. Table 3 further supports this, where we see that the adaptive RDA algorithms outperform their non-adaptive counterparts both in terms of sparsity (the proportion of non-zero rows) and test set error rates.

6.4 Income Prediction

The KDD census income data set from the UCI repository (Asuncion and Newman, 2007) contains census data extracted from 1994 and 1995 population surveys conducted by the U.S. Census Bureau. The data consists of 40 demographic and employment related variables which are used to predict whether a respondent has income above or below $50,000. We quantize each feature into bins (5 per feature for continuous features) and take products of features to give a 4001 dimensional feature space with 0/1 features. The data is divided into a training set of 199,523 instances and test set of 99,762 test instances.

As in the prequel, we compare AROW, PA, RDA, and adaptive RDA with and without ℓ_1-regularization on this data set. We use the first 10,000 examples of the training set to select the step size parameters λ for AROW and PA and η for RDA. We perform ten experiments on random shuffles of the training data. Each experiment consists of a training pass through some proportion of the data (.05, .1, .25, .5, or the entire training set) and computing the test set error rate of the learned predictor. Table 4 and Figure 6 summarize the results of these experiments. The variance of the test error rates is on the order of 10^{-6} so we do not report it. As earlier, the table and figure make it clear that the adaptive methods (AROW and ADAGRAD-RDA) give better performance than non-adaptive methods. Further, as detailed in the table, the ADAGRAD methods can give extremely sparse predictors that still give excellent test set performance. This is consistent with the experiments we have seen to this point, where ADAGRAD gives sparse but highly accurate predictors.

6.5 Experiments with Sparsity-Accuracy Tradeoffs

In our final set of experiments, we investigate the tradeoff between the level of sparsity and the classification accuracy for the ADAGRAD-RDA algorithms. Using the same experimental setup as for the initial text classification experiments described in Section 6.1, we record the average test-set performance of ADAGRAD-RDA versus the proportion of features that are non-zero in the predictor ADAGRAD outputs after a single pass through the training data. To achieve this, we run

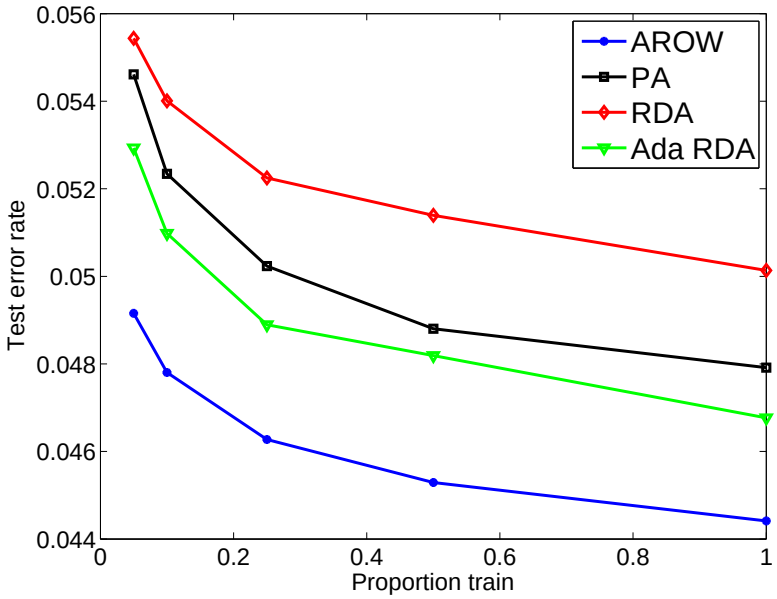


Figure 6: Test set error rates as function of proportion of training data seen on Census Income data set.

Prop. Train	0.05	0.10	0.25	0.50	1.00
AROW	0.049	0.048	0.046	0.045	0.044
PA	0.055	0.052	0.050	0.049	0.048
RDA	0.055	0.054	0.052	0.051	0.050
Ada-RDA	0.053	0.051	0.049	0.048	0.047
ℓ_1 RDA	0.056 (0.075)	0.054 (0.066)	0.053 (0.058)	0.052 (0.053)	0.051 (0.050)
ℓ_1 Ada-RDA	0.052 (0.062)	0.051 (0.053)	0.050 (0.044)	0.050 (0.040)	0.049 (0.037)

Table 4: Test set error rates as function of proportion of training data seen (proportion of non-zeros in parenthesis where appropriate) on Census Income data set.

ADAGRAD with ℓ_1-regularization, and we sweep the regularization multiplier λ from 10^{-8} to 10^{-1}. These values result in predictors ranging from a completely dense predictor to an all-zeros predictor, respectively.

We summarize our results in Figure 7, which shows the test set performance of ADAGRAD for each of the four categories ECAT, CCAT, GCAT, and MCAT. Within each plot, the horizontal black line labeled AROW designates the baseline performance of AROW on the text classification task, though we would like to note that AROW generates fully dense predictors. The plots all portray a similar story. With high regularization values, ADAGRAD exhibits, as expected, poor performance as it retains no predictive information from the learning task. Put another way, when the regularization value is high ADAGRAD is confined to an overly sparse predictor which exhibits poor generalization. However, as the regularization multiplier λ decreases, the learned predictor becomes less sparse and eventually the accuracy of ADAGRAD exceeds AROW's accuracy. It is interesting to note that for these experiments, as soon as the predictor resulting from a *single* pass

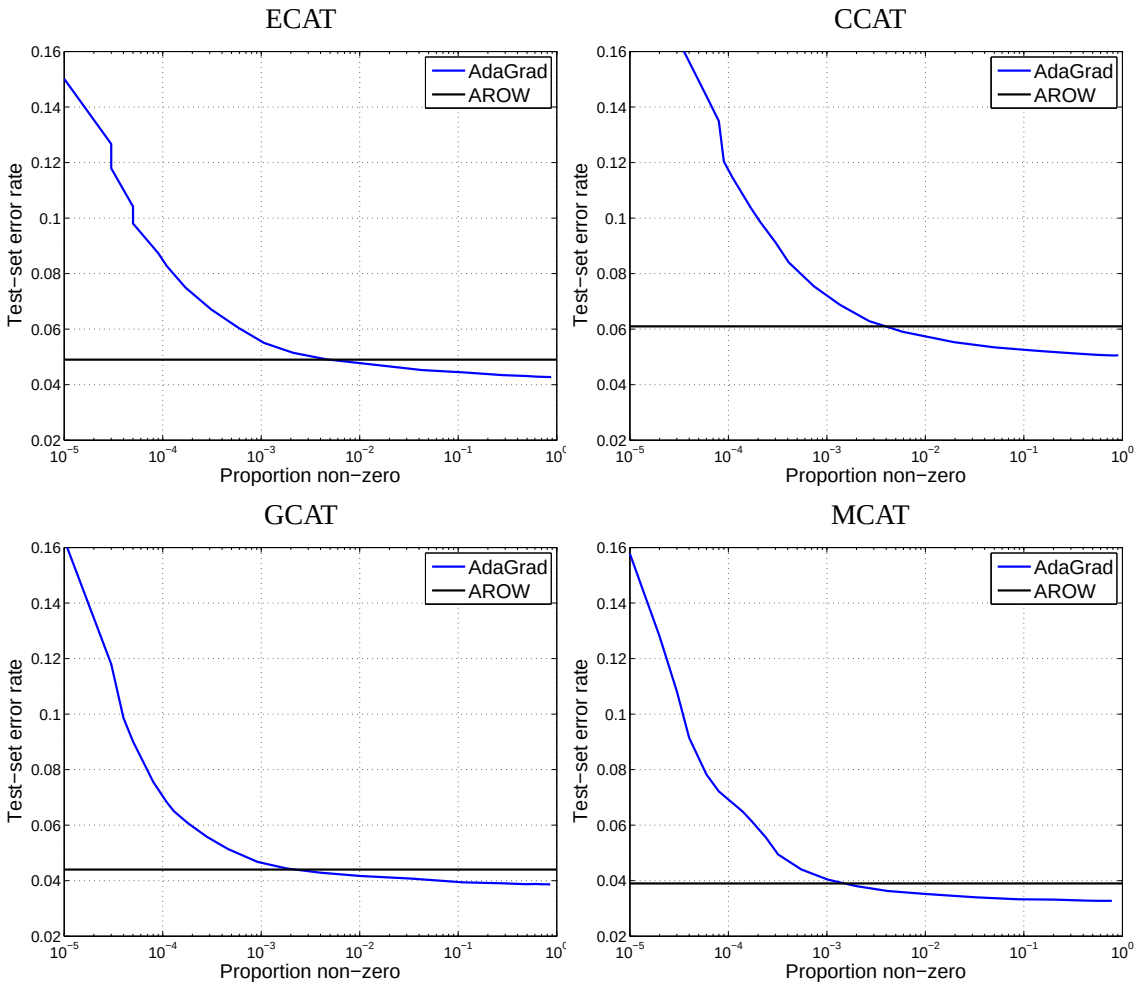


Figure 7: Test set error rates as a function of proportion of non-zeros in predictor x output by ADAGRAD (AROW plotted for reference).

through the data has more than 1% non-zero coefficients, ADAGRAD's performance matches that of AROW. We also would like to note that the variance in the test-set error rates for these experiments is on the order of 10^{-6}, and we thus do not draw error bars in the graphs. The performance of ADAGRAD as a function of regularization for other sparse data sets, especially in relation to that of AROW, was qualitatively similar to this experiment.

7. Conclusions

We presented a paradigm that adapts subgradient methods to the geometry of the problem at hand. The adaptation allows us to derive strong regret guarantees, which for some natural data distributions achieve better performance guarantees than previous algorithms. Our online regret bounds can be naturally converted into rate of convergence and generalization bounds (Cesa-Bianchi et al., 2004). Our experiments show that adaptive methods, specifically ADAGRAD-FOBOS, ADAGRAD-RDA, and AROW clearly outperform their non-adaptive counterparts. Furthermore, the ADAGRAD fam-

ily of algorithms naturally incorporates regularization and gives very sparse solutions with similar performance to dense solutions. Our experiments with adaptive methods use a diagonal approximation to the matrix obtained by taking outer products of subgradients computed along the run of the algorithm. It remains to be tested whether using the full outer product matrix can further improve performance.

To conclude we would like to underscore a possible elegant generalization that interpolates between full-matrix proximal functions and diagonal approximations using block diagonal matrices. Specifically, for $v \in \mathbb{R}^d$ let $v = [v_{[1]}^\top \ \cdots \ v_{[k]}^\top]^\top$ where $v_{[i]} \in \mathbb{R}^{d_i}$ are subvectors of v with $\sum_{i=1}^k d_i = d$. We can define the associated block-diagonal approximation to the outer product matrix $\sum_{\tau=1}^t g_\tau g_\tau^\top$ by

$$G_t = \sum_{\tau=1}^t \begin{bmatrix} g_{\tau,[1]} g_{\tau,[1]}^\top & 0 & \cdots & 0 \\ 0 & g_{\tau,[2]} g_{\tau,[2]}^\top & \ddots & 0 \\ \vdots & \ddots & \ddots & 0 \\ 0 & \cdots & 0 & g_{\tau,[k]} g_{\tau,[k]}^\top \end{bmatrix}.$$

In this case, a combination of Theorems 5 and 7 gives the next corollary.

Corollary 12 *Let G_t be the block-diagonal outer product matrix defined above and the sequence $\{x_t\}$ be defined by the RDA update of (3) with $\psi_t(x) = \left\langle x, G_t^{1/2} x \right\rangle$. Then, for any $x^* \in \mathcal{X}$,*

$$R_\phi(T) \leq \frac{1}{\eta} \max_i \left\| x_{[i]}^* \right\|_2^2 \operatorname{tr}(G_T^{1/2}) + \eta \operatorname{tr}(G_T^{1/2}).$$

A similar bound holds for composite mirror-descent updates, and it is straightforward to get infimal equalities similar to those in Corollary 11 with the infimum taken over block-diagonal matrices. Such an algorithm can interpolate between the computational simplicity of the diagonal proximal functions and the ability of full matrices to capture correlation in the gradient vectors.

A few open questions stem from this line of research. The first is whether we can *efficiently* use full matrices in the proximal functions, as in Section 4. A second open issue is whether non-Euclidean proximal functions, such as the relative entropy, can be used. We also think that the strongly convex case—when f_t or φ is strongly convex—presents interesting challenges that we have not completely resolved. We hope to investigate both empirical and formal extensions of this work in the near future.

Acknowledgments

There are many people to whom we owe our sincere thanks for this research. Fernando Pereira helped push us in the direction of working on adaptive online methods and has been a constant source of discussion and helpful feedback. Samy Bengio provided us with a processed version of the ImageNet data set and was instrumental in helping to get our experiments running, and Adam Sadovsky gave many indispensable coding suggestions. The anonymous reviewers also gave several suggestions that improved the quality of the paper. Lastly, Sam Roweis was a sounding board for some of our earlier ideas on the subject, and we will miss him dearly.

Appendix A. Full Matrix Motivating Example

As in the diagonal case, as the adversary we choose $\eta > 0$ and on rounds $t = 1, \ldots, \eta^2/\varepsilon^2$ play the vector $\pm v_1$. After the first η^2/ε^2 rounds, the adversary simply cycles through the vectors $v_2, \ldots, v_d$. Thus, for Zinkevich's projected gradient, we have $x_t = \alpha_{t,1} v_1$ for some multiplier $\alpha_{t,1} > 0$ when $t \le \eta^2/\varepsilon^2$. After the first η^2/ε^2 rounds, we perform the updates

$$x_{t+1} = \Pi_{\|x\|_2 \le \sqrt{d}} \left(x_t + \frac{\eta}{\sqrt{t}} v_i \right)$$

for some index i, but as in the diagonal case, $\eta/\sqrt{t} \le \varepsilon$, and by orthogonality of v_i, v_j, we have $x_t = V\alpha_t$ for some $\alpha_t \succeq 0$, and the projection step can only shrink the multiplier $\alpha_{t,i}$ for index i. Thus, each coordinate incurs loss at least $1/(2\varepsilon)$, and projected gradient descent suffers losses $\Omega(d/\varepsilon)$.

On the other hand, ADAGRAD suffers loss at most d. Indeed, since $g_1 = v_1$ and $\|v_1\|_2 = 1$, we have $G_1^2 = v_1 v_1^\top v_1 v_1^\top = v_1 v_1^\top = G_1$, so $G_1 = G_1^\dagger = G_1^{\frac{1}{2}}$, and

$$x_2 = x_1 + G_1^\dagger = x_1 + v_1 v_1^\top v_1 = x_1 + v_1.$$

Since $\langle x_2, v_1 \rangle = 1$, we see that ADAGRAD suffers no loss (and $G_t = G_1$) until a vector $z_t = \pm v_i$ for $i \neq 1$ is played by the adversary. However, an identical argument shows that G_t is simply updated to $v_1 v_1^\top + v_i v_i^\top$, in which case $x_t = v_1 + v_i$. Indeed, an inductive argument shows that until all the vectors v_i are seen, we have $\|x_t\|_2 < \sqrt{d}$ by orthogonality, and eventually we have

$$x_t = \sum_{i=1}^d v_i \quad \text{and} \quad \|x_t\|_2 = \sqrt{\sum_{i=1}^d \|v_i\|_2^2} = \sqrt{d}$$

so that $x_t \in \mathcal{X} = \{x : \|x\|_2 \le \sqrt{d}\}$ for ADAGRAD for all t. All future predictions thus achieve margin 1 and suffer no loss.

Appendix B. Technical Lemmas

Lemma 13 *Let $A \succeq B \succeq 0$ be symmetric $d \times d$ PSD matrices. Then $A^{1/2} \succeq B^{1/2}$.*

Proof This is Example 3 of Davis (1963). We include a proof for convenience of the reader. Let λ be any eigenvalue (with corresponding eigenvector x) of $A^{1/2} - B^{1/2}$; we show that $\lambda \ge 0$. Clearly $A^{1/2}x - \lambda x = B^{1/2}x$. Taking the inner product of both sides with $A^{1/2}x$, we have $\left\|A^{1/2}x\right\|_2^2 - \lambda \left\langle A^{1/2}x, x \right\rangle = \left\langle A^{1/2}x, B^{1/2}x \right\rangle$. We use the Cauchy-Schwarz inequality:

$$\left| \left\|A^{1/2}x\right\|_2^2 - \lambda \left\langle A^{1/2}x, x \right\rangle \right| \le \left\|A^{1/2}x\right\|_2 \left\|B^{1/2}x\right\|_2 = \sqrt{\langle Ax, x\rangle \langle Bx, x\rangle} \le \langle Ax, x\rangle = \left\|A^{1/2}x\right\|_2^2$$

where the last inequality follows from the assumption that $A \succeq B$. Thus we must have $\lambda \left\langle A^{1/2}x, x \right\rangle \ge 0$, which implies $\lambda \ge 0$. ■

The gradient of the function $\mathrm{tr}(X^p)$ is easy to compute for integer values of p. However, when p is real we need the following lemma. The lemma tacitly uses the fact that there is a unique positive semidefinite X^p when $X \succeq 0$ (Horn and Johnson, 1985, Theorem 7.2.6).

Lemma 14 *Let $p \in \mathbb{R}$ and $X \succ 0$. Then $\nabla_X \operatorname{tr}(X^p) = pX^{p-1}$.*

Proof We do a first order expansion of $(X+A)^p$ when $X \succ 0$ and A is symmetric. Let $X = U\Lambda U^\top$ be the symmetric eigen-decomposition of X and $VDV^\top$ be the decomposition of $\Lambda^{-1/2}U^\top AU\Lambda^{-1/2}$. Then

$$\begin{aligned}(X+A)^p &= (U\Lambda U^\top + A)^p = U(\Lambda + U^\top AU)^p U^\top = U\Lambda^{p/2}(I + \Lambda^{-1/2}U^\top AU\Lambda^{-1/2})^p\Lambda^{p/2}U^\top \\ &= U\Lambda^{p/2}V^\top(I+D)^pV\Lambda^{p/2}U^\top = U\Lambda^{p/2}V^\top(I + pD + o(D))V\Lambda^{p/2}U^\top \\ &= U\Lambda^p U^\top + pU\Lambda^{p/2}V^\top DV\Lambda^{p/2}U^\top + o(U\Lambda^{-/2}V^\top DV\Lambda^{p/2}U^\top) \\ &= X^p + U\Lambda^{(p-1)/2}U^\top AU\Lambda^{(p-1)/2}U^\top + o(A) = X^p + pX^{(p-1)/2}AX^{(p-1)/2} + o(A).\end{aligned}$$

In the above, $o(A)$ is a matrix that goes to zero faster than $A \to 0$, and the second line follows via a first-order Taylor expansion of $(1+d_i)^p$. From the above, we immediately have

$$\operatorname{tr}((X+A)^p) = \operatorname{tr}X^p + p\operatorname{tr}(X^{p-1}A) + o(\operatorname{tr}A),$$

which completes the proof. ■

Appendix C. Proof of Lemma 4

We prove the lemma by considering an arbitrary real-valued sequence $\{a_i\}$ and its vector representation $a_{1:i} = [a_1 \ \cdots \ a_i]$. We are next going to show that

$$\sum_{t=1}^{T} \frac{a_t^2}{\|a_{1:t}\|_2} \le 2\|a_{1:T}\|_2 \ , \tag{24}$$

where we define $\frac{0}{0} = 0$. We use induction on T to prove inequality (24). For $T = 1$, the inequality trivially holds. Assume the bound (24) holds true for $T-1$, in which case

$$\sum_{t=1}^{T} \frac{a_t^2}{\|a_{1:t}\|_2} = \sum_{t=1}^{T-1} \frac{a_t^2}{\|a_{1:t}\|_2} + \frac{a_T^2}{\|a_{1:T}\|_2} \le 2\|a_{1:T-1}\|_2 + \frac{a_T^2}{\|a_{1:T}\|_2} \ ,$$

where the inequality follows from the inductive hypothesis. We define $b_T = \sum_{t=1}^{T} a_t^2$ and use concavity to obtain that $\sqrt{b_T - a_T^2} \le \sqrt{b_T} - a_T^2 \frac{1}{2\sqrt{b_T}}$ so long as $b_T - a_T^2 \ge 0$.[2] Thus,

$$2\|a_{1:T-1}\|_2 + \frac{a_T^2}{\|a_{1:T}\|_2} = 2\sqrt{b_T - a_T^2} + \frac{a_T^2}{\sqrt{b_T}} \le 2\sqrt{b_T} = 2\|a_{1:T}\|_2 \ .$$

Having proved the bound (24), we note that by construction that $s_{t,i} = \|g_{1:t,i}\|_2$, so

$$\sum_{t=1}^{T} \left\langle g_t, \operatorname{diag}(s_t)^{-1} g_t \right\rangle = \sum_{t=1}^{T}\sum_{i=1}^{d} \frac{g_{t,i}^2}{\|g_{1:t,i}\|_2} \le 2\sum_{i=1}^{d} \|g_{1:T,i}\|_2 \ .$$

2. We note that we use an identical technique in the full-matrix case. See Lemma 8.

Appendix D. Proof of Lemmas 8 and 9

We begin with the more difficult proof of Lemma 8.

Proof of Lemma 8 The core of the proof is based on the concavity of the function $\mathrm{tr}(A^{1/2})$. However, careful analysis is required as A might not be strictly positive definite. We also use the previous lemma which implies that the gradient of $\mathrm{tr}(A^{1/2})$ is $\frac{1}{2}A^{-1/2}$ when $A \succ 0$.

First, A^p is matrix-concave for $A \succ 0$ and $0 \leq p \leq 1$ (see, for example, Corollary 4.1 in Ando, 1979 or Theorem 16.1 in Bondar, 1994). That is, for $A, B \succ 0$ and $\alpha \in [0,1]$ we have

$$(\alpha A + (1-\alpha)B)^p \succeq \alpha A^p + (1-\alpha)B^p \; . \tag{25}$$

Now suppose simply $A, B \succeq 0$ (but neither is necessarily strict). Then for any $\delta > 0$, we have $A + \delta I \succ 0$ and $B + \delta I \succ 0$ and therefore

$$(\alpha(A+\delta I) + (1-\alpha)(B+\delta I))^p \succeq \alpha(A+\delta I)^p + (1-\alpha)(B+\delta I)^p \succeq \alpha A^p + (1-\alpha)B^p \; ,$$

where we used Lemma 13 for the second matrix inequality. Moreover, $\alpha A + (1-\alpha)B + \delta I \to \alpha A + (1-\alpha)B$ as $\delta \to 0$. Since A^p is continuous (when we use the unique PSD root), this line of reasoning proves that (25) holds for $A, B \succeq 0$. Thus, we proved that

$$\mathrm{tr}((\alpha A + (1-\alpha)B)^p) \geq \alpha \, \mathrm{tr}(A^p) + (1-\alpha)\,\mathrm{tr}(B^p) \;\; \text{for } 0 \leq p \leq 1 \; .$$

Recall now that Lemma 14 implies that the gradient of $\mathrm{tr}(A^{1/2})$ is $\frac{1}{2}A^{-1/2}$ when $A \succ 0$. Therefore, from the concavity of $A^{1/2}$ and the form of its gradient, we can use the standard first-order inequality for concave functions so that for any $A, B \succ 0$,

$$\mathrm{tr}(A^{1/2}) \leq \mathrm{tr}(B^{1/2}) + \frac{1}{2}\,\mathrm{tr}(B^{-1/2}(A-B)) \; . \tag{26}$$

Let $A = B - \nu g g^\top \succeq 0$ and suppose only that $B \succeq 0$. We must take some care since $B^{-1/2}$ may not necessarily exist, and the above inequality does not hold true in the pseudo-inverse sense when $B \not\succ 0$. However, for any $\delta > 0$ we know that $2\nabla_B \,\mathrm{tr}((B+\delta I)^{1/2}) = (B+\delta I)^{-1/2}$, and $A - B = -\nu g g^\top$. From (26) and Lemma 13, we have

$$\begin{aligned} 2\,\mathrm{tr}(B - t g g^\top)^{1/2} &= 2\,\mathrm{tr}(A^{1/2}) \;\leq\; 2\,\mathrm{tr}((A+\delta I)^{1/2}) \\ &\leq 2\,\mathrm{tr}(B+\delta I)^{1/2} - \nu\,\mathrm{tr}((B+\delta I)^{-1/2} g g^\top) \; . \end{aligned} \tag{27}$$

Note that $g \in \mathrm{Range}(B)$, because if it were not, we could choose some u with $Bu = 0$ and $\langle g, u\rangle \neq 0$, which would give $\langle u, (B - c g g^\top) u\rangle = -c\,\langle g, u\rangle^2 < 0$, a contradiction. Now let $B = V\,\mathrm{diag}(\lambda) V^\top$ be the eigen-decomposition of B. Since $g \in \mathrm{Range}(B)$,

$$\begin{aligned} g^\top (B+\delta I)^{-1/2} g &= g^\top V \,\mathrm{diag}\left(1/\sqrt{\lambda_i + \delta}\right) V^\top g \\ &= \sum_{i:\lambda_i > 0} \frac{1}{\sqrt{\lambda_i + \delta}} (g^\top v_i)^2 \;\; \underset{\delta \downarrow 0}{\longrightarrow} \;\; \sum_{i:\lambda_i > 0} \lambda_i^{-1/2} (g^\top v_i)^2 \;=\; g^\top (B^\dagger)^{1/2} g \; . \end{aligned}$$

Thus, by taking $\delta \downarrow 0$ in (27), and since both $\mathrm{tr}(B+\delta I)^{1/2}$ and $\mathrm{tr}((B+\delta I)^{-1/2} g g^\top)$ are evidently continuous in δ, we complete the proof. ■

Proof of Lemma 9 We begin by noting that $\delta^2 I \succeq gg^\top$, so from Lemma 13 we get $(A+gg^\top)^{1/2} \preceq (A+\delta^2 I)^{1/2}$. Since A and I are simultaneously diagonalizable, we can generalize the inequality $\sqrt{a+b} \leq \sqrt{a}+\sqrt{b}$, which holds for $a,b \geq 0$, to positive semi-definite matrices, thus,

$$(A+\delta^2 I)^{1/2} \preceq A^{1/2} + \delta I \ .$$

Therefore, if $A+gg^\top$ is of full rank, we have $(A+gg^\top)^{-1/2} \succeq (A^{1/2}+\delta I)^{-1}$ (Horn and Johnson, 1985, Corollary 7.7.4(a)). Since $g \in \text{Range}((A+gg^\top)^{1/2})$, we can apply an analogous limiting argument to the one used in the proof of Lemma 8 and discard all zero eigenvalues of $A+gg^\top$, which completes the lemma. ■

Appendix E. Solution to Problem (15)

We prove here a technical lemma that is useful in characterizing the solution of the optimization problem below. Note that the second part of the lemma implies that we can treat the inverse of the solution matrix S^{-1} as $S^\dagger$. We consider solving

$$\min_S \ \text{tr}(S^{-1}A) \ \ \text{subject to} \ \ S \succeq 0, \ \text{tr}(S) \leq c \ \ \text{where} \ \ A \succeq 0 \ . \tag{28}$$

Lemma 15 *If A is of full rank, then the minimizer of (28) is $S = cA^{\frac{1}{2}}/\text{tr}(A^{\frac{1}{2}})$. If A is not of full rank, then setting $S = cA^{\frac{1}{2}}/\text{tr}(A^{\frac{1}{2}})$ gives*

$$\text{tr}(S^\dagger A) = \inf_S \left\{ \text{tr}(S^{-1}A) : S \succeq 0, \ \text{tr}(S) \leq c \right\} \ .$$

In either case, $\text{tr}(S^\dagger A) = \text{tr}(A^{\frac{1}{2}})^2/c$.

Proof Both proofs rely on constructing the Lagrangian for (28). We introduce $\theta \in \mathbb{R}_+$ for the trace constraint and $Z \succeq 0$ for the positive semidefinite constraint on S. In this case, the Lagrangian is

$$\mathcal{L}(S,\theta,Z) = \text{tr}(S^{-1}A) + \theta(\text{tr}(S)-c) - \text{tr}(SZ).$$

The derivative of $\mathcal{L}$ with respect to S is

$$-S^{-1}AS^{-1} + \theta I - Z. \tag{29}$$

If S is full rank, then to satisfy the generalized complementarity conditions for the problem (Boyd and Vandenberghe, 2004), we must have $Z = 0$. Therefore, we get $S^{-1}AS^{-1} = \theta I$. We now can multiply by S on the right and the left to get that $A = \theta S^2$, which implies that $S \propto A^{\frac{1}{2}}$. If A is of full rank, the optimal solution for $S \succ 0$ forces θ to be positive so that $\text{tr}(S) = c$. This yields the solution $S = cA^{\frac{1}{2}}/\text{tr}(A^{\frac{1}{2}})$. In order to verify optimality of this solution, we set $Z = 0$ and $\theta = c^{-2}\text{tr}(A^{1/2})^2$ which gives $\nabla_S \mathcal{L}(S,\theta,Z) = 0$, as is indeed required.

Suppose now that A is not full rank and that

$$A = Q \begin{bmatrix} \Lambda & 0 \\ 0 & 0 \end{bmatrix} Q^\top$$

is the eigen-decomposition of A. Let n be the dimension of the null-space of A (so the rank of A is $d-n$). Define the variables

$$Z(\theta) = \begin{bmatrix} 0 & 0 \\ 0 & \theta I \end{bmatrix}, \quad S(\theta,\delta) = \frac{1}{\sqrt{\theta}} Q \begin{bmatrix} \Lambda^{\frac{1}{2}} & 0 \\ 0 & \delta I \end{bmatrix} Q^\top, \quad S(\delta) = \frac{c}{\operatorname{tr}(A^{\frac{1}{2}}) + \delta n} Q \begin{bmatrix} \Lambda^{\frac{1}{2}} & 0 \\ 0 & \delta I \end{bmatrix} Q^\top.$$

It is easy to see that $\operatorname{tr} S(\delta) = c$, and

$$\lim_{\delta \to 0} \operatorname{tr}(S(\delta)^{-1} A) = \operatorname{tr}(S(0)^\dagger A) = \operatorname{tr}(A^{\frac{1}{2}}) \operatorname{tr}(\Lambda^{\frac{1}{2}})/c = \operatorname{tr}(A^{\frac{1}{2}})^2/c.$$

Further, let $g(\theta) = \inf_S \mathcal{L}(S,\theta,Z(\theta))$ be the dual of (28). From the above analysis and (29), it is evident that

$$-S(\theta,\delta)^{-1} A S(\theta,\delta)^{-1} + \theta I - Z(\theta) = -\theta Q \begin{bmatrix} \Lambda^{-\frac{1}{2}} \Lambda \Lambda^{-\frac{1}{2}} & 0 \\ 0 & \delta^{-2} I \cdot 0 \end{bmatrix} Q^\top + \theta I - \begin{bmatrix} 0 & 0 \\ 0 & \theta I \end{bmatrix} = 0.$$

So $S(\theta,\delta)$ achieves the infimum in the dual for *any* $\delta > 0$, $\operatorname{tr}(S(0)Z(\theta)) = 0$, and

$$g(\theta) = \sqrt{\theta} \operatorname{tr}(\Lambda^{\frac{1}{2}}) + \sqrt{\theta} \operatorname{tr}(\Lambda^{\frac{1}{2}}) + \sqrt{\theta}\delta n - \theta c.$$

Setting $\theta = \operatorname{tr}(\Lambda^{\frac{1}{2}})^2/c^2$ gives $g(\theta) = \operatorname{tr}(\Lambda^{\frac{1}{2}})^2/c - \delta n \operatorname{tr}(\Lambda^{\frac{1}{2}})/c$. Taking $\delta \to 0$ gives $g(\theta) = \operatorname{tr}(A^{\frac{1}{2}})^2/c$, which means that $\lim_{\delta\to 0} \operatorname{tr}(S(\delta)^{-1}A) = \operatorname{tr}(A^{\frac{1}{2}})^2/c = g(\theta)$. Thus the duality gap for the original problem is 0 so $S(0)$ is the limiting solution.

The last statement of the lemma is simply plugging $S^\dagger = (A^\dagger)^{\frac{1}{2}} \operatorname{tr}(A^{\frac{1}{2}})/c$ in to the objective being minimized. ■

Appendix F. Proofs of Propositions 2 and 3

We begin with the proof of Proposition 2. The proof essentially builds upon Xiao (2010) and Nesterov (2009), with some modification to deal with the indexing of ψ_t. We include the proof for completeness.

Proof of Proposition 2 Define ψ_t^* to be the conjugate dual of $t\varphi(x) + \psi_t(x)/\eta$:

$$\psi_t^*(g) = \sup_{x \in \mathcal{X}} \left\{ \langle g, x \rangle - t\varphi(x) - \frac{1}{\eta}\psi_t(x) \right\}.$$

Since ψ_t/η is $1/\eta$-strongly convex with respect to the norm $\|\cdot\|_{\psi_t}$, the function ψ_t^* has η-Lipschitz continuous gradients with respect to $\|\cdot\|_{\psi_t^*}$:

$$\|\nabla\psi_t^*(g_1) - \nabla\psi_t^*(g_2)\|_{\psi_t} \le \eta \|g_1 - g_2\|_{\psi_t^*} \tag{30}$$

for any g_1, g_2 (see, e.g., Nesterov, 2005, Theorem 1 or Hiriart-Urruty and Lemaréchal, 1996, Chapter X). Further, a simple argument with the fundamental theorem of calculus gives that if f has L-Lipschitz gradients, $f(y) \le f(x) + \langle \nabla f(x), y - x \rangle + (L/2)\|y - x\|^2$, and

$$\nabla\psi_t^*(g) = \operatorname*{argmin}_{x \in \mathcal{X}} \left\{ -\langle g, x \rangle + t\varphi(x) + \frac{1}{\eta}\psi_t(x) \right\}. \tag{31}$$

Using the bound (30) and identity (31), we can give the proof of the corollary. Indeed, letting $g_t \in \partial f_t(x_t)$ and defining $z_t = \sum_{\tau=1}^{t} g_\tau$, we have

$$\begin{aligned}
&\sum_{t=1}^{T} f_t(x_t) + \varphi(x_t) - f_t(x^*) - \varphi(x^*) \\
&\le \sum_{t=1}^{T} \langle g_t, x_t - x^* \rangle - \varphi(x^*) + \varphi(x_t) \\
&\le \sum_{t=1}^{T} \langle g_t, x_t \rangle + \varphi(x_t) + \sup_{x \in \mathcal{X}} \left\{ -\sum_{t=1}^{T} \langle g_t, x \rangle - T\varphi(x) - \frac{1}{\eta}\psi_T(x) \right\} + \psi_T(x^*) \\
&= \frac{1}{\eta}\psi_T(x^*) + \sum_{t=1}^{T} \langle g_t, x_t \rangle + \varphi(x_t) + \psi_T^*(-z_T).
\end{aligned}$$

Since $\psi_{t+1} \ge \psi_t$, it is clear that

$$\begin{aligned}
\psi_T^*(-z_T) &= -\sum_{t=1}^{T} \langle g_t, x_{T+1} \rangle - T\varphi(x_{T+1}) - \frac{1}{\eta}\psi_T(x_{T+1}) \\
&\le -\sum_{t=1}^{T} \langle g_t, x_{T+1} \rangle - (T-1)\varphi(x_{T+1}) - \varphi(x_{T+1}) - \frac{1}{\eta}\psi_{T-1}(x_{T+1}) \\
&\le \sup_{x \in \mathcal{X}} \left(-\langle z_T, x \rangle - (T-1)\varphi(x) - \frac{1}{\eta}\psi_{T-1}(x) \right) - \varphi(x_{T+1}) = \psi_{T-1}^*(-z_T) - \varphi(x_{T+1}).
\end{aligned}$$

The Lipschitz continuity of $\nabla\psi_t^*$, the identity (31), and the fact that $z_T - z_{T-1} = -g_T$ give

$$\begin{aligned}
&\sum_{t=1}^{T} f_t(x_t) + \varphi(x_{t+1}) - f_t(x^*) - \varphi(x^*) \\
&\le \frac{1}{\eta}\psi_T(x^*) + \sum_{t=1}^{T} \langle g_t, x_t \rangle + \varphi(x_{t+1}) + \psi_{T-1}^*(-z_T) - \varphi(x_{T+1}) \\
&\le \frac{1}{\eta}\psi_T(x^*) + \sum_{t=1}^{T} \langle g_t, x_t \rangle + \varphi(x_{t+1}) - \varphi(x_{T+1}) \\
&\qquad + \psi_{T-1}^*(-z_{T-1}) - \left\langle \nabla\psi_{T-1}^*(z_{T-1}), g_T \right\rangle + \frac{\eta}{2}\|g_T\|_{\psi_{T-1}^*}^2 \\
&= \frac{1}{\eta}\psi_T(x^*) + \sum_{t=1}^{T-1} \langle g_t, x_t \rangle + \varphi(x_{t+1}) + \psi_{T-1}^*(-z_{T-1}) + \frac{\eta}{2}\|g_T\|_{\psi_{T-1}^*}^2.
\end{aligned}$$

We can repeat the same sequence of steps that gave the last equality to see that

$$\sum_{t=1}^{T} f_t(x_t) + \varphi(x_{t+1}) - f_t(x^*) - \varphi(x^*) \le \frac{1}{\eta}\psi_T(x^*) + \frac{\eta}{2}\sum_{t=1}^{T} \|g_t\|_{\psi_{t-1}^*}^2 + \psi_0^*(-z_0).$$

Recalling that $x_1 = \operatorname{argmin}_{x \in \mathcal{X}}\{\varphi(x)\}$ and that $\psi_0^*(0) = 0$ completes the proof. ■

We now turn to the proof of Proposition 3. We begin by stating and fully proving an (essentially) immediate corollary to Lemma 2.3 of Duchi et al. (2010).

Lemma 16 *Let $\{x_t\}$ be the sequence defined by the update (4) and assume that $B_{\psi_t}(\cdot,\cdot)$ is strongly convex with respect to a norm $\|\cdot\|_{\psi_t}$. Let $\|\cdot\|_{\psi_t^*}$ be the associated dual norm. Then for any x^*,*

$$\eta\left(f_t(x_t) - f_t(x^*)\right) + \eta\left(\varphi(x_{t+1}) - \varphi(x^*)\right) \leq B_{\psi_t}(x^*, x_t) - B_{\psi_t}(x^*, x_{t+1}) + \frac{\eta^2}{2}\left\|f_t'(x_t)\right\|_{\psi_t^*}^2$$

Proof The optimality of x_{t+1} for (4) implies for all $x \in \mathcal{X}$ and $\varphi'(x_{t+1}) \in \partial\varphi(x_{t+1})$

$$\left\langle x - x_{t+1}, \eta f'(x_t) + \nabla\psi_t(x_{t+1}) - \nabla\psi_t(x_t) + \eta\varphi'(x_{t+1})\right\rangle \geq 0. \tag{32}$$

In particular, this obtains for $x = x^*$. From the subgradient inequality for convex functions, we have $f_t(x^*) \geq f_t(x_t) + \langle f_t'(x_t), x^* - x_t\rangle$, or $f_t(x_t) - f_t(x^*) \leq \langle f_t'(x_t), x_t - x^*\rangle$, and likewise for $\varphi(x_{t+1})$. We thus have

$$\begin{aligned}
&\eta\left[f_t(x_t) + \varphi(x_{t+1}) - f_t(x^*) - \varphi(x^*)\right] \\
&\quad\leq \eta\left\langle x_t - x^*, f_t'(x_t)\right\rangle + \eta\left\langle x_{t+1} - x^*, \varphi'(x_{t+1})\right\rangle \\
&\quad= \eta\left\langle x_{t+1} - x^*, f_t'(x_t)\right\rangle + \eta\left\langle x_{t+1} - x^*, \varphi'(x_{t+1})\right\rangle + \eta\left\langle x_t - x_{t+1}, f_t'(x_t)\right\rangle \\
&\quad= \left\langle x^* - x_{t+1}, \nabla\psi_t(x_t) - \nabla\psi_t(x_{t+1}) - \eta f_t'(x_t) - \eta\varphi'(x_{t+1})\right\rangle \\
&\qquad + \left\langle x^* - x_{t+1}, \nabla\psi_t(x_{t+1}) - \nabla\psi_t(x_t)\right\rangle + \eta\left\langle x_t - x_{t+1}, f_t'(x_t)\right\rangle.
\end{aligned}$$

Now, by (32), the first term in the last equation is non-positive. Thus we have that

$$\begin{aligned}
&\eta\left[f_t(x_t) + \varphi(x_{t+1}) - f_t(x^*) - \varphi(x^*)\right] \\
&\quad\leq \left\langle x^* - x_{t+1}, \nabla\psi_t(x_{t+1}) - \nabla\psi_t(x_t)\right\rangle + \eta\left\langle x_t - x_{t+1}, f_t'(x_t)\right\rangle \\
&\quad= B_{\psi_t}(x^*, x_t) - B_{\psi_t}(x_{t+1}, x_t) - B_{\psi_t}(x^*, x_{t+1}) + \eta\left\langle x_t - x_{t+1}, f_t'(x_t)\right\rangle \\
&\quad= B_{\psi_t}(x^*, x_t) - B_{\psi_t}(x_{t+1}, x_t) - B_{\psi_t}(x^*, x_{t+1}) + \eta\left\langle \eta^{-\frac{1}{2}}(x_t - x_{t+1}), \sqrt{\eta} f_t'(x_t)\right\rangle \\
&\quad\leq B_{\psi_t}(x^*, x_t) - B_{\psi_t}(x_{t+1}, x_t) - B_{\psi_t}(x^*, x_{t+1}) + \frac{1}{2}\left\|x_t - x_{t+1}\right\|_{\psi_t}^2 + \frac{\eta^2}{2}\left\|f_t'(x_t)\right\|_{\psi_t^*}^2 \\
&\quad\leq B_{\psi_t}(x^*, x_t) - B_{\psi_t}(x^*, x_{t+1}) + \frac{\eta^2}{2}\left\|f_t'(x_t)\right\|_{\psi_t^*}^2.
\end{aligned}$$

In the above, the first equality follows from simple algebra with Bregman divergences, the second to last inequality follows from Fenchel's inequality applied to the conjugate functions $\frac{1}{2}\|\cdot\|_{\psi_t}^2$ and $\frac{1}{2}\|\cdot\|_{\psi_t^*}^2$ (Boyd and Vandenberghe, 2004, Example 3.27), and the last inequality follows from the assumed strong convexity of B_{ψ_t} with respect to the norm $\|\cdot\|_{\psi_t}$. ■

Proof of Proposition 3 Sum the equation in the conclusion of Lemma 16. ■

Appendix G. Derivations of Algorithms

In this appendix, we give the formal derivations of the solution to the ADAGRAD update for ℓ_1-regularization and projection to an ℓ_1-ball, as described originally in Section 5.

G.1 ℓ_1-regularization

We give the derivation for the primal-dual subgradient update, as composite mirror-descent is entirely similar. We need to solve update (3), which amounts to

$$\min_x \ \eta \langle \bar{g}_t, x \rangle + \frac{1}{2t}\delta \|x\|_2^2 + \frac{1}{2t} \langle x, \mathrm{diag}(s_t)x \rangle + \eta\lambda \|x\|_1 \ .$$

Let $\hat{x}$ denote the optimal solution of the above optimization problem. Standard subgradient calculus implies that when $|\bar{g}_{t,i}| \leq \lambda$ the solution is $\hat{x}_i = 0$. Similarly, when $\bar{g}_{t,i} < -\lambda$, then $\hat{x}_i > 0$, the objective is differentiable, and the solution is obtained by setting the gradient to zero:

$$\eta\bar{g}_{t,i} + \frac{H_{t,ii}}{t}\hat{x}_i + \eta\lambda = 0 \ , \quad \text{so that} \quad \hat{x}_i = \frac{\eta t}{H_{t,ii}} \left(-\bar{g}_{t,i} - \lambda\right) \ .$$

Likewise, when $\bar{g}_{t,i} > \lambda$ then $\hat{x}_i < 0$, and the solution is $\hat{x}_i = \frac{\eta t}{H_{t,ii}}(-\bar{g}_{t,i} + \lambda)$. Combining the three cases, we obtain the simple update (19) for $x_{t+1,i}$.

G.2 ℓ_1-ball projections

The derivation we give is somewhat terse, and we refer the interested reader to Brucker (1984) or Pardalos and Rosen (1990) for more depth. Recall that our original problem (20) is symmetric in its objective and constraints, so we assume without loss of generality that $v \succeq 0$ (otherwise, we reverse the sign of each negative component in v, then flip the sign of the corresponding component in the solution vector). This gives

$$\min_z \ \frac{1}{2}\|z - v\|_2^2 \ \text{ s.t. } \langle a, z \rangle \leq c, \ z \succeq 0 \ .$$

Clearly, if $\langle a, v \rangle \leq c$ the optimal $z^* = v$, hence we assume that $\langle a, v \rangle > c$. We also assume without loss of generality that $v_i/a_i \geq v_{i+1}/a_{i+1}$ for simplicity of our derivation. (We revisit this assumption at the end of the derivation.) Introducing Lagrange multipliers $\theta \in \mathbb{R}_+$ for the constraint that $\langle a, z \rangle \leq c$ and $\alpha \in \mathbb{R}_+^d$ for the positivity constraint on z, we get

$$\mathcal{L}(z, \alpha, \theta) = \frac{1}{2}\|z - v\|_2^2 + \theta(\langle a, z \rangle - c) - \langle \alpha, z \rangle \, .$$

Computing the gradient of $\mathcal{L}$, we have $\nabla_z \mathcal{L}(z, \alpha, \theta) = z - v + \theta a - \alpha$. Suppose that we knew the optimal $\theta^* \geq 0$. Using the complementarity conditions on z and α for optimality of z (Boyd and Vandenberghe, 2004), we see that the solution z_i^* satisfies

$$z_i^* = \begin{cases} v_i - \theta^* a_i & \text{if } v_i \geq \theta^* a_i \\ 0 & \text{otherwise} \ . \end{cases}$$

Analogously, the complimentary conditions on $\langle a, z \rangle \leq c$ show that given θ^*, we have

$$\sum_{i=1}^d a_i \left[v_i - \theta^* a_i\right]_+ = c \quad \text{or} \quad \sum_{i=1}^d a_i^2 \left[\frac{v_i}{a_i} - \theta^*\right]_+ = c \ .$$

Conversely, had we obtained a value $\theta \geq 0$ satisfying the above equation, then θ would evidently induce the optimal z^* through the equation $z_i = [v_i - \theta a_i]_+$.

Now, let ρ be the largest index in $\{1,\ldots,d\}$ such that $v_i - \theta^* a_i > 0$ for $i \le \rho$ and $v_i - \theta^* a_i \le 0$ for $i > \rho$. From the assumption that $v_i/a_i \le v_{i+1}/a_{i+1}$, we have $v_{\rho+1}/a_{\rho+1} \le \theta^* < v_\rho/a_\rho$. Thus, had we known the last non-zero index ρ, we would have obtained

$$\sum_{i=1}^{\rho} a_i v_i - \frac{v_\rho}{a_\rho} \sum_{i=1}^{\rho} a_i^2 = \sum_{i=1}^{\rho} a_i^2 \left(\frac{v_i}{a_i} - \frac{v_\rho}{a_\rho}\right) < c \ ,$$
$$\sum_{i=1}^{\rho} a_i v_i - \frac{v_{\rho+1}}{a_{\rho+1}} \sum_{i=1}^{\rho} a_i^2 = \sum_{i=1}^{\rho+1} a_i^2 \left(\frac{v_i}{a_i} - \frac{v_{\rho+1}}{a_{\rho+1}}\right) \ge c \ .$$

Given ρ satisfying the above inequalities, we can reconstruct the optimal θ^* by noting that the latter inequality should equal c exactly when we replace v_ρ/a_ρ with θ, that is,

$$\theta^* = \frac{\sum_{i=1}^{\rho} a_i v_i - c}{\sum_{i=1}^{\rho} a_i^2} \ . \tag{33}$$

The above derivation results in the following procedure (when $\langle a, v\rangle > c$). We sort v in descending order of v_i/a_i and find the largest index ρ such that $\sum_{i=1}^{\rho} a_i v_i - (v_\rho/a_\rho)\sum_{i=1}^{\rho-1} a_i^2 < c$. We then reconstruct θ^* using equality (33) and return the soft-thresholded values of v_i (see Algorithm 3). It is easy to verify that the algorithm can be implemented in $O(d \log d)$ time. A randomized search with bookkeeping (Pardalos and Rosen, 1990) can be straightforwardly used to derive a linear time algorithm.

References

J. Abernethy, P. L. Bartlett, A. Rakhlin, and A. Tewari. Optimal strategies and minimax lower bounds for online convex games. In *Proceedings of the Twenty First Annual Conference on Computational Learning Theory*, 2008.

T. Ando. Concavity of certain maps on positive definite matrices and applications to Hadamard products. *Linear Algebra and its Applications*, 26:203–241, 1979.

A. Asuncion and D. J. Newman. UCI machine learning repository, 2007. URL `http://www.ics.uci.edu/~mlearn/MLRepository.html`.

P. Auer and C. Gentile. Adaptive and self-confident online learning algorithms. In *Proceedings of the Thirteenth Annual Conference on Computational Learning Theory*, 2000.

P. L. Bartlett, E. Hazan, and A. Rakhlin. Adaptive online gradient descent. In *Advances in Neural Information Processing Systems 20*, 2007.

A. Beck and M. Teboulle. Mirror descent and nonlinear projected subgradient methods for convex optimization. *Operations Research Letters*, 31:167–175, 2003.

J. V. Bondar. Comments on and complements to *Inequalities: Theory of Majorization and Its Applications*. *Linear Algebra and its Applications*, 199:115–129, 1994.

A. Bordes, L. Bottou, and P. Gallinari. Sgd-qn: Careful quasi-newton stochastic gradient descent. *Journal of Machine Learning Research*, 10:1737–1754, 2009.